

```

1  /* =====
2  NaqshAI — O'zbek milliy naqshlari studiyasi
3  Elektron hisoblash mashinalari (EHM) uchun yaratilgan dastur
4  -----
5  Dasturlash tili : JavaScript (ECMAScript, ECMA-262 / ISO/IEC 16262 standarti)
6  Muallif       : EGAMNAZAROVA SEVARAXON XASANBOY QIZI
7  Talabnoma     : DT 202610311/2
8  Fayl         : naqshai.js
9  Hajmi        : 9 008 qator (ushbu sarlavha bilan birga)
10 -----
11 IZOH. Ushbu matn — dasturning BOSHLANG'ICH MATNI (source code).
12 U hech qanday kompilyator yoki obfuskator tomonidan qayta ishlanmagan:
13 o'zgaruvchi va funksiya nomlari to'liq, izohlar (kommentariyalar) saqlangan,
14 qator tuzilishi asl holida.
15
16 Ushbu faylga FAQAT muallifning shaxsiy kodi kiritilgan. Dastur tarkibidagi
17 uchinchi tomon kutubxonalari — Three.js r128 (MIT litsenziyasi) va
18 Supabase JS v2 (MIT litsenziyasi) — muallifning ijodiy mehnati natijasi
19 emas, shuning uchun bu yerga KIRITILMAGAN.
20 ===== */
21
22 'use strict';
23 /* =====
24 NaqshAI — O'zbek milliy naqshlari studiyasi
25 Yagona fayl: naqsh yadrosi, 3D, AR, Chat, Eksport
26 ===== */
27
28 const TAU = Math.PI * 2;
29
30 /* ----- Global holat ----- */
31 const P = {
32   type:'girihi', sym:8, cmp:4, sw:2.2, ir:0.38,
33   spir:0.34, dens:0.55, lay:2, skew:0, rot:0,
34   pal:'gold', fill:'dual',
35   dvar:0, weave:true, /* PIC: kontakt burchagi varianti va to'qish */
36   _bg:'#f7f1e3'
37 };
38 let TESS = { on:false, group:'p4', cell:150, showCell:false, bleed:true, noring:true };
39 let NORING = 0; /* >0 bo'lsa: markazdagi shu radiusdan katta doiralar (ramkalar) chizilmaydi */
40 const T3D = { tess:false, group:'p4' };
41
42 /* ----- Palitralar (6 ta, har biri 5 rang) ----- */
43 const PAL = {
44   gold: {nm:"Oltin",      bg:'#f7f1e3', c:['#b8912f', '#1d3a63', '#6e5312', '#3f6a95', '#efe2bc']},
45   turquoise: {nm:"Feruza", bg:'#f0f7f6', c:['#0f8b8b', '#b8912f', '#0a5a60', '#6cc0b6', '#e2f2ee']},
46   emerald: {nm:"Zumrad",  bg:'#eff5ec', c:['#1e7a4c', '#b8912f', '#0d4227', '#5cb886', '#d7ecdd']},
47   ruby: {nm:"Yoqut",     bg:'#f9f0ec', c:['#9c2331', '#b8912f', '#57101a', '#cf6572', '#f2d9d6']},
48   navy: {nm:"To'q ko'k",  bg:'#eef1f6', c:['#173263', '#b8912f', '#0c1e40', '#5f7cb4', '#e6e0cb']},
49   earth: {nm:"Tuproq",    bg:'#f6f0e6', c:['#7a5230', '#5f6b4a', '#452e14', '#b08050', '#e6d3af']}
50 };
51
52 /* ----- Rang helperlari ----- */
53 function hex2rgb(h){
54   h = (h||'#000').replace('#','');
55   if(h.length===3) h = h[0]+h[0]+h[1]+h[1]+h[2]+h[2];
56   return [parseInt(h.slice(0,2),16)|0, parseInt(h.slice(2,4),16)|0, parseInt(h.slice(4,6),16)|0];
57 }
58 function lerpColor(c1,c2,t){
59   const a=hex2rgb(c1), b=hex2rgb(c2);
60   const m=i=>Math.round(a[i]+(b[i]-a[i])*t);
61   return 'rgb('+m(0)+' '+m(1)+' '+m(2)+' )';
62 }
63 /* Qoraytirish — lenta konturlari uchun */
64 function shade(c,t){ return lerpColor(c,'#201a0c',t); }
65
66 /* ----- Geometriya helperlari ----- */
67 function pol(a,r){ return [Math.cos(a)*r, Math.sin(a)*r]; }
68 function rayX(p,a,q,b){
69   const dx=Math.cos(a), dy=Math.sin(a), ex=Math.cos(b), ey=Math.sin(b);
70   const den = dx*ey - dy*ex;
71   if(Math.abs(den) < 1e-9) return null;
72   const t = ((q[0]-p[0])*ey - (q[1]-p[1])*ex) / den;
73   return [p[0]+dx*t, p[1]+dy*t];
74 }
75 /* Ikki kesma kesishmasi */
76 function segX(A,B,C,D){
77   const d1x=B[0]-A[0], d1y=B[1]-A[1], d2x=D[0]-C[0], d2y=D[1]-C[1];
78   const den = d1x*d2y - d1y*d2x;
79   if(Math.abs(den)<1e-9) return null;
80   const t = ((C[0]-A[0])*d2y - (C[1]-A[1])*d2x)/den;
81   return [A[0]+d1x*t, A[1]+d1y*t];
82 }
83 /* Doira — 4 kubik Bezier bilan */
84 function circle(ctx,x,y,r){
85   if(NORING && x===0 && y===0 && r >= NORING) return; /* tessellatsiyada doira ramka olib tashlanadi */
86   const k = 0.5522847498*r;
87   ctx.moveTo(x+r,y);
88   ctx.bezierCurveTo(x+r,y+k, x+k,y+r, x,y+r);
89   ctx.bezierCurveTo(x-k,y+r, x-r,y+k, x-r,y);
90   ctx.bezierCurveTo(x-r,y-k, x-k,y-r, x,y-r);
91   ctx.bezierCurveTo(x+k,y-r, x+r,y-k, x+r,y);

```

```

92   ctx.closePath();
93 }
94 /* Yoy — Bezier segmentlar bilan (4/3·tan(Δ/4) formulasi) */
95 function arcB(ctx,cx,cy,r,a0,a1,noMove){
96   let da = a1-a0;
97   const segs = Math.max(1, Math.ceil(Math.abs(da)/(Math.PI/2)));
98   const step = da/segs;
99   const s0 = [cx+Math.cos(a0)*r, cy+Math.sin(a0)*r];
100  if(!noMove) ctx.moveTo(s0[0],s0[1]);
101  for(let i=0;i<segs;i++){
102    const t0=a0+step*i, t1=t0+step;
103    const k = 4/3*Math.tan((t1-t0)/4)*r;
104    ctx.bezierCurveTo(
105      cx+Math.cos(t0)*r - Math.sin(t0)*k, cy+Math.sin(t0)*r + Math.cos(t0)*k,
106      cx+Math.cos(t1)*r + Math.sin(t1)*k, cy+Math.sin(t1)*r - Math.cos(t1)*k,
107      cx+Math.cos(t1)*r, cy+Math.sin(t1)*r);
108  }
109 }
110 function ellipsePath(ctx,x,y,rx,ry,rot){
111   ctx.save(); ctx.translate(x,y); ctx.rotate(rot||0); ctx.scale(rx,ry);
112   circle(ctx,0,0,1);
113   ctx.restore();
114 }
115 /* Catmull-Rom → kubik Bezier */
116 function smoothPath(ctx,pts,closed){
117   const n = pts.length;
118   if(n < 2) return;
119   const get = i => closed ? pts[((i%n)+n)%n] : pts[Math.max(0,Math.min(n-1,i))];
120   ctx.moveTo(pts[0][0], pts[0][1]);
121   const last = closed ? n : n-1;
122   for(let i=0;i<last;i++){
123     const p0=get(i-1), p1=get(i), p2=get(i+1), p3=get(i+2);
124     ctx.bezierCurveTo(
125       p1[0]+(p2[0]-p0[0])/6, p1[1]+(p2[1]-p0[1])/6,
126       p2[0]-(p3[0]-p1[0])/6, p2[1]-(p3[1]-p1[1])/6,
127       p2[0], p2[1]);
128   }
129   if(closed) ctx.closePath();
130 }
131 function polyPath(ctx,pts){
132   if(!pts.length) return;
133   ctx.moveTo(pts[0][0],pts[0][1]);
134   for(let i=1;i<pts.length;i++) ctx.lineTo(pts[i][0],pts[i][1]);
135   ctx.closePath();
136 }
137 function starPath(ctx,n,rOut,rIn,rot0){
138   rot0 = rot0||0;
139   const pts=[];
140   for(let i=0;i<n*2;i++) pts.push(pol(rot0 + Math.PI*i/n, i%2 ? rIn : rOut));
141   polyPath(ctx,pts);
142 }
143 /* Barg — yopiq tomchi (2 kubik Bezier) */
144 function leaf(ctx,x,y,ang,len,wid){
145   ctx.save(); ctx.translate(x,y); ctx.rotate(ang);
146   ctx.moveTo(0,0);
147   ctx.bezierCurveTo(len*0.22,-wid, len*0.72,-wid*0.7, len,0);
148   ctx.bezierCurveTo(len*0.72, wid*0.7, len*0.22, wid, 0,0);
149   ctx.closePath();
150   ctx.restore();
151 }
152 /* Gulbarg — egri yonli, o'tkir uchli */
153 function petalPath(ctx,a,rIn,rOut,halfw){
154   const bl = pol(a-halfw, rIn), br = pol(a+halfw, rIn), tip = pol(a, rOut);
155   const cl1 = pol(a-halfw*1.12, rIn+(rOut-rIn)*0.42), cl2 = pol(a-halfw*0.5, rOut*0.93);
156   const cr2 = pol(a+halfw*1.12, rIn+(rOut-rIn)*0.42), cr1 = pol(a+halfw*0.5, rOut*0.93);
157   ctx.moveTo(bl[0],bl[1]);
158   ctx.bezierCurveTo(cl1[0],cl1[1], cl2[0],cl2[1], tip[0],tip[1]);
159   ctx.bezierCurveTo(cr1[0],cr1[1], cr2[0],cr2[1], br[0],br[1]);
160   const cb = pol(a, rIn*0.86);
161   ctx.bezierCurveTo(cb[0],cb[1], cb[0],cb[1], bl[0],bl[1]);
162   ctx.closePath();
163 }
164 /* Logarifmik spiral: r(θ)=a·e^(bθ) */
165 function spiralPts(b,thMax,L,mirror,phase){
166   const pts=[], a = L/Math.exp(b*thMax), mir = mirror ? -1 : 1;
167   const steps = Math.max(10, Math.round(thMax/0.14));
168   for(let i=0;i<=steps;i++){
169     const th = thMax*i/steps;
170     const r = a*Math.exp(b*th);
171     pts.push([Math.cos(mir*th+(phase||0))*r, Math.sin(mir*th+(phase||0))*r]);
172   }
173   return pts;
174 }
175 function cubicAt(p0,p1,p2,p3,t){
176   const u=1-t;
177   return [
178     u*u*u*p0[0] + 3*u*u*t*p1[0] + 3*u*t*t*p2[0] + t*t*t*p3[0],
179     u*u*u*p0[1] + 3*u*u*t*p1[1] + 3*u*t*t*p2[1] + t*t*t*p3[1]
180   ];
181 }
182

```

```

183  /* ----- Chizish helperlari ----- */
184  function st(ctx,col,w){
185      ctx.save();
186      ctx.strokeStyle=col; ctx.lineWidth=Math.max(0.15,w);
187      ctx.lineCap='round'; ctx.lineJoin='round';
188      ctx.stroke();
189      ctx.restore();
190  }
191  /* Interlacing: pastki chiziq kesishuvda uziladi */
192  function iStroke(ctx,col,w){
193      if(!ctx._svg){
194          ctx.save();
195          ctx.strokeStyle=P._bg; ctx.lineWidth=Math.max(0.4,w)*2.6;
196          ctx.lineCap='round'; ctx.lineJoin='round';
197          ctx.stroke();
198          ctx.restore();
199      }
200      st(ctx,col,w);
201  }
202  /* == LENTA (RIBBON) – sanoat sifatidagi tasma: ikki kontur orasida band ==
203      Haqiqiy giring/knotwork ana shunday ko'rinadi */
204  function ribbon(ctx,band,edge,w){
205      ctx.save();
206      ctx.lineCap='round'; ctx.lineJoin='round';
207      ctx.strokeStyle = edge;
208      ctx.lineWidth = w;
209      ctx.stroke();
210      ctx.strokeStyle = band;
211      ctx.lineWidth = Math.max(0.6, w - Math.max(1.8, w*0.42));
212      ctx.stroke();
213      /* glazur yaltirog'i – lenta o'rtasidagi nozik yorug' chiziq */
214      ctx.strokeStyle = lerpColor(band, '#ffffff', 0.5);
215      ctx.lineWidth = Math.max(0.35, w*0.11);
216      ctx.stroke();
217      ctx.restore();
218  }
219  /* Lenta + interlacing (pastdagi lentani uzadi → ustidan o'tish effekti) */
220  function ribbonI(ctx,band,edge,w){
221      if(!ctx._svg){
222          ctx.save();
223          ctx.strokeStyle = P._bg;
224          ctx.lineWidth = w + Math.max(3, w*0.55);
225          ctx.lineCap='round'; ctx.lineJoin='round';
226          ctx.stroke();
227          ctx.restore();
228      }
229      ribbon(ctx,band,edge,w);
230  }
231  function fillA(ctx,col,alpha){
232      if(P.fill==='line') return;
233      ctx.save();
234      ctx.globalAlpha = (P.fill==='fill' ? 0.94 : 0.62) * (alpha===undefined?1:alpha);
235      ctx.fillStyle = col;
236      ctx.fill();
237      ctx.restore();
238  }
239  function fillHard(ctx,col,alpha){
240      ctx.save();
241      ctx.globalAlpha = alpha===undefined?1:alpha;
242      ctx.fillStyle = col;
243      ctx.fill();
244      ctx.restore();
245  }
246  function dot(ctx,x,y,r,col){
247      ctx.beginPath(); circle(ctx,x,y,r);
248      ctx.save(); ctx.fillStyle=col; ctx.fill(); ctx.restore();
249  }
250
251  /* =====
252      A. O'ZBEK GEOMETRIK NAQSHLARI (8) – kanonik qurilish
253      ===== */
254
255  /* Rozetta geometriyasi: yulduz o'zagi + kite-gulbarglar (klassik islomiy rozetta) */
256  function rosetteGeo(n,R,ir){
257      const R0 = R*0.92; /* gulbarg tashqi radiusi */
258      const r2 = R*(0.36 + ir*0.28); /* yulduz uchlari */
259      const r1 = r2*0.55; /* yulduz ichki */
260      const rm = r2 + (R0-r2)*0.52; /* gulbarg yelkasi */
261      const w = (Math.PI/n)*0.86; /* gulbarg yarim eni */
262      return {R0,r2,r1,rm,w};
263  }
264  /* Kite-gulbarg – biroz bo'rtgan yonlar bilan */
265  function kitePath(ctx,a,g){
266      const A = pol(a, g.r2), L = pol(a-g.w, g.rm), T = pol(a, g.R0), Rr = pol(a+g.w, g.rm);
267      const bow = 0.14;
268      const q=(P1,P2)=>{ /* biroz tashqariga bo'rtgan Bezier yon */
269          const mx=(P1[0]+P2[0])/2, my=(P1[1]+P2[1])/2;
270          const d=Math.hypot(mx,my)||1;
271          const cx2=mx*(1+bow*40/d), cy2=my*(1+bow*40/d);
272          ctx.bezierCurveTo(P1[0]+(cx2-P1[0])*0.6,P1[1]+(cy2-P1[1])*0.6, P2[0]+(cx2-P2[0])*0.6,P2[1]+(cy2-P2[1])*0.6, P2
[0],P2[1]);

```

```

273     };
274     ctx.moveTo(A[0],A[1]); q(A,L); q(L,T); q(T,Rr); q(Rr,A);
275     ctx.closePath();
276 }
277
278 /* 1. GIRIH – rozetta strapwork: to'qilgan lentali yulduz + kite-gulbarglar */
279 function drawGirih(ctx,R,p,sw,c0,c1,c2,c3){
280     const n = Math.max(6, p.sym);
281     const g = rosetteGeo(n,R,p.ir);
282     g.w *= (0.72 + p.dens*0.55); /* zichlik → gulbarg eni */
283     g.r1 = g.r2*(0.66 - p.spir*0.28); /* spiral → yulduz o'tkirligi */
284     const edge = shade(c0,0.5);
285     const bw = sw*2.2 + 1.5;
286     /* 1) kite-gulbarglar foni */
287     for(let i=0;i<n;i++){
288         const a = TAU*i/n - Math.PI/2;
289         ctx.beginPath(); kitePath(ctx,a,g);
290         fillA(ctx, i%2? c3 : lerpColor(c3,c1,0.25), 0.75);
291         st(ctx, shade(c2,0.25), sw*0.45);
292         /* o'rta tomir – chizmachil aniqlik chizig'i */
293         const v0 = pol(a, g.r2*1.05), v1 = pol(a, g.R0*0.93);
294         ctx.beginPath();
295         ctx.moveTo(v0[0],v0[1]);
296         ctx.bezierCurveTo(v0[0],v0[1], v1[0],v1[1], v1[0],v1[1]);
297         st(ctx, shade(c2,0.12), sw*0.3);
298     }
299     /* 2) yulduz o'zagi to'ldirilgan */
300     ctx.beginPath(); starPath(ctx, n, g.r2, g.r1, -Math.PI/2);
301     fillA(ctx, c1, 0.9);
302     /* 3) LENTALAR: {n/k} vatarlar, ikki o'tishda to'qiladi */
303     const k = Math.max(2, Math.min(n-2, Math.floor(n/2)-1));
304     const pts=[]; for(let i=0;i<n;i++) pts.push(pol(TAU*i/n - Math.PI/2, g.R0));
305     const chord=(i)>=>{ const A=pts[i], B=pts[(i+k)%n]; ctx.beginPath(); ctx.moveTo(A[0],A[1]); ctx.lineTo(B[0],B[1])
};
306     for(let pass=0; pass<2; pass++){
307         for(let i=pass; i<n; i+=2){ chord(i); ribbonI(ctx, c0, edge, bw); }
308     }
309     /* 4) markaziy vatarlarning o'rta bo'laklari – to'qishni yakunlash */
310     for(let i=0;i<n;i+=2){
311         const A=pts[i], B=pts[(i+k)%n];
312         ctx.beginPath();
313         ctx.moveTo(A[0]+(B[0]-A[0])*0.42, A[1]+(B[1]-A[1])*0.42);
314         ctx.lineTo(A[0]+(B[0]-A[0])*0.58, A[1]+(B[1]-A[1])*0.58);
315         ribbonI(ctx, c0, edge, bw);
316     }
317     /* 4b) gulbarglar orasidagi interstitsial nayzalar */
318     for(let i=0;i<n;i++){
319         const a = TAU*(i+0.5)/n - Math.PI/2;
320         ctx.beginPath();
321         const A2 = pol(a-g.w*0.5, g.rm), B2 = pol(a+g.w*0.5, g.rm), T2 = pol(a, g.R0*0.995), I2 = pol(a, g.r2*1.12);
322         ctx.moveTo(I2[0],I2[1]);
323         ctx.bezierCurveTo(A2[0],A2[1], A2[0],A2[1], T2[0],T2[1]);
324         ctx.bezierCurveTo(B2[0],B2[1], B2[0],B2[1], I2[0],I2[1]);
325         ctx.closePath();
326         fillA(ctx, c0, 0.4);
327         st(ctx, shade(c2,0.2), sw*0.35);
328     }
329     /* 5) tashqi halqa lentasi + tugunlarda mini-yulduzlar */
330     ctx.beginPath(); circle(ctx,0,0,g.R0); ribbon(ctx, c1, shade(c1,0.45), bw*0.85);
331     ctx.beginPath(); circle(ctx,0,0,R); st(ctx, c2, sw*0.6);
332     for(let i=0;i<n;i++){
333         const q = pol(TAU*i/n - Math.PI/2, g.R0);
334         ctx.beginPath(); starPath(ctx, 8, bw*1.05, bw*0.5, 0);
335         ctx.save(); ctx.translate(q[0],q[1]);
336         ctx.beginPath(); starPath(ctx, 8, bw*1.05, bw*0.5, 0);
337         fillHard(ctx, c3, 0.95); st(ctx, shade(c0,0.35), sw*0.35);
338         ctx.restore();
339     }
340     dot(ctx,0,0,g.r1*0.35,c0);
341     /* murakkablik ≥5 → ichki mini-rozetta halqasi */
342     if(p.cmp>=5){
343         ctx.save();
344         ctx.scale(0.4,0.4);
345         ctx.rotate(Math.PI/n);
346         const g2 = rosetteGeo(n, R, p.ir);
347         for(let i=0;i<n;i++){
348             ctx.beginPath(); kitePath(ctx, TAU*i/n - Math.PI/2, g2);
349             fillA(ctx, i%2? c1 : c3, 0.65);
350             st(ctx, shade(c2,0.25), sw*0.9);
351         }
352         ctx.beginPath(); starPath(ctx, n, g2.r2, g2.r2*0.5, -Math.PI/2);
353         fillA(ctx, c0, 0.85);
354         st(ctx, edge, sw*0.8);
355         ctx.restore();
356     }
357 }
358
359 /* 2. XATAM – 8 qirrali yulduz: ikki to'qilgan kvadrat lenta */
360 function drawXatam(ctx,R,p,sw,c0,c1,c2,c3){
361     const edge = shade(c0,0.5);
362     const bw = sw*2.4 + 1.5;

```

```

363 const levels = 1 + (p.cmp>=5 ? 1 : 0);
364 for(let lev=0; lev<levels; lev++){
365   const r = R*0.9*Math.pow(0.5, lev);
366   const rot = -Math.PI/4 + lev*Math.PI/8;
367   const sq = (rr,rt)=>{ const q=[]; for(let i=0;i<4;i++) q.push(pol(rt+TAU*i/4, rr)); return q; };
368   const A = sq(r, rot), B = sq(r, rot+Math.PI/4);
369   /* 8 qirrali yulduz foni */
370   ctx.beginPath(); starPath(ctx, 8, r, r*0.765, rot);
371   fillA(ctx, lev? c3 : c1, 0.55);
372   /* ichki sakkizburchak */
373   ctx.beginPath();
374   const oct=[]; for(let i=0;i<8;i++) oct.push(pol(rot+Math.PI/8+TAU*i/8, r*0.545));
375   polyPath(ctx,oct);
376   fillA(ctx, lev? c1 : c3, 0.8);
377   st(ctx, shade(c2,0.25), sw*0.5);
378   /* TO'QILGAN kvadratlar: A pastda, B ustida, so'ng A ning 2 qarama-qarshi
379      burchagi qayta ustiga – navbatlashgan haqiqiy weave */
380   ctx.beginPath(); polyPath(ctx,A); ribbon(ctx, c0, edge, bw);
381   ctx.beginPath(); polyPath(ctx,B); ribbonI(ctx, lev? c0 : c1, shade(lev? c0 : c1, 0.5), bw);
382   for(const j of [0,2]){
383     ctx.beginPath();
384     const P1=A[j], P2=A[(j+1)%4];
385     ctx.moveTo(P1[0]+(P2[0]-P1[0])*0.28, P1[1]+(P2[1]-P1[1])*0.28);
386     ctx.lineTo(P1[0]+(P2[0]-P1[0])*0.72, P1[1]+(P2[1]-P1[1])*0.72);
387     const P3=A[(j+2)%4], P4=A[(j+3)%4];
388     ctx.moveTo(P3[0]+(P4[0]-P3[0])*0.28, P3[1]+(P4[1]-P3[1])*0.28);
389     ctx.lineTo(P3[0]+(P4[0]-P3[0])*0.72, P3[1]+(P4[1]-P3[1])*0.72);
390     ribbonI(ctx, c0, edge, bw);
391     break;
392   }
393 }
394 /* yulduz uchlaridan tashqi halqaga bog'lovchi nurlar */
395 ctx.beginPath();
396 for(let i=0;i<8;i++){
397   const a = -Math.PI/4 + TAU*i/8;
398   const A2 = pol(a, R*0.9), B2 = pol(a, R);
399   ctx.moveTo(A2[0],A2[1]); ctx.lineTo(B2[0],B2[1]);
400 }
401 st(ctx, c2, sw*0.6);
402 ctx.beginPath(); circle(ctx,0,0,R); ribbon(ctx, c1, shade(c1,0.45), sw*1.6+1);
403 dot(ctx,0,0,R*0.05,c0);
404 }
405
406 /* 3. ROZETTA – gulbarg halqalari, konsentrik bandlar (loyqasiz) */
407 function drawRozetta(ctx,R,p,sw,c0,c1,c2,c3){
408   const n = Math.max(5,p.sym);
409   const w = (Math.PI/n)*(0.55 + p.dens*0.35);
410   /* tashqi gulbarg halqasi */
411   for(let i=0;i<n;i++){
412     const a = TAU*i/n - Math.PI/2;
413     ctx.beginPath(); petalPath(ctx, a, R*0.46, R*0.95, w);
414     fillA(ctx, i%2? c1 : c3, 0.65);
415     st(ctx, shade(c0,0.35), sw*0.7);
416     /* o'rta tomir */
417     const b0 = pol(a, R*0.5), b1 = pol(a, R*0.86);
418     ctx.beginPath(); ctx.moveTo(b0[0],b0[1]);
419     ctx.bezierCurveTo(b0[0],b0[1], b1[0],b1[1], b1[0],b1[1]);
420     st(ctx, shade(c2,0.15), sw*0.35);
421   }
422   /* oraliq kichik gulbarglar – o'z bandida */
423   for(let i=0;i<n;i++){
424     const a = TAU*(i+0.5)/n - Math.PI/2;
425     ctx.beginPath(); petalPath(ctx, a, R*0.24, R*0.5, w*0.62);
426     fillA(ctx, c0, 0.55);
427     st(ctx, shade(c0,0.4), sw*0.5);
428   }
429   /* markaz: halqa + yulduzcha – ichki radius slayderiga bog'liq */
430   const rc = R*(0.13 + p.ir*0.28);
431   ctx.beginPath(); circle(ctx,0,0,rc); ribbon(ctx, c1, shade(c1,0.45), sw*1.6+1);
432   ctx.beginPath(); starPath(ctx, n, rc*0.72, rc*0.38, -Math.PI/2);
433   fillHard(ctx, c0, 0.9); st(ctx, shade(c0,0.4), sw*0.4);
434   /* murakkablik → nuqta halqalari */
435   const dotRings = Math.max(0, Math.round(p.cmp/2) - 1);
436   for(let k=0;k<dotRings;k++){
437     const rd = rc + (R*0.46 - rc)*(k+1)/(dotRings+1);
438     for(let i=0;i<n*2;i++){
439       const q = pol(TAU*(i+0.5)/(n*2) - Math.PI/2, rd);
440       dot(ctx, q[0], q[1], sw*0.7, k%2? c2 : c1);
441     }
442   }
443   ctx.beginPath(); circle(ctx,0,0,R); st(ctx, c2, sw*0.55);
444   dot(ctx,0,0,rc*0.16,c1);
445 }
446
447 /* 4. YULDUZ – to'qilgan {n/k} lenta yulduz */
448 function drawYulduz(ctx,R,p,sw,c0,c1,c2,c3){
449   const n = Math.max(5,p.sym);
450   const k = Math.max(2, Math.min(n-2, Math.floor(n/2)-1 + Math.round(p.spir*1.5)));
451   const r = R*0.9;
452   const edge = shade(c0,0.5);
453   const bw = sw*2.4 + 1.5;

```

```

454 const pts=[]; for(let i=0;i<n;i++) pts.push(pol(TAU*i/n - Math.PI/2, r));
455 /* markaziy n-burchak o'zak: vatarlar ichki kesishmalaridan */
456 const inner=[];
457 for(let i=0;i<n;i++){
458   const X = segX(pts[i], pts[(i+k)%n], pts[(i+1)%n], pts[(i+1-k+n)%n]);
459   if(X) inner.push(X);
460 }
461 if(inner.length>=3){
462   ctx.beginPath(); polyPath(ctx, inner);
463   fillA(ctx, c1, 0.85);
464 }
465 /* to'qilgan lenta vatarlar — ikki o'tish + o'rta segment ustidan */
466 const chord=(i)=>{ const A=pts[i],B=pts[(i+k)%n]; ctx.beginPath(); ctx.moveTo(A[0],A[1]); ctx.lineTo(B[0],B[1]);
};
467 for(let pass=0; pass<2; pass++){
468   for(let i=pass; i<n; i+=2){ chord(i); ribbonI(ctx, i%2? c1 : c0, shade(i%2? c1 : c0, 0.5), bw); }
469 }
470 for(let i=0;i<n;i+=2){
471   const A=pts[i], B=pts[(i+k)%n];
472   ctx.beginPath();
473   ctx.moveTo(A[0]+(B[0]-A[0])*0.44, A[1]+(B[1]-A[1])*0.44);
474   ctx.lineTo(A[0]+(B[0]-A[0])*0.56, A[1]+(B[1]-A[1])*0.56);
475   ribbonI(ctx, c0, edge, bw);
476 }
477 /* tashqi halqa lentasi + tugun nuqtalar */
478 ctx.beginPath(); circle(ctx,0,0,r); ribbon(ctx, c1, shade(c1,0.45), bw*0.8);
479 for(const q of pts) dot(ctx,q[0],q[1], bw*0.45, c0);
480 ctx.beginPath(); circle(ctx,0,0,R); st(ctx, c2, sw*0.5);
481 dot(ctx,0,0,R*0.04,c0);
482 }
483
484 /* 5. MUQARNAS — toza pog'onali arkada: ravoq lentalar + tokchalar */
485 function drawMuqarnas(ctx,R,p,sw,c0,c1,c2,c3){
486   const levels = Math.max(2, Math.min(6, 1 + p.cmp));
487   for(let l=0;l<levels;l++){
488     const r = R*0.94*Math.pow(0.74, l);
489     const m = Math.max(6, p.sym + l*2);
490     const drop = r*(0.14 + p.ir*0.2);
491     const t = l/(levels-1||1);
492     const band = lerpColor(c0,c1,t);
493     /* tokcha (nisha) fonlari */
494     for(let i=0;i<m;i++){
495       const a0 = TAU*i/m - Math.PI/2, a1 = TAU*(i+1)/m - Math.PI/2;
496       const A = pol(a0,r), B = pol(a1,r), M = pol((a0+a1)/2, r-drop);
497       ctx.beginPath();
498       ctx.moveTo(A[0],A[1]);
499       ctx.bezierCurveTo(A[0]*(1-drop/r*0.5),A[1]*(1-drop/r*0.5), M[0]+(A[0]-B[0])*0.15,M[1]+(A[1]-B[1])*0.15, M[0]
,M[1]);
500       ctx.bezierCurveTo(M[0]+(B[0]-A[0])*0.15,M[1]+(B[1]-A[1])*0.15, B[0]*(1-drop/r*0.5),B[1]*(1-drop/r*0.5), B[0]
,B[1]);
501       arcB(ctx,0,0,r,a1,a0,true);
502       ctx.closePath();
503       fillA(ctx, i%2? c3 : lerpColor(c3,c1,0.3), 0.4 + t*0.2);
504     }
505     /* ravoq lentasi */
506     ctx.beginPath();
507     for(let i=0;i<m;i++){
508       const a0 = TAU*i/m - Math.PI/2, a1 = TAU*(i+1)/m - Math.PI/2;
509       const A = pol(a0,r), B = pol(a1,r), M = pol((a0+a1)/2, r-drop);
510       ctx.moveTo(A[0],A[1]);
511       ctx.bezierCurveTo(A[0]*(1-drop/r*0.5),A[1]*(1-drop/r*0.5), M[0]+(A[0]-B[0])*0.15,M[1]+(A[1]-B[1])*0.15, M[0]
,M[1]);
512       ctx.bezierCurveTo(M[0]+(B[0]-A[0])*0.15,M[1]+(B[1]-A[1])*0.15, B[0]*(1-drop/r*0.5),B[1]*(1-drop/r*0.5), B[0]
,B[1]);
513     }
514     ribbon(ctx, band, shade(band,0.45), sw*1.5+0.8);
515     /* osilgan tomchi nuqtalari */
516     for(let i=0;i<m;i++){
517       const M = pol(TAU*(i+0.5)/m - Math.PI/2, r-drop);
518       dot(ctx, M[0], M[1], sw*0.9+0.6, shade(band,0.3));
519     }
520   }
521   ctx.beginPath(); circle(ctx,0,0,R); st(ctx,c2,sw*0.55);
522   const core = R*Math.pow(0.74, levels);
523   ctx.beginPath(); starPath(ctx, p.sym, core, core*0.5, -Math.PI/2);
524   fillHard(ctx, c0, 0.9);
525 }
526
527 /* 6. KATAK-8 — 4.8.8 Arximed kafeli, aniq hizalangan */
528 function drawKatak8(ctx,R,p,sw,c0,c1,c2,c3){
529   const FIELD = !!p.field;
530   const cells = FIELD ? Math.max(2, Math.round(R*1.7/(p._cellpx||150))) : Math.max(2, Math.round(2 + p.dens*2));
531   const s = R*1.7/cells;
532   const oR = s*0.5/Math.cos(Math.PI/8); /* sakkizburchak "radiusi" */
533   const half = cells*s/2;
534   ctx.save();
535   if(!FIELD){
536     ctx.beginPath(); circle(ctx,0,0,R*0.98);
537     if(!ctx._svg) ctx.clip();
538   }
539   for(let gy=0; gy<=cells; gy++){

```

```

540     for(let gx=0; gx<=cells; gx++){
541         const x = -half + gx*s, y = -half + gy*s;
542         if(Math.hypot(x,y) > R*1.05) continue;
543         /* sakkizburchak */
544         const oct=[]; for(let i=0;i<8;i++) oct.push([x+Math.cos(Math.PI/8+TAU*i/8)*oR, y+Math.sin(Math.PI/8+TAU*i/8)
*oR]);
545         ctx.beginPath(); polyPath(ctx,oct);
546         fillA(ctx, (gx+gy)%2? c3 : lerpColor(c3,'#ffffff',0.4), 0.8);
547         ribbon(ctx, c0, shade(c0,0.45), sw*1.4+0.8);
548         /* ichki yulduzcha */
549         if(p.cmp>=3){
550             ctx.beginPath();
551             const sp=[]; for(let i=0;i<16;i++) sp.push([x+Math.cos(Math.PI/8+Math.PI*i/8)*(i%2? oR*0.22 : oR*0.5), y+M
ath.sin(Math.PI/8+Math.PI*i/8)*(i%2? oR*0.22 : oR*0.5)]);
552             polyPath(ctx,sp);
553             fillA(ctx, c1, 0.8); st(ctx, shade(c1,0.35), sw*0.4);
554         }
555     }
556 }
557 /* kichik kvadratlar – panjara tugunlari */
558 const q = s*0.5 - oR*Math.cos(Math.PI/8);
559 for(let gy=0; gy<cells; gy++){
560     for(let gx=0; gx<cells; gx++){
561         const x = -half + gx*s + s/2, y = -half + gy*s + s/2;
562         if(Math.hypot(x,y) > R*1.05) continue;
563         ctx.beginPath();
564         polyPath(ctx,[[x,y-q*1.35],[x+q*1.35,y],[x,y+q*1.35],[x-q*1.35,y]]);
565         fillHard(ctx, c0, 0.9);
566         st(ctx, shade(c0,0.45), sw*0.5);
567     }
568 }
569 ctx.restore();
570 if(!FIELD){
571     ctx.beginPath(); circle(ctx,0,0,R*0.98); ribbon(ctx, c1, shade(c1,0.45), sw*1.8+1);
572 }
573 }
574
575 /* 7. OLTİ BURCHAK – asal uyasi, toza lenta konturlar */
576 function drawHexa(ctx,R,p,sw,c0,c1,c2,c3){
577     const FIELD = !!p._field;
578     const s = FIELD ? Math.max(26,(p._cellpx||150)*0.42) : R/(2.0 + p.dens*2.0);
579     const dx = s*Math.sqrt(3), dy = s*1.5;
580     ctx.save();
581     if(!FIELD){
582         ctx.beginPath(); circle(ctx,0,0,R*0.98);
583         if(!ctx._svg) ctx.clip();
584     }
585     const NN = FIELD ? Math.ceil(R/(s*1.5))+1 : 5;
586     for(let gy=-NN; gy<=NN; gy++){
587         for(let gx=-NN; gx<=NN; gx++){
588             const x = gx*dx + (gy%2 ? dx/2 : 0), y = gy*dy;
589             if(Math.hypot(x,y) > R*1.08) continue;
590             const pts=[]; for(let i=0;i<6;i++) pts.push([x+Math.cos(Math.PI/6+TAU*i/6)*s, y+Math.sin(Math.PI/6+TAU*i/6)*
s]);
591             ctx.beginPath(); polyPath(ctx,pts);
592             const ci = ((gx%3)+3+(gy%3)*2)%3;
593             fillA(ctx, [c3, lerpColor(c3,'#ffffff',0.45), lerpColor(c3,c1,0.25)][ci], 0.7);
594             ribbon(ctx, c0, shade(c0,0.45), sw*1.3+0.7);
595             if(p.cmp>=4){
596                 for(let i=0;i<6;i++){
597                     ctx.beginPath();
598                     leaf(ctx, x, y, Math.PI/6+TAU*i/6, s*0.52, s*0.14);
599                     fillA(ctx, c1, 0.75); st(ctx, shade(c1,0.35), sw*0.35);
600                 }
601                 dot(ctx,x,y,s*0.1,c0);
602             }
603         }
604     }
605     ctx.restore();
606     if(!FIELD){
607         ctx.beginPath(); circle(ctx,0,0,R*0.98); ribbon(ctx, c1, shade(c1,0.45), sw*1.8+1);
608     }
609 }
610
611 /* 8. ZANJIRA – bir-biriga o'tkazilgan halqalar zanjiri (haqiqiy interlock) */
612 function drawZanjira(ctx,R,p,sw,c0,c1,c2,c3){
613     const n = Math.max(6, p.sym);
614     const rc = R*0.68;
615     const rr = Math.PI*rc/n * (0.92 + p.dens*0.35); /* qo'shnilar bilan kesishadi */
616     const bw = sw*2.2 + 1.5;
617     const centers=[]; for(let i=0;i<n;i++) centers.push(pol(TAU*i/n - Math.PI/2, rc));
618     /* har halqa – lenta; keyingisi oldingisini kesib o'tadi */
619     for(let i=0;i<n;i++){
620         const c = centers[i], col = i%2? c1 : c0;
621         ctx.beginPath(); circle(ctx, c[0], c[1], rr);
622         ribbonI(ctx, col, shade(col,0.5), bw);
623     }
624     /* birinchi halqaning oxirgi halqa tomonidagi yoyi – zanjirni ulash */
625     const a0 = Math.atan2(centers[0][1], centers[0][0]) + Math.PI; /* markazga qaragan */
626     const dirLast = Math.atan2(centers[n-1][1]-centers[0][1], centers[n-1][0]-centers[0][0]);
627     ctx.beginPath();

```

```

628 arcB(ctx, centers[0][0], centers[0][1], rr, dirLast-0.7, dirLast+0.7);
629 ribbonI(ctx, c0, shade(c0,0.5), bw);
630 /* markaziy medalyon */
631 const ri = rc - rr - bw;
632 if(ri > R*0.1){
633   ctx.beginPath(); circle(ctx,0,0,ri*0.9); ribbon(ctx, c2, shade(c2,0.4), sw*1.3+0.7);
634   ctx.beginPath(); starPath(ctx, n, ri*0.62, ri*0.34, -Math.PI/2);
635   fillA(ctx, c1, 0.8); st(ctx, shade(c1,0.4), sw*0.5);
636   dot(ctx,0,0,ri*0.14,c0);
637 }
638 ctx.beginPath(); circle(ctx,0,0,R); st(ctx, c2, sw*0.5);
639 }
640
641 /* =====
642 B. ISLIMIY ORGANIK NAQSHLAR (8) – nafis novda va barglar
643 ===== */
644
645 /* Nozik torayuvchi novda: 3 o'tishli qalinlik gradatsiyasi */
646 function stem(ctx,pts,col,w){
647   const edge = shade(col,0.35);
648   ctx.beginPath(); smoothPath(ctx,pts,false);
649   st(ctx, edge, w);
650   const half = pts.slice(0, Math.ceil(pts.length*0.6));
651   ctx.beginPath(); smoothPath(ctx,half,false);
652   st(ctx, edge, w*1.55);
653   ctx.beginPath(); smoothPath(ctx,pts,false);
654   st(ctx, col, Math.max(0.4,w*0.5));
655 }
656 /* Nafis barg: to'ldirish + kontur + o'rta tomir */
657 function fineLeaf(ctx,x,y,ang,len,wid,fillC,edgeC,sw){
658   ctx.beginPath(); leaf(ctx,x,y,ang,len,wid);
659   fillA(ctx, fillC, 0.85);
660   st(ctx, edgeC, sw);
661   ctx.beginPath();
662   ctx.save(); ctx.translate(x,y); ctx.rotate(ang);
663   ctx.moveTo(len*0.08,0);
664   ctx.bezierCurveTo(len*0.3,-wid*0.12, len*0.55,-wid*0.1, len*0.8,0);
665   ctx.restore();
666   st(ctx, edgeC, sw*0.55);
667 }
668 /* Uch bargli palmetta kurtak */
669 function budAt(ctx,x,y,ang,L,fillC,edgeC,sw){
670   for(const [da,f] of [[-0.55,0.72],[0.55,0.72],[0,1]]){
671     ctx.beginPath(); leaf(ctx,x,y,ang+da,L*f,L*f*0.3);
672     fillA(ctx, f===1? fillC : lerpColor(fillC,edgeC,0.25), 0.9);
673     st(ctx, shade(edgeC,0.2), sw*0.5);
674   }
675 }
676
677 /* 9. ISLIMIY – nafis spiral novdalar */
678 function drawIslimiy(ctx,R,p,sw,c0,c1,c2,c3){
679   const n = Math.max(3, Math.min(10, p.sym));
680   const b = 0.1 + p.spir*0.28;
681   const thMax = Math.PI*(1.2 + p.cmp*0.18);
682   for(let i=0;i<n;i++){
683     ctx.save(); ctx.rotate(TAU*i/n);
684     const pts = spiralPts(b, thMax, R*0.88, i%2===1, 0);
685     stem(ctx, pts, c0, sw*1.1);
686     /* barglar – kamroq, kattaroq, uchga qarab o'sadi */
687     const every = Math.max(5, 11 - Math.round(p.dens*5));
688     for(let j=every; j<pts.length-3; j+=every){
689       const t2 = j/pts.length;
690       const a = pts[j], b2 = pts[j+1];
691       const ang = Math.atan2(b2[1]-a[1], b2[0]-a[0]);
692       const side = (Math.floor(j/every)%2)? 1 : -1;
693       fineLeaf(ctx, a[0], a[1], ang + side*Math.PI/2.5, R*0.1*(0.5+t2), R*0.03*(0.5+t2), c1, shade(c0,0.25), sw*0.
45);
694     }
695     const e = pts[pts.length-1], e2 = pts[pts.length-3];
696     budAt(ctx, e[0], e[1], Math.atan2(e[1]-e2[1],e[0]-e2[0]), R*0.16, c1, c0, sw);
697     /* markazdagi kichik qarshi gajak */
698     if(p.cmp>=3){
699       const pts2 = spiralPts(b*1.35, thMax*0.42, R*0.3, i%2===0, 0.4);
700       stem(ctx, pts2, c2, sw*0.7);
701       const f = pts2[pts2.length-1], f2 = pts2[pts2.length-2];
702       fineLeaf(ctx, f[0], f[1], Math.atan2(f[1]-f2[1],f[0]-f2[0]), R*0.09, R*0.028, c3, shade(c2,0.2), sw*0.4);
703     }
704     ctx.restore();
705   }
706   ctx.beginPath(); circle(ctx,0,0,R*0.05+sw); fillHard(ctx,c0,0.95);
707   ctx.beginPath(); circle(ctx,0,0,R); st(ctx,c2,sw*0.5);
708 }
709
710 /* -----
711 O'YMA PANEL (kartush) – Xiva/Farg'ona yog'och o'ymakorligi uslubi
712 -----
713 Ustun qirralaridagi va eshik tabaqalaridagi naqsh radial medalyon EMAS:
714 u tik KARTUSH – ramkali panel, ichida o'qqa nisbatan KO'ZGU-SIMMETRIK
715 islamiy. Uch xususiyati bu uslubni tanitadi va shu yerda takrorlanadi:
716
717 1) kartush ramkasi – yon tomonlari tik, uchlari siniq (ogee) yoy;

```



```

718     2) NUQTALI FON ("chizma" / pargori) – o'yilgan pastki qatlam;
719     3) RELYEF – har kontur ostiga yorug' va soya nusxasi, shunda naqsh
720         yassi chiziqli emas, o'yilgan sirt bo'lib ko'rinadi.
721
722     Barcha shakllar oddiy yo'l buyruqlari bilan chiziladi (offscreen canvas
723     yoki filtr ishlatilmaydi), shuning uchun SVG/DXF eksporti ham ishlaydi.
724     ----- */
725     function oymaCartouche(ctx, w, h){
726         const tip = h*0.22;                /* uchdagi yoy balandligi */
727         ctx.moveTo(-w, -h + tip);
728         ctx.lineTo(-w, h - tip);
729         /* yuqori siniq yoy: nazorat nuqtalari cho'qqida qarama-qarshi yo'nalgani
730             uchun uchi yumaloq emas, O'TKIR chiqadi */
731         ctx.bezierCurveTo(-w, h - tip*0.32, -w*0.13, h*1.02, 0, h);
732         ctx.bezierCurveTo(w*0.13, h*1.02, w, h - tip*0.32, w, h - tip);
733         ctx.lineTo(w, -h + tip);
734         ctx.bezierCurveTo(w, -h + tip*0.32, w*0.13, -h*1.02, 0, -h);
735         ctx.bezierCurveTo(-w*0.13, -h*1.02, -w, -h + tip*0.32, -w, -h + tip);
736         ctx.closePath();
737     }
738
739     function drawOyma(ctx, R, p, sw, c0, c1, c2, c3){
740         const hw = R*0.60, hh = R*0.95;
741         const fw = R*0.070;                /* ramka eni */
742         const lw = Math.max(0.6, sw*0.85);
743         const iw = hw - fw*2.1, ih = hh - fw*2.1; /* ichki maydon */
744         const dark = shade(c0, 0.55), light = lerpColor(c1, 'ffffff', 0.72);
745
746         /* --- RELYEF yordamchisi: konturni yorug' va soya nusxasi bilan --- */
747         const d = Math.max(0.55, lw*0.55);
748         /* Vektor eksportida (SVG/DXF/EPS) yorug'-soya nusxalari CHIZILMAYDI:
749             ular ekranda o'yma taassurotini beradi, lekin lazer uchun bu HAR
750             CHIZIQNI UCH MARTA kesish degani – material kuyadi va ish vaqti uch
751             barobar oshadi. SvgCtx o'zini ctx._svg bilan tanitadi. */
752         const vec = !!ctx._svg;
753         const relief = (path, wid)=>{
754             if(!vec){
755                 ctx.save(); ctx.translate(-d, -d); ctx.beginPath(); path(); st(ctx, light, wid); ctx.restore();
756                 ctx.save(); ctx.translate(d, d); ctx.beginPath(); path(); st(ctx, dark, wid); ctx.restore();
757             }
758             ctx.beginPath(); path(); st(ctx, c0, wid);
759         };
760
761         /* --- 1) NUQTALI FON – o'yilgan pastki qatlam ---
762             Bu SIRT FAKTURASI, kesish konturi emas. Vektor eksportida chizilmaydi:
763             aks holda lazerga minglab mayda doira (har biri alohida teshik) ketardi –
764             o'lchandi: 180x120 mm o'yma panelda 3019 kontur, 2.5 MB DXF. */
765         if(!vec && p.dens > 0.15){
766             ctx.save();
767             ctx.beginPath(); oymaCartouche(ctx, iw, ih); ctx.clip();
768             const step = Math.max(R*0.026, R*0.055*(1.15 - p.dens));
769             const rr = Math.max(0.5, step*0.17);
770             /* deterministik siljish – Math.random ishlatilmaydi, aks holda har
771                 chizishda boshqa natija chiqib, eksport takrorlanmas bo'lib qolardi */
772             const jit = (i,j)=>{ const s = Math.sin(i*127.1 + j*311.7)*43758.5453; return s - Math.floor(s); };
773             for(let j = -Math.ceil(ih/step); j <= Math.ceil(ih/step); j++){
774                 for(let i = -Math.ceil(iw/step); i <= Math.ceil(iw/step); i++){
775                     const x = i*step + (j%2 ? step*0.5 : 0) + (jit(i,j)-0.5)*step*0.28;
776                     const y = j*step + (jit(j,i)-0.5)*step*0.28;
777                     ctx.beginPath(); circle(ctx, x, y, rr); fillHard(ctx, shade(c2, 0.30), 0.55);
778                 }
779             }
780             ctx.restore();
781         }
782
783         /* --- 2) MARKAZIY KOMPOZITSIYA – o'qqa nisbatan ko'zgu-simmetrik ---
784             Tuzilma: har yon tomonda ILON-IZI NOVDA pastdan tepaga o'tadi, uning
785             har burilishida palmetta, oralarida barglar. Novda panel enining
786             deyarli hammasini egallagani uchun maydon TEKIS to'ladi.
787
788             (Avvalgi urinishda har tugundan logarifmik spiral chiqarilgandi: katta
789             L da spiralning tashqi o'rami yirik halqaga aylanib, panel o'rtaida
790             chalkash tugun hosil qilardi, yon tomonlar esa bo'sh qolardi. Ilon-izi
791             novda bu muammoni butunlay yo'q qiladi.) */
792         const waves = 3 + Math.min(2, Math.max(0, (p.cmp[0] - 3)));
793         const half = (sgn)=>{
794             ctx.save(); ctx.scale(sgn, 1);
795         };
796         /* --- ilon-izi novda --- */
797         const vine = [];
798         const N = 120;
799         for(let i = 0; i <= N; i++){
800             const t = i/N;                /* 0 → pastdan tepaga */
801             const y = -ih*0.86 + t*ih*1.72;
802             /* novda o'qni KESIB O'TMAYDI: aks holda ikki ko'zgu nusxa markazda
803                 chalkashib, tugun hosil qilardi */
804             const x = iw*0.16 + (0.5 + 0.5*Math.sin(t*Math.PI*waves - Math.PI/2))*iw*0.62;
805             vine.push([x, y]);
806         }
807         relief(()=>smoothPath(ctx, vine, false), lw*1.15);
808     }

```

```

809     /* burilish nuqtalari – palmetta va qarshi gajak */
810     for(let w = 0; w < waves; w++){
811         const t = (w + 0.5)/waves;          /* to'liq cho'qqisi */
812         const i = Math.round(t*N);
813         const a = vine[Math.max(0,i-1)], b2 = vine[Math.min(N,i+1)];
814         const ang = Math.atan2(b2[1]-a[1], b2[0]-a[0]);
815         const q = vine[i];
816         /* palmetta TASHQARIGA qaragan – o'q tomonga emas, aks holda ikki
817            ko'zgu nusxaning palmettalari markazda ustma-ust tushadi */
818         budAt(ctx, q[0], q[1], 0, iw*0.30, c1, c0, lw*0.9);
819     }
820
821     /* novda bo'ylab barglar – ikki tomonga navbatma-navbat */
822     const step = Math.max(5, 13 - Math.round(p.dens*7));
823     for(let i = step; i < N - step; i += step){
824         const a = vine[i-1], b2 = vine[i+1], q = vine[i];
825         const ang = Math.atan2(b2[1]-a[1], b2[0]-a[0]);
826         const side = (Math.floor(i/step) % 2) ? 1 : -1;
827         fineLeaf(ctx, q[0], q[1], ang + side*Math.PI/2.3, iw*0.20, iw*0.068, c1, dark, lw*0.42);
828     }
829     ctx.restore();
830 };
831 half(1); half(-1);
832
833     /* o'q bo'ylab ingichka poya – ikki novdani bog'laydi */
834     const spine = [];
835     for(let i = 0; i <= 24; i++) spine.push([0, -ih*0.88 + (i/24)*ih*1.76]);
836     relief(()=>smoothPath(ctx, spine, false), lw*0.8);
837
838     /* uchlardagi kurtaklar va markaziy palmetta */
839     budAt(ctx, 0, ih*0.90, -Math.PI/2, ih*0.13, c1, c0, lw);
840     budAt(ctx, 0, -ih*0.90, Math.PI/2, ih*0.13, c1, c0, lw);
841     for(const s2 of [-1, 1]){
842         fineLeaf(ctx, 0, 0, s2>0 ? 0 : Math.PI, iw*0.34, iw*0.13, c1, dark, lw*0.55);
843         fineLeaf(ctx, 0, 0, s2>0 ? -Math.PI/2 : Math.PI/2, iw*0.22, iw*0.085, c3, dark, lw*0.45);
844     }
845     ctx.beginPath(); circle(ctx, 0, 0, iw*0.075); fillHard(ctx, c3, 0.95); st(ctx, dark, lw*0.7);
846
847     /* --- 3) KARTUSH RAMKASI – eng ustida, relyef bilan --- */
848     relief(()=>oymaCartouche(ctx, iw, ih), lw*0.9);
849     relief(()=>oymaCartouche(ctx, hw, hh), lw*1.35);
850     /* ramka ichidagi ingichka yo'l */
851     ctx.beginPath(); oymaCartouche(ctx, (hw+iw)/2, (hh+ih)/2); st(ctx, c2, lw*0.55);
852 }
853
854     /* Rumi motivi – ilmoqli buralgan barg (klassik "yaprak") */
855     function rumiLeafPath(ctx,L){
856         ctx.moveTo(0,0);
857         ctx.bezierCurveTo(L*0.12,-L*0.36, L*0.68,-L*0.42, L*0.94,-L*0.1);
858         ctx.bezierCurveTo(L*1.04, L*0.04, L*0.9, L*0.2, L*0.7, L*0.13);
859         ctx.bezierCurveTo(L*0.52, L*0.06, L*0.42, L*0.2, L*0.28, L*0.16);
860         ctx.bezierCurveTo(L*0.12, L*0.1, L*0.04, L*0.05, 0,0);
861         ctx.closePath();
862     }
863
864     /* 10. RUMI – halqa bo'ylab ilmoqli barglar + tok chizig'i */
865     function drawRumi(ctx,R,p,sw,c0,c1,c2,c3){
866         const n = Math.max(5, p.sym);
867         const rc = R*0.62;
868         /* tok halqasi (lenta) */
869         ctx.beginPath(); circle(ctx,0,0,rc);
870         ribbon(ctx, c2, shade(c2,0.4), sw*1.4+0.8);
871         const L = R*0.36*(0.8 + p.dens*0.35);
872         for(let i=0;i<n;i++){
873             const a = TAU*i/n - Math.PI/2;
874             const c = pol(a, rc);
875             /* tashqariga qaragan katta rumi barg */
876             ctx.save(); ctx.translate(c[0],c[1]); ctx.rotate(a - Math.PI/2 + 0.35 + p.skew*0.4);
877             ctx.beginPath(); rumiLeafPath(ctx, L);
878             fillA(ctx, i%2? c1 : c0, 0.85);
879             st(ctx, shade(i%2? c1 : c0, 0.4), sw*0.7);
880             /* ichki ilmoq chizig'i */
881             ctx.beginPath();
882             ctx.moveTo(L*0.1,-L*0.06);
883             ctx.bezierCurveTo(L*0.35,-L*0.22, L*0.62,-L*0.24, L*0.78,-L*0.06);
884             st(ctx, shade(c3,0.1), sw*0.4);
885             ctx.restore();
886             /* ichkariga qaragan kichik barg */
887             ctx.save(); ctx.translate(c[0],c[1]); ctx.rotate(a + Math.PI/2 - 0.35 - p.skew*0.4);
888             ctx.beginPath(); rumiLeafPath(ctx, L*0.55);
889             fillA(ctx, c3, 0.85);
890             st(ctx, shade(c2,0.25), sw*0.5);
891             ctx.restore();
892             dot(ctx, c[0], c[1], sw*1.1+0.6, shade(c0,0.2));
893         }
894     }
895
896     /* TASHQI BOG'LOVCHI NOVDALAR – katta barglar orasidan chetgacha S-novda:
897        tessellatsiyada qo'shni naqsh novdalari bilan tutashib UZLUKSIZ gulchambar hosil qiladi */
898     for(let i=0;i<n;i++){
899         const a = TAU*(i+0.5)/n - Math.PI/2;
900         const sgn = i%2 ? 1 : -1;
901         const pts = [];
902         for(let j=0;j<=10;j++){

```

```

900     const t = j/10;
901     const rr = rc*0.96 + (R*1.06 - rc*0.96)*t;
902     const wob = Math.sin(t*Math.PI)*0.16*sgn; /* yengil S-egri */
903     pts.push(pol(a + wob, rr));
904 }
905 stem(ctx, pts, c2, sw*0.85);
906 /* o'rtadagi yon barg */
907 const m1 = pts[5], m2 = pts[6];
908 const mAng = Math.atan2(m2[1]-m1[1], m2[0]-m1[0]);
909 fineLeaf(ctx, m1[0], m1[1], mAng + sgn*Math.PI/2.4, L*0.34, L*0.11, c3, shade(c2,0.2), sw*0.4);
910 /* uchidagi barg – plitka chegarasidan o'tib qo'shnisiga yetadi */
911 const e = pts[10], e2 = pts[9];
912 fineLeaf(ctx, e[0], e[1], Math.atan2(e[1]-e2[1], e[0]-e2[0]), L*0.42, L*0.14, i%2? c1 : c0, shade(c0,0.3), sw*
0.5);
913 }
914 /* markaz rozetkasi */
915 const ri = rc - L*0.75;
916 if(ri > R*0.08){
917     ctx.beginPath(); starPath(ctx, n, ri*0.7, ri*0.4, -Math.PI/2);
918     fillA(ctx, c1, 0.8); st(ctx, shade(c1,0.35), sw*0.5);
919     dot(ctx,0,0,ri*0.16,c0);
920 }
921 ctx.beginPath(); circle(ctx,0,0,R); st(ctx,c2,sw*0.5);
922 }
923
924 /* 11. HATAYI – lotus: 3-segmentli gulbarglar, toza bandlar */
925 function hatayiPetal(ctx,a,rIn,rOut,w){
926     const bl=pol(a-w,rIn), br=pol(a+w,rIn);
927     const wl=pol(a-w*0.62, rIn+(rOut-rIn)*0.58), wr=pol(a+w*0.62, rIn+(rOut-rIn)*0.58);
928     const tip=pol(a,rOut);
929     ctx.moveTo(bl[0],bl[1]);
930     const clp=pol(a-w*1.25, rIn+(rOut-rIn)*0.28);
931     ctx.bezierCurveTo(clp[0],clp[1], wl[0],wl[1], wl[0],wl[1]);
932     const c2p=pol(a-w*0.12, rOut*0.99);
933     ctx.bezierCurveTo(wl[0],wl[1], c2p[0],c2p[1], tip[0],tip[1]);
934     const c3p=pol(a+w*0.12, rOut*0.99);
935     ctx.bezierCurveTo(c3p[0],c3p[1], wr[0],wr[1], wr[0],wr[1]);
936     const c4p=pol(a+w*1.25, rIn+(rOut-rIn)*0.28);
937     ctx.bezierCurveTo(wr[0],wr[1], c4p[0],c4p[1], br[0],br[1]);
938     const cb=pol(a, rIn*0.84);
939     ctx.bezierCurveTo(cb[0],cb[1], cb[0],cb[1], bl[0],bl[1]);
940     ctx.closePath();
941 }
942 function drawHatayi(ctx,R,p,sw,c0,c1,c2,c3){
943     const m = 5 + (p.cmp>=5 ? 2 : (p.cmp>=3 ? 1 : 0));
944     const w = (Math.PI/m)*(0.62+p.dens*0.25);
945     /* tashqi gulbarglar */
946     for(let i=0;i<m;i++){
947         const a = TAU*i/m - Math.PI/2;
948         ctx.beginPath(); hatayiPetal(ctx, a, R*0.36, R*0.94, w);
949         fillA(ctx, i%2? c1 : lerpColor(c1,c3,0.4), 0.7);
950         st(ctx, shade(c0,0.35), sw*0.7);
951         const t0 = pol(a, R*0.42), t1 = pol(a, R*0.82);
952         ctx.beginPath(); ctx.moveTo(t0[0],t0[1]);
953         ctx.bezierCurveTo(t0[0],t0[1], t1[0],t1[1], t1[0],t1[1]);
954         st(ctx, shade(c2,0.15), sw*0.35);
955     }
956     /* oraliq kichik gulbarglar */
957     for(let i=0;i<m;i++){
958         ctx.beginPath(); hatayiPetal(ctx, TAU*(i+0.5)/m - Math.PI/2, R*0.2, R*0.46, w*0.66);
959         fillA(ctx, c3, 0.8); st(ctx, shade(c2,0.25), sw*0.5);
960     }
961     /* urug'don halqasi */
962     ctx.beginPath(); circle(ctx,0,0,R*0.2); ribbon(ctx, c1, shade(c1,0.4), sw*1.3+0.7);
963     const seeds = Math.max(6, m);
964     for(let i=0;i<seeds;i++){
965         const c = pol(TAU*i/seeds, R*0.13);
966         dot(ctx,c[0],c[1],R*0.024,shade(c0,0.15));
967     }
968     dot(ctx,0,0,R*0.055,c0);
969     ctx.beginPath(); circle(ctx,0,0,R); st(ctx,c2,sw*0.5);
970 }
971
972 /* 12. PALAK – nilufar: 3 toza gulbarg halqasi */
973 function drawPalak(ctx,R,p,sw,c0,c1,c2,c3){
974     const n = Math.max(6, p.sym);
975     const rings = [
976         {rIn:R*0.55, rOut:R*0.95, w:(Math.PI/n)*0.8, off:0, f:c1, e:c0},
977         {rIn:R*0.32, rOut:R*0.6, w:(Math.PI/n)*0.72, off:0.5, f:c3, e:c2},
978         {rIn:R*0.14, rOut:R*0.36, w:(Math.PI/n)*0.62, off:0, f:lerpColor(c1,'#ffffff',0.3), e:c0}
979     ];
980     const cnt = 1 + Math.min(2, Math.round(p.cmp/3)+1);
981     for(let k=0;k<Math.min(rings.length,cnt+1);k++){
982         const rg = rings[k];
983         for(let i=0;i<n;i++){
984             const a = TAU*(i+rg.off)/n - Math.PI/2;
985             ctx.beginPath(); petalPath(ctx, a, rg.rIn, rg.rOut, rg.w);
986             fillA(ctx, i%2? rg.f : lerpColor(rg.f, rg.e, 0.18), 0.75);
987             st(ctx, shade(rg.e,0.3), sw*0.6*(1-k*0.12));
988         }
989     }

```

```

990   ctx.beginPath(); circle(ctx,0,0,R*0.13); ribbon(ctx, c0, shade(c0,0.45), sw*1.2+0.6);
991   dot(ctx,0,0,R*0.05,c1);
992   ctx.beginPath(); circle(ctx,0,0,R); st(ctx,c2,sw*0.5);
993 }
994
995 /* 13. SHAMSA — quyosh medalyoni: yulduz + nurlar + gulli tashqi band */
996 function drawShamsa(ctx,R,p,sw,c0,c1,c2,c3){
997   const n = Math.max(6, p.sym);
998   /* tashqi bandlar */
999   ctx.beginPath(); circle(ctx,0,0,R*0.98); ribbon(ctx, c1, shade(c1,0.45), sw*1.7+1);
1000  ctx.beginPath(); circle(ctx,0,0,R*0.8); st(ctx, c2, sw*0.5);
1001  /* tashqi gulbarg bandi 0.8R-0.97R */
1002  for(let i=0;i<n*2;i++){
1003    const a = TAU*(i+0.5)/(n*2) - Math.PI/2;
1004    ctx.beginPath(); petalPath(ctx, a, R*0.8, R*0.955, (Math.PI/(n*2))*0.8);
1005    fillA(ctx, i%2? c3 : c1, 0.65);
1006    st(ctx, shade(c0,0.3), sw*0.4);
1007  }
1008  /* nurlar — 0.42R dan 0.76R gacha, uzun/qisqa navbat */
1009  for(let i=0;i<n*2;i++){
1010    const a = TAU*i/(n*2) - Math.PI/2;
1011    const long = i%2===0;
1012    const r0 = R*0.4, len = R*(long? 0.36 : 0.22);
1013    const s0 = pol(a,r0);
1014    ctx.beginPath(); leaf(ctx, s0[0], s0[1], a, len, len*0.14);
1015    fillA(ctx, long? c0 : c2, 0.9);
1016    st(ctx, shade(long? c0 : c2, 0.3), sw*0.4);
1017  }
1018  /* markaziy yulduz o'zagi */
1019  ctx.beginPath(); circle(ctx,0,0,R*0.42); ribbon(ctx, c2, shade(c2,0.4), sw*1.2+0.6);
1020  ctx.beginPath(); starPath(ctx, n, R*0.36, R*0.17, -Math.PI/2);
1021  fillA(ctx, c0, 0.92);
1022  st(ctx, shade(c0,0.45), sw*0.7);
1023  ctx.beginPath(); starPath(ctx, n, R*0.2, R*0.1, -Math.PI/2 + Math.PI/n);
1024  fillA(ctx, c1, 0.95);
1025  dot(ctx,0,0,R*0.055,c3);
1026 }
1027
1028 /* 14. ARABESQUE — ogee to'r: S-egri lentalar + palmettalar */
1029 function drawArabesque(ctx,R,p,sw,c0,c1,c2,c3){
1030   const n = Math.max(5, p.sym);
1031   const rIn = R*0.4, rOut = R*0.88;
1032   /* ogee to'ri: har ichki tugundan ikki qo'shni tashqi tugunga S-egri */
1033   const inner=[], outer=[];
1034   for(let i=0;i<n;i++){
1035     inner.push(pol(TAU*i/n - Math.PI/2, rIn));
1036     outer.push(pol(TAU*(i+0.5)/n - Math.PI/2, rOut));
1037   }
1038   const sCurve=(A,B,dir)=>{
1039     const mx=(A[0]+B[0])/2, my=(A[1]+B[1])/2;
1040     const nx=-(B[1]-A[1]), ny=(B[0]-A[0]);
1041     const L2=Math.hypot(nx,ny)||1;
1042     const off = R*0.09*dir*(0.6+p.spir);
1043     ctx.moveTo(A[0],A[1]);
1044     ctx.bezierCurveTo(
1045       A[0]*0.6+mx*0.4 + nx/L2*off, A[1]*0.6+my*0.4 + ny/L2*off,
1046       B[0]*0.6+mx*0.4 - nx/L2*off, B[1]*0.6+my*0.4 - ny/L2*off,
1047       B[0],B[1]);
1048   };
1049   ctx.beginPath();
1050   for(let i=0;i<n;i++){ sCurve(inner[i], outer[i], 1); }
1051   ribbon(ctx, c0, shade(c0,0.45), sw*1.6+0.8);
1052   ctx.beginPath();
1053   for(let i=0;i<n;i++){ sCurve(inner[(i+1)%n], outer[i], -1); }
1054   ribbonI(ctx, c1, shade(c1,0.45), sw*1.6+0.8);
1055   /* tashqi tugunlarda palmetta */
1056   for(let i=0;i<n;i++){
1057     const a = TAU*(i+0.5)/n - Math.PI/2;
1058     budAt(ctx, outer[i][0], outer[i][1], a, R*0.14, c1, c0, sw);
1059   }
1060   /* ichki tugunlarda barg juftligi */
1061   for(let i=0;i<n;i++){
1062     const a = TAU*i/n - Math.PI/2;
1063     fineLeaf(ctx, inner[i][0], inner[i][1], a+Math.PI*0.82, R*0.1, R*0.03, c3, shade(c2,0.2), sw*0.4);
1064     fineLeaf(ctx, inner[i][0], inner[i][1], a-Math.PI*0.82+Math.PI*2, R*0.1, R*0.03, c3, shade(c2,0.2), sw*0.4);
1065   }
1066   /* markaz */
1067   ctx.beginPath(); circle(ctx,0,0,rIn*0.55); ribbon(ctx, c2, shade(c2,0.4), sw*1.2+0.6);
1068   ctx.beginPath(); starPath(ctx, n, rIn*0.4, rIn*0.22, -Math.PI/2);
1069   fillA(ctx,c0,0.9);
1070   dot(ctx,0,0,rIn*0.12,c1);
1071   ctx.beginPath(); circle(ctx,0,0,R); st(ctx,c2,sw*0.5);
1072 }
1073
1074 /* 15. SCROLL — bargli novda: S-poya + gajak uchi */
1075 function drawScroll(ctx,R,p,sw,c0,c1,c2,c3){
1076   const n = Math.max(3, Math.min(9, p.sym));
1077   for(let i=0;i<n;i++){
1078     ctx.save(); ctx.rotate(TAU*i/n);
1079     const p0=[R*0.16,0], p1=[R*0.36,-R*0.3*(1+p.skew*0.4)], p2=[R*0.58,R*0.3*(1+p.skew*0.4)], p3=[R*0.8,0];
1080     /* poya nuqtalari */

```

```

1081     const stemPts=[]; for(let j=0;j<=16;j++) stemPts.push(cubicAt(p0,p1,p2,p3,j/16));
1082     stem(ctx, stemPts, i%2? c1 : c0, sw*1.05);
1083     /* gajak uchi — kichik spiral */
1084     const endAng = Math.atan2(stemPts[16][1]-stemPts[15][1], stemPts[16][0]-stemPts[15][0]);
1085     const curl = spiralPts(0.22, Math.PI*1.6, R*0.14, i%2===1, 0)
1086     .map(q=>{
1087         const ca=Math.cos(endAng), sa=Math.sin(endAng);
1088         return [p3[0]+q[0]*ca-q[1]*sa, p3[1]+q[0]*sa+q[1]*ca];
1089     });
1090     stem(ctx, curl, c2, sw*0.7);
1091     /* navbatlashgan barglar */
1092     const leaves = 2 + Math.round(p.dens*3);
1093     for(let j=1;j<=leaves;j++){
1094         const t = j/(leaves+1);
1095         const q = cubicAt(p0,p1,p2,p3,t);
1096         const q2 = cubicAt(p0,p1,p2,p3,Math.min(1,t+0.02));
1097         const ang = Math.atan2(q2[1]-q[1], q2[0]-q[0]);
1098         fineLeaf(ctx, q[0], q[1], ang + (j%2? 1:-1)*Math.PI/2.3, R*0.12*(0.7+t*0.5), R*0.035, j%2? c3:c1, shade(c0,0
1099     .25), sw*0.45);
1100     }
1101     ctx.restore();
1102     }
1103     ctx.beginPath(); circle(ctx,0,0,R*0.1); ribbon(ctx, c0, shade(c0,0.45), sw*1.2+0.6);
1104     dot(ctx,0,0,R*0.04,c1);
1105     ctx.beginPath(); circle(ctx,0,0,R); st(ctx,c2,sw*0.5);
1106 }
1107 /* 16. MEDALYON — ko'p bandli markaziy medalyon */
1108 function drawMedallion(ctx,R,p,sw,c0,c1,c2,c3){
1109     const n = Math.max(6, p.sym);
1110     /* tashqi band + scallop */
1111     ctx.beginPath(); circle(ctx,0,0,R*0.98); ribbon(ctx, c1, shade(c1,0.45), sw*1.7+1);
1112     ctx.beginPath();
1113     for(let i=0;i<n*2;i++){
1114         const a0 = TAU*i/(n*2)-Math.PI/2, a1 = TAU*(i+1)/(n*2)-Math.PI/2;
1115         const A=pol(a0,R*0.9), M=pol((a0+a1)/2, R*0.82);
1116         if(i===0) ctx.moveTo(A[0],A[1]);
1117         const B=pol(a1,R*0.9);
1118         ctx.bezierCurveTo(A[0]*0.94,A[1]*0.94, M[0],M[1], M[0],M[1]);
1119         ctx.bezierCurveTo(M[0],M[1], B[0]*0.94,B[1]*0.94, B[0],B[1]);
1120     }
1121     ctx.closePath();
1122     ribbon(ctx, c2, shade(c2,0.4), sw*1.1+0.5);
1123     /* gulbarg bandi */
1124     for(let i=0;i<n;i++){
1125         const a = TAU*i/n - Math.PI/2;
1126         ctx.beginPath(); petalPath(ctx, a, R*0.5, R*0.8, (Math.PI/n)*0.7);
1127         fillA(ctx, i%2? c1 : c3, 0.65);
1128         st(ctx, shade(c0,0.3), sw*0.5);
1129     }
1130     /* girih o'zagi */
1131     ctx.save(); ctx.scale(0.5,0.5);
1132     drawGirih(ctx, R*0.95, {...p, sym:n}, sw*1.6, c0, c1, c2, c3);
1133     ctx.restore();
1134 }
1135
1136 /* — CHINNI HOSHIYA BANDI — Rishton lagani uslubidagi ikki qatorli hoshiya — */
1137 function ceramicBorder(ctx,R,p,pal){
1138     const sw = p.sw;
1139     /* tashqi qalin band */
1140     ctx.beginPath(); circle(ctx,0,0,R*0.985); ribbon(ctx, pal.c[1], shade(pal.c[1],0.45), sw*2.6+2);
1141     ctx.beginPath(); circle(ctx,0,0,R*0.885); ribbon(ctx, pal.c[0], shade(pal.c[0],0.45), sw*1.4+0.8);
1142     /* band ichida: mini rozetta + romb navbatlashuvi */
1143     const m = Math.max(12, p.sym*3);
1144     const rb = R*0.935;
1145     for(let i=0;i<m;i++){
1146         const a = TAU*i/m - Math.PI/2;
1147         const q = pol(a, rb);
1148         ctx.save(); ctx.translate(q[0],q[1]); ctx.rotate(a+Math.PI/2);
1149         if(i%2===0){
1150             ctx.beginPath(); starPath(ctx, 8, R*0.033, R*0.015, 0);
1151             fillHard(ctx, pal.c[4], 0.95); st(ctx, shade(pal.c[0],0.35), sw*0.35);
1152             dot(ctx,0,0,R*0.008, pal.c[0]);
1153         } else {
1154             ctx.beginPath();
1155             polyPath(ctx, [[0, -R*0.028], [R*0.02,0], [0,R*0.028], [-R*0.02,0]]);
1156             fillHard(ctx, pal.c[3], 0.9); st(ctx, shade(pal.c[2],0.25), sw*0.3);
1157         }
1158         ctx.restore();
1159     }
1160     /* ichki nozik band: scallop + nuqta zanjiri */
1161     ctx.beginPath();
1162     const ms = m;
1163     for(let i=0;i<ms;i++){
1164         const a0 = TAU*i/ms - Math.PI/2, a1 = TAU*(i+1)/ms - Math.PI/2;
1165         const A = pol(a0, R*0.84), M = pol((a0+a1)/2, R*0.8), B = pol(a1, R*0.84);
1166         if(i===0) ctx.moveTo(A[0],A[1]);
1167         ctx.bezierCurveTo(A[0]*0.97,A[1]*0.97, M[0],M[1], M[0],M[1]);
1168         ctx.bezierCurveTo(M[0],M[1], B[0]*0.97,B[1]*0.97, B[0],B[1]);
1169     }
1170     ctx.closePath();

```

```

1171   st(ctx, shade(pal.c[2],0.15), sw*0.6);
1172   for(let i=0;i<m;i++){
1173     const q = pol(TAU*(i+0.5)/ms - Math.PI/2, R*0.815);
1174     dot(ctx, q[0], q[1], sw*0.65, pal.c[1]);
1175   }
1176   ctx.beginPath(); circle(ctx,0,0,R*0.72); st(ctx, pal.c[2], sw*0.5);
1177 }
1178 /* Chinni (lagan/piyola) kompozitsiyasi: hoshiya bandlari + markaziy medalyon */
1179 function drawPlateComposition(ctx,S,p){
1180   const pal = PAL[p.pal] || PAL.gold;
1181   ctx.save();
1182   ctx.translate(S/2, S/2);
1183   ceramicBorder(ctx, S*0.49, p, pal);
1184   ctx.restore();
1185   drawPattern(ctx, S/2, S/2, S*0.315, p);
1186 }
1187
1188 /* =====
1189 HANKIN USULI — "poligonlar aloqasi" (polygons-in-contact)
1190 Har plitka qirrasining O'RTASIDAN kontakt burchagi ostida ikki nur
1191 chiqadi; qo'shni nurlar kesishmasi strapwork chizig'ini beradi.
1192 ===== */
1193 function polyArea(poly){
1194   let a=0;
1195   for(let i=0;i<poly.length;i++){
1196     const A=poly[i], B=poly[(i+1)%poly.length];
1197     a += A[0]*B[1] - B[0]*A[1];
1198   }
1199   return a/2;
1200 }
1201 function hankinStraps(polyIn, ang){
1202   const poly = polyArea(polyIn) < 0 ? polyIn.slice().reverse() : polyIn;
1203   const m = poly.length, straps=[];
1204   const mids=[], dirs=[];
1205   for(let i=0;i<m;i++){
1206     const A=poly[i], B=poly[(i+1)%m];
1207     mids.push([(A[0]+B[0])/2, (A[1]+B[1])/2]);
1208     dirs.push(Math.atan2(B[1]-A[1], B[0]-A[0]));
1209   }
1210   for(let i=0;i<m;i++){
1211     const j=(i+1)%m;
1212     const X = rayX(mids[i], dirs[i]+ang, mids[j], dirs[j]+Math.PI-ang);
1213     if(X) straps.push([mids[i], X, mids[j]]);
1214   }
1215   return straps;
1216 }
1217 function strokeStraps(ctx, straps, band, edge, w, weave){
1218   const draw = (sset)=>{
1219     for(const sp of sset){
1220       ctx.beginPath();
1221       ctx.moveTo(sp[0][0],sp[0][1]);
1222       ctx.lineTo(sp[1][0],sp[1][1]);
1223       ctx.lineTo(sp[2][0],sp[2][1]);
1224       weave ? ribbonI(ctx, band, edge, w) : ribbon(ctx, band, edge, w);
1225     }
1226   };
1227   draw(straps);
1228 }
1229 /* 4.8.8 plitkalash geometriyasi (sakkizburchak + kvadrat) */
1230 function tiles488(R, cells){
1231   const s = R*1.9/cells;
1232   const oR = s*0.5/Math.cos(Math.PI/8);
1233   const octs=[], sqs=[];
1234   const n2 = Math.ceil(cells/2)+1;
1235   for(let gy=-n2; gy<=n2; gy++){
1236     for(let gx=-n2; gx<=n2; gx++){
1237       const x=gx*s, y=gy*s;
1238       if(Math.hypot(x,y) < R*1.15){
1239         const o=[]; for(let i=0;i<8;i++) o.push([x+Math.cos(Math.PI/8+TAU*i/8)*oR, y+Math.sin(Math.PI/8+TAU*i/8)*o
R]);
1240         octs.push(o);
1241       }
1242       const qx=x+s/2, qy=y+s/2;
1243       if(Math.hypot(qx,qy) < R*1.15){
1244         const q = oR*Math.sin(Math.PI/8)*Math.SQRT2;
1245         sqs.push([[qx,qy-q],[qx+q,qy],[qx,qy+q],[qx-q,qy]]);
1246       }
1247     }
1248   }
1249   return {octs, sqs, s, oR};
1250 }
1251 /* GIRIH HANKIN 4.8.8 — yulduz-xoch klassikasi (Registon poli uslubi) */
1252 function drawHankin8(ctx,R,p,sw,c0,c1,c2,c3){
1253   const FIELD = !!p._field;
1254   const cells = FIELD ? Math.max(2, Math.round(R*1.9/(p._cellpx||150))) : 2 + Math.round(p.dens*2);
1255   const T = tiles488(R, cells);
1256   const ang = Math.PI*(0.2 + p.spir*0.18); /* kontakt burchagi ≈ 36°–68° */
1257   const edge = shade(c0, 0.5);
1258   const bw = sw*2.0 + 1.2;
1259   ctx.save();
1260   if(!FIELD){

```

```

1261     ctx.beginPath(); circle(ctx,0,0,R*0.985);
1262     if(!ctx._svg) ctx.clip();
1263   }
1264   /* plitka fonlari */
1265   for(let i=0;i<T.octs.length;i++){
1266     ctx.beginPath(); polyPath(ctx, T.octs[i]);
1267     fillA(ctx, i%2? c3 : lerpColor(c3,'#ffffff',0.4), 0.6);
1268   }
1269   for(const q of T.sqs){
1270     ctx.beginPath(); polyPath(ctx, q);
1271     fillA(ctx, c1, 0.55);
1272   }
1273   /* Hankin straplari: oq/och lenta – haqiqiy girih ko'rinishi */
1274   const band = lerpColor(c3,'#ffffff',0.75);
1275   for(const o of T.octs) strokeStraps(ctx, hankinStraps(o, ang), band, edge, bw, false);
1276   for(const q of T.sqs) strokeStraps(ctx, hankinStraps(q, ang), band, edge, bw, true);
1277   ctx.restore();
1278   if(!FIELD){
1279     ctx.beginPath(); circle(ctx,0,0,R*0.985); ribbon(ctx, c1, shade(c1,0.45), sw*2+1.2);
1280     ctx.beginPath(); circle(ctx,0,0,R*0.9); st(ctx, c2, sw*0.5);
1281   }
1282 }
1283 /* GIRIH HANKIN 6.6.6 – geksagonal to'rdan 6-yulduzlar */
1284 function drawHankin6(ctx,R,p,sw,c0,c1,c2,c3){
1285   const FIELD = !!p._field;
1286   const s = FIELD ? Math.max(30,(p._cellpx||150)*0.5) : R/(1.5 + p.dens*1.6);
1287   const dx = s*Math.sqrt(3), dy = s*1.5;
1288   const ang = Math.PI*(0.24 + p.spir*0.16);
1289   const edge = shade(c0, 0.5);
1290   const bw = sw*2.0 + 1.2;
1291   const band = lerpColor(c3,'#ffffff',0.75);
1292   ctx.save();
1293   if(!FIELD){
1294     ctx.beginPath(); circle(ctx,0,0,R*0.985);
1295     if(!ctx._svg) ctx.clip();
1296   }
1297   const hexes=[];
1298   const NN = FIELD ? Math.ceil(R/(s*1.5))+1 : 5;
1299   for(let gy=-NN; gy<=NN; gy++){
1300     for(let gx=-NN; gx<=NN; gx++){
1301       const x = gx*dx + (gy%2 ? dx/2 : 0), y = gy*dy;
1302       if(Math.hypot(x,y) > R*1.15) continue;
1303       const h=[]; for(let i=0;i<6;i++) h.push([x+Math.cos(Math.PI/6+TAU*i/6)*s, y+Math.sin(Math.PI/6+TAU*i/6)*s]);
1304       hexes.push({h, gx, gy});
1305     }
1306   }
1307   for(const {h,gx,gy} of hexes){
1308     ctx.beginPath(); polyPath(ctx, h);
1309     fillA(ctx, ((gx+gy*2)%3+3)%3===0 ? c1 : ((gx+gy*2)%3+3)%3===1 ? c3 : lerpColor(c3,'#ffffff',0.4), 0.55);
1310   }
1311   for(const {h} of hexes) strokeStraps(ctx, hankinStraps(h, ang), band, edge, bw, false);
1312   ctx.restore();
1313   if(!FIELD){
1314     ctx.beginPath(); circle(ctx,0,0,R*0.985); ribbon(ctx, c1, shade(c1,0.45), sw*2+1.2);
1315   }
1316 }
1317 /* SAKKIZ QIRRALI GIRIH – klassik kvadrat o'quv paneli
1318 (markaziy 8-yulduz rozetta + kvadrat ramka, to'liq bo'yalgan) */
1319 function drawPanelGirih(ctx,R,p,sw,c0,c1,c2,c3){
1320   const H = R*0.94;
1321   const edge = shade(c0, 0.5);
1322   const bw = sw*2.4 + 1.6;
1323   const band = lerpColor(c3,'#ffffff',0.8);
1324   /* tashqi ramka bandlari */
1325   ctx.beginPath(); polyPath(ctx, [[-H,-H],[H,-H],[H,H],[-H,H]]);
1326   ribbon(ctx, c1, shade(c1,0.45), bw*1.1);
1327   const H2 = H*0.88;
1328   ctx.beginPath(); polyPath(ctx, [[-H2,-H2],[H2,-H2],[H2,H2],[-H2,H2]]);
1329   fillA(ctx, c3, 0.45);
1330   ribbon(ctx, c0, edge, bw*0.7);
1331   /* ramka orasidagi mini-rozetkalar */
1332   const mm = 4 + Math.round(p.dens*3);
1333   const rimR = (H+H2)/2;
1334   for(let side=0; side<4; side++){
1335     for(let i=1;i<mm;i++){
1336       const t = i/mm*2 - 1;
1337       let x,y;
1338       if(side===0){ x=t*rimR; y=-rimR; } else if(side===1){ x=rimR; y=t*rimR; }
1339       else if(side===2){ x=t*rimR; y=rimR; } else { x=-rimR; y=t*rimR; }
1340       dot(ctx,x,y,(H-H2)*0.22, i%2? c0 : band);
1341     }
1342   }
1343   /* burchak choraklari */
1344   for(const [sx,sy] of [[-1,-1],[1,-1],[1,1],[-1,1]]){
1345     ctx.beginPath();
1346     ctx.moveTo(sx*H2, sy*H2);
1347     ctx.lineTo(sx*H2, sy*H2*0.55);
1348     ctx.bezierCurveTo(sx*H2*0.8, sy*H2*0.6, sx*H2*0.6, sy*H2*0.8, sx*H2*0.55, sy*H2);
1349     ctx.closePath();
1350     fillA(ctx, c0, 0.6);
1351     st(ctx, edge, sw*0.5);

```

```

1352 }
1353 /* MARKAZIY 8-YULDUZ ROZETTA: ikki kvadrat + kite gulbarglar */
1354 const r = H2*0.66;
1355 /* kite gulbarglar (yulduz uchlaridan tashqariga) */
1356 const g = {R0:H2*0.92, r2:r*0.78, rm:r*0.9, w:Math.PI/8*0.8};
1357 for(let i=0;i<8;i++){
1358   const a = TAU*i/8 - Math.PI/2;
1359   ctx.beginPath(); kitePath(ctx, a, g);
1360   fillA(ctx, i%2? c1 : c3, 0.75);
1361   st(ctx, shade(c2,0.2), sw*0.4);
1362 }
1363 /* yulduz maydoni */
1364 ctx.beginPath(); starPath(ctx, 8, r, r*0.765, -Math.PI/2);
1365 fillA(ctx, c1, 0.92);
1366 /* ikki kvadrat — oq lenta straplar (haqiqiy panel uslubi) */
1367 const sq=(rot)=>{ const q=[]; for(let i=0;i<4;i++) q.push(pol(rot+TAU*i/4, r)); return q; };
1368 ctx.beginPath(); polyPath(ctx, sq(-Math.PI/2));
1369 ribbon(ctx, band, edge, bw);
1370 ctx.beginPath(); polyPath(ctx, sq(-Math.PI/2+Math.PI/4));
1371 ribbonI(ctx, band, edge, bw);
1372 /* birinchi kvadratning ikki qarama-qarshi tomoni qayta ustidan — weave */
1373 const A = sq(-Math.PI/2);
1374 for(const j of [0,2]){
1375   const P1=A[j], P2=A[(j+1)%4];
1376   ctx.beginPath();
1377   ctx.moveTo(P1[0]+(P2[0]-P1[0])*0.3, P1[1]+(P2[1]-P1[1])*0.3);
1378   ctx.lineTo(P1[0]+(P2[0]-P1[0])*0.7, P1[1]+(P2[1]-P1[1])*0.7);
1379   ribbonI(ctx, band, edge, bw);
1380 }
1381 /* markaziy sakkizburchak + gul */
1382 const oct=[]; for(let i=0;i<8;i++) oct.push(pol(-Math.PI/2+Math.PI/8+TAU*i/8, r*0.545));
1383 ctx.beginPath(); polyPath(ctx, oct);
1384 fillA(ctx, c0, 0.9);
1385 st(ctx, edge, sw*0.5);
1386 for(let i=0;i<8;i++){
1387   ctx.beginPath();
1388   petalPath(ctx, TAU*i/8 - Math.PI/2, r*0.1, r*0.42, Math.PI/8*0.62);
1389   fillA(ctx, band, 0.9);
1390   st(ctx, shade(c1,0.3), sw*0.35);
1391 }
1392 dot(ctx,0,0,r*0.09,c1);
1393 }
1394 /* STAKAN uchun FRIZ kompozitsiyasi — gorizontal band, chokka mos takror */
1395 function drawFriezeComposition(ctx,S,p){
1396   const pal = PAL[p.pal] || PAL.gold;
1397   const line=(y,w,col)=>{
1398     ctx.beginPath(); ctx.moveTo(-6,y); ctx.lineTo(S+6,y);
1399     ribbon(ctx, col, shade(col,0.45), w);
1400   };
1401   line(S*0.14, p.sw*2.4+1.6, pal.c[1]);
1402   line(S*0.225, p.sw*1.2+0.6, pal.c[0]);
1403   line(S*0.775, p.sw*1.2+0.6, pal.c[0]);
1404   line(S*0.86, p.sw*2.4+1.6, pal.c[1]);
1405   /* band foni — naqsh tagida yengil tus */
1406   ctx.save();
1407   ctx.globalAlpha = 0.38;
1408   ctx.fillStyle = pal.c[4];
1409   ctx.fillRect(0, S*0.27, S, S*0.46);
1410   ctx.restore();
1411   /* motif qatori — zichroq, teng qadam: chokda uzilmaydi */
1412   const m = Math.max(6, Math.min(12, p.sym));
1413   for(let i=0;i<m;i++){
1414     drawPattern(ctx, (i+0.5)*S/m, S*0.5, (S/m)*0.42, {...p, lay:1});
1415   }
1416   /* ajratkich romblar (chok nuqtasida ham) */
1417   for(let i=0;i<m;i++){
1418     const x = i*S/m;
1419     ctx.beginPath();
1420     polyPath(ctx, [[x,S*0.5-S*0.045],[x+S*0.026,S*0.5],[x,S*0.5+S*0.045],[x-S*0.026,S*0.5]]);
1421     fillHard(ctx, pal.c[1], 0.95);
1422     st(ctx, shade(pal.c[1],0.4), p.sw*0.4);
1423   }
1424   /* yuqori nuqta zanjiri */
1425   for(let i=0;i<m*3;i++){
1426     const x = (i+0.5)*S/(m*3);
1427     dot(ctx, x, S*0.185, p.sw*0.7, pal.c[0]);
1428     dot(ctx, x, S*0.815, p.sw*0.7, pal.c[0]);
1429   }
1430 }
1431 /* =====
1432 C. KOMBINATSIYALAR (4) — tashqi uslub + ichki uslub (0.5R)
1433 ===== */
1434 function makeCombo(fA,fB){
1435   return function(ctx,R,p,sw,c0,c1,c2,c3){
1436     fA(ctx,R,p,sw,c0,c1,c2,c3);
1437     ctx.save();
1438     ctx.scale(0.5,0.5);
1439     ctx.rotate(Math.PI/Math.max(3,p.sym));
1440     fB(ctx,R,p,sw*1.7,c1,c0,c3,c2);
1441     ctx.restore();
1442   };

```



```

1443 }
1444 function drawGirihIslimiy(ctx,R,p,sw,c0,c1,c2,c3){ makeCombo(drawGirih,drawIslimiy)(ctx,R,p,sw,c0,c1,c2,c3); }
1445 function drawShamsaGirih(ctx,R,p,sw,c0,c1,c2,c3){ makeCombo(drawShamsa,drawGirih)(ctx,R,p,sw,c0,c1,c2,c3); }
1446 function drawMuqarnasIslimiy(ctx,R,p,sw,c0,c1,c2,c3){ makeCombo(drawMuqarnas,drawIslimiy)(ctx,R,p,sw,c0,c1,c2,c3); }
1447 }
1448 function drawXatamRumi(ctx,R,p,sw,c0,c1,c2,c3){ makeCombo(drawXatam,drawRumi)(ctx,R,p,sw,c0,c1,c2,c3); }
1449
1450 /* =====
1451 D. DUNYO NAQSHLARI (8)
1452 ===== */
1453
1454 /* 21. Celtic – 2 tolali torus o'rmasi: haqiqiy navbatlashgan to'qish */
1455 function drawCeltic(ctx,R,p,sw,c0,c1,c2,c3){
1456   const k = Math.max(4, p.sym);
1457   const rm = R*0.66, a = R*(0.14 + p.dens*0.1);
1458   const bw = sw*2.6 + 2;
1459   const strandSeg = (phase, j)=>{ /* j-segment:  $\theta \in [j\pi/k, (j+1)\pi/k]$  */
1460     const pts=[];
1461     const steps = 14;
1462     for(let i=0;i<=steps;i++){
1463       const th = (j + i/steps)*Math.PI/k;
1464       pts.push(pol(th - Math.PI/2, rm + a*Math.sin(k*th + phase)));
1465     }
1466     ctx.beginPath(); smoothPath(ctx, pts, false);
1467   };
1468   /* ikki tola, 2k segment; naqsh navbatlashadi: (tola + j) juft → ostda, toq → ustda */
1469   for(let pass=0; pass<2; pass++){
1470     for(let strand=0; strand<2; strand++){
1471       const phase = strand*Math.PI;
1472       const col = strand? c1 : c0;
1473       for(let j=0; j<2*k; j++){
1474         if(((strand + j) % 2) !== pass) continue;
1475         strandSeg(phase, j);
1476         if(pass===0) ribbon(ctx, col, shade(col,0.5), bw);
1477         else ribbonI(ctx, col, shade(col,0.5), bw);
1478       }
1479     }
1480   }
1481   /* ichki/tashqi chegara bandlari */
1482   ctx.beginPath(); circle(ctx,0,0, rm - a - bw*0.9); ribbon(ctx, c2, shade(c2,0.4), sw*1.2+0.6);
1483   ctx.beginPath(); circle(ctx,0,0, rm + a + bw*0.9); ribbon(ctx, c2, shade(c2,0.4), sw*1.2+0.6);
1484   /* markaziy tugma */
1485   if(p.cmp>=3){
1486     ctx.beginPath(); starPath(ctx, k, (rm-a)*0.5, (rm-a)*0.28, -Math.PI/2);
1487     fillA(ctx, c3, 0.8); st(ctx, shade(c0,0.35), sw*0.5);
1488     dot(ctx,0,0,(rm-a)*0.12,c0);
1489   }
1490   ctx.beginPath(); circle(ctx,0,0,R); st(ctx,c2,sw*0.5);
1491 }
1492
1493 /* 22. Mandala – konsentrik halqalar, har halqada boshqa element */
1494 function drawMandala(ctx,R,p,sw,c0,c1,c2,c3){
1495   const rings = Math.max(6, Math.min(12, 5 + p.cmp + Math.round(p.dens*2)));
1496   const cols = [c0,c1,c2,c3];
1497   for(let k=0;k<rings;k++){
1498     const t = k/(rings-1);
1499     const r = R*(0.12 + 0.85*t);
1500     const m = Math.max(6, Math.round(p.sym*(1 + t*1.5)));
1501     const col = cols[k%4];
1502     const kind = k%5;
1503     if(kind===0){
1504       for(let i=0;i<m;i++){ const c=pol(TAU*i/m, r); dot(ctx,c[0],c[1], R*0.014+sw*0.5, col); }
1505     } else if(kind===1){
1506       const step = R*0.85/(rings-1);
1507       for(let i=0;i<m;i++){
1508         ctx.beginPath();
1509         petalPath(ctx, TAU*i/m, r-step*0.5, r+step*0.55, (Math.PI/m)*0.75);
1510         fillA(ctx,col,0.5); st(ctx, c0, sw*0.5);
1511       }
1512     } else if(kind===2){
1513       ctx.beginPath();
1514       const pts=[];
1515       for(let i=0;i<=m*6;i++){
1516         const a = TAU*i/(m*6);
1517         pts.push(pol(a, r*(1 + 0.035*Math.sin(a*m + k))));
1518       }
1519       smoothPath(ctx,pts,true);
1520       st(ctx,col,sw*0.7);
1521     } else if(kind===3){
1522       const step = R*0.85/(rings-1);
1523       for(let i=0;i<m;i++){
1524         const a = TAU*(i+0.5)/m, c = pol(a,r);
1525         ctx.beginPath(); leaf(ctx, c[0], c[1], a, step*0.9, step*0.28*(1+p.spir));
1526         fillA(ctx,col,0.7); st(ctx,c0,sw*0.4);
1527       }
1528     } else {
1529       ctx.beginPath(); circle(ctx,0,0,r); st(ctx,col,sw*0.55);
1530     }
1531   }
1532   dot(ctx,0,0,R*0.05,c0);
1533 }

```

```

1533
1534 /* 23. Penrose – kite/dart aperiodik subdivide, oltin nisbat */
1535 function drawPenrose(ctx,R,p,sw,c0,c1,c2,c3){
1536   const PHI = 1.6180339887;
1537   let tris = [];
1538   for(let i=0;i<10;i++){
1539     let B = pol(TAU*(2*i-1)/20 - Math.PI/2, R*0.98);
1540     let C = pol(TAU*(2*i+1)/20 - Math.PI/2, R*0.98);
1541     if(i%2===0){ const t=B; B=C; C=t; }
1542     tris.push([0,[0,0],B,C]);
1543   }
1544   const depth = Math.max(1, Math.min(6, p.cmp));
1545   const div = (x1,x2,f)=>[x1[0]+(x2[0]-x1[0])/f, x1[1]+(x2[1]-x1[1])/f];
1546   for(let d=0; d<depth; d++){
1547     const next=[];
1548     for(const T of tris){
1549       const [ty,A,B,C] = T;
1550       if(ty===0){
1551         const Pp = div(A,B,PHI);
1552         next.push([0,C,Pp,B],[1,Pp,C,A]);
1553       } else {
1554         const Q = div(B,A,PHI), Rr = div(B,C,PHI);
1555         next.push([1,Rr,C,A],[0,Q,Rr,B],[1,Rr,Q,A]);
1556       }
1557     }
1558     tris = next;
1559   }
1560   for(const T of tris){
1561     const [ty,A,B,C] = T;
1562     ctx.beginPath(); polyPath(ctx,[A,B,C]);
1563     fillA(ctx, ty? c1:c3, 0.75);
1564     st(ctx, c0, sw*0.4);
1565   }
1566   ctx.beginPath(); circle(ctx,0,0,R*0.99); st(ctx,c2,sw);
1567 }
1568
1569 /* 24. Zellige – Marokash kafel: egri tomonli romblar, 5 rang */
1570 function drawZellige(ctx,R,p,sw,c0,c1,c2,c3){
1571   const FIELD = !!p._field;
1572   const palc = PAL[p.pal] ? PAL[p.pal].c : [c0,c1,c2,c3,c0];
1573   const s = FIELD ? Math.max(30,(p._cellpx||150)*0.55) : R/(1.6 + p.dens*2.2);
1574   const bow = s*0.2*(0.4 + p.spir);
1575   const curvedQuad = (pts)=>{
1576     ctx.moveTo(pts[0][0],pts[0][1]);
1577     for(let i=0;i<4;i++){
1578       const A=pts[i], B=pts[(i+1)%4];
1579       const mx=(A[0]+B[0])/2, my=(A[1]+B[1])/2;
1580       const nx=-(B[1]-A[1]), ny=(B[0]-A[0]);
1581       const L=Math.hypot(nx,ny)||1;
1582       const sgn = i%2? 1:-1;
1583       const cx2=mx+nx/L*bow*sgn, cy2=my+ny/L*bow*sgn;
1584       ctx.bezierCurveTo(
1585         A[0]+(cx2-A[0])*0.7, A[1]+(cy2-A[1])*0.7,
1586         B[0]+(cx2-B[0])*0.7, B[1]+(cy2-B[1])*0.7,
1587         B[0],B[1]);
1588     }
1589     ctx.closePath();
1590   };
1591   const NZ = FIELD ? Math.ceil(R/(s*0.5))+2 : 4;
1592   for(let gy=-NZ; gy<=NZ; gy++){
1593     for(let gx=-NZ; gx<=NZ; gx++){
1594       const x = (gx+gy)*s*0.5*1.6, y = (gy-gx)*s*0.5;
1595       if(Math.hypot(x,y) > R*(FIELD?1.5:1.03)) continue;
1596       const q=[[x,y-s*0.5],[x+s*0.8,y],[x,y+s*0.5],[x-s*0.8,y]];
1597       ctx.beginPath(); curvedQuad(q);
1598       const ci = ((gx%5)+(gy*2%5)+10)%5;
1599       fillA(ctx, palc[ci], 0.85);
1600       st(ctx, c0, sw*0.6);
1601       if(p.cmp>=5){ dot(ctx,x,y,s*0.07,palc[(ci+2)%5]); }
1602     }
1603   }
1604   if(!FIELD){ ctx.beginPath(); circle(ctx,0,0,R); st(ctx,c0,sw); }
1605 }
1606
1607 /* 25. Batik – to'lqinli konsentrik halqalar + gul motivlari */
1608 function drawBatik(ctx,R,p,sw,c0,c1,c2,c3){
1609   const rings = 3 + Math.min(5, p.cmp);
1610   const n = Math.max(4,p.sym);
1611   const A = R*0.035*(0.6 + p.dens);
1612   for(let k=0;k<rings;k++){
1613     const rk = R*(0.18 + 0.8*k/(rings-1||1));
1614     const pts=[];
1615     const steps = Math.max(48, n*10);
1616     for(let i=0;i<steps;i++){
1617       const a = TAU*i/steps;
1618       pts.push(pol(a, rk + A*Math.sin(n*a + k*1.3 + p.skew)));
1619     }
1620     ctx.beginPath(); smoothPath(ctx,pts,true);
1621     st(ctx, k%2? c1:c0, sw*(0.7 + 0.3*(k%2)));
1622     if(k%2===1 && k<rings-1){
1623       const m = n + k;

```

```

1624     for(let i=0;i<m;i++){
1625         const c = pol(TAU*(i+0.3)/m, rk + R*0.35/rings);
1626         for(let j=0;j<5;j++){
1627             const q = pol(TAU*j/5, R*0.028*(1+p.spir));
1628             dot(ctx, c[0]+q[0], c[1]+q[1], R*0.014, c3);
1629         }
1630         dot(ctx,c[0],c[1],R*0.012,c2);
1631     }
1632 }
1633 }
1634 dot(ctx,0,0,R*0.05*p.ir*2+2,c0);
1635 }
1636
1637 /* 26. Paisley – buta/tomchi: 4 Bezier teardrop, ichida spiral */
1638 function drawPaisley(ctx,R,p,sw,c0,c1,c2,c3){
1639     const n = Math.max(3,p.sym);
1640     for(let i=0;i<n;i++){
1641         ctx.save();
1642         ctx.rotate(TAU*i/n);
1643         ctx.translate(R*0.52, 0);
1644         ctx.rotate(Math.PI/2 + p.skew*0.6);
1645         const L = R*0.42*(0.8+p.dens*0.4), W = L*0.58;
1646         /* boteh: 4 Bezier – asimmetrik tomchi, uchi bukilgan */
1647         ctx.beginPath();
1648         ctx.moveTo(0, L*0.5);
1649         ctx.bezierCurveTo(-W, L*0.5, -W, -L*0.35, -W*0.25, -L*0.62);
1650         ctx.bezierCurveTo(-W*0.05, -L*0.75, W*0.3, -L*0.78, W*0.38, -L*0.6);
1651         ctx.bezierCurveTo(W*0.44, -L*0.46, W*0.1, -L*0.42, W*0.12, -L*0.28);
1652         ctx.bezierCurveTo(W*0.55, -L*0.05, W*0.62, L*0.32, 0, L*0.5);
1653         ctx.closePath();
1654         fillA(ctx, i%2?c1:c3, 0.55);
1655         ribbonI(ctx, c0, shade(c0,0.5), sw*1.8+1);
1656         /* ichki hoshiya konturi */
1657         ctx.save(); ctx.scale(0.82,0.82);
1658         ctx.beginPath();
1659         ctx.moveTo(0, L*0.5);
1660         ctx.bezierCurveTo(-W, L*0.5, -W, -L*0.35, -W*0.25, -L*0.62);
1661         ctx.bezierCurveTo(-W*0.05, -L*0.75, W*0.3, -L*0.78, W*0.38, -L*0.6);
1662         ctx.bezierCurveTo(W*0.44, -L*0.46, W*0.1, -L*0.42, W*0.12, -L*0.28);
1663         ctx.bezierCurveTo(W*0.55, -L*0.05, W*0.62, L*0.32, 0, L*0.5);
1664         ctx.closePath();
1665         ctx.restore();
1666         st(ctx, shade(c2,0.2), sw*0.45);
1667         /* ichki spiral */
1668         const sp = spiralPts(0.16+p.spir*0.2, Math.PI*2.1, L*0.42, i%2===1, 0);
1669         ctx.beginPath(); smoothPath(ctx, sp.map(q=>[q[0], q[1]+L*0.05]), false);
1670         st(ctx, c2, sw*0.6);
1671         /* uchidagi 5-barg gul */
1672         for(let j=0;j<5;j++){
1673             const q = pol(TAU*j/5, L*0.1);
1674             dot(ctx, q[0]-W*0.05, q[1]-L*0.7, L*0.055, c1);
1675         }
1676         dot(ctx, -W*0.05, -L*0.7, L*0.045, c0);
1677         /* chekka nuqta bezaklari */
1678         if(p.cmp>=4){
1679             for(let j=0;j<7;j++){
1680                 const t = j/6;
1681                 dot(ctx, -W*(1.12), L*0.4 - t*L*0.8, sw*0.8, c2);
1682             }
1683         }
1684         ctx.restore();
1685     }
1686     ctx.beginPath(); circle(ctx,0,0,R*p.ir*0.4+2); st(ctx,c0,sw*0.7);
1687 }
1688
1689 /* 27. Greek Key – meander labirint, konsentrik kvadrat ramkalar */
1690 function drawGreek(ctx,R,p,sw,c0,c1,c2,c3){
1691     const frames = 2 + Math.min(4, Math.round(p.cmp/2));
1692     const meanderSide = (x0,y0,x1,y1,units,h)=>{
1693         const dx=(x1-x0)/units, dy=(y1-y0)/units;
1694         const nx=-dy, ny=dx; /* ichkariga normal */
1695         const L=Math.hypot(nx,ny)||1;
1696         const hx=nx/L*h, hy=ny/L*h;
1697         for(let u=0;u<units;u++){
1698             const bx=x0+dx*u, by=y0+dy*u;
1699             const s=[[0,0],[0.72,0],[0.72,0.72],[0.24,0.72],[0.24,0.3],[0.48,0.3],[0.48,0.52],[1,0.52]];
1700             ctx.moveTo(bx,by);
1701             for(const q of s){
1702                 ctx.lineTo(bx+dx*q[0]+hx*q[1], by+dy*q[0]+hy*q[1]);
1703             }
1704         }
1705     };
1706     for(let k=0;k<frames;k++){
1707         const half = R*0.68*(1 - k*0.24);
1708         if(half < R*0.1) break;
1709         const units = Math.max(3, Math.round(p.sym*(0.5+p.dens*0.5)) - k);
1710         const h = half*0.28;
1711         const c=[[-half,-half],[half,-half],[half,half],[-half,half]];
1712         ctx.beginPath();
1713         for(let e=0;e<4;e++){
1714             const A=c[e], B=c[(e+1)%4];

```

```

1715     meanderSide(A[0],A[1],B[0],B[1],units,h);
1716   }
1717   st(ctx, k%2?c1:c0, sw*(1-k*0.1));
1718   ctx.beginPath(); polyPath(ctx,c);
1719   st(ctx, c2, sw*0.5);
1720 }
1721 ctx.beginPath(); starPath(ctx, 4, R*p.ir*0.35, R*p.ir*0.15, Math.PI/4);
1722 fillA(ctx,c3,0.8); st(ctx,c0,sw*0.7);
1723 }
1724
1725 /* 28. Xitoy panjara – fretwork kataklar, T-shakl bo'linmalar */
1726 function drawXitoy(ctx,R,p,sw,c0,c1,c2,c3){
1727   const FIELD = !!p._field;
1728   const half = FIELD ? R*1.42 : R*0.92;
1729   const cells = FIELD ? Math.max(2, Math.round(half*2/(p._cellpx||150))) : Math.max(2, Math.round(2 + p.dens*3));
1730   const s = half*2/cells;
1731   ctx.beginPath();
1732   polyPath(ctx,[[ -half,-half],[half,-half],[half,half],[ -half,half]]);
1733   st(ctx,c0,sw*1.4);
1734   ctx.beginPath();
1735   for(let i=1;i<cells;i++){
1736     ctx.moveTo(-half+i*s,-half); ctx.lineTo(-half+i*s,half);
1737     ctx.moveTo(-half,-half+i*s); ctx.lineTo(half,-half+i*s);
1738   }
1739   st(ctx,c0,sw*0.9);
1740   /* har katakda T-motivlar navbatlashgan yo'nalishda */
1741   const tLen = s*0.34*(0.6+p.ir), tBar = s*0.24;
1742   for(let gy=0; gy<cells; gy++){
1743     for(let gx=0; gx<cells; gx++){
1744       const cx = -half + gx*s + s/2, cy = -half + gy*s + s/2;
1745       const rot = ((gx+gy)%4) * Math.PI/2 + p.skew*0.4;
1746       ctx.save(); ctx.translate(cx,cy); ctx.rotate(rot);
1747       ctx.beginPath();
1748       /* T ning oyog'i katak CHEGARASIGACHA boradi – panjara bilan
1749          tutashadi. Ilgari u faqat tLen gacha chizilar va motiv katak
1750          ichida "suzib" qolardi: naqsh uzilgan ko'rinardi, lazerda
1751          kesilganda esa bunday bo'lak paneldan tushib ketardi. */
1752       ctx.moveTo(0,-s/2); ctx.lineTo(0,tLen*0.2);
1753       ctx.moveTo(-tBar,tLen*0.2); ctx.lineTo(tBar,tLen*0.2);
1754       ctx.moveTo(-tBar,tLen*0.2); ctx.lineTo(-tBar,tLen*0.62);
1755       ctx.moveTo(tBar,tLen*0.2); ctx.lineTo(tBar,tLen*0.62);
1756       ctx.restore();
1757       st(ctx, (gx+gy)%2? c1:c2, sw*0.75);
1758       /* Burchak qavslari – T0'RTALA burchakda, katakdan biroz ichkarida.
1759          Ilgari faqat ikki qarama-qarshi burchakda chizilar va ular
1760          panjara chizig'i ustiga tushib, tasodifiy chiziqchalardek
1761          ko'rinardi. To'rttasi birgalikda ichki ramka hosil qiladi –
1762          xitoy fretworkidagi klassik motiv. */
1763       if(p.cmp>=4){
1764         ctx.beginPath();
1765         const q = s*0.15, inn = s*0.06, h2 = s/2 - inn;
1766         for(const [sx2, sy2] of [[-1,-1],[1,-1],[1,1],[-1,1]]){
1767           const px = cx + sx2*h2, py = cy + sy2*h2;
1768           ctx.moveTo(px - sx2*q, py); ctx.lineTo(px, py); ctx.lineTo(px, py - sy2*q);
1769         }
1770         st(ctx,c3,sw*0.6);
1771       }
1772     }
1773   }
1774 }
1775
1776 /* =====
1777 E. MATEMATIK FRAKTALLAR (7)
1778 ===== */
1779
1780 /* 29. Koch qor parchasi */
1781 function drawKoch(ctx,R,p,sw,c0,c1,c2,c3){
1782   const depth = Math.max(1, Math.min(5, p.cmp));
1783   let pts = [];
1784   for(let i=0;i<3;i++) pts.push(pol(TAU*i/3 - Math.PI/2, R*0.85));
1785   for(let d=0; d<depth; d++){
1786     const next=[];
1787     for(let i=0;i<pts.length;i++){
1788       const A=pts[i], B=pts[(i+1)%pts.length];
1789       const dx=(B[0]-A[0])/3, dy=(B[1]-A[1])/3;
1790       const P1=[A[0]+dx, A[1]+dy], P3=[A[0]+2*dx, A[1]+2*dy];
1791       const ang = Math.atan2(dy,dx) - Math.PI/3;
1792       const L = Math.hypot(dx,dy);
1793       const P2=[P1[0]+Math.cos(ang)*L, P1[1]+Math.sin(ang)*L];
1794       next.push(A,P1,P2,P3);
1795     }
1796     pts = next;
1797   }
1798   ctx.beginPath(); polyPath(ctx,pts);
1799   fillA(ctx,c3,0.35); iStroke(ctx,c0,sw*0.8);
1800   ctx.beginPath(); circle(ctx,0,0,R*p.ir*0.5);
1801   st(ctx,c2,sw*0.6);
1802   dot(ctx,0,0,R*0.03,c1);
1803 }
1804
1805 /* 30. Sierpinski uchburchagi */

```

```

1806 function drawSierpinski(ctx,R,p,sw,c0,c1,c2,c3){
1807   const depth = Math.max(1, Math.min(6, p.cmp));
1808   const A=pol((-Math.PI/2,R*0.92), B=pol(Math.PI/6,R*0.92), C=pol(Math.PI*5/6,R*0.92));
1809   const mid=(u,v)=>[(u[0]+v[0])/2,(u[1]+v[1])/2];
1810   const rec=(a,b,c,d)=>{
1811     if(d===0){
1812       ctx.beginPath(); polyPath(ctx,[a,b,c]);
1813       fillA(ctx, d%2?c1:c3, 0.7); st(ctx,c0,sw*0.35);
1814       return;
1815     }
1816     rec(a,mid(a,b),mid(a,c),d-1);
1817     rec(mid(a,b),b,mid(b,c),d-1);
1818     rec(mid(a,c),mid(b,c),c,d-1);
1819   };
1820   rec(A,B,C,depth);
1821   ctx.beginPath(); polyPath(ctx,[A,B,C]); st(ctx,c0,sw);
1822 }
1823
1824 /* 31. Dragon curve – L-sistema: F→F+G, G→F-G */
1825 function drawDragon(ctx,R,p,sw,c0,c1,c2,c3){
1826   const iters = 8 + Math.min(5, p.cmp);
1827   let seq = 'F';
1828   for(let d=0; d<iters; d++){
1829     let next='';
1830     for(const ch of seq){
1831       if(ch==='F') next += 'F+G';
1832       else if(ch==='G') next += 'F-G';
1833       else next += ch;
1834     }
1835     seq = next;
1836     if(seq.length > 40000) break;
1837   }
1838   let x=0,y=0,ang=0;
1839   const pts=[[0,0]];
1840   let minX=0,maxX=0,minY=0,maxY=0;
1841   for(const ch of seq){
1842     if(ch==='F' || ch==='G'){
1843       x += Math.cos(ang); y += Math.sin(ang);
1844       pts.push([x,y]);
1845       if(x<minX)minX=x; if(x>maxX)maxX=x;
1846       if(y<minY)minY=y; if(y>maxY)maxY=y;
1847     }
1848     else if(ch==='+') ang += Math.PI/2;
1849     else if(ch==='-') ang -= Math.PI/2;
1850   }
1851   const w=maxX-minX||1, h=maxY-minY||1;
1852   const sc = R*1.7/Math.max(w,h);
1853   const ox=(minX+maxX)/2, oy=(minY+maxY)/2;
1854   ctx.beginPath();
1855   ctx.moveTo((pts[0][0]-ox)*sc, (pts[0][1]-oy)*sc);
1856   for(let i=1;i<pts.length;i++) ctx.lineTo((pts[i][0]-ox)*sc, (pts[i][1]-oy)*sc);
1857   st(ctx, c0, sw*0.6);
1858   ctx.beginPath(); circle(ctx,0,0,R); st(ctx,c2,sw*0.5);
1859 }
1860
1861 /* 32. Hilbert space-filling curve */
1862 function drawHilbert(ctx,R,p,sw,c0,c1,c2,c3){
1863   const order = Math.max(2, Math.min(6, 1 + Math.round(p.cmp*0.8)));
1864   const N = 1<<order;
1865   const d2xy = (d)=>{
1866     let rx, ry, t=d, x=0, y=0;
1867     for(let s=1; s<N; s*=2){
1868       rx = 1 & (t/2); ry = 1 & (t ^ rx);
1869       if(ry===0){
1870         if(rx===1){ x=s-1-x; y=s-1-y; }
1871         const tmp=x; x=y; y=tmp;
1872       }
1873       x += s*rx; y += s*ry;
1874       t = Math.floor(t/4);
1875     }
1876     return [x,y];
1877   };
1878   const total = N*N;
1879   const sc = R*1.7/(N-1);
1880   ctx.beginPath();
1881   for(let d=0; d<total; d++){
1882     const q = d2xy(d);
1883     const px = (q[0]-(N-1)/2)*sc, py=(q[1]-(N-1)/2)*sc;
1884     if(d===0) ctx.moveTo(px,py); else ctx.lineTo(px,py);
1885   }
1886   st(ctx, c0, sw*0.55);
1887   ctx.beginPath();
1888   polyPath(ctx,[[ -R*0.92,-R*0.92],[R*0.92,-R*0.92],[R*0.92,R*0.92],[ -R*0.92,R*0.92]]);
1889   st(ctx,c2,sw*0.6);
1890 }
1891
1892 /* 33. Spirograph – epitrochoid */
1893 function drawSpirograph(ctx,R,p,sw,c0,c1,c2,c3){
1894   const R0 = R*0.62;
1895   /* Gipotroxoida: r = R0·a/b (a va b o'zaro tub) → egri b ta cho'qqili
1896     bo'lib, a ta aylanishdan keyin yopiladi.

```

```

1897     DIQQAT – DEGENERATSIYA:  $b = 2a$  bo'lganda  $(R0-r)/r = 1$  va egri
1898     ELLIPSGA aylanadi;  $d \approx r$  bo'lsa esa TO'G'RI CHIZIQA siqiladi.
1899     Eski formulada  $kNum = sym-2$ ,  $kDen = sym+cmp$  edi va standart
1900     qiymatlarda ( $sym = 8$ ,  $cmp = 4$ ) aynan  $kDen = 2 \cdot kNum$  chiqib qolardi –
1901     shuning uchun spirograf o'rniga to'g'ri chiziqlar chizilardi.
1902     Endi bu holat aniq chetlab o'tiladi. */
1903     const gcd = (x, y) => { x = Math.abs(x); y = Math.abs(y); while(y) { const t = y; y = x % y; x = t; } return x || 1
; };
1904     let b = Math.max(3, p.sym | 0);
1905     let a = 1 + ((p.cmp | 0) % Math.max(1, b - 1));
1906     const g0 = gcd(a, b); a = a/g0; b = b/g0;
1907     if(b === 2*a) b += 1; /* ellips/chiziqa aylanmasin */
1908     if(a >= b) a = 1;
1909     const r = R0*a/b;
1910     const d = r*(0.45 + p.spir*1.1);
1911     const loops = a;
1912     const steps = Math.max(480, 180*b);
1913     const pts=[];
1914     let maxR=0;
1915     for(let i=0; i<=steps; i++){
1916         const t = TAU*loops*i/steps;
1917         const x=(R0-r)*Math.cos(t) + d*Math.cos((R0-r)*t/r);
1918         const y=(R0-r)*Math.sin(t) - d*Math.sin((R0-r)*t/r);
1919         pts.push([x,y]);
1920         const rr=Math.hypot(x,y); if(rr>maxR) maxR=rr;
1921     }
1922     const sc = R*0.95/(maxR||1);
1923     ctx.beginPath();
1924     smoothPath(ctx, pts.map(q=>[q[0]*sc, q[1]*sc]), false);
1925     st(ctx, c0, sw*0.55);
1926     ctx.beginPath(); circle(ctx, 0, 0, R*0.98); st(ctx, c2, sw*0.5);
1927     dot(ctx, 0, 0, R*0.03, c1);
1928 }
1929
1930 /* 34. Lissajous */
1931 function drawLissajous(ctx, R, p, sw, c0, c1, c2, c3){
1932     const a = Math.max(1, p.sym-3), b = a + Math.max(1, Math.round(p.cmp/2));
1933     const del = Math.PI/2 * (1 + p.skew);
1934     const steps = 400;
1935     const pts=[];
1936     for(let i=0; i<=steps; i++){
1937         const t = TAU*i/steps;
1938         pts.push([R*0.88*Math.sin(a*t+del), R*0.88*Math.sin(b*t)]);
1939     }
1940     ctx.beginPath(); smoothPath(ctx, pts, true);
1941     st(ctx, c0, sw*0.6);
1942     if(p.dens>0.5){
1943         const pts2 = pts.map(q=>[q[0]*0.7, q[1]*0.7]);
1944         ctx.beginPath(); smoothPath(ctx, pts2, true);
1945         st(ctx, c1, sw*0.45);
1946     }
1947     ctx.beginPath();
1948     polyPath(ctx, [[-R*0.95, -R*0.95], [R*0.95, -R*0.95], [R*0.95, R*0.95], [-R*0.95, R*0.95]]);
1949     st(ctx, c2, sw*0.4);
1950 }
1951
1952 /* 35. Voronoi – seed nuqtalardan hujayra mozaika */
1953 function mulberry32(seed){
1954     let t = seed>>>0;
1955     return function(){
1956         t += 0x6D2B79F5;
1957         let x = Math.imul(t ^ (t>>>15), 1 | t);
1958         x ^= x + Math.imul(x ^ (x>>>7), 61 | x);
1959         return ((x ^ (x>>>14)) >>> 0) / 4294967296;
1960     };
1961 }
1962 function clipHalfPlane(poly, px, py, nx, ny){
1963     /* n·(q - p) <= 0 tomonini saqlaydi */
1964     const out=[];
1965     for(let i=0; i<poly.length; i++){
1966         const A=poly[i], B=poly[(i+1)%poly.length];
1967         const da=(A[0]-px)*nx + (A[1]-py)*ny;
1968         const db=(B[0]-px)*nx + (B[1]-py)*ny;
1969         if(da<=0) out.push(A);
1970         if((da<0 && db>0) || (da>0 && db<0)){
1971             const t = da/(da-db);
1972             out.push([A[0]+(B[0]-A[0])*t, A[1]+(B[1]-A[1])*t]);
1973         }
1974     }
1975     return out;
1976 }
1977 function drawVoronoi(ctx, R, p, sw, c0, c1, c2, c3){
1978     const palc = PAL[p.pal] ? PAL[p.pal].c : [c0, c1, c2, c3, c0];
1979     const rnd = mulberry32(1000 + p.sym*31 + p.cmp*7 + Math.round(p.dens*100)*13 + Math.round(p.spir*100)*3);
1980     const m = 8 + p.cmp*4 + Math.round(p.dens*14);
1981     const seeds=[];
1982     for(let i=0; i<m; i++){
1983         const a = rnd()*TAU, rr = Math.sqrt(rnd())*R*0.92;
1984         seeds.push(pol(a, rr));
1985     }
1986     /* boshlang'ich poligon – R ga chizilgan 16-burchak */

```

```

1987 const base=[]; for(let i=0;i<16;i++) base.push(pol(TAU*i/16, R*0.98));
1988 for(let i=0;i<m;i++){
1989   let cell = base.slice();
1990   const S = seeds[i];
1991   for(let j=0;j<m && cell.length;j++){
1992     if(j===i) continue;
1993     const T = seeds[j];
1994     const mx=(S[0]+T[0])/2, my=(S[1]+T[1])/2;
1995     const nx=T[0]-S[0], ny=T[1]-S[1];
1996     cell = clipHalfPlane(cell, mx,my, nx,ny);
1997   }
1998   if(cell.length<3) continue;
1999   ctx.beginPath(); polyPath(ctx,cell);
2000   fillA(ctx, palc[i%5], 0.75);
2001   st(ctx, c0, sw*0.55);
2002   if(p.cmp>=5) dot(ctx, S[0], S[1], sw*0.9, c2);
2003 }
2004 ctx.beginPath(); circle(ctx,0,0,R*0.99); st(ctx,c0,sw*0.9);
2005 }
2006
2007 /* =====
2008 NaqshAI — PIC GEOMETRIYA YADROSI (Polygons-In-Contact / Hankin usuli)
2009 -----
2010 Klassik g'irih qurilishi. Bosqichlar:
2011 1. PLITKALASH — davriy (yoki radial) plitka to'ri, qat'iy qirra-qirraga
2012 2. KONTAKT — har qirra o'rtasidan 6 burchagi ostida ikki nur
2013 3. GLOBAL GRAF — barcha segmentlar bitta grafda, dedup + tugun birlashtirish
2014 4. IP (STRAND) — kesishishlarda davomiylik bo'yicha uzluksiz ip ajratish
2015 5. TO'QISH — har ip bo'ylab ust/ost navbatlashishi
2016 Chiqishi sof ma'lumot: {tiles, strands, crossings, cores, period}
2017 Chizish bu ma'lumotdan keyin, alohida bosqichda (picRender).
2018 ===== */
2019 var TAU2 = Math.PI * 2;
2020 var SQ3 = Math.sqrt(3);
2021
2022 /* ----- kichik vektor yordamchilari ----- */
2023 function vAdd(a,b){ return [a[0]+b[0], a[1]+b[1]]; }
2024 function vSub(a,b){ return [a[0]-b[0], a[1]-b[1]]; }
2025 function vMul(a,k){ return [a[0]*k, a[1]*k]; }
2026 function vLen(a){ return Math.hypot(a[0],a[1]); }
2027 function vNorm(a){ var l=vLen(a)||1; return [a[0]/l, a[1]/l]; }
2028 function vDot(a,b){ return a[0]*b[0]+a[1]*b[1]; }
2029 function polyCentroid(p){
2030   var x=0,y=0;
2031   for(var i=0;i<p.length;i++){ x+=p[i][0]; y+=p[i][1]; }
2032   return [x/p.length, y/p.length];
2033 }
2034 function ngon(cx,cy,r,n,rot){
2035   var q=[];
2036   for(var i=0;i<n;i++){ var a=rot+TAU2*i/n; q.push([cx+Math.cos(a)*r, cy+Math.sin(a)*r]); }
2037   return q;
2038 }
2039 /* Ikki nur kesishmasi (nuqta + yo'nalish vektori) */
2040 function rayCross(p,d,q,e){
2041   var den = d[0]*e[1] - d[1]*e[0];
2042   if(Math.abs(den) < 1e-12) return null;
2043   var t = ((q[0]-p[0])*e[1] - (q[1]-p[1])*e[0]) / den;
2044   return [p[0]+d[0]*t, p[1]+d[1]*t];
2045 }
2046 /* bbox ni qoplaydigan panjara nuqtalari: P = i.v1 + j.v2 */
2047 function latticePoints(v1,v2,bb,margin){
2048   var det = v1[0]*v2[1] - v1[1]*v2[0];
2049   if(Math.abs(det) < 1e-9) return [];
2050   var m = margin||0;
2051   var x0=bb.x0-m, y0=bb.y0-m, x1=bb.x1+m, y1=bb.y1+m;
2052   var corners=[[x0,y0],[x1,y0],[x1,y1],[x0,y1]];
2053   var iMin=Infinity, iMax=-Infinity, jMin=Infinity, jMax=-Infinity;
2054   for(var k=0;k<4;k++){
2055     var c=corners[k];
2056     var i=( c[0]*v2[1] - c[1]*v2[0])/det;
2057     var j=(-c[0]*v1[1] + c[1]*v1[0])/det;
2058     iMin=Math.min(iMin,i); iMax=Math.max(iMax,i);
2059     jMin=Math.min(jMin,j); jMax=Math.max(jMax,j);
2060   }
2061   var out=[];
2062   for(var jj=Math.floor(jMin)-1; jj<=Math.ceil(jMax)+1; jj++)
2063     for(var ii=Math.floor(iMin)-1; ii<=Math.ceil(iMax)+1; ii++)
2064       out.push({i:ii, j:jj, p:[ii*v1[0]+jj*v2[0], ii*v1[1]+jj*v2[1]]});
2065   return out;
2066 }
2067 function inBB(p,bb,pad){
2068   var q=pad||0;
2069   return p[0]>=bb.x0-q && p[0]<=bb.x1+q && p[1]>=bb.y0-q && p[1]<=bb.y1+q;
2070 }
2071
2072 /* =====
2073 1) PLITKALASHLAR
2074 Har biri {tiles:[{poly,kind}], period:[w,h]|null, unit:<o'lcham>}
2075 qaytaradi. `kind` — rang guruhi (0 = asosiy, 1 = ikkilamchi).
2076 Barchasi QIRRA-QIRRAGA: har ichki qirra aynan ikki plitkaga tegishli,
2077 shu sabab tugunlar bir-biriga aniq tushadi va iplar uzilmaydi.

```

```

2078 ////////////////////////////////////////////////// */
2079 var PIC_TILINGS = {
2080
2081   /* 4.4.4.4 – kvadrat to'r (6=45° klassik sakkizburchak-xoch beradi) */
2082   sq: {
2083     nm: "Kvadrat to'r (4.4.4.4)", deltas:[30,45,60,22.5], sides:4,
2084     build: function(s, bb){
2085       var tiles=[], pts=latticePoints([s,0],[0,s], bb, s*1.2);
2086       for(var k=0;k<pts.length;k++){
2087         var x=pts[k].p[0], y=pts[k].p[1];
2088         tiles.push({poly:[x,y],[x+s,y],[x+s,y+s],[x,y+s]], kind:(pts[k].i+pts[k].j)&1});
2089       }
2090       return {tiles:tiles, period:[s,s], unit:s};
2091     }
2092   },
2093
2094   /* 4.8.8 – sakkizburchak + kvadrat: Registon panellarining asosi */
2095   t488: {
2096     nm: "Sakkizburchak + kvadrat (4.8.8)", deltas:[45,67.5,22.5], sides:8,
2097     build: function(s, bb){
2098       var oR = (s*0.5)/Math.cos(Math.PI/8);          /* sakkizburchak tashqi radiusi */
2099       var a = 2*oR*Math.sin(Math.PI/8);             /* qirra uzunligi */
2100       var qR = a/Math.SQRT2;                         /* kvadrat tashqi radiusi (45° burilgan) */
2101       var tiles=[], pts=latticePoints([s,0],[0,s], bb, s*1.6);
2102       for(var k=0;k<pts.length;k++){
2103         var x=pts[k].p[0], y=pts[k].p[1];
2104         tiles.push({poly:ngon(x,y,oR,8,Math.PI/8), kind:0});
2105         tiles.push({poly:ngon(x+s/2,y+s/2,qR,4,-Math.PI/2), kind:1});
2106       }
2107       return {tiles:tiles, period:[s,s], unit:s};
2108     }
2109   },
2110
2111   /* 6.6.6 – asal uyasi (p6m) */
2112   t666: {
2113     nm: "Olti burchak to'ri (6.6.6)", deltas:[30,60,45,15], sides:6,
2114     build: function(s, bb){
2115       var v1=[SQ3*s,0], v2=[SQ3*s/2, 1.5*s];
2116       var tiles=[], pts=latticePoints(v1,v2, bb, s*2);
2117       for(var k=0;k<pts.length;k++){
2118         var c=pts[k].p;
2119         tiles.push({poly:ngon(c[0],c[1],s,6,Math.PI/6), kind:((pts[k].i+2*pts[k].j)%3+3)%3===0?0:1});
2120       }
2121       return {tiles:tiles, period:[SQ3*s, 3*s], unit:s};
2122     }
2123   },
2124
2125   /* 3.6.3.6 – olti burchak + uchburchak (trigeksagonal) */
2126   t3636: {
2127     nm: "Olti burchak + uchburchak (3.6.3.6)", deltas:[45,30,60], sides:6,
2128     build: function(a, bb){
2129       var d=2*a, v1=[d,0], v2=[d/2, d*SQ3/2];
2130       var tiles=[], pts=latticePoints(v1,v2, bb, d*2);
2131       var mid=function(A,B){ return [(A[0]+B[0])/2,(A[1]+B[1])/2] };
2132       for(var k=0;k<pts.length;k++){
2133         var P=pts[k].p;
2134         tiles.push({poly:ngon(P[0],P[1],a,6,0), kind:0});
2135         /* bo'shliq uchburchaklari = panjara uchburchagining qirra o'rtalari */
2136         var A=P, B=vAdd(P,v1), C=vAdd(P,v2), D=vAdd(B,v2);
2137         tiles.push({poly:[mid(A,B),mid(B,C),mid(A,C)], kind:1});
2138         tiles.push({poly:[mid(B,D),mid(C,D),mid(B,C)], kind:1});
2139       }
2140       return {tiles:tiles, period:[d, d*SQ3], unit:a};
2141     }
2142   },
2143
2144   /* 3.12.12 – o'n ikki burchak + uchburchak: Temuriylar 12-nurli yulduzi */
2145   t3122: {
2146     nm: "O'n ikki burchak (3.12.12)", deltas:[45,75,30,60,15], sides:12,
2147     build: function(a, bb){
2148       var R12 = a/(2*Math.sin(Math.PI/12));
2149       var r12 = a/(2*Math.tan(Math.PI/12));
2150       var d = 2*r12, v1=[d,0], v2=[d/2, d*SQ3/2];
2151       var tR = a/SQ3;          /* uchburchak tashqi radiusi */
2152       var tiles=[], pts=latticePoints(v1,v2, bb, d*1.6);
2153       for(var k=0;k<pts.length;k++){
2154         var P=pts[k].p;
2155         tiles.push({poly:ngon(P[0],P[1],R12,12,Math.PI/12), kind:0});
2156         var Gu=vAdd(P, vMul(vAdd(v1,v2), 1/3));
2157         var Gd=vAdd(P, vMul(vAdd(v1,v2), 2/3));
2158         tiles.push({poly:ngon(Gu[0],Gu[1],tR,3,-Math.PI/2), kind:1});
2159         tiles.push({poly:ngon(Gd[0],Gd[1],tR,3, Math.PI/2), kind:1});
2160       }
2161       return {tiles:tiles, period:[d, d*SQ3], unit:a};
2162     }
2163   },
2164
2165   /* SHAMSA-10 – HAQIQIY GIRIH TILE TO'PLAMI (o'n burchakli medalyon).
2166   Markazda o'n burchak (ichki burchagi 144°), uning har qirrasiga tashqi
2167   tomondan muntazam beshburchak (108°) ulanadi. O'n burchak uchida
2168   burchaklar yig'indisi 144° + 108° + 108° = 360° – ya'ni beshburchaklar

```



```

2169     halqasi BO'SHLIQSIZ yopiladi. Bu Registon va Shohi Zinda panellaridagi
2170     klassik shamsa qurilishi; barcha qirralar teng (a), barcha burchaklar
2171     36° ga karrali. Aynan shu sabab PIC lentalarini butun medalyon bo'ylab
2172     uzluksiz oqadi va 10 nurli yulduz markazda aniq yopiladi. */
2173     shamsa10: {
2174         nm: "Shamsa — o'n burchak + beshburchak", deltas:[72,54,63,36], sides:10, radial:true,
2175         build: function(unit, bb, opt){
2176             opt = opt || {};
2177             var R = opt.R || unit*4;
2178             /* R — medalyonning tashqi radiusi. Beshburchak halqasining tashqi
2179             chegarasi markazdan rOut = rd + 2·rp masofada (rd — o'n burchak
2180             ichki radiusi, rp — beshburchak ichki radiusi) → a ni shundan topamiz. */
2181             var rdK = 1/(2*Math.tan(Math.PI/10)); /* o'n burchak ichki radiusi / a */
2182             var rpK = 1/(2*Math.tan(Math.PI/5)); /* beshburchak ichki radiusi / a */
2183             var RpK = 1/(2*Math.sin(Math.PI/5)); /* beshburchak tashqi radiusi / a */
2184             var a = R/(rdK + rpK + RpK);
2185             var rd = rdK*a, rp = rpK*a, Rp = RpK*a;
2186             var Rd = a/(2*Math.sin(Math.PI/10));
2187             var tiles=[], i, j;
2188             /* markaziy o'n burchak: uchlari 18°+36k, qirra o'rtalari 36k */
2189             tiles.push({poly: ngon(0,0,Rd,10,Math.PI/10), kind:0});
2190             for(i=0;i<10;i++){
2191                 var th = TAU2*i/10; /* qirra normalining yo'nalishi */
2192                 var C = [Math.cos(th)*(rd+rp), Math.sin(th)*(rd+rp)];
2193                 /* beshburchakning ichki qirrasini o'rtasi C dan (th+180°) tomonda →
2194                 uchlari th + 180° + 36° + 72°j burchaklarda */
2195                 var poly=[];
2196                 for(j=0;j<5;j++){
2197                     var aa = th + Math.PI + Math.PI/5 + TAU2*j/5;
2198                     poly.push([C[0]+Math.cos(aa)*Rp, C[1]+Math.sin(aa)*Rp]);
2199                 }
2200                 tiles.push({poly:poly, kind:1});
2201             }
2202             return {tiles:tiles, period:null, unit:a, radius:rd+rp+Rp, radial:true};
2203         }
2204     }
2205 };
2206
2207 /* =====
2208 2) KONTAKT NURLARI + 3) GLOBAL GRAF (dedup, tugun birlashtirish)
2209 =====
2210 Har plitkani har qirrasini o'rtasidan ikki nur chiqadi:
2211     d1 = +cosδ·u + sinδ·nIn      d2 = -cosδ·u + sinδ·nIn
2212     (u — qirra yo'nalishi, nIn — plitka ichiga qaragan normal)
2213     Qo'shni plitkada u va nIn ikkisi ham teskari bo'ladi, shuning uchun
2214     uning nurlari -d1, -d2 ga teng → qirra o'rtasida chiziqlar TO'G'RI
2215     davom etadi. Aynan shu xossa "yamoq" effektini yo'q qiladi.
2216 ===== */
2217 function picBuild(tg, deltaDeg, opt){
2218     opt = opt || {};
2219     var tiles = tg.tiles, unit = tg.unit || 1;
2220     var EPS = unit * 1e-3; /* tugun birlashtirish toleransi */
2221     var delta = deltaDeg * Math.PI/180;
2222     var cs = Math.cos(delta), sn = Math.sin(delta);
2223
2224     var nodes=[], nodeMap={}, edges=[], edgeSet={};
2225     /* Tugun turi muhim:
2226     mid — plitka qirrasining o'rtasi (ichkarida 4 darajali KESISHISH,
2227     patch chegarasida 2 darajali TUGATUVCHI)
2228     x — plitka ichidagi uchrashuv nuqtasi (lentaning haqiqiy BURCHAGI) */
2229     function nodeAt(p, isMid){
2230         var kx = Math.round(p[0]/EPS), ky = Math.round(p[1]/EPS);
2231         var key = kx+'|'+ky;
2232         var id = nodeMap[key];
2233         if(id === undefined){
2234             /* qo'shni kataklarni ham tekshiramiz — yaxlitlash chegarasida bo'linib
2235             ketmasligi uchun (aks holda bir tugun ikkiga ajralib, ip uziladi) */
2236             for(var dx=-1; dx<=1 && id===undefined; dx++){
2237                 for(var dy=-1; dy<=1 && id===undefined; dy++){
2238                     var alt = nodeMap[(kx+dx)+'|'+(ky+dy)];
2239                     if(alt !== undefined && Math.hypot(nodes[alt].p[0]-p[0], nodes[alt].p[1]-p[1]) < EPS) id = alt;
2240                 }
2241             }
2242             if(id === undefined){ id = nodes.length; nodes.push({p:p, e:[], mid:false}); }
2243             nodeMap[key] = id;
2244         }
2245         if(isMid) nodes[id].mid = true;
2246         return id;
2247     }
2248     function addEdge(a,b){
2249         if(a===b) return;
2250         var key = a<b ? a+'-'+b : b+'-'+a;
2251         if(edgeSet[key] !== undefined) return; /* DEDUP: umumiy qirradagi takror segment */
2252         edgeSet[key] = edges.length;
2253         edges.push({a:a, b:b});
2254         nodes[a].e.push(edges.length-1);
2255         nodes[b].e.push(edges.length-1);
2256     }
2257
2258     /* Konveks plitka ichidaligini tekshirish (yarim tekislik testi) */
2259     function insideConvex(X, poly, cen, tol){
2260         for(var i=0;i<poly.length;i++){

```

```

2260     var A=poly[i], B=poly[(i+1)%poly.length];
2261     var u=vNorm(vSub(B,A));
2262     var nIn=[-u[1],u[0]];
2263     if(vDot(nIn, vSub(cen,A)) < 0) nIn=[u[1],-u[0]];
2264     if(vDot(nIn, vSub(X,A)) < -tol) return false;
2265 }
2266 return true;
2267 }
2268
2269 var cores = [];
2270 for(var t=0; t<tiles.length; t++){
2271     var poly = tiles[t].poly, m = poly.length;
2272     var cen = polyCentroid(poly);
2273     /* har qirra o'rtasidan ikki nur – jami 2m nur */
2274     var rays=[], i, j;
2275     for(i=0;i<m;i++){
2276         var A = poly[i], B = poly[(i+1)%m];
2277         var M = [(A[0]+B[0])/2, (A[1]+B[1])/2];
2278         var u = vNorm(vSub(B,A));
2279         var nIn = [-u[1], u[0]];
2280         if(vDot(nIn, vSub(cen,M)) < 0) nIn = [u[1], -u[0]]; /* ichkariga qaratish */
2281         rays.push({p:M, d:[ cs*u[0] + sn*nIn[0], cs*u[1] + sn*nIn[1], e:i});
2282         rays.push({p:M, d:[-cs*u[0] + sn*nIn[0], -cs*u[1] + sn*nIn[1], e:i});
2283     }
2284     /* HANKIN QOIDASI + O'ZARO JUFTLASH.
2285     Har nur plitka ichida boshqa nur bilan uchrashadi. Juftlash O'ZARO
2286     bo'lishi shart: aks holda nurlar har biri o'z nuqtasida tugab,
2287     yulduz yopilmaydi va bo'sh uchlar ("tikan") paydo bo'ladi.
2288     Shu sabab barcha nomzod kesishmalar masofa bo'yicha tartiblanib,
2289     ochko'z (greedy) usulda o'zaro juftlanadi. */
2290     var tol = unit*1e-6, minT = unit*1e-4;
2291     var cand=[];
2292     for(i=0;i<rays.length;i++){
2293         for(j=i+1;j<rays.length;j++){
2294             if(rays[i].e === rays[j].e) continue; /* bir qirraning nurlari juftlashmaydi */
2295             var X = rayCross(rays[i].p, rays[i].d, rays[j].p, rays[j].d);
2296             if(!X) continue;
2297             var ti = vDot(vSub(X, rays[i].p), rays[i].d);
2298             var tj = vDot(vSub(X, rays[j].p), rays[j].d);
2299             if(ti < minT || tj < minT) continue; /* orqaga qarab yechim */
2300             if(!insideConvex(X, poly, cen, unit*1e-3)) continue;
2301             /* t ni KVANTLAB saqlaymiz: "chiroyli" 8 larda bir nechta nomzod
2302             bir xil masofada bo'ladi (teng holat). Xom suzuvchi nuqta bilan
2303             taqqoslasak, panjaraning turli joyidagi bir xil plitkalar teng
2304             holatni har xil hal qiladi – natijada naqsh davriyligi buzilib,
2305             chok-suz tiling chetida chiziqlar mos kelmaydi.
2306             Kvantlash + indeks bo'yicha uzil-kesil tartib buni yo'q qiladi. */
2307             cand.push({i:i, j:j, x:X, t:Math.max(ti,tj), tq:Math.round(Math.max(ti,tj)/(unit*1e-7))});
2308         }
2309     }
2310     cand.sort(function(A2,B2){ return (A2.tq - B2.tq) || (A2.i - B2.i) || (A2.j - B2.j); });
2311     var taken = new Array(rays.length).fill(false);
2312     var meet = new Array(rays.length).fill(null);
2313     for(i=0;i<cand.length;i++){
2314         var cd=cand[i];
2315         if(taken[cd.i] || taken[cd.j]) continue;
2316         taken[cd.i]=taken[cd.j]=true;
2317         meet[cd.i]=meet[cd.j]=cd.x;
2318     }
2319     var cp=[];
2320     for(i=0;i<rays.length;i++){
2321         if(!meet[i]) continue;
2322         addEdge(nodeAt(rays[i].p, true), nodeAt(meet[i], false));
2323         cp.push(meet[i]);
2324     }
2325     /* yulduz o'zagi: uchrashuv nuqtalari markaz atrofida burchak bo'yicha tartiblanadi */
2326     if(cp.length >= 3){
2327         var uniq=[], seen={};
2328         for(i=0;i<cp.length;i++){
2329             var kk = Math.round(cp[i][0]/(unit*1e-3))+'|'+Math.round(cp[i][1]/(unit*1e-3));
2330             if(seen[kk]) continue;
2331             seen[kk]=1; uniq.push(cp[i]);
2332         }
2333         if(uniq.length >= 3){
2334             uniq.sort(function(P,Q){
2335                 return Math.atan2(P[1]-cen[1],P[0]-cen[0]) - Math.atan2(Q[1]-cen[1],Q[0]-cen[0]);
2336             });
2337             cores.push({poly:uniq, kind:tiles[t].kind});
2338         }
2339     }
2340 }
2341
2342 /* ---- 4) IP AJRATISH: tugunda davomiylik = eng qarama-qarshi qirra ---- */
2343 function dirOf(eIdx, fromNode){
2344     var e = edges[eIdx];
2345     var o = e.a === fromNode ? e.b : e.a;
2346     return vNorm(vSub(nodes[o].p, nodes[fromNode].p));
2347 }
2348 function partner(nodeId, eIdx){
2349     var nd = nodes[nodeId], inc = nd.e;
2350     if(inc.length < 2) return -1;

```

```

2351     /* Lentaning burchagi (plitka ichidagi uchrashuv nuqtasi): davomiylik
2352     yagona – burchak qanchalik o'tkir bo'lsa ham ip UZILMAYDI. */
2353     if(!nd.mid && inc.length === 2) return inc[0] === eIdx ? inc[1] : inc[0];
2354     /* Patch chegarasidagi qirra o'rtasi: faqat bitta plitka nur bergan →
2355     ip shu yerda TUGAYDI (aks holda orqaga qaytib, iplar qo'shilib ketadi
2356     va to'qish paritysi buziladi). */
2357     if(nd.mid && inc.length < 4) return -1;
2358     /* Haqiqiy kesishish: chiziq TO'G'RI davom etadi (-dot ≈ 1) */
2359     var din = dirOf(eIdx, nodeId);
2360     var best=-1, bestDot=0.55;
2361     for(var k=0; k<inc.length; k++){
2362         if(inc[k] === eIdx) continue;
2363         var sc = -vDot(din, dirOf(inc[k], nodeId));
2364         if(sc > bestDot){ bestDot = sc; best = inc[k]; }
2365     }
2366     return best;
2367 }
2368 var used = new Array(edges.length).fill(false);
2369 var strands=[];
2370 function walk(startE, startN){
2371     var pts=[nodes[startN].p], eIdx=startE, cur=startN, guard=0;
2372     var eList=[];
2373     while(eIdx >= 0 && !used[eIdx] && guard++ < 20000){
2374         used[eIdx]=true; eList.push(eIdx);
2375         var e=edges[eIdx];
2376         var nxt = e.a===cur ? e.b : e.a;
2377         pts.push(nodes[nxt].p);
2378         cur = nxt;
2379         eIdx = partner(cur, eIdx);
2380     }
2381     return {pts:pts, edges:eList, endNode:cur};
2382 }
2383 for(var ei=0; ei<edges.length; ei++){
2384     if(used[ei]) continue;
2385     /* ikki yo'nalishga yurib, ipni to'liq yig'amiz */
2386     var fwd = walk(ei, edges[ei].a);
2387     var backStart = partner(edges[ei].a, ei);
2388     var pts = fwd.pts;
2389     if(backStart >= 0 && !used[backStart]){
2390         var bwd = walk(backStart, edges[ei].a);
2391         bwd.pts.reverse();
2392         pts = bwd.pts.slice(0, bwd.pts.length-1).concat(fwd.pts);
2393     }
2394     var closed = pts.length>2 && Math.hypot(pts[0][0]-pts[pts.length-1][0], pts[0][1]-pts[pts.length-1][1]) < EPS*
4;
2395     strands.push({pts:pts, closed:closed});
2396 }
2397
2398 /* ---- 5) KESISHISHLAR + TO'QISH (ust/ost navbatlashishi) ---- */
2399 var cross=[];
2400 for(var ni=0; ni<nodes.length; ni++){
2401     if(nodes[ni].e.length === 4 && nodes[ni].mid) cross.push({node:ni, p:nodes[ni].p, over:-1, on:[]});
2402 }
2403 var crossOfNode={};
2404 for(var ci=0; ci<cross.length; ci++) crossOfNode[cross[ci].node]=ci;
2405 /* har ipni kuzatib, u kesib o'tgan tugunlarni tartib bilan yozamiz */
2406 for(var si=0; si<strands.length; si++){
2407     var sp = strands[si].pts, seq=[];
2408     for(var pi=0; pi<sp.length; pi++){
2409         var id = nodeMap[Math.round(sp[pi][0]/EPS)+'|'+Math.round(sp[pi][1]/EPS)];
2410         if(id === undefined) continue;
2411         var c = crossOfNode[id];
2412         if(c !== undefined) seq.push(c);
2413     }
2414     strands[si].cross = seq;
2415     for(var qi=0; qi<seq.length; qi++) cross[seq[qi]].on.push({s:si, k:qi});
2416 }
2417 /* ---- TO'QISH: 2-bo'yash (XOR cheklovlar grafi) ----
2418 Har kesishishda ikki ip uchrashadi. Haqiqiy to'qish shartini
2419 ip bo'ylab ust/ost QAT'IY almashishi belgilaydi:
2420     over_s(c1) XOR over_s(c2) = 1 (ketma-ket kesishishlar)
2421     x_c = 1 bo'lsa kichik indeksli ip ustidan o'tadi, deb belgilaymiz.
2422     over_s(c) = x_c XOR (s ≠ A(c)) bo'lgani uchun cheklov quyidagiga aylanadi:
2423     x_c1 XOR x_c2 = 1 XOR b1 XOR b2
2424 Bu – grafni ikki rangga bo'yash masalasi; BFS bilan tarqatib yechiladi.
2425 (Ochko'z "birinchi kelgan yutadi" usuli navbatlashishni buzadi.) */
2426 for(ci=0; ci<cross.length; ci++){
2427     var c3 = cross[ci], sa=-1, sb=-1;
2428     for(var oi=0; oi<c3.on.length; oi++){
2429         var sid=c3.on[oi].s;
2430         if(sa===-1) sa=sid;
2431         else if(sid!==sa && sb===-1) sb=sid;
2432     }
2433     if(sb===-1) sb=sa;
2434     c3.A = Math.min(sa,sb); c3.B = Math.max(sa,sb);
2435     c3.x = -1;
2436 }
2437 var adj=[];
2438 for(ci=0; ci<cross.length; ci++) adj.push([]);
2439 function bOf(c, s){ return (s !== c.A) ? 1 : 0; }
2440 for(si=0; si<strands.length; si++){

```

```

2441     var seq3 = strands[si].cross || [];
2442     for(var q3=0; q3+1<seq3.length; q3++){
2443         var i1=seq3[q3], i2=seq3[q3+1];
2444         if(i1===i2) continue;
2445         var C1=cross[i1], C2=cross[i2];
2446         if(C1.A===C1.B || C2.A===C2.B) continue;      /* o'z-o'zini kesish – cheklovsiz */
2447         var par = 1 ^ b0f(C1,si) ^ b0f(C2,si);
2448         adj[i1].push({to:i2, p:par});
2449         adj[i2].push({to:i1, p:par});
2450     }
2451 }
2452 var conflicts=0;
2453 for(ci=0; ci<cross.length; ci++){
2454     if(cross[ci].x !== -1) continue;
2455     cross[ci].x = 1;
2456     var queue=[ci];
2457     while(queue.length){
2458         var cu=queue.pop();
2459         for(var ai=0; ai<adj[cu].length; ai++){
2460             var lnk=adj[cu][ai];
2461             var want = cross[cu].x ^ lnk.p;
2462             if(cross[lnk.to].x === -1){ cross[lnk.to].x = want; queue.push(lnk.to); }
2463             else if(cross[lnk.to].x !== want) conflicts++;
2464         }
2465     }
2466 }
2467 for(ci=0; ci<cross.length; ci++){
2468     var c4=cross[ci];
2469     c4.over = c4.x === 1 ? c4.A : c4.B;
2470 }
2471
2472 return {
2473     tiles: tiles, cores: cores, strands: strands, cross: cross,
2474     nodes: nodes, edges: edges, period: tg.period, unit: unit, weaveConflicts: conflicts, radial: !!tg.radial,
2475     radius: tg.radius, delta: deltaDeg
2476 };
2477 }
2478
2479 /* =====
2480 6) MODEL YASASH – plitkalash + 6 tanlash bitta joyda
2481 6 ("kontakt burchagi") KVANTLANGAN: girih faqat simmetriyaga bog'langan
2482 diskret burchaklarda "yopiladi". Erkin slayder oradagi qiymatni bersa,
2483 yulduz uchlari mos kelmaydi va mayda noto'g'ri uchburchaklar chiqadi –
2484 shu sabab foydalanuvchiga variant tanlash beriladi, xom burchak emas.
2485 ===== */
2486 function picDeltas(tileId, n){
2487     var T = PIC_TILINGS[tileId] || PIC_TILINGS.t488;
2488     if(T.deltas && T.deltas.length) return T.deltas;    /* plitkalashning kanonik burchaklari */
2489     if(T.radial){
2490         var nn = Math.max(5, Math.round(n||10));
2491         return [90 - 180/nn, 45, 60, 180/nn*1.5].map(function(d){
2492             return Math.max(8, Math.min(82, Math.round(d*10)/10));
2493         });
2494     }
2495     return T.deltas;
2496 }
2497 function picModel(spec){
2498     var id = PIC_TILINGS[spec.tile] ? spec.tile : 't488';
2499     var T = PIC_TILINGS[id];
2500     var ds = picDeltas(id, spec.n);
2501     var delta = ds[[(spec.dvar[0] % ds.length + ds.length) % ds.length];
2502     var bb = spec.bb || {x0:-spec.R, y0:-spec.R, x1:spec.R, y1:spec.R};
2503     var tg = T.build(spec.unit, bb, {n:spec.n, R:spec.R, rings:spec.rings, ir:spec.ir});
2504     var M = picBuild(tg, delta, spec);
2505     M.tileId = id; M.tileNm = T.nm; M.deltaList = ds;
2506     picFaces(M);
2507     return M;
2508 }
2509
2510 /* =====
2511 7) RENDER – TEKIS RANG (alfa aralashmasi yo'q)
2512 Haqiqiy koshinkorlikda ranglar to'liq qoplama, chegarada yupqa to'q
2513 kontur bo'ladi. Yarim shaffof qatlamlar bir-birining ustiga tushib
2514 "akvarel" tusini bergani uchun bu yerda globalAlpha ISHLATILMAYDI.
2515 To'g'ish: 1-o'tishda barcha iplar, 2-o'tishda kesishishlarda faqat
2516 USTIDAN o'tuvchi ipning qisqa bo'lagi qayta chiziladi → ost ip uziladi.
2517 Bu usul SVG eksportida ham aynan ishlaydi (chizish tartibi saqlanadi).
2518 ===== */
2519 function picPolyPath(ctx, pts, closed){
2520     if(!pts.length) return;
2521     ctx.moveTo(pts[0][0], pts[0][1]);
2522     for(var i=1; i<pts.length; i++) ctx.lineTo(pts[i][0], pts[i][1]);
2523     if(closed) ctx.closePath();
2524 }
2525 function picFlatFill(ctx, col){
2526     ctx.save(); ctx.globalAlpha = 1; ctx.fillStyle = col; ctx.fill(); ctx.restore();
2527 }
2528 function picStroke(ctx, col, w){
2529     ctx.save();
2530     ctx.globalAlpha = 1; ctx.strokeStyle = col;
2531     ctx.lineWidth = Math.max(0.12, w);

```

```

2532     ctx.lineCap = 'butt'; ctx.lineJoin = 'miter'; ctx.miterLimit = 6;
2533     ctx.stroke(); ctx.restore();
2534 }
2535 /* Bitta lenta: to'q kontur (keng) + ochiq band (tor). Ikki o'tish — chegara
2536     avtomatik bir tekis qalinlikda chiqadi, alohida offset hisoblash kerak emas. */
2537 function picRibbon(ctx, pts, band, edge, w, closed){
2538     ctx.beginPath(); picPolyPath(ctx, pts, closed);
2539     picStroke(ctx, edge, w);
2540     ctx.beginPath(); picPolyPath(ctx, pts, closed);
2541     picStroke(ctx, band, Math.max(0.4, w - Math.max(1.1, w*0.34)));
2542 }
2543 /* Kesishish atrofidagi qisqa bo'lak — ust ipni qayta chizish uchun */
2544 function picLocalPiece(pts, idx, reach){
2545     var out=[pts[idx]], i, acc;
2546     acc=0;
2547     for(i=idx-1;i>=0;i--){
2548         acc += Math.hypot(pts[i+1][0]-pts[i][0], pts[i+1][1]-pts[i][1]);
2549         out.unshift(pts[i]);
2550         if(acc >= reach) break;
2551     }
2552     acc=0;
2553     for(i=idx+1;i<pts.length;i++){
2554         acc += Math.hypot(pts[i][0]-pts[i-1][0], pts[i][1]-pts[i-1][1]);
2555         out.push(pts[i]);
2556         if(acc >= reach) break;
2557     }
2558     return out;
2559 }
2560 function picRender(ctx, M, style){
2561     var s = style || {};
2562     var bw = Math.max(0.8, s.bandW || M.unit*0.16);
2563     var band = s.band || '#fdfaf0';
2564     var edge = s.edge || '#241a08';
2565     var fills = s.fills || ['#efe2bc', '#3f6a95'];
2566     var core = s.core || '#1d3a63';
2567     var mode = s.mode || 'dual'; /* 'dual' | 'fill' | 'line' */
2568     var weave = s.weave !== false;
2569     var i, k;
2570
2571     /* 1) FON + YUZALAR — har shakl o'z rangida (haqiqiy sirlangan koshin) */
2572     if(mode !== 'line'){
2573         for(i=0;i<M.tiles.length;i++){
2574             ctx.beginPath(); picPolyPath(ctx, M.tiles[i].poly, true);
2575             picFlatFill(ctx, fills[0]);
2576         }
2577         var fc = M.faces || [];
2578         var pl = s.regionPal || [fills[0], core, fills[1] || core, band];
2579         for(i=0;i<fc.length;i++){
2580             ctx.beginPath(); picPolyPath(ctx, fc[i].poly, true);
2581             picFlatFill(ctx, pl[fc[i].rank % pl.length]);
2582         }
2583     }
2584
2585     /* 3) faqat chiziq rejimi — texnik chizma (CNC/lazer uchun) */
2586     if(mode === 'line'){
2587         for(i=0;i<M.strands.length;i++){
2588             ctx.beginPath(); picPolyPath(ctx, M.strands[i].pts, false);
2589             picStroke(ctx, edge, Math.max(0.3, bw*0.18));
2590         }
2591         return;
2592     }
2593
2594     /* 4) barcha iplar — asosiy o'tish */
2595     for(i=0;i<M.strands.length;i++){
2596         picRibbon(ctx, M.strands[i].pts, band, edge, bw, false);
2597     }
2598
2599     /* 5) TO'QISH — kesishishda ustidan o'tuvchi ipni qayta chizish.
2600         Ost ip shu joyda ust lentaning tagida qolib, uzilgan ko'rinadi. */
2601     if(weave){
2602         var reach = bw*1.15;
2603         for(k=0;k<M.cross.length;k++){
2604             var c = M.cross[k];
2605             if(c.over < 0) continue;
2606             var str = M.strands[c.over];
2607             if(!str) continue;
2608             /* ipdagi shu kesishishning o'rnini topamiz */
2609             var seq = str.cross || [], pos = -1;
2610             for(i=0;i<seq.length;i++) if(seq[i] === k){ pos = i; break; }
2611             if(pos < 0) continue;
2612             /* ip nuqtalari orasidan kesishish koordinatasiga eng yaqinini olamiz */
2613             var bi=-1, bd=Infinity;
2614             for(i=0;i<str.pts.length;i++){
2615                 var d = Math.hypot(str.pts[i][0]-c.p[0], str.pts[i][1]-c.p[1]);
2616                 if(d < bd){ bd = d; bi = i; }
2617             }
2618             if(bi < 0 || bd > M.unit*0.02) continue;
2619             picRibbon(ctx, picLocalPiece(str.pts, bi, reach), band, edge, bw, false);
2620         }
2621     }
2622 }

```

```

2623
2624 /* =====
2625 8) YUZALARNI AJRATISH (planar faces) – HAQIQIY KOSHIN BO'YASH
2626 -----
2627 Lentalarning o'q chiziqlari tekislikni yopiq yuzalarga bo'ladi:
2628 yulduzlar, ko'rshapalaklar (bowtie), oltiburchaklar, romblar...
2629 Chinnisozlikda AYNAN shu yuzalar sirlanadi va bir xil shakl doim
2630 bir xil rang oladi. Yuzalar yarim qirralar (half-edge) aylanishi
2631 bilan topiladi: har dartning davomi – tugunda teskari dartdan
2632 soat yo'nalishi bo'yicha keyingi dart.
2633 ===== */
2634 function picFaces(M){
2635   var nodes=M.nodes, edges=M.edges;
2636   var darts=[], dartAt=[];
2637   var i, k;
2638   for(i=0;i<nodes.length;i++) dartAt.push([]);
2639   for(i=0;i<edges.length;i++){
2640     darts.push({from:edges[i].a, to:edges[i].b, tw:-1});
2641     darts.push({from:edges[i].b, to:edges[i].a, tw:-1});
2642     darts[darts.length-2].tw = darts.length-1;
2643     darts[darts.length-1].tw = darts.length-2;
2644     dartAt[edges[i].a].push(darts.length-2);
2645     dartAt[edges[i].b].push(darts.length-1);
2646   }
2647   function ang(d){
2648     var A=nodes[darts[d].from].p, B=nodes[darts[d].to].p;
2649     return Math.atan2(B[1]-A[1], B[0]-A[0]);
2650   }
2651   var order={};
2652   for(i=0;i<nodes.length;i++){
2653     var lst = dartAt[i].slice();
2654     lst.sort(function(p,q){ return ang(p)-ang(q); });
2655     dartAt[i]=lst;
2656     for(k=0;k<lst.length;k++) order[lst[k]] = k;
2657   }
2658   /* dart u→v ning davomi: v tugunida (v→u) dartdan soat yo'nalishida keyingisi */
2659   function nextDart(d){
2660     var v = darts[d].to, rev = darts[d].tw;
2661     var lst = dartAt[v], m = lst.length;
2662     var idx = order[rev];
2663     if(idx === undefined) return -1;
2664     return lst[(idx - 1 + m) % m];
2665   }
2666   var seen = new Array(darts.length).fill(false);
2667   var faces=[];
2668   for(i=0;i<darts.length;i++){
2669     if(seen[i]) continue;
2670     var cyc=[], d=i, guard=0;
2671     while(d>=0 && !seen[d] && guard++ < 100000){
2672       seen[d]=true; cyc.push(d); d = nextDart(d);
2673     }
2674     if(cyc.length < 3) continue;
2675     var pts = cyc.map(function(q){ return nodes[darts[q].from].p; });
2676     var a2=0;
2677     for(k=0;k<pts.length;k++){
2678       var A=pts[k], B=pts[(k+1)%pts.length];
2679       a2 += A[0]*B[1] - B[0]*A[1];
2680     }
2681     a2 /= 2;
2682     if(a2 <= 0) continue; /* tashqi yuza / teskari aylanish */
2683     faces.push({poly:pts, area:a2, n:pts.length});
2684   }
2685   /* Bir xil shakl → bir xil rang: yuza (area) va qirra soni bo'yicha guruh.
2686      Kvantlash toleransi hisoblash xatolarini yutadi. */
2687   var u2 = M.unit*M.unit;
2688   var groups={}, gList=[];
2689   /* Radial medalyonda kataklar tashqariga kattalashadi, shuning uchun yuza
2690      bo'yicha guruhlash ranglarni chalkashtiradi – u holda faqat SHAKL
2691      (qirra soni) hisobga olinadi va halqalar bo'ylab rang izchil qoladi. */
2692   var byShapeOnly = !!M.radial;
2693   for(i=0;i<faces.length;i++){
2694     var key = byShapeOnly ? String(faces[i].n) : (faces[i].n + ':' + Math.round(faces[i].area/u2*160));
2695     if(groups[key] === undefined){ groups[key] = gList.length; gList.push({key:key, area:faces[i].area, n:faces[i].n, cnt:0}); }
2696     faces[i].g = groups[key];
2697     gList[groups[key]].cnt++;
2698   }
2699   /* guruhlarni kattaligi bo'yicha tartiblab, barqaror rang indeksini beramiz
2700      (eng katta yuza = asosiy yulduz maydoni) */
2701   var idx = gList.map(function(g,j){ return j; });
2702   idx.sort(function(p,q){ return gList[q].area - gList[p].area; });
2703   var rank={};
2704   for(i=0;i<idx.length;i++) rank[idx[i]] = i;
2705   for(i=0;i<faces.length;i++) faces[i].rank = rank[faces[i].g];
2706   M.faces = faces;
2707   M.faceGroups = gList.length;
2708   return faces;
2709 }
2710
2711 /* =====
2712 9) ILOVA KO'PRIGI – PIC yadrosini NaqshAI parametrlariga bog'lash

```

```

2713  */
2714  /* Plitkash o'lchov koeffitsienti: har to'rda `unit` boshqa ma'noni
2715  bildiradi (qirra uzunligi yoki katak qadami), shuning uchun ekranda
2716  taxminan bir xil zichlik chiqishi uchun moslanadi. */
2717  var PIC_USCALE = { sq:1.0, t488:1.0, t666:0.66, t3636:0.52, t3122:0.5, shamsa10:1.0 };
2718  var PIC_LIST = [
2719    {id:'sq',      nm:'Kvadrat to\'r (4.4.4.4)'},
2720    {id:'t488',    nm:'Sakkizburchak + kvadrat (4.8.8)'},
2721    {id:'t666',    nm:'Olti burchak (6.6.6)'},
2722    {id:'t3636',   nm:'Olti burchak + uchburchak (3.6.3.6)'},
2723    {id:'t3122',   nm:'O\'n ikki burchak (3.12.12)'},
2724    {id:'shamsa10', nm:'Shamsa - o\'n nurli medalyon'}
2725  ];
2726  function picIsRadial(tileId){
2727    var T = PIC_TILINGS[tileId];
2728    return !(T && T.radial);
2729  }
2730  /* Naqsh parametrlaridan PIC spetsifikatsiyasi */
2731  function picSpecFrom(p, R, opt){
2732    opt = opt || {};
2733    var tile = PIC_TILINGS[p.tile] ? p.tile : 't488';
2734    var us = PIC_USCALE[tile] || 1;
2735    var unit;
2736    if(opt.cellPx){
2737      unit = opt.cellPx * us; /* tessellatsiyada katak slayderi */
2738    } else {
2739      unit = R * (0.62 - Math.min(1, Math.max(0, p.dens))) * 0.40 * us;
2740    }
2741    unit = Math.max(R*0.06, unit);
2742    var bb = opt.bb || {x0:-R, y0:-R, x1:R, y1:R};
2743    return {
2744      tile: tile, unit: unit, R: R, bb: bb,
2745      dvar: p.dvar||0,
2746      n: 10,
2747      ir: 0.12 + Math.min(0.8, Math.max(0.1, p.ir))*0.3
2748    };
2749  }
2750  /* Palitradan render uslubi. Alfa yo'q - ranglar to'liq qoplama. */
2751  function picStyleFrom(pal, p, unit){
2752    var band = lerpColor(pal.c[4], 'ffffff', 0.62);
2753    var edge = shade(pal.c[2], 0.30);
2754    var mode = p.fill === 'line' ? 'line' : 'dual';
2755    /* Yuza ranglari: eng katta yuza (yulduz maydoni) - asosiy urg'u.
2756       Medalyonda markaz ko'zga tashlanishi kerak, maydonda esa fon ochiq
2757       bo'lgani ma'qul, shuning uchun tartib turiga qarab o'zgaradi. */
2758    var regionPal = picIsRadial(p.tile)
2759      ? [pal.c[1], pal.c[3], pal.c[4], pal.c[0], pal.c[2]]
2760      : [pal.c[4], pal.c[1], pal.c[3], pal.c[0], pal.c[2]];
2761    return {
2762      band: band, edge: edge,
2763      fills: [pal.c[4], pal.c[3]],
2764      core: pal.c[1], core2: pal.c[3],
2765      regionPal: regionPal,
2766      bandW: Math.max(1.1, unit*0.16*(p.sw/2.2)),
2767      mode: mode,
2768      weave: p.weave !== false
2769    };
2770  }
2771  /* Model keshi - slayder surilganda har kadrda qayta qurmaslik uchun */
2772  var PIC_CACHE = {};
2773  function picGet(spec){
2774    var key = [spec.tile, Math.round(spec.unit*100), spec.dvar,
2775      Math.round(spec.bb.x0), Math.round(spec.bb.y0),
2776      Math.round(spec.bb.x1), Math.round(spec.bb.y1), Math.round(spec.ir*1000)].join('|');
2777    var hit = PIC_CACHE[key];
2778    if(hit) return hit;
2779    var M = picModel(spec);
2780    var keys = Object.keys(PIC_CACHE);
2781    if(keys.length > 24) delete PIC_CACHE[keys[0]];
2782    PIC_CACHE[key] = M;
2783    return M;
2784  }
2785
2786  /* ---- Yakka naqsh (medalyon) sifatida chizish: PATTERNS registriga ulanadi ---- */
2787  function drawPICPattern(ctx, R, p, sw, c0, c1, c2, c3){
2788    var pal = PAL[p.pal] || PAL.gold;
2789    var spec, M;
2790    if(p._field){
2791      /* Uzluksiz maydon: butun kanvas bo'ylab yaxlit naqsh (medalyon takrori emas) */
2792      var Rf = R;
2793      spec = picSpecFrom(p, Rf, {cellPx: p._cellpx, bb:{x0:-Rf, y0:-Rf, x1:Rf, y1:Rf}});
2794      M = picGet(spec);
2795      picRender(ctx, M, picStyleFrom(pal, p, spec.unit));
2796      return;
2797    }
2798    spec = picSpecFrom(p, R);
2799    M = picGet(spec);
2800    var style = picStyleFrom(pal, p, spec.unit);
2801    ctx.save();
2802    if(!ctx._svg && !picIsRadial(p.tile)){
2803      /* maydon turlari doira ichida kesiladi - medalyon o'z chegarasiga ega */

```

```

2804     ctx.beginPath(); circle(ctx, 0, 0, R*0.995); ctx.clip();
2805     ctx.beginPath(); circle(ctx, 0, 0, R*0.995);
2806     picFlatFill(ctx, pal.c[4]);
2807 }
2808 picRender(ctx, M, style);
2809 ctx.restore();
2810 if(!picIsRadial(p.tile) && !NORING){
2811     ctx.beginPath(); circle(ctx, 0, 0, R*0.995);
2812     picStroke(ctx, style.edge, Math.max(0.6, style.bandW*0.45));
2813 }
2814 }
2815
2816 /* =====
2817 10) CHOK-SUZ TESSELLATSIYA – MATEMATIK ANIQ
2818 -----
2819 Avvalgi usul "bleed" (katakdan kattaroq radius) bilan chokni YASHIRAR edi
2820 – bu taxminiy va eksportda ko'rinib qolardi. To'g'ri yechim:
2821 naqsh panjaraning ANIQ davriga teng offscreen kanvasga chiziladi, geometriya
2822 esa 3x3 davr bo'ylab yaratiladi → kanvas chetidan chiqqan lentalar qarama-
2823 qarshi chetdan aynan davom etadi. So'ng createPattern('repeat') bilan
2824 istalgan o'lchamga cho'ziladi: chok nolga TENG, taxmin yo'q.
2825 ===== */
2826 function picSeamlessTile(p, cellPx, dpr){
2827     var tile = PIC_TILINGS[p.tile] ? p.tile : 't488';
2828     if(picIsRadial(tile)) return null; /* medalyon davriy emas */
2829     var us = PIC_USCALE[tile] || 1;
2830     var unit = Math.max(12, cellPx*us);
2831     /* davrni bilish uchun kichik namuna quramiz */
2832     var probe = PIC_TILINGS[tile].build(unit, {x0:0,y0:0,x1:1,y1:1});
2833     if(!probe.period) return null;
2834     var pw = probe.period[0], ph = probe.period[1];
2835     if(!(pw > 2 && ph > 2)) return null;
2836     var spec = {
2837         tile: tile, unit: unit, R: Math.max(pw,ph),
2838         bb: {x0:-pw, y0:-ph, x1:2*pw, y1:2*ph},
2839         dvar: p.dvar||0, n:10, ir:0.3
2840     };
2841     var M = picGet(spec);
2842     var cv = document.createElement('canvas');
2843     cv.width = Math.max(2, Math.round(pw*dpr));
2844     cv.height = Math.max(2, Math.round(ph*dpr));
2845     var c = cv.getContext('2d');
2846     if(!c) return null;
2847     var pal = PAL[p.pal] || PAL.gold;
2848     c.setTransform(dpr, 0, 0, dpr, 0, 0);
2849     c.fillStyle = pal.c[4];
2850     c.fillRect(0, 0, pw, ph);
2851     picRender(c, M, picStyleFrom(pal, p, unit));
2852     return {canvas: cv, w: pw, h: ph};
2853 }
2854
2855 /* =====
2856 REGISTR + DISPATCHER + TESSELLATSIYA
2857 ===== */
2858 const CATS = {A:"O'zbek geometrik", B:"Islimiy organik", C:"Kombinatsiyalar", D:"Dunyo naqshlari", E:"Matematik fr
aktallar"};
2859 const PATTERNS = [
2860     {id:'giriH', nm:'GiriH (4.8.8)', cat:'A', fn:drawPICPattern, pic:'t488', th:"PIC (polygons-in-contact) yad
rosi, 4.8.8 plitkalash: har qirra o'rtasidan 6 burchagi ostida ikki nur; nurlar plitka ichida o'zaro uchrashib glo
bal lenta to'rini beradi. Lentalar plitka chegarasida UZILMAYDI – qo'shni plitkani nurlari aynan bir to'g'ri chi
ziqda davom etadi."},
2861     {id:'hankin8', nm:'GiriH – kvadrat to'r', cat:'A', fn:drawPICPattern, pic:'sq', th:"PIC kvadrat to'rda (4.4.4
.4): 8=45° klassik sakkizburchak-xoch naqshini, boshqa variantlar rom to'rlarini beradi."},
2862     {id:'hankin6', nm:'GiriH – olti burchak', cat:'A', fn:drawPICPattern, pic:'t666', th:"PIC olti burchakli to'rd
a (6.6.6): 6 olti nurli yulduz o'tkirligini belgilaydi; barcha variantlar geometrik jihatdan yopiladi."},
2863     {id:'panelgiriH', nm:'GiriH – 12 nurli', cat:'A', fn:drawPICPattern, pic:'t3122', th:"PIC 3.12.12 plitkalashda: o
'n ikki burchak + uchburchak → Temuriylar davri 12 nurli yulduzi (Bibixonim, Go'ri Amir panellari)."},
2864     {id:'pic3636', nm:'GiriH – 6+3 to'r', cat:'A', fn:drawPICPattern, pic:'t3636', th:"Hankin PIC 3.6.3.6 to'rida:
olti burchak va uchburchak qirralari o'rtasidan 6 burchagi ostidagi nurlar 6 nurli yulduzlar va ko'rshapalak yuza
larini beradi."},
2865     {id:'shamsal0', nm:'Shamsa – 10 nurli', cat:'A', fn:drawPICPattern, pic:'shamsal0', th:"Haqiqiy giriH tile to'pl
ami: markazda o'n burchak, atrofida beshburchaklar halqasi. Uchdagi burchaklar 144°+108°+108°=360° – halqa bo'shli
qsiz yopiladi (Registon, Shohi Zinda uslubi)."},
2866     {id:'xatam', nm:'Xatam', cat:'A', fn:drawXatam, th:"8 qirrali yulduz: ikki kvadrat 45° ga buralib u
stma-ust qo'yiladi; kvadratlar to'ri bilan davom etadi."},
2867     {id:'rozetta', nm:'Rozetta', cat:'A', fn:drawRozetta, th:"Markaziy gul: n barg, har barg 2 kubik Bezier y
oy bilan, uchi o'tkir."},
2868     {id:'yulduz', nm:'Yulduz', cat:'A', fn:drawYulduz, th:"{n/k} yulduz poligoni: nuqta_i dan nuqta_(i+k)
ga chiziqli, k=[n/2]-1."},
2869     {id:'muqarnas', nm:'Muqarnas', cat:'A', fn:drawMuqarnas, th:"Rekursiv pog'onali ravoq: har daraja 0.72 massh
tabda, orasida stalaktit yoylar."},
2870     {id:'katak8', nm:'Katak-8', cat:'A', fn:drawKatak8, th:"4.8.8 Arximed tessellatsiyasi: sakkiz burchak +
kvadrat to'ri."},
2871     {id:'hexa', nm:'Olti burchak', cat:'A', fn:drawHexa, th:"Asal uyasi to'ri (p6m simmetriya), har katakda
ichki bezak."},
2872     {id:'zanjira', nm:'Zanjira', cat:'A', fn:drawZanjira, th:"Halqalar zanjiri: Bezier ovallar radial nurlar
bo'ylab, o'zaro to'qilgan."},
2873     {id:'islmiy', nm:'Islimiy', cat:'B', fn:drawIslimiy, th:"Logarifmik spiral: r(θ)=a·e^(bθ), b=0.08+spiral
·0.38; har novda uchida barg."},
2874     {id:'rumi', nm:'Rumi', cat:'B', fn:drawRumi, th:"S-shakl qo'sh spiral: ikki qarama-qarshi log-sp
iral tutashgan, uchlarida buralgan barglar."},

```



```

2875   {id:'hatayi', nm:'Hatayi', cat:'B', fn:drawHatayi, th:"Stilizatsiyalangan lotus: 5–7 gulbarg, har biri
      3 Bezier segment; markazda urug'don doiralari."},
2876   {id:'palak', nm:'Palak', cat:'B', fn:drawPalak, th:"Nilufar: konsentrik gulbarg halqalari, tashqari
da katta, ichkarida kichik barglar."},
2877   {id:'shamsa', nm:'Shamsa', cat:'B', fn:drawShamsa, th:"Quyosh medalyoni: markaziy yulduz + alanga-nurl
ar + tashqi gul halqasi (3 qatlam)."},
2878   {id:'arabesk', nm:'Arabesque', cat:'B', fn:drawArabesque, th:"0'zaro to'qilgan spiral novdalar to'ri, har tut
ashuvda barg."},
2879   {id:'scroll', nm:'Bargli novda', cat:'B', fn:drawScroll, th:"Asosiy S-egri (kubik Bezier) + navbatlashgan yo
n barglar;  $B(t) = \sum C(3,i)(1-t)^{3-i} \cdot t^i \cdot P_i$ ."},
2880   {id:'medallion', nm:'Medalyon', cat:'B', fn:drawMedallion, th:"Yulduz + gul + halqa kombinatsiyasi – markaziy
murakkab medalyon."},
2881   {id:'oyma', nm:'O'yma panel', cat:'B', fn:drawOyma, th:"Yog'och o'ymakorligi kartushi (Xiva/Farg'ona u
slubi). Radial medalyon emas – TIK PANEL: yon tomonlari tik, uchlari siniq (ogee) yoy bilan yopilgan ramka; ichida
o'qqa nisbatan ko'zgu-simmetrik islamiy novda. Uslubni tanitadigan uch belgi: kartush ramkasi, nuqtali fon (\chi
zma"/pargori – o'yilgan pastki qatlam) va relyef – har kontur ostiga yorug' va soya nusxasi qo'yiladi, shunda naq
sh yassi chiziq emas, o'yilgan sirt bo'lib ko'rinadi. Ustun qirralari va eshik tabaqalari aynan shunday panellarda
n yig'iladi."},
2882   {id:'gi', nm:'Girih+Islimiy', cat:'C', fn:drawGirihIslimiy, th:"Tashqi qatlam – girih strapwork, ichki
qatlam (0.5R) – islamiy spirallar."},
2883   {id:'sg', nm:'Shamsa+Girih', cat:'C', fn:drawShamsaGirih, th:"Tashqi shamsa medalyoni ichida girih y
ulduz-tugun."},
2884   {id:'mi', nm:'Muqarnas+Islimiy', cat:'C', fn:drawMuqarnasIslimiy, th:"Muqarnas pog'onalar ichida islamiy no
vdalar."},
2885   {id:'xr', nm:'Xatam+Rumi', cat:'C', fn:drawXatamRumi, th:"Xatam yulduzi ichida rumi qo'sh spiral
lar."},
2886   {id:'celtic', nm:'Celtic', cat:'D', fn:drawCeltic, th:"Knotwork: aylana bo'ylab kesishgan Bezier lenta
lar, interlacing (ustma-ust to'qilish)."},
2887   {id:'mandala', nm:'Mandala', cat:'D', fn:drawMandala, th:"6–12 konsentrik halqa, har halqada boshqa eleme
nt: nuqta, barg, yoy, gulbarg."},
2888   {id:'penrose', nm:'Penrose', cat:'D', fn:drawPenrose, th:"Kite/dart aperiodik subdivide, oltin nisbat  $\phi = (1+\sqrt{5})/2 \approx 1.618$ ; rekursiya darajasi = murakkablik."},
2889   {id:'zellige', nm:'Zellige', cat:'D', fn:drawZellige, th:"Marokash kafeli: 4 Bezier tomonli egri romblar,
5 rangli davriy almashinuv."},
2890   {id:'batik', nm:'Batik', cat:'D', fn:drawBatik, th:"To'lqinli halqalar:  $r(\theta) = r_0 + A \cdot \sin(n\theta)$  modulyats
iyasi, orasida gul motivlari."},
2891   {id:'paisley', nm:'Paisley', cat:'D', fn:drawPaisley, th:"Buta/tomchi: asimmetrik teardrop (4 Bezier), ic
hida log-spiral, uchida 5-barg gul."},
2892   {id:'greek', nm:'Greek Key', cat:'D', fn:drawGreek, th:"Meander: to'g'ri burchakli labirint halqalar, k
onsentrik kvadrat ramkalarda."},
2893   {id:'xitoy', nm:'Xitoy panjara', cat:'D', fn:drawXitoy, th:"Fretwork: to'g'ri burchakli kataklar + T-shakl
ichki bo'linmalar."},
2894   {id:'koch', nm:'Koch', cat:'E', fn:drawKoch, th:"Qor parchasi: har segment 4 ga bo'linadi; frakt
al o'lcham  $D = \log 4 / \log 3 \approx 1.262$ ."},
2895   {id:'sierpinski', nm:'Sierpinski', cat:'E', fn:drawSierpinski, th:"Uchburchak fraktal:  $D = \log 3 / \log 2 \approx 1.585$ ."},
2896   {id:'dragon', nm:'Dragon', cat:'E', fn:drawDragon, th:"L-sistema: F→F+G, G→F–G, burchak 90°."},
2897   {id:'hilbert', nm:'Hilbert', cat:'E', fn:drawHilbert, th:"Fazoni to'ldiruvchi egri:  $4^n$  nuqta, Gray-kod a
sosida  $d \rightarrow (x,y)$ ."},
2898   {id:'spirograph', nm:'Spirograph', cat:'E', fn:drawSpirograph, th:"Epitrochoid:  $x = (R-r)\cos t + d \cdot \cos((R-r)t/r)$ ,  $y = (R-r)\sin t - d \cdot \sin((R-r)t/r)$ ."},
2899   {id:'lissajous', nm:'Lissajous', cat:'E', fn:drawLissajous, th:" $x = A \cdot \sin(at + \phi)$ ,  $y = B \cdot \sin(bt)$  – chastotalar nisbat
i figurani belgilaydi."},
2900   {id:'voronoi', nm:'Voronoi', cat:'E', fn:drawVoronoi, th:"Seed nuqtalardan hujayra mozaika: har hujayra y
arim tekisliklar kesishmasi."},
2901   ];
2902   function getPattern(t){ return PATTERNS.find(x=>x.id===t) || PATTERNS[0]; }
2903   function getPatternDrawFn(t){ return getPattern(t).fn; }
2904
2905   /* To'r/fraktal turlarida qatlamlash chiziqlarni buzadi – ular yakka qatlamda chiziladi */
2906   /* PIC yadrosi bilan chiziladigan naqshlar (girih oilasi) */
2907   const PIC_PATTERN = {girih:1, hankin8:1, hankin6:1, panelgirih:1, pic3636:1, shamsa10:1};
2908   function picTileOf(type){ const pt = getPattern(type); return (pt && pt.pic) ? pt.pic : 't488'; }
2909   const SINGLE_LAYER = {girih:1, hankin8:1, hankin6:1, panelgirih:1, pic3636:1, shamsa10:1, katak8:1, hexa:1, zellige:
1, xitoy:1, penrose:1, voronoi:1, hilbert:1, dragon:1, greek:1, koch:1, sierpinski:1, batik:1, hankin8:1, hankin6:1, panelgiri
h:1};
2910
2911   /* Umumiy kirish nuqtasi: qatamlar simmetriyaga mos YARIM QADAM bilan buriladi –
2912   chiziqlar bir-biriga uyg'un, interlock bo'ladi */
2913   function drawPattern(ctx, cx, cy, R, p){
2914     const pal = PAL[p.pal] || PAL.gold;
2915     const fn = getPatternDrawFn(p.type);
2916     /* PIC (girih oilasi) naqshlari o'z plitka turini p.tile orqali oladi –
2917     bu yerda o'rnatilmasa, drawPICPattern har doim standart 't488' ga
2918     tushib qoladi va Girih/Shamsa variantlari bir xil chiqadi. */
2919     if(PIC_PATTERN[p.type]) p.tile = picTileOf(p.type);
2920     ctx.save();
2921     ctx.translate(cx, cy);
2922     ctx.rotate((p.rot||0)*Math.PI/180);
2923     if(p.skew) ctx.transform(1, 0, Math.tan(p.skew*0.35), 1, 0, 0);
2924     /* Qatamlar = ICHMA-ICH medalyonlar (0.42 masshtab) – ustma-ust loyqa emas,
2925     har qatlam oldingisining markaziy bo'shlig'ida joylashadi */
2926     const layers = SINGLE_LAYER[p.type] ? 1 : Math.max(1, p.lay);
2927     const halfStep = Math.PI/Math.max(3, p.sym);
2928     for(let l=0; l<layers; l++){
2929       const s = Math.pow(0.42, l);
2930       if(R*s < 14) break;
2931       const t = layers>1 ? l/(layers-1) : 0;
2932       const c0 = lerpColor(pal.c[0], pal.c[2], t*0.6);
2933       const c1 = lerpColor(pal.c[1], pal.c[3], t*0.6);
2934       ctx.save();
2935       ctx.scale(s, s);

```

```

2936     ctx.rotate((l%2)*halfStep); /* yarim qadam – qatlamlar interlock */
2937     fn(ctx, R, p, Math.max(0.3, p.sw*(1 - 0.15*l)/Math.sqrt(s)), c0, c1, pal.c[2], pal.c[4]);
2938     ctx.restore();
2939 }
2940 ctx.restore();
2941 }
2942
2943 /* Tashqi halqa bezagi – yakka rejimda naqshni ramkalaydi */
2944 function drawBorderRing(ctx,R,p){
2945     const pal = PAL[p.pal] || PAL.gold;
2946     ctx.beginPath(); circle(ctx,0,0,R*1.06); st(ctx, pal.c[0], p.sw*0.8);
2947     ctx.beginPath(); circle(ctx,0,0,R*1.105); st(ctx, pal.c[2], p.sw*0.45);
2948     const n = Math.max(12, p.sym*2);
2949     ctx.beginPath();
2950     for(let i=0;i<n;i++){
2951         const a = TAU*i/n;
2952         const A = pol(a, R*1.06), B = pol(a, R*1.105);
2953         ctx.moveTo(A[0],A[1]); ctx.lineTo(B[0],B[1]);
2954     }
2955     st(ctx, pal.c[1], p.sw*0.5);
2956 }
2957
2958 /* =====
2959 TESSELLATSIYA – PATH-BASED SEAMLESS TILING
2960 T(n,m) = n·v1 + m·v2 – Wallpaper guruhlari:
2961     p4: v1=(c,0), v2=(0,c)
2962     p4m: p4 + ko'zgu (shaxmat tartibida aks)
2963     p6: v1=(c,0), v2=(c/2, c·√3/2) – geksagonal panjara
2964     p6m: p6 + ko'zgu
2965 drawImage EMAS – har katak to'g'ridan drawPattern bilan chiziladi,
2966 R_tile > c/2 (bleed) → naqsh qo'shni katakka kiradi → chiziqlar UZILMAYDI
2967 ===== */
2968 const GRIDS = {
2969     p4: {v1:c=>[c,0], v2:c=>[0,c], mirror:false, hex:false},
2970     p4m: {v1:c=>[c,0], v2:c=>[0,c], mirror:true, hex:false},
2971     p6: {v1:c=>[c,0], v2:c=>[c/2, c*Math.sqrt(3)/2], mirror:false, hex:true},
2972     p6m: {v1:c=>[c,0], v2:c=>[c/2, c*Math.sqrt(3)/2], mirror:true, hex:true}
2973 };
2974 /* og'ir fraktallar tessellatsiyada yengillashtiriladi */
2975 const HEAVY_TESS = {dragon:1, hilbert:1, voronoi:1, penrose:1, hankin8:1, hankin6:1};
2976 /* siyrak chiziqli naqshlar – o'z radiusining kichik qismini "siyoh" bilan
2977 to'ldiradi, shuning uchun tessellatsiyada zichroq joylashtiriladi
2978 (pastdagi drawTessellation'da SPARSE_TESS izohiga qarang) */
2979 const SPARSE_TESS = {koch:1, sierpinski:1, dragon:1, hilbert:1, voronoi:1, penrose:1, lissajous:1, spirograph:1};
2980 /* Grid-naqshlar tiling rejimida UZLUKSIZ YAXLIT MAYDON sifatida chiziladi –
2981 medalyon takrori emas, haqiqiy chinnisozlikdagidek chok-suz naqsh */
2982 const FIELD_TESS = {giri:1, hankin8:1, hankin6:1, panelgiri:1, pic3636:1, katak8:1, hexa:1, zellige:1, xitoy:1};
2983 function drawTessellation(ctx,W,H,p){
2984     /* PIC naqshlari: davriy panjara → MATEMATIK ANIQ chok-suz to'ldirish */
2985     if(PIC_PATTERN[p.type] && picDrawTessellation(ctx,W,H,p)) return;
2986     if(FIELD_TESS[p.type]){
2987         const fp = {...p, _field:true, _cellpx:Math.max(60, TESS.cell||150), lay:1};
2988         drawPattern(ctx, W/2, H/2, Math.hypot(W,H)/2*1.02, fp);
2989         if(TESS.showCell){
2990             ctx.save();
2991             ctx.strokeStyle='rgba(74,124,89,.4)'; ctx.lineWidth=1; ctx.setLineDash([4,4]);
2992             const c2 = fp._cellpx;
2993             for(let x=W/2 % c2; x<W; x+=c2){ ctx.beginPath(); ctx.moveTo(x,0); ctx.lineTo(x,H); ctx.stroke(); }
2994             for(let y=H/2 % c2; y<H; y+=c2){ ctx.beginPath(); ctx.moveTo(0,y); ctx.lineTo(W,y); ctx.stroke(); }
2995             ctx.setLineDash([]); ctx.restore();
2996         }
2997         return;
2998     }
2999     const g = GRIDS[TESS.group] || GRIDS.p4;
3000     const cFull = Math.max(60, TESS.cell || Math.round(Math.min(W,H)/3));
3001     /* SIYRAK naqshlar (fraktal chiziqlar – Koch, Sierpinski, Drakon, Hilbert,
3002     Voronoi, Penrose, Lissaju, Spirograf): ularning "siyoh"i o'z radiusining
3003     kichik qismini egallaydi, shuning uchun oddiy bleed ORTIRISH yordam
3004     bermaydi – bo'sh joy naqsh o'zida, katak orasidagi masofada emas.
3005     Bunday tur'lar uchun katak QADAMINI qisqartirib, nusxalarni zich
3006     joylashtiramiz – shundagina qo'shnilar haqiqatda ustma-ust tushadi. */
3007     const c = SPARSE_TESS[p.type] ? cFull*0.4 : cFull;
3008     const v1 = g.v1(c), v2 = g.v2(c);
3009     /* bleed → seamless: kvadrat to'rda (p4/p4m) eng uzoq "bo'sh nuqta" diagonal
3010     qo'shnigacha c·√2/2 ≈ 0.707c masofada, olti burchakli to'rda (p6/p6m)
3011     esa uch qo'shni orasidagi markaz c/√3 ≈ 0.577c da – shu chegaradan
3012     past bleed diagonal/markaziy uchburchak bo'shliqlarida UZILISH
3013     (medalyonlar orasida oq bo'shliq) qoldiradi. */
3014     const Rt = cFull*(TESS.bleed === false ? 0.5 : (g.hex ? 0.64 : 0.76));
3015     /* tessellatsiyada har katak to'g'ridan chiziladi – samaradorlik uchun
3016     qatlamlar 2 taga, og'ir fraktallar cmp≤3 ga cheklanadi */
3017     let tp = {...p, lay:Math.min(2, p.lay)};
3018     if(HEAVY_TESS[p.type]) tp.cmp = Math.min(3, tp.cmp);
3019     const mX = W/2 + Rt*1.7, mY = H/2 + Rt*1.7;
3020     const nMax = Math.min(80, Math.ceil(Math.hypot(W,H)/c) + 2);
3021     NORING = TESS.noring === false ? 0 : Rt*0.84; /* medalyonlar ramkasiz – tabiiy tutashadi */
3022     for(let n=-nMax; n<=nMax; n++){
3023         for(let m=-nMax; m<=nMax; m++){
3024             const tx = n*v1[0] + m*v2[0], ty = n*v1[1] + m*v2[1];
3025             if(Math.abs(tx) > mX || Math.abs(ty) > mY) continue;
3026             const mirror = g.mirror && (((n+m)%2 + 2)%2 !== 0);

```

```

3027     ctx.save();
3028     ctx.translate(W/2 + tx, H/2 + ty);
3029     if(mirror) ctx.scale(-1,1); /* p4m/p6m — ko'zgu aksi */
3030     drawPattern(ctx, 0, 0, Rt, tp);
3031     ctx.restore();
3032   }
3033 }
3034 NORING = 0;
3035 if(TESS.showCell){
3036   ctx.save();
3037   ctx.strokeStyle = 'rgba(74,124,89,.45)';
3038   ctx.lineWidth = 1;
3039   ctx.setLineDash([4,4]);
3040   for(let n=-nMax; n<=nMax; n++){
3041     for(let m=-nMax; m<=nMax; m++){
3042       const tx = n*v1[0] + m*v2[0], ty = n*v1[1] + m*v2[1];
3043       if(Math.abs(tx) > mX || Math.abs(ty) > mY) continue;
3044       const X = W/2 + tx, Y = H/2 + ty;
3045       ctx.beginPath();
3046       if(g.hex){
3047         const hr = c/Math.sqrt(3);
3048         for(let i=0;i<6;i++){
3049           const a = Math.PI/6 + TAU*i/6;
3050           if(i===0) ctx.moveTo(X+Math.cos(a)*hr, Y+Math.sin(a)*hr);
3051           else ctx.lineTo(X+Math.cos(a)*hr, Y+Math.sin(a)*hr);
3052         }
3053         ctx.closePath();
3054       } else {
3055         ctx.rect(X-c/2, Y-c/2, c, c);
3056       }
3057       ctx.stroke();
3058     }
3059   }
3060   ctx.setLineDash([]);
3061   ctx.restore();
3062 }
3063 }
3064 /* =====
3065    UI YORDAMCHILARI
3066    ===== */
3067 function $(id){ return document.getElementById(id); }
3068
3069 /* =====
3070    PIC — CHOK-SUZ TO'LDIRISH VA BOSHQARUV
3071    ===== */
3072 var PIC_TILE_CACHE = {};
3073 /* Davriy PIC naqshini butun kanvasga chok-suz yoyish.
3074    true qaytarsa — chizish bajarildi; false bo'lsa umumiy yo'l ishlaydi. */
3075 function picDrawTessellation(ctx, W, H, p){
3076   var tile = picTileOf(p.type);
3077   if(picIsRadial(tile)) return false; /* medalyon davriy emas */
3078   var pp = Object.assign({}, p, {tile:tile});
3079   /* SVG/DXF eksportida createPattern yo'q → maydonni to'g'ridan chizamiz
3080      (natija aynan bir xil geometriya, faqat takrorlanish vektor sifatida) */
3081   if(ctx.svg){
3082     var Rf = Math.hypot(W,H)/2*1.02;
3083     var spec = picSpecFrom(pp, Rf, {cellPx:Math.max(40,TESS.cell||150), bb:{x0:-Rf,y0:-Rf,x1:Rf,y1:Rf}});
3084     var M = picGet(spec);
3085     ctx.save(); ctx.translate(W/2,H/2);
3086     ctx.rotate((p.rot||0)*Math.PI/180);
3087     picRender(ctx, M, picStyleFrom(PAL[p.pal]||PAL.gold, pp, spec.unit));
3088     ctx.restore();
3089     return true;
3090   }
3091   var dpr = Math.min(2, (typeof window!=='undefined' && window.devicePixelRatio) || 1);
3092   var cell = Math.max(40, TESS.cell || 150);
3093   var key = [tile, cell, pp.dvar||0, pp.pal, pp.fill, pp.sw, pp.weave!=='false', Math.round(dpr*100)].join('|');
3094   var T = PIC_TILE_CACHE[key];
3095   if(T === undefined){
3096     T = picSeamlessTile(pp, cell, dpr) || null;
3097     var ks = Object.keys(PIC_TILE_CACHE);
3098     if(ks.length > 10) delete PIC_TILE_CACHE[ks[0]];
3099     PIC_TILE_CACHE[key] = T;
3100   }
3101   if(!T) return false;
3102   var pat = ctx.createPattern(T.canvas, 'repeat');
3103   if(!pat) return false;
3104   /* Naqsh matritsasi: offscreen kanvas dpr da chizilgani uchun teskari
3105      mashtab beriladi — shunda ekranda aynan bir davr = T.w × T.h bo'ladi. */
3106   if(pat.setTransform && typeof DOMMatrix === 'function'){
3107     try { pat.setTransform(new DOMMatrix([1/dpr,0,0,1/dpr,0,0])); } catch(e){}
3108   }
3109   ctx.save();
3110   ctx.translate(W/2, H/2);
3111   ctx.rotate((p.rot||0)*Math.PI/180);
3112   var D = Math.hypot(W,H)*1.1;
3113   ctx.fillStyle = pat;
3114   ctx.fillRect(-D, -D, D*2, D*2);
3115   ctx.restore();
3116   if(TESS.showCell){
3117     ctx.save();

```

```

3118     ctx.strokeStyle='rgba(74,124,89,.45)'; ctx.lineWidth=1; ctx.setLineDash([4,4]);
3119     for(var x=(W/2)%T.w; x<W; x+=T.w){ ctx.beginPath(); ctx.moveTo(x,0); ctx.lineTo(x,H); ctx.stroke(); }
3120     for(var y=(H/2)%T.h; y<H; y+=T.h){ ctx.beginPath(); ctx.moveTo(0,y); ctx.lineTo(W,y); ctx.stroke(); }
3121     ctx.setLineDash([]); ctx.restore();
3122 }
3123 return true;
3124 }
3125 /* Girih boshqaruvi: kontakt burchagi variantlari + to'qish */
3126 function buildGirihUI(){
3127     var box = $('acc-girih');
3128     var isPic = !!PIC_PATTERN[P.type];
3129     if(box) box.style.display = isPic ? '' : 'none';
3130     if(!isPic) return;
3131     var tile = picTileOf(P.type);
3132     var ds = picDeltas(tile, 10);
3133     var seg = $('seg-dvar');
3134     if(seg){
3135         seg.innerHTML = '';
3136         var cur = ((P.dvar|0) % ds.length + ds.length) % ds.length;
3137         ds.forEach(function(d, i){
3138             var b = document.createElement('button');
3139             b.textContent = d + '\u00b0';
3140             if(i === cur) b.className = 'sel';
3141             b.addEventListener('click', function(){
3142                 P.dvar = i;
3143                 buildGirihUI();
3144                 redraw(true);
3145             });
3146             seg.appendChild(b);
3147         });
3148         var lab = $('v-dvar');
3149         if(lab) lab.textContent = '\u03b4 = ' + ds[cur] + '\u00b0';
3150     }
3151     var ww = $('chk-weave');
3152     if(ww) ww.checked = P.weave !== false;
3153     var nt = $('girih-note');
3154     if(nt){
3155         nt.textContent = (PIC_TILINGS[tile] ? PIC_TILINGS[tile].nm : tile) +
3156             '\u2014 kontakt burchagi faqat shu diskret qiymatlarda naqshni to'liq yopadi, ' +
3157             'shuning uchun erkin slayder emas, variant tanlanadi.';
3158     }
3159 }
3160 function wireGirihUI(){
3161     var ww = $('chk-weave');
3162     if(ww) ww.addEventListener('change', function(){ P.weave = ww.checked; PIC_TILE_CACHE = {}; redraw(); });
3163 }
3164
3165 function ntf(msg, type){
3166     const wrap = $('toasts');
3167     if(!wrap) return;
3168     const t = document.createElement('div');
3169     t.className = 'toast ' + (type==='err' ? 'err' : 'ok');
3170     t.textContent = msg;
3171     wrap.appendChild(t);
3172     setTimeout(()=>{ t.style.opacity='0'; t.style.transition='opacity .3s'; }, 2600);
3173     setTimeout(()=>{ if(t.parentNode) t.parentNode.removeChild(t); }, 3000);
3174 }
3175
3176 /* ----- Studio canvas ----- */
3177 let cvEl=null, cvCtx=null;
3178 function resizeCanvas(){
3179     if(!cvEl) return;
3180     const wrap = $('cv-wrap');
3181     if(!wrap) return;
3182     const dpr = Math.min(2, window.devicePixelRatio||1);
3183     const w = wrap.clientWidth||600, h = wrap.clientHeight||500;
3184     cvEl.width = Math.max(50, Math.round(w*dpr));
3185     cvEl.height = Math.max(50, Math.round(h*dpr));
3186     cvCtx = cvEl.getContext('2d');
3187     if(cvCtx) cvCtx.setTransform(dpr,0,0,dpr,0,0);
3188     redraw();
3189 }
3190
3191 const REVEAL = { raf:0 };
3192 /* Naqsh O'ZI CHIZILADI: spiral-sektor ochilish + oltin mo'yqalam uchi */
3193 function startReveal(){
3194     if(!cvCtx || !cvEl) return;
3195     if(window.matchMedia('& matchMedia('(prefers-reduced-motion: reduce)').matches) return;
3196     if(typeof cancelAnimationFrame === 'function') cancelAnimationFrame(REVEAL.raf);
3197     const snap = document.createElement('canvas');
3198     snap.width = cvEl.width; snap.height = cvEl.height;
3199     const sctx = snap.getContext('2d');
3200     if(!sctx) return;
3201     sctx.drawImage(cvEl, 0, 0);
3202     const pal = PAL[P.pal] || PAL.gold;
3203     const nowFn = ()=> (window.performance ? performance.now() : Date.now());
3204     const t0 = nowFn(), dur = 800;
3205     const cx = cvEl.width/2, cy = cvEl.height/2;
3206     const Rm = Math.hypot(cx, cy)*1.05;
3207     const step = ()=>{
3208         const t = Math.min(1, (nowFn()-t0)/dur);

```

```

3209     const e = 1 - Math.pow(1-t, 3);
3210     cvCtx.save();
3211     cvCtx.setTransform(1,0,0,1,0,0);
3212     cvCtx.clearRect(0,0,cvEl.width,cvEl.height);
3213     cvCtx.fillStyle = pal.bg;
3214     cvCtx.fillRect(0,0,cvEl.width,cvEl.height);
3215     const a1 = -Math.PI/2 + e*TAU;
3216     const rr = Rm*Math.min(1, 0.3 + e*0.85);
3217     cvCtx.beginPath();
3218     cvCtx.moveTo(cx,cy);
3219     cvCtx.arc(cx,cy,rr,-Math.PI/2,a1);
3220     cvCtx.closePath();
3221     cvCtx.clip();
3222     cvCtx.drawImage(snap,0,0);
3223     cvCtx.restore();
3224     if(t < 1){
3225         /* mo'yqalam uchi — porloq oltin nuqta chizish chetida yuradi */
3226         const br = rr*0.66;
3227         const bx = cx + Math.cos(a1)*br, by = cy + Math.sin(a1)*br;
3228         cvCtx.save();
3229         cvCtx.setTransform(1,0,0,1,0,0);
3230         cvCtx.shadowColor = 'rgba(224,189,106,.95)';
3231         cvCtx.shadowBlur = 20;
3232         cvCtx.fillStyle = '#e0bd6a';
3233         cvCtx.beginPath(); cvCtx.arc(bx,by,5.5,0,TAU); cvCtx.fill();
3234         cvCtx.restore();
3235         REVEAL.raf = requestAnimationFrame(step);
3236     } else {
3237         cvCtx.save();
3238         cvCtx.setTransform(1,0,0,1,0,0);
3239         cvCtx.clearRect(0,0,cvEl.width,cvEl.height);
3240         cvCtx.drawImage(snap,0,0);
3241         cvCtx.restore();
3242     }
3243 };
3244 REVEAL.raf = requestAnimationFrame(step);
3245 }
3246 function redraw(animate){
3247     if(!cvCtx || !cvEl) return;
3248     const dpr = Math.min(2, window.devicePixelRatio||1);
3249     const W = cvEl.width/dpr, H = cvEl.height/dpr;
3250     const pal = PAL[P.pal] || PAL.gold;
3251     P._bg = pal.bg;
3252     cvCtx.save();
3253     cvCtx.setTransform(dpr,0,0,dpr,0,0);
3254     cvCtx.clearRect(0,0,W,H);
3255     cvCtx.fillStyle = pal.bg;
3256     cvCtx.fillRect(0,0,W,H);
3257     if(TESS.on){
3258         drawTessellation(cvCtx, W, H, P);
3259     } else {
3260         const grad = cvCtx.createRadialGradient(W/2,H/2,10, W/2,H/2, Math.max(W,H)*0.7);
3261         grad.addColorStop(0,'rgba(255,255,255,0.35)');
3262         grad.addColorStop(1,'rgba(0,0,0,0.03)');
3263         cvCtx.fillStyle = grad;
3264         cvCtx.fillRect(0,0,W,H);
3265         const Rb = Math.min(W,H)*0.40;
3266         cvCtx.save();
3267         cvCtx.translate(W/2, H/2);
3268         cvCtx.rotate((P.rot||0)*Math.PI/180);
3269         drawBorderRing(cvCtx, Rb*1.04, P);
3270         cvCtx.restore();
3271         drawPattern(cvCtx, W/2, H/2, Rb, P);
3272     }
3273     cvCtx.restore();
3274     updateInfoBar();
3275     if(cvEl.classList){
3276         cvEl.classList.remove('pulse');
3277         void cvEl.offsetWidth;
3278         cvEl.classList.add('pulse');
3279     }
3280     if(threeReady){
3281         /* 3D teksturani debounce bilan yangilash — slayder surilganda ortiqcha yuk bo'lmasin */
3282         if(redraw._t3) clearTimeout(redraw._t3);
3283         redraw._t3 = setTimeout(()=>{ if(threeReady) update3DTexture(); }, 180);
3284     }
3285     arNeedsRedraw = true;
3286     if(animate) startReveal();
3287 }
3288 function updateInfoBar(){
3289     const el = $('info-name');
3290     if(!el) return;
3291     const pt = getPattern(P.type), pal = PAL[P.pal] || PAL.gold;
3292     if(PIC_PATTERN[P.type]){
3293         const tl = picTileOf(P.type);
3294         const ds = picDeltas(tl, 10);
3295         const dv = ds[(P.dvar||0) % ds.length + ds.length] % ds.length;
3296         el.innerHTML = '<b>'+pt.nm+'</b> · PIC yadrosi · kontakt burchagi δ='+dv+'° · '+pal.nm+
3297             (P.weave===false ? '' : ' · to\'qilgan') +
3298             (TESS.on ? ' · chok-suz tiling · katak '+TESS.cell+'px' : '');
3299         return;

```

```

3300     }
3301     el.innerHTML = '<b>'+pt.nm+'</b> · '+P.sym+' simmetriya · '+pal.nm+' · qatlam: '+P.lay + (TESS.on ? ' · tessella
tisiya '+TESS.group+' · katak '+TESS.cell+'px' : '');
3302 }
3303
3304 /* ----- Slayderlar ----- */
3305 const SLIDERS = [
3306   {k:'sym', nm:'Simmetriya (n)', min:3, max:16, step:1},
3307   {k:'cmp', nm:'Murakkablik', min:1, max:8, step:1},
3308   {k:'sw', nm:'Chiziq qalinligi', min:0.5, max:6, step:0.1},
3309   {k:'ir', nm:'Ichki radius', min:0.1, max:0.8, step:0.01},
3310   {k:'spir', nm:'Spiral kuchi', min:0, max:1, step:0.01},
3311   {k:'dens', nm:'Zichlik', min:0, max:1, step:0.01},
3312   {k:'lay', nm:'Qatlamlar', min:1, max:5, step:1},
3313   {k:'skew', nm:'Buriilish (skew)', min:-1, max:1, step:0.01},
3314   {k:'rot', nm:'Aylanish (°)', min:0, max:360, step:1}
3315 ];
3316 function buildSliders(){
3317   const box = $('sliders');
3318   if(!box) return;
3319   box.innerHTML = '';
3320   SLIDERS.forEach(s=>{
3321     const row = document.createElement('div');
3322     row.className = 'sl-row';
3323     row.innerHTML = '<label>'+s.nm+' <b id="v-'+s.k+'">'+P[s.k]+'</b></label>'+
3324       '<input type="range" id="sl-'+s.k+' " min="'+s.min+' " max="'+s.max+' " step="'+s.step+' " value="'+P[s.k]+'">';
3325     box.appendChild(row);
3326     const inp = row.querySelector('input');
3327     if(inp) inp.addEventListener('input', ()=>{
3328       P[s.k] = parseFloat(inp.value);
3329       if(s.k==='sw' || s.k==='dens') PIC_TILE_CACHE = {};
3330       const v = $('v-'+s.k);
3331       if(v) v.textContent = (s.step<1 ? P[s.k].toFixed(2) : P[s.k]);
3332       redraw();
3333     });
3334   });
3335 }
3336 function syncSliders(){
3337   SLIDERS.forEach(s=>{
3338     const inp = $('sl-'+s.k), v = $('v-'+s.k);
3339     if(inp) inp.value = P[s.k];
3340     if(v) v.textContent = (s.step<1 ? Number(P[s.k]).toFixed(2) : P[s.k]);
3341   });
3342   const fillSel = $('sel-fill'); if(fillSel) fillSel.value = P.fill;
3343   const curPt = getPattern(P.type);
3344   if(curPt && curPt.cat !== CUR_CAT){ CUR_CAT = curPt.cat; }
3345   renderPatGrid();
3346   document.querySelectorAll('.pal-chip').forEach(b=>{
3347     b.classList.toggle('sel', b.dataset.pal===P.pal);
3348   });
3349   applyParamAvailability();
3350 }
3351
3352 /* ----- Naqsh ro'yxati: kategoriya tablari + o'ralgan chip to'ri
3353 (barcha 5 kategoriya – O'zbek, Islimiy, Kombo, DUNYO, Matematik – bir bosishda) ----- */
3354 const CAT_SHORT = {A:"O'zbek", B:"Islimiy", C:"Kombo", D:"Dunyo", E:"Matematik"};
3355 let CUR_CAT = 'A';
3356 function renderPatGrid(){
3357   const grid = $('pat-grid-box');
3358   if(!grid) return;
3359   grid.innerHTML = '';
3360   PATTERNS.filter(pt=>pt.cat===CUR_CAT).forEach(pt=>{
3361     const b = document.createElement('button');
3362     b.textContent = pt.nm;
3363     b.dataset.id = pt.id;
3364     if(pt.id===P.type) b.classList.add('sel');
3365     b.addEventListener('click', ()=>{
3366       P.type = pt.id;
3367       syncSliders();
3368       redraw(true);
3369     });
3370     grid.appendChild(b);
3371   });
3372   document.querySelectorAll('#cat-tabs button').forEach(b=>{
3373     b.classList.toggle('sel', b.dataset.cat===CUR_CAT);
3374   });
3375 }
3376 function buildPatternList(){
3377   const box = $('pat-list');
3378   if(!box) return;
3379   box.innerHTML = '';
3380   const cur = getPattern(P.type);
3381   CUR_CAT = cur ? cur.cat : 'A';
3382   const tabs = document.createElement('div');
3383   tabs.className = 'cat-tabs';
3384   tabs.id = 'cat-tabs';
3385   Object.keys(CATS).forEach(cat=>{
3386     const b = document.createElement('button');
3387     b.textContent = CAT_SHORT[cat] || cat;
3388     b.title = CATS[cat];
3389     b.dataset.cat = cat;

```

```

3390     if(cat===CUR_CAT) b.classList.add('sel');
3391     b.addEventListener('click', ()=>{
3392         CUR_CAT = cat;
3393         renderPatGrid();
3394     });
3395     tabs.appendChild(b);
3396 });
3397 box.appendChild(tabs);
3398 const grid = document.createElement('div');
3399 grid.className = 'pat-wrap';
3400 grid.id = 'pat-grid-box';
3401 box.appendChild(grid);
3402 renderPatGrid();
3403 }
3404
3405 /* ----- Qaysi slayder qaysi naqshda ishlaydi ----- */
3406 const PARAM_MAP = {
3407     xatam:['cmp'],
3408     rozetta:['sym','cmp','ir','dens'], yulduz:['sym','spir'],
3409     muqarnas:['sym','cmp','ir'], katak8:['cmp','dens'],
3410     hexa:['cmp','dens'], zanjira:['sym','dens'],
3411     girih:['dens'], hankin8:['dens'], hankin6:['dens'], panelgirih:['dens'],
3412     pic3636:['dens'], shamsal0:[],
3413     islmiy:['sym','cmp','spir','dens'], rumi:['sym','dens'],
3414     hatayi:['sym','cmp','dens'], palak:['sym','cmp'],
3415     shamsa:['sym'], arabesk:['sym','cmp','spir'],
3416     scroll:['sym','dens','spir'], medallion:['sym','cmp','ir','spir','dens'],
3417     gi: '*', sg: '*', mi: '*', xr: '*',
3418     celtic:['sym','cmp','dens'], mandala:['sym','cmp','dens','spir'],
3419     penrose:['cmp'], zellige:['cmp','dens','spir'],
3420     batik:['sym','cmp','dens','spir'], paisley:['sym','cmp','dens','spir'],
3421     greek:['sym','cmp','ir','dens'], xitoy:['cmp','ir','dens'],
3422     koch:['cmp','ir'], sierpinski:['cmp'], dragon:['cmp'], hilbert:['cmp'],
3423     spirograph:['sym','cmp','spir'], lissajous:['sym','cmp','dens'],
3424     voronoi:['sym','cmp','dens','spir']
3425 };
3426 function applyParamAvailability(){
3427     buildGirihUI();
3428     const spec = PARAM_MAP[P.type];
3429     const act = (spec==='*' || !spec) ? ['sym','cmp','ir','spir','dens'] : spec;
3430     ['sym','cmp','ir','spir','dens'].forEach(k=>{
3431         const inp = $('sl-'+k);
3432         const row = inp ? inp.parentNode : null;
3433         if(row && row.classList) row.classList.toggle('off', act.indexOf(k)<0);
3434     });
3435     const layInp = $('sl-lay');
3436     const layRow = layInp ? layInp.parentNode : null;
3437     if(layRow && layRow.classList) layRow.classList.toggle('off', !!SINGLE_LAYER[P.type]);
3438 }
3439
3440 /* ----- Palitralar ----- */
3441 function buildPalettes(){
3442     const row = $('pal-row');
3443     if(!row) return;
3444     row.innerHTML = '';
3445     Object.keys(PAL).forEach(key=>{
3446         const b = document.createElement('button');
3447         b.className = 'pal-chip' + (key===P.pal ? ' sel' : '');
3448         b.dataset.pal = key;
3449         b.title = PAL[key].nm;
3450         b.setAttribute('aria-label', PAL[key].nm);
3451         PAL[key].c.forEach(c=>{
3452             const s = document.createElement('span');
3453             s.style.background = c;
3454             b.appendChild(s);
3455         });
3456         b.addEventListener('click', ()=>{
3457             P.pal = key;
3458             PIC_TILE_CACHE = {};
3459             syncSliders();
3460             redraw();
3461         });
3462         row.appendChild(b);
3463     });
3464 }
3465
3466 /* ----- Tasodifiy ----- */
3467 function randomize(){
3468     const ids = PATTERNS.map(x=>x.id);
3469     const pals = Object.keys(PAL);
3470     P.type = ids[Math.floor(Math.random()*ids.length)];
3471     P.pal = pals[Math.floor(Math.random()*pals.length)];
3472     P.sym = 3 + Math.floor(Math.random()*14);
3473     P.cmp = 1 + Math.floor(Math.random()*8);
3474     P.sw = +(0.8 + Math.random()*3.5).toFixed(1);
3475     P.ir = +(0.15 + Math.random()*0.6).toFixed(2);
3476     P.spir = +(Math.random()).toFixed(2);
3477     P.dens = +(0.2 + Math.random()*0.7).toFixed(2);
3478     P.lay = 1 + Math.floor(Math.random()*4);
3479     P.skew = +(Math.random()-0.5)*0.8).toFixed(2);
3480     P.rot = Math.floor(Math.random()*360);

```

```

3481   P.dvar = Math.floor(Math.random()*4);
3482   syncSliders();
3483   redraw(true);
3484   ntf('Tasodifiy naqsh: ' + getPattern(P.type).nm, 'ok');
3485 }
3486
3487 /* ----- Saqlangan naqshlar galereyasi ----- */
3488 function lsGet(key, fallback){
3489   try{ const v = localStorage.getItem(key); return v ? JSON.parse(v) : fallback; }
3490   catch(e){ return fallback; }
3491 }
3492 function lsSet(key, val){
3493   try{ localStorage.setItem(key, JSON.stringify(val)); return true; }
3494   catch(e){ return false; }
3495 }
3496 /* =====
3497   SAQLANGAN NAQSHLAR – BULUT (Supabase) yoki lokal
3498   -----
3499   Supabase rejimida va foydalanuvchi kirgan bo'lsa naqshlar `public.patterns`
3500   jadvalida saqlanadi: shunda ular BOSHQA QURILMADA ham ochiladi. Aks holda
3501   (server sozlanmagan yoki mehmon) – localStorage.
3502
3503   Xavfsizlik serverda: RLS siyosati `user_id = auth.uid() and is_active()`
3504   shartini qo'yadi, ya'ni har kim faqat O'Z naqshlarini ko'radi va
3505   o'zgartiradi; bloklangan foydalanuvchi esa hech narsa yoza olmaydi
3506   (supabase/schema.sql). Mijozdagi tekshiruvlar faqat qulaylik uchun.
3507
3508   CLOUD.patterns – jadvalning xotiradagi nusxasi. Interfeys sinxron
3509   getSavedList() ni chaqiradi, shuning uchun ro'yxat kirishda bir marta
3510   yuklanadi va har o'zgarishdan keyin yangilanadi.
3511   ===== */
3512 const CLOUD = { patterns:null, loading:false, err:'' };
3513 function cloudActive(){ return (typeof sbEnabled === 'function') && sbEnabled() && !!AUTH.user; }
3514
3515 function cloudRowToItem(r){
3516   return { id:r.id, p:r.params || {}, thumb:r.thumb || '', t:Date.parse(r.created_at) || Date.now() };
3517 }
3518 async function cloudLoadPatterns(){
3519   if(!cloudActive()){ CLOUD.patterns = null; return; }
3520   CLOUD.loading = true; CLOUD.err = '';
3521   try{
3522     const { data, error } = await AUTH.client
3523       .from('patterns')
3524       .select('id, params, thumb, created_at')
3525       .order('created_at', { ascending:false })
3526       .limit(200);
3527     if(error) throw error;
3528     CLOUD.patterns = (data || []).map(cloudRowToItem);
3529   }catch(e){
3530     console.error('[cloud] load', e);
3531     CLOUD.err = (e && e.message) || 'yuklab bo'lmadi';
3532     CLOUD.patterns = CLOUD.patterns || [];
3533   }
3534   CLOUD.loading = false;
3535 }
3536 async function cloudSavePattern(item){
3537   const { data, error } = await AUTH.client
3538     .from('patterns')
3539     .insert({ user_id: AUTH.user.id, name: (item.p && item.p.type) || null,
3540       params: item.p, thumb: item.thumb })
3541     .select('id, params, thumb, created_at')
3542     .single();
3543   if(error) throw error;
3544   CLOUD.patterns = CLOUD.patterns || [];
3545   CLOUD.patterns.unshift(cloudRowToItem(data));
3546   return true;
3547 }
3548 async function cloudDeletePattern(id){
3549   const { error } = await AUTH.client.from('patterns').delete().eq('id', id);
3550   if(error) throw error;
3551   CLOUD.patterns = (CLOUD.patterns || []).filter(x=>x.id !== id);
3552   return true;
3553 }
3554 /* Mehmon sifatida yasalgan naqshlar kirishda YO'QOLMASIN – bir marta
3555   bulutga ko'chiriladi va lokal ro'yxat tozalanadi. */
3556 async function cloudMigrateGuest(){
3557   if(!cloudActive()) return 0;
3558   let guest = [];
3559   try{ guest = lsGet('naqsh_saved', []) || []; }catch(e){ }
3560   if(!guest.length) return 0;
3561   let n = 0;
3562   for(const it of guest.slice(0, 48)){
3563     try{ await cloudSavePattern(it); n++; }catch(e){ console.error('[cloud] migrate', e); break; }
3564   }
3565   if(n){ try{ localStorage.removeItem('naqsh_saved'); }catch(e){ } }
3566   return n;
3567 }
3568
3569 /* Saqlangan naqshlar: kirgan foydalanuvchining hisobida, aks holda mehmon ro'yxatida */
3570 function getSavedList(){
3571   if(cloudActive()) return CLOUD.patterns || [];

```



```

3572     const u = (typeof currentUser==='function') ? currentUser() : null;
3573     if(u){ const users = usersGet(); return (users[u.login] && users[u.login].patterns) || []; }
3574     return lsGet('naqsh_saved', []);
3575 }
3576 function setSavedList(arr){
3577     const u = (typeof currentUser==='function') ? currentUser() : null;
3578     if(u){
3579         const users = usersGet();
3580         if(users[u.login]){ users[u.login].patterns = arr; return usersSet(users); }
3581     }
3582     return lsSet('naqsh_saved', arr);
3583 }
3584 /* Yon paneldagi PROFIL KARTASI — kim kirgan va naqshlar qayerda saqlanmoqda.
3585     Nomi renderProfileCard: pastda profil OYNASI uchun renderProfile() bor va
3586     ikkinchi e'lon birinchisini bosib ketardi (funksiya deklaratsiyasi
3587     ko'tariladi) — shu sababli karta umuman chizilmasdi. */
3588 function renderProfileCard(){
3589     const box = $('profile-box');
3590     if(!box) return;
3591     const u = (typeof currentUser === 'function') ? currentUser() : null;
3592     if(!u){
3593         box.innerHTML = '<div class="prof"><div class="prof-av">?</div><div class="prof-i"> ' +
3594             '<div class="prof-n">Mehmon</div> ' +
3595             '<div class="prof-m">Naqshlar faqat shu brauzerda saqlanadi</div> ' +
3596             '<span class="prof-b local">lokal</span></div></div>';
3597         return;
3598     }
3599     const cloud = cloudActive();
3600     const initials = (u.name || u.login || '?').trim().charAt(0).toUpperCase();
3601     const prov = u.provider === 'google' ? 'Google' : 'Email';
3602     const n = getSavedList().length;
3603     const since = u.created ? new Date(u.created).toLocaleDateString('uz-UZ') : '';
3604     box.innerHTML =
3605         '<div class="prof"> ' +
3606         '<div class="prof-av">' + escHTML(initials) + '</div> ' +
3607         '<div class="prof-i"> ' +
3608         '<div class="prof-n">' + escHTML(u.name || u.login) + '</div> ' +
3609         '<div class="prof-m">' + escHTML(u.login) + ' · ' + prov +
3610         '(since ? ' + escHTML(since) : '') + '</div> ' +
3611         '<span class="prof-b ' + (cloud ? 'cloud' : 'local') + '"> ' +
3612         (cloud ? 'hisobda saqlanadi' : 'shu qurilmada') +
3613         '</span> <span class="prof-b">' + n + ' ta naqsh</span> ' +
3614         '</div> ' +
3615         '</div>';
3616 }
3617 function renderGallery(){
3618     const grid = $('gal-grid'), empty = $('gal-empty');
3619     if(!grid) return;
3620     const saved = getSavedList();
3621     grid.innerHTML = '';
3622     if(empty) empty.style.display = saved.length ? 'none' : 'block';
3623     saved.forEach((item, idx)=>{
3624         const d = document.createElement('div');
3625         d.className = 'gal-item';
3626         d.title = 'Yuklash: ' + (getPattern(item.p.type).nm);
3627         const img = document.createElement('img');
3628         img.src = item.thumb;
3629         img.alt = getPattern(item.p.type).nm;
3630         d.appendChild(img);
3631         const del = document.createElement('button');
3632         del.className = 'del';
3633         del.textContent = 'x';
3634         del.setAttribute('aria-label', 'O\'chirish');
3635         del.addEventListener('click', (e)=>{
3636             e.stopPropagation();
3637             if(cloudActive() && item.id !== undefined){
3638                 cloudDeletePattern(item.id)
3639                     .then(()=>{ renderGallery(); ntf('Naqsh o\'chirildi', 'ok'); })
3640                     .catch(err=>{
3641                         console.error('[cloud] delete', err);
3642                         ntf('O\'chirib bo'lmadi: ' + ((err && err.message) || 'noma'lum'), 'err');
3643                     });
3644             return;
3645         })
3646         const arr = getSavedList();
3647         arr.splice(idx,1);
3648         setSavedList(arr);
3649         renderGallery();
3650         ntf('Naqsh o\'chirildi', 'ok');
3651     });
3652     d.appendChild(del);
3653     d.addEventListener('click', ()=>{
3654         Object.assign(P, item.p);
3655         syncSliders();
3656         redraw(true);
3657         ntf('Saqlangan naqsh yuklandi', 'ok');
3658     });
3659     grid.appendChild(d);
3660 });
3661 renderProfileCard();
3662 }

```

```

3663 function saveCurrent(){
3664   const off = document.createElement('canvas');
3665   off.width = off.height = 144;
3666   const octx = off.getContext('2d');
3667   if(octx){
3668     const pal = PAL[P.pal] || PAL.gold;
3669     octx.fillStyle = pal.bg;
3670     octx.fillRect(0,0,144,144);
3671     drawPattern(octx, 72, 72, 66, P);
3672   }
3673   const snap = {...P}; delete snap._bg;
3674   /* Ko'rinish rasmi JPEG (0.85) — galereyada farqi bilinmaydi, lekin hajmi
3675   PNG dan ~3.6 barobar kichik (o'lchandi: 38.9 KB → 10.9 KB). Bulutda bu
3676   to'g'ridan-to'g'ri sig'imga aylanadi: bepul 500 MB da ~273 o'rniga
3677   ~975 to'la foydalanuvchi. Fon baribir to'ldirilgani uchun JPEG da
3678   shaffoflik yo'qligi muammo emas. */
3679   const item = { p:snap, thumb: off.toDataURL('image/jpeg', 0.85), t:Date.now() };
3680   if(cloudActive()){
3681     cloudSavePattern(item)
3682       .then(()=>{ renderGallery(); ntf('Naqsh hisobingizga saqlandi 📁', 'ok'); })
3683       .catch(e=>{
3684         console.error(['cloud] save', e);
3685         ntf('Bulutga saqlab bo'lmadi: ' + ((e && e.message) || 'noma\'lum'), 'err');
3686       });
3687     return;
3688   }
3689   const arr = getSavedList();
3690   arr.unshift(item);
3691   if(arr.length > 48) arr.length = 48;
3692   if(setSavedList(arr)){
3693     renderGallery();
3694     ntf('Naqsh saqlandi 📁', 'ok');
3695   } else {
3696     ntf('Saqlab bo'lmadi (localStorage mavjud emas)', 'err');
3697   }
3698 }
3699
3700 /* =====
3701 EKSPORT — Canvas API interceptor (SVG/DXF), PNG
3702 ===== */
3703 /* 2x3 affin matritsa: [a,b,c,d,e,f] */
3704 function matMul(m,n){
3705   return [
3706     m[0]*n[0]+m[2]*n[1],      m[1]*n[0]+m[3]*n[1],
3707     m[0]*n[2]+m[2]*n[3],      m[1]*n[2]+m[3]*n[3],
3708     m[0]*n[4]+m[2]*n[5]+m[4], m[1]*n[4]+m[3]*n[5]+m[5]
3709   ];
3710 }
3711 function SvgCtx(){
3712   this._svg = true;
3713   this.m = [1,0,0,1,0,0];
3714   this.stack = [];
3715   this.paths = []; /* {cmds:[['M',x,y][['L',x,y][['C',...][['Z']], w, op:'stroke'|'fill']} */
3716   this.cur = []; /* joriy path buyruqlari */
3717   this.strokeStyle = '#000'; this.fillStyle = '#000';
3718   this.lineWidth = 1; this.globalAlpha = 1;
3719   this.lineCap = 'round'; this.lineJoin = 'round';
3720   this.font = '10px sans-serif';
3721 }
3722 SvgCtx.prototype.tp = function(x,y){
3723   const m = this.m;
3724   return [m[0]*x + m[2]*y + m[4], m[1]*x + m[3]*y + m[5]];
3725 };
3726 SvgCtx.prototype.scaleFactor = function(){
3727   const m = this.m;
3728   return Math.sqrt(Math.abs(m[0]*m[3] - m[1]*m[2])) || 1;
3729 };
3730 SvgCtx.prototype.save = function(){ this.stack.push(this.m.slice()); };
3731 SvgCtx.prototype.restore = function(){ const m = this.stack.pop(); if(m) this.m = m; };
3732 SvgCtx.prototype.translate = function(x,y){ this.m = matMul(this.m, [1,0,0,1,x,y]); };
3733 SvgCtx.prototype.rotate = function(a){ const c=Math.cos(a), s=Math.sin(a); this.m = matMul(this.m, [c,s,-s,c,0,0])
3734 ; };
3735 SvgCtx.prototype.scale = function(x,y){ this.m = matMul(this.m, [x,0,0,y,0,0]); };
3736 SvgCtx.prototype.transform = function(a,b,c,d,e,f){ this.m = matMul(this.m, [a,b,c,d,e,f]); };
3737 SvgCtx.prototype.setTransform = function(a,b,c,d,e,f){ this.m = [a,b,c,d,e,f]; };
3738 SvgCtx.prototype.beginPath = function(){ this.cur = []; };
3739 SvgCtx.prototype.moveTo = function(x,y){ const p = this.tp(x,y); this.cur.push(['M',p[0],p[1]]); };
3740 SvgCtx.prototype.lineTo = function(x,y){ const p = this.tp(x,y); this.cur.push(['L',p[0],p[1]]); };
3741 SvgCtx.prototype.bezierCurveTo = function(x1,y1,x2,y2,x,y){
3742   const a = this.tp(x1,y1), b = this.tp(x2,y2), c = this.tp(x,y);
3743   this.cur.push(['C',a[0],a[1],b[0],b[1],c[0],c[1]]);
3744 };
3745 SvgCtx.prototype.quadraticCurveTo = function(qx,qy,x,y){
3746   /* kvadratik → kubik konversiya */
3747   let sx = 0, sy = 0;
3748   for(let i=this.cur.length-1; i>=0; i--){
3749     const c = this.cur[i];
3750     if(c[0]=== 'M' || c[0]=== 'L'){ sx=c[1]; sy=c[2]; break; }
3751     if(c[0]=== 'C'){ sx=c[5]; sy=c[6]; break; }
3752   }
3753   const q = this.tp(qx,qy), e = this.tp(x,y);

```

```

3753     this.cur.push(['C',
3754         sx + 2/3*(q[0]-sx), sy + 2/3*(q[1]-sy),
3755         e[0] + 2/3*(q[0]-e[0]), e[1] + 2/3*(q[1]-e[1]),
3756         e[0], e[1]]);
3757 };
3758 SvgCtx.prototype.closePath = function(){ this.cur.push(['Z']); };
3759 SvgCtx.prototype.arc = function(x,y,r,a0,a1,ccw){
3760     /* yoy → 4/3·tan(Δθ/4) Bezier approksimatsiyasi */
3761     if(ccw){ const t=a0; a0=a1; a1=t + TAU; }
3762     let da = a1 - a0;
3763     if(da < 0) da += TAU;
3764     if(da > TAU) da = TAU;
3765     const segs = Math.max(1, Math.ceil(da / (Math.PI/2)));
3766     const step = da/segs;
3767     let sp = this.tp(x + Math.cos(a0)*r, y + Math.sin(a0)*r);
3768     this.cur.push([this.cur.length ? 'L' : 'M', sp[0], sp[1]]);
3769     for(let i=0;i<segs;i++){
3770         const t0 = a0 + step*i, t1 = t0 + step;
3771         const k = 4/3*Math.tan((t1-t0)/4)*r;
3772         const p1 = this.tp(x+Math.cos(t0)*r - Math.sin(t0)*k, y+Math.sin(t0)*r + Math.cos(t0)*k);
3773         const p2 = this.tp(x+Math.cos(t1)*r + Math.sin(t1)*k, y+Math.sin(t1)*r - Math.cos(t1)*k);
3774         const p3 = this.tp(x+Math.cos(t1)*r, y+Math.sin(t1)*r);
3775         this.cur.push(['C',p1[0],p1[1],p2[0],p2[1],p3[0],p3[1]]);
3776     }
3777 };
3778 SvgCtx.prototype.ellipse = function(x,y,rx,ry,rot,a0,a1,ccw){
3779     this.save();
3780     this.translate(x,y); this.rotate(rot||0); this.scale(rx,ry);
3781     this.arc(0,0,1,a0===undefined?0:a0, a1===undefined?TAU:a1, ccw);
3782     this.restore();
3783 };
3784 SvgCtx.prototype.rect = function(x,y,w,h){
3785     this.moveTo(x,y); this.lineTo(x+w,y); this.lineTo(x+w,y+h); this.lineTo(x,y+h); this.closePath();
3786 };
3787 SvgCtx.prototype.stroke = function(){
3788     if(this.cur.length) this.paths.push({cmds:this.cur.slice(), w:this.lineWidth*this.scaleFactor(), op:'stroke'});
3789 };
3790 SvgCtx.prototype.fill = function(){
3791     if(this.cur.length) this.paths.push({cmds:this.cur.slice(), w:0.5, op:'fill'});
3792 };
3793 SvgCtx.prototype.clearRect = function({});
3794 SvgCtx.prototype.fillRect = function({});
3795 SvgCtx.prototype.strokeRect = function(x,y,w,h){ this.beginPath(); this.rect(x,y,w,h); this.stroke(); };
3796 SvgCtx.prototype.clip = function({});
3797 SvgCtx.prototype.drawImage = function({});
3798 SvgCtx.prototype.setLineDash = function({});
3799 SvgCtx.prototype.fillText = function({});
3800 SvgCtx.prototype.measureText = function(){ return {width:0}; };
3801 SvgCtx.prototype.createLinearGradient = function(){ return {addColorStop:function({})}; };
3802 SvgCtx.prototype.createRadialGradient = function(){ return {addColorStop:function({})}; };
3803
3804 function num(v){ return (Math.round(v*100)/100).toString(); }
3805 function cmdsToD(cmds){
3806     let d = '';
3807     for(const c of cmds){
3808         if(c[0]=== 'M') d += 'M ' + num(c[1]) + ' ' + num(c[2]) + ' ';
3809         else if(c[0]=== 'L') d += 'L ' + num(c[1]) + ' ' + num(c[2]) + ' ';
3810         else if(c[0]=== 'C') d += 'C ' + num(c[1]) + ' ' + num(c[2]) + ' ' + num(c[3]) + ' ' + num(c[4]) + ' ' + num(c[
3811         5]) + ' ' + num(c[6]) + ' ';
3812         else if(c[0]=== 'Z') d += 'Z ';
3813     }
3814     return d.trim();
3815 }
3816 /* Naqshni vektor sifatida yozib olish.
3817 bw × bh — taxta o'lchami ICHKI birlikda (uzun tomon 1000).
3818 fill:
3819     'tile' — naqsh butun yuzani to'ldiradi (tessellatsiya);
3820     'medal' — markazda doira medalyon (eski xatti-harakat).
3821     Ilgari faqat 500,500,470 — ya'ni HAR DOIM kvadrat taxtdagi DOIRA edi,
3822     shuning uchun foydalanuvchi qanday o'lcham tanlasa ham doira chiqardi. */
3823 function recordPattern(bw, bh, fill){
3824     bw = bw || 1000; bh = bh || bw;
3825     const fake = new SvgCtx();
3826     const saveTess = TESS.on;
3827     if(fill === 'medal'){
3828         drawPattern(fake, bw/2, bh/2, Math.min(bw, bh)*0.47, P);
3829     } else {
3830         TESS.on = true;
3831         drawTessellation(fake, bw, bh, P);
3832     }
3833     TESS.on = saveTess;
3834     return fake.paths;
3835 }
3836 /* Fizik o'lchamdan ichki taxta o'lchamini chiqarish: uzun tomon 1000
3837 birlik, kalta tomon nisbatan. Shunday qilinsa naqsh chizish miqyosi
3838 avvalgidek qoladi (kvadrat 100 mm da aynan 1000×1000), lekin
3839 to'rtburchak o'lchamlar ham to'g'ri chiqadi. */
3840 function boardFor(wMm, hMm){
3841     const S = 1000;
3842     const w = Math.max(1, wMm || 100), h = Math.max(1, hMm || w);
3843     const mx = Math.max(w, h);

```

```

3843   return { bw: S*w/mx, bh: S*h/mx, k: mx/S }; /* k – birlik → mm */
3844 }
3845 /* Bezier → 32 nuqtali tabaqalash */
3846 function flattenCmds(cmds){
3847   const polys = [];
3848   let cur = null, sx = 0, sy = 0;
3849   for(const c of cmds){
3850     if(c[0]=== 'M'){
3851       if(cur && cur.pts.length>1) polys.push(cur);
3852       cur = {pts:[c[1],c[2]], closed:false};
3853       sx=c[1]; sy=c[2];
3854     } else if(c[0]=== 'L'){
3855       if(!cur) cur = {pts:[sx,sy], closed:false};
3856       cur.pts.push([c[1],c[2]]);
3857       sx=c[1]; sy=c[2];
3858     } else if(c[0]=== 'C'){
3859       if(!cur) cur = {pts:[sx,sy], closed:false};
3860       const p0=[sx,sy], p1=[c[1],c[2]], p2=[c[3],c[4]], p3=[c[5],c[6]];
3861       /* Adaptiv tabaqalash: segmentlar soni egri chiziq uzunligidan
3862        chiqadi. Ilgari har egri qat'iy 32 bo'lakka bo'linardi – mayda
3863        egrilarda bu keraksiz nuqtalar (fayl hajmi va ishlov vaqti),
3864        yirik egrilarda esa aniqlik yetmasligi mumkin edi.
3865         $n \approx \sqrt{(2 \cdot L)}$  da xato  $\approx L/(8n^2) \approx 1/16$  birlik = 100 mm detalda
3866        0.006 mm – lazer aniqligidan ancha mayda. */
3867       const L = Math.hypot(p1[0]-p0[0], p1[1]-p0[1]) +
3868         Math.hypot(p2[0]-p1[0], p2[1]-p1[1]) +
3869         Math.hypot(p3[0]-p2[0], p3[1]-p2[1]);
3870       const n = Math.max(3, Math.min(32, Math.ceil(Math.sqrt(2*L))));
3871       for(let i=1;i<=n;i++){
3872         cur.pts.push(cubicAt(p0,p1,p2,p3,i/n));
3873       }
3874       sx=c[5]; sy=c[6];
3875     } else if(c[0]=== 'Z'){
3876       if(cur){ cur.closed = true; polys.push(cur); cur = null; }
3877     }
3878   }
3879   if(cur && cur.pts.length>1) polys.push(cur);
3880   return polys;
3881 }
3882 /* -----
3883 LAZER GEOMETRIYASINI TAYYORLASH
3884 Ikki o'lchangan muammo tuzatiladi:
3885
3886 1) ISH MAYDONIDAN CHIQISH. Tekislikni to'ldiruvchi naqshlar (giri
3887 oilasi, katak8, hexa, zellige) berilgan radiusdan tashqariga ham
3888 chizadi. O'lchov: panelgiri 1000 birlik taxtada -2657...3880 gacha
3889 chiqib ketgan – ya'ni 100 mm deb e'lon qilingan detal aslida 654 mm.
3890 Bunday fayl lazerda material chetidan tashqarida kesadi. Shuning
3891 uchun butun geometriya taxtaga QIRQILADI: yopiq konturlar
3892 Sutherland–Hodgman bilan (yopiqliciga qoladi, chegara bo'ylab
3893 yuradi), ochiq chiziqlar Liang–Barsky bilan bo'laklanadi.
3894
3895 2) TAKRORIY KONTURLAR. Qo'shni plitkalar umumiy qirrani ikki marta
3896 chizadi (o'lchovda hexa'da 82%, celtic'da 65%). Lazer o'sha chiziqni
3897 ikki marta kesadi: material kuyadi va ish vaqti bekorga oshadi.
3898 Bir xil konturlar bitta qoldiriladi.
3899 ----- */
3900 const BOARD = { x0:0, y0:0, x1:1000, y1:1000 };
3901
3902 function _ix(a, b, axis, val){
3903   const t = (val - a[axis]) / (b[axis] - a[axis]);
3904   return [a[0] + (b[0]-a[0])*t, a[1] + (b[1]-a[1])*t];
3905 }
3906 /* yopiq kontur – Sutherland–Hodgman */
3907 function clipClosed(pts, R){
3908   const edges = [
3909     [p => p[0] >= R.x0, (a,b)=>_ix(a,b,0,R.x0)],
3910     [p => p[0] <= R.x1, (a,b)=>_ix(a,b,0,R.x1)],
3911     [p => p[1] >= R.y0, (a,b)=>_ix(a,b,1,R.y0)],
3912     [p => p[1] <= R.y1, (a,b)=>_ix(a,b,1,R.y1)]
3913   ];
3914   let out = pts;
3915   for(const [inside, isect] of edges){
3916     const src = out; out = [];
3917     const n = src.length;
3918     if(!n) return null;
3919     for(let i = 0; i < n; i++){
3920       const a = src[i], b = src[(i+1) % n];
3921       const ia = inside(a), ib = inside(b);
3922       if(ia) out.push(a);
3923       if(ia !== ib) out.push(isect(a, b));
3924     }
3925     if(!out.length) return null;
3926   }
3927   return out.length > 2 ? out : null;
3928 }
3929 /* bitta kesma – Liang–Barsky */
3930 function clipSeg(a, b, R){
3931   let t0 = 0, t1 = 1;
3932   const dx = b[0]-a[0], dy = b[1]-a[1];
3933   const P = [-dx, dx, -dy, dy];

```

```

3934 const Q = [a[0]-R.x0, R.x1-a[0], a[1]-R.y0, R.y1-a[1]];
3935 for(let i = 0; i < 4; i++){
3936   if(P[i] === 0){ if(Q[i] < 0) return null; continue; }
3937   const r = Q[i]/P[i];
3938   if(P[i] < 0){ if(r > t1) return null; if(r > t0) t0 = r; }
3939   else { if(r < t0) return null; if(r < t1) t1 = r; }
3940 }
3941 return [[a[0]+dx*t0, a[1]+dy*t0], [a[0]+dx*t1, a[1]+dy*t1]];
3942 }
3943 function clipOpen(pts, R){
3944   const out = []; let cur = null;
3945   const same = (p,q)=> Math.abs(p[0]-q[0]) < 1e-6 && Math.abs(p[1]-q[1]) < 1e-6;
3946   for(let i = 1; i < pts.length; i++){
3947     const seg = clipSeg(pts[i-1], pts[i], R);
3948     if(!seg){ if(cur && cur.length > 1) out.push(cur); cur = null; continue; }
3949     if(cur && same(cur[cur.length-1], seg[0])) cur.push(seg[1]);
3950     else { if(cur && cur.length > 1) out.push(cur); cur = [seg[0], seg[1]]; }
3951   }
3952   if(cur && cur.length > 1) out.push(cur);
3953   return out;
3954 }
3955 function clipPolys(polys, R){
3956   const out = [];
3957   for(const pl of polys){
3958     if(!pl.pts || pl.pts.length < 2) continue;
3959     let inside = true;
3960     for(const q of pl.pts){
3961       if(q[0] < R.x0-1e-9 || q[0] > R.x1+1e-9 || q[1] < R.y0-1e-9 || q[1] > R.y1+1e-9){ inside = false; break; }
3962     }
3963     if(inside){ out.push(pl); continue; }
3964     if(pl.closed){
3965       const c = clipClosed(pl.pts, R);
3966       if(c) out.push({ pts:c, closed:true });
3967     } else {
3968       for(const seg of clipOpen(pl.pts, R)) out.push({ pts:seg, closed:false });
3969     }
3970   }
3971   return out;
3972 }
3973 /* bir xil konturlarni bittaga tushirish – kalit nuqtalar to'plamidan
3974 yasaladi, shuning uchun yo'nalish va boshlang'ich nuqtaga bog'liq emas */
3975 function dedupePolys(polys){
3976   /* Kalit nuqtalar TO'PLAMIdan yasaladi (yig'indi va XOR – ikkalasi ham
3977 tartibdan mustaqil), shuning uchun yo'nalish va boshlang'ich nuqtaga
3978 bog'liq emas. Ilgari bu yerda har kontur uchun matn massivi qurilib
3979 saralanardi – zich naqshlarda eksportning asosiy sekinlik sababi
3980 shu edi. */
3981 const seen = new Set(), out = [];
3982 for(const pl of polys){
3983   let h1 = 0, h2 = 0;
3984   for(const q of pl.pts){
3985     const a = Math.round(q[0]*100) | 0, b = Math.round(q[1]*100) | 0;
3986     let v = (Math.imul(a, 73856093) ^ Math.imul(b, 19349663)) >>> 0;
3987     v = (v ^ (v >>> 13)) >>> 0;
3988     h1 = (h1 + v) >>> 0;
3989     h2 = (h2 ^ v) >>> 0;
3990   }
3991   const key = (pl.closed ? 'C' : 'O') + pl.pts.length + '|' + h1 + '|' + h2;
3992   if(seen.has(key)) continue;
3993   seen.add(key);
3994   out.push(pl);
3995 }
3996 return out;
3997 }
3998 /* USTMA-UST TUSHGAN KESMALARNI BIRLASHTIRISH.
3999 Qo'shni plitkalar umumiy qirrani ikki marta chizadi. Bir xil KONTURNI
4000 olib tashlash yetarli emas: ko'pincha ikki HAR XIL kontur bitta qirrani
4001 baham ko'radi. Lazer uchun bu chiziq ikki marta kesiladi – material
4002 kuyadi, ish vaqti bekorga oshadi.
4003 Shuning uchun geometriya kesmalarga ajratiladi, takrorlari tashlanadi,
4004 qolganlari esa uzluksiz zanjirlarga qayta yig'iladi (kesuvchi bosh
4005 kamroq sakraydi). Yaxlitlash 0.1 birlik = 100 mm detalda 0.01 mm –
4006 lazer aniqligidan ancha mayda. */
4007 function mergeOverlaps(polys){
4008   const nk = p => Math.round(p[0]*10) + ',' + Math.round(p[1]*10);
4009   const seen = new Set(), segs = [];
4010   for(const pl of polys){
4011     const pts = pl.closed ? pl.pts.concat([pl.pts[0]]) : pl.pts;
4012     for(let i = 1; i < pts.length; i++){
4013       const a = pts[i-1], b = pts[i];
4014       const ka = nk(a), kb = nk(b);
4015       if(ka === kb) continue; /* nol uzunlikdagi kesma */
4016       const key = ka < kb ? ka + '|' + kb : kb + '|' + ka;
4017       if(seen.has(key)) continue;
4018       seen.add(key);
4019       segs.push([a, b]);
4020     }
4021   }
4022   /* zanjirlash */
4023   const adj = new Map();
4024   segs.forEach((sg, i)=>{

```

```

4025     for(const p of sg){
4026         const k = nk(p);
4027         if(!adj.has(k)) adj.set(k, []);
4028         adj.get(k).push(i);
4029     }
4030 });
4031 const used = new Uint8Array(segs.length);
4032 /* Tugundan davom ettirish uchun kesma tanlash.
4033    Ilgari ro'yxatdan oxirgisi olinardi. Bu o'zini kesib o'tuvchi
4034    naqshlarda (dragon: 122 947 ta to'rt tarmoqli kesishish) har
4035    kesishishda boshqa tarmoqqa burilib ketardi va uzluksiz chiziq
4036    minglab bo'lakka sochilardi — o'lchov: 1 764 kontur → 32 309 bo'lak.
4037    Lazer uchun har bo'lak — alohida bosh ko'tarish/tushirish.
4038    Endi kirish yo'nalishini eng yaxshi davom ettiradigan kesma
4039    tanlanadi, ya'ni chiziq o'z tabiiy yo'lidan yuradi. */
4040 const takeAt = (k, dx, dy)=>{
4041     const list = adj.get(k);
4042     if(!list) return -1;
4043     let w = 0;
4044     for(let i = 0; i < list.length; i++) if(!used[list[i]]) list[w++] = list[i];
4045     list.length = w;
4046     if(!w) return -1;
4047     if(w === 1 || (dx === 0 && dy === 0)) return list[0];
4048     const dl = Math.hypot(dx, dy) || 1;
4049     let best = list[0], bestC = -Infinity;
4050     for(let i = 0; i < w; i++){
4051         const j = list[i], sg = segs[j];
4052         const head = nk(sg[0]) === k;
4053         const a = head ? sg[0] : sg[1], b = head ? sg[1] : sg[0];
4054         const ex = b[0]-a[0], ey = b[1]-a[1];
4055         const el = Math.hypot(ex, ey) || 1;
4056         const c = (dx*ex + dy*ey)/(dl*el);
4057         if(c > bestC){ bestC = c; best = j; }
4058     }
4059     return best;
4060 };
4061 const out = [];
4062 for(let i = 0; i < segs.length; i++){
4063     if(used[i]) continue;
4064     used[i] = 1;
4065     const chain = [segs[i][0], segs[i][1]];
4066     for(;;){
4067         const n = chain.length, e = chain[n-1], pv = chain[n-2];
4068         const j = takeAt(nk(e), e[0]-pv[0], e[1]-pv[1]);
4069         if(j < 0) break;
4070         used[j] = 1;
4071         const k = nk(e);
4072         chain.push(nk(segs[j][0]) === k ? segs[j][1] : segs[j][0]);
4073     }
4074     for(;;){
4075         const e = chain[0], pv = chain[1];
4076         const j = takeAt(nk(e), e[0]-pv[0], e[1]-pv[1]);
4077         if(j < 0) break;
4078         used[j] = 1;
4079         const k = nk(e);
4080         chain.unshift(nk(segs[j][0]) === k ? segs[j][1] : segs[j][0]);
4081     }
4082     let closed = false;
4083     if(chain.length > 3 && nk(chain[0]) === nk(chain[chain.length-1])){ chain.pop(); closed = true; }
4084     if(chain.length > 1) out.push({ pts:chain, closed });
4085 }
4086 return out;
4087 }
4088
4089 /* -----
4090    NUQTALARNI SODDALASHTIRISH (Douglas-Peucker)
4091    -----
4092    O'lchov natijasi: 120×90 mm taxtda spirograph naqshi 1 727 016 vertex
4093    berardi — DXF fayli 89 MB. Lazer apparatlarining import moduli bunday
4094    faylni ochmaydi yoki soatlab o'ylaydi; ochsa ham kesuvchi bosh har
4095    mikron uchun to'xtab-to'xtab yuradi. Sabab — egri chiziqlar qat'iy
4096    qadam bilan tekislanadi, natijada deyarli bir to'g'ri chiziqda yotgan
4097    o'nlab keraksiz nuqta qoladi.
4098    Douglas-Peucker aynan shu keraksiz nuqtalarni olib tashlaydi: kontur
4099    shakli berilgan chegaradan (mm) ko'p og'masa, oraliq nuqta tushiriladi.
4100    Boshlang'ich chegara 0.02 mm — lazerning o'z pozitsiya xatosidan
4101    (~0.05 mm) va kerf kengligidan (~0.1–0.2 mm) mayda, ya'ni ko'zga ham,
4102    materialga ham bilinmaydi.
4103    Rekursiya emas, o'z stegi ishlatiladi: 1.2 mln nuqtali konturda
4104    rekursiya chaqiruvlar stegini to'ldirib yuboradi. */
4105 function dpSimplify(pts, eps){
4106     const n = pts.length;
4107     if(n < 3 || !(eps > 0)) return pts;
4108     const keep = new Uint8Array(n);
4109     keep[0] = 1; keep[n-1] = 1;
4110     const e2 = eps*eps;
4111     const stack = [0, n-1];
4112     while(stack.length){
4113         const i1 = stack.pop(), i0 = stack.pop();
4114         if(i1 <= i0 + 1) continue;
4115         const ax = pts[i0][0], ay = pts[i0][1];

```

```

4116     const dx = pts[i1][0] - ax, dy = pts[i1][1] - ay;
4117     const dd = dx*dx + dy*dy;
4118     let best = -1, bd = e2;
4119     for(let i = i0 + 1; i < i1; i++){
4120         const px = pts[i][0] - ax, py = pts[i][1] - ay;
4121         let d;
4122         if(dd > 0){
4123             let t = (px*dx + py*dy)/dd;
4124             if(t < 0) t = 0; else if(t > 1) t = 1;
4125             const ex = px - dx*t, ey = py - dy*t;
4126             d = ex*ex + ey*ey;
4127         } else d = px*px + py*py;
4128         if(d > bd){ bd = d; best = i; }
4129     }
4130     if(best < 0) continue;
4131     keep[best] = 1;
4132     stack.push(i0, best, best, i1);
4133 }
4134 const out = [];
4135 for(let i = 0; i < n; i++) if(keep[i]) out.push(pts[i]);
4136 return out;
4137 }
4138 function simplifyPolys(polys, eps){
4139     const out = [];
4140     for(const pl of polys){
4141         if(pl.pts.length < 3){ out.push(pl); continue; }
4142         if(pl.closed){
4143             /* yopiq konturda oxirgi (yopuvchi) kesma ham soddalashtirilishi
4144             uchun birinchi nuqta vaqtincha oxiriga qo'shiladi */
4145             const s = dpSimplify(pl.pts.concat([pl.pts[0]]), eps);
4146             s.pop();
4147             out.push(s.length > 2 ? { pts:s, closed:true } : pl);
4148         } else {
4149             out.push({ pts: dpSimplify(pl.pts, eps), closed:false });
4150         }
4151     }
4152     return out;
4153 }
4154 function countVerts(polys){
4155     let n = 0;
4156     for(const pl of polys) n += pl.pts.length;
4157     return n;
4158 }
4159 /* ENG QISQA KESMA CHEGARASI.
4160 Douglas-Peucker faqat "deyarli to'g'ri" joylarni siqadi. Fraktal
4161 naqshlarda (dragon, koch) esa nuqtalar haqiqatan ham burchak yasaydi,
4162 lekin ular orasidagi masofa 0.01–0.05 mm – lazer kerfi (0.1–0.2 mm)
4163 dan mayda, ya'ni apparat baribir chiza olmaydi: o'lchov natijasida
4164 dragon 32 310 kontur va 22 MB fayl berardi.
4165 Shuning uchun oldingi saqlangan nuqtadan minSeg dan yaqin nuqtalar
4166 tushiriladi. Konturning oxirgi nuqtasi doim saqlanadi – shakl uzunligi
4167 qisqarmasligi uchun. */
4168 function decimatePts(pts, minSeg){
4169     if(!(minSeg > 0) || pts.length < 3) return pts;
4170     const m2 = minSeg*minSeg;
4171     const out = [pts[0]];
4172     let lx = pts[0][0], ly = pts[0][1];
4173     for(let i = 1; i < pts.length - 1; i++){
4174         const dx = pts[i][0]-lx, dy = pts[i][1]-ly;
4175         if(dx*dx + dy*dy >= m2){ out.push(pts[i]); lx = pts[i][0]; ly = pts[i][1]; }
4176     }
4177     out.push(pts[pts.length-1]);
4178     return out;
4179 }
4180 /* Lazer fayli uchun cheklovlar.
4181 Zinapoya: avval eng mayda (ko'zga ko'rinmas) chegara sinaladi; fayl
4182 apparat uchun og'ir bo'lsa, keyingi zinaga o'tiladi. Eng qo'pol zinada
4183 ham og'ish 0.25 mm – 120 mm detalda 0.2%, kerf kengligidan kichik. */
4184 const LASER_LADDER = [
4185     { eps:0.02, seg:0 },
4186     { eps:0.04, seg:0.05 },
4187     { eps:0.07, seg:0.08 },
4188     { eps:0.10, seg:0.12 },
4189     { eps:0.15, seg:0.20 },
4190     { eps:0.25, seg:0.30 }
4191 ];
4192 /* Vertex byudjeti. 190 000 ≈ 11 MB DXF – eski import modullari ham
4193 bemalol ochadi (muammo 89 MB da boshlangan edi). Undan pastga
4194 tushirilsa koch kabi fraktal naqshlarda mayda tishlar yo'qoladi
4195 (o'lchov: byudjet 150 000 da koch kesish uzunligining 11.6% ini
4196 yo'qotardi), shuning uchun byudjet aynan shu chegarada. */
4197 const LASER_VERT_MAX = 190000;
4198 let LASER_INFO = null; /* oxirgi eksport statistikasi (konsol uchun) */
4199
4200 /* Lazer eksporti uchun yakuniy geometriya:
4201 tekislash → taxtaga qirgish → bir xil konturlarni tashlash →
4202 ustma-ust kesmalarni birlashtirish → mayda parchalarni olib tashlash.
4203 Oxirgi bosqich: qirgishdan qolgan 0.3 mm dan qisqa parchalar lazerda
4204 kesma emas, KUYGAN NUQTA beradi (kerf o'zi ~0.1–0.2 mm). */
4205 function laserPolys(wMm, hMm, fill){
4206     const B = boardFor(wMm, hMm);

```

```

4207 const board = { x0:0, y0:0, x1:B.bw, y1:B.bh };
4208 const polys = [];
4209 for(const pth of recordPattern(B.bw, B.bh, fill)){
4210   for(const pl of flattenCmds(pth.cmds)) if(pl.pts.length > 1) polys.push(pl);
4211 }
4212 const minU = 0.3/B.k; /* 0.3 mm — birlikda */
4213 const longEnough = pl=>{
4214   let L = 0;
4215   for(let i = 1; i < pl.pts.length; i++)
4216     L += Math.hypot(pl.pts[i][0]-pl.pts[i-1][0], pl.pts[i][1]-pl.pts[i-1][1]);
4217   if(pl.closed && pl.pts.length > 1)
4218     L += Math.hypot(pl.pts[0][0]-pl.pts[pl.pts.length-1][0], pl.pts[0][1]-pl.pts[pl.pts.length-1][1]);
4219   return L >= minU;
4220 };
4221 /* 1-BOSQICH — to'liq aniqlikdagi geometriyada ustma-ustlikni tozalash.
4222 Bu aynan shu yerda bo'lishi shart: qo'shni plitkalarining umumiy
4223 qirralari BIR XIL nuqtalar bilan chizilgan, shuning uchun ular
4224 ishonchli aniqlanadi. Agar avval soddalashtirilsa, ikki nusxada
4225 turli nuqtalar saqlanib qolib, qirra takrorlanadi (o'lchov: girih
4226 yo'li 50 509 → 85 760 birlikka, ya'ni 70% ortiqcha kesishga
4227 ko'tarilgan edi). */
4228 const merged = mergeOverlaps(dedupePolys(clipPolys(polys, board)));
4229 /* 2-BOSQICH — keraksiz nuqtalarni tashlash, so'ng USTMA-USTLIKNI
4230 QAYTA TOZALASH. Soddalashtirish A-B-C ni A-C ga aylantiradi va
4231 boshqa zanjirda ham xuddi shunday A-C paydo bo'lishi mumkin —
4232 ya'ni soddalashtirishning O'ZI yangi takror kesma yaratadi
4233 (o'lchov: girih 4%, panelgirih 8.4%). Ikkinchi birlashtirish
4234 ularni yo'q qiladi. */
4235 let res = null, step = LASER_LADDER[0];
4236 for(let i = 0; i < LASER_LADDER.length; i++){
4237   step = LASER_LADDER[i];
4238   const segU = step.seg/B.k, epsU = step.eps/B.k;
4239   const dec = step.seg > 0
4240     ? merged.map(pl=>({ pts: decimatePts(pl.pts, segU), closed: pl.closed })))
4241     : merged.filter(pl=> pl.closed ? pl.pts.length > 2 : pl.pts.length > 1)
4242   ; merged;
4243   res = mergeOverlaps(simplifyPolys(dec, epsU)).filter(longEnough);
4244   if(countVerts(res) <= LASER_VERT_MAX) break;
4245 }
4246 LASER_INFO = { polys: res.length,
4247               verts: countVerts(res),
4248               vertsRaw: countVerts(merged),
4249               epsMm: step.eps, segMm: step.seg };
4250 return res;
4251 }
4252
4253 /* DXF AC1015 (AutoCAD 2000), $INSUNITS=4 (mm). sizeMm — chetma-chet o'lcham.
4254 Konturlar yopiq LWPOLYLINE sifatida chiqadi (lazer uchun shart). */
4255 /* -----
4256 DXF — R12 (AC1009)
4257 -----
4258 Ilgari AC1015 (AutoCAD 2000) va LWPOLYLINE ishlatilardi. Lazer
4259 apparatlarining import moduli ko'pincha ANCHA ESKI bo'ladi va faqat R12 ni
4260 to'liq o'qiydi: LWPOLYLINE R14 dan boshlab paydo bo'lgan, handle (5) va
4261 AcDbEntity sinf belgilari esa R13+ ga tegishli — eski o'qigich ularda
4262 to'xtaydi yoki bo'sh chizma ochadi.
4263 R12 — eng past umumiy maxraj: uni R12 dan keyingi HAMMA dastur o'qiydi.
4264 Shuning uchun bu yerda POLYLINE/VERTEX/SEQEND ishlatiladi, handle va
4265 sinf belgilari yo'q.
4266 ----- */
4267 function buildDXF(wMm, hMm, fill){
4268   const B = boardFor(wMm, hMm);
4269   const polys = laserPolys(wMm, hMm, fill);
4270   const L = [];
4271   const push = (...a)=>{ for(const x of a) L.push(x); };
4272   const MM = B.k; /* ichki birlik → mm */
4273   const BH = B.bh;
4274   const W = B.bw*MM, H = BH*MM;
4275
4276   /* HEADER — o'lchov birligi metrik ekani ikki xil kalit bilan aytiladi:
4277   $INSUNITS (R13+) va $MEASUREMENT (R12 o'qigichlari shuni qaraydi) */
4278   push('0', 'SECTION', '2', 'HEADER',
4279         '9', '$ACADVER', '1', 'AC1009',
4280         '9', '$INSUNITS', '70', '4',
4281         '9', '$MEASUREMENT', '70', '1',
4282         '9', '$EXTMIN', '10', '0.0', '20', '0.0', '30', '0.0',
4283         '9', '$EXTMAX', '10', num(W), '20', num(H), '30', '0.0',
4284         '0', 'ENDSEC');
4285
4286   /* TABLES — R12 shaklida: handle yo'q, sinf belgisi yo'q */
4287   push('0', 'SECTION', '2', 'TABLES');
4288   push('0', 'TABLE', '2', 'LTYPE', '70', '1',
4289         '0', 'LTYPE', '2', 'CONTINUOUS', '70', '0', '3', 'Solid line', '72', '65', '73', '0', '40', '0.0',
4290         '0', 'ENDTAB');
4291   push('0', 'TABLE', '2', 'LAYER', '70', '2',
4292         '0', 'LAYER', '2', 'Naqsh', '70', '0', '62', '1', '6', 'CONTINUOUS',
4293         '0', 'LAYER', '2', 'Chegara', '70', '0', '62', '7', '6', 'CONTINUOUS',
4294         '0', 'ENDTAB');
4295   push('0', 'ENDSEC');
4296   push('0', 'SECTION', '2', 'BLOCKS', '0', 'ENDSEC');
4297

```



```

4298  /* ENTITIES */
4299  push('0','SECTION','2','ENTITIES');
4300  const emitPoly = (pts, closed, layer)=>{
4301    push('0','POLYLINE','8',layer,'66','1','70', closed ? '1' : '0',
4302      '10','0.0','20','0.0','30','0.0');
4303    for(const q of pts){
4304      /* DXF y o'qi yuqoriga qaragan – shuning uchun ag'dariladi */
4305      push('0','VERTEX','8',layer,
4306        '10', num(q[0]*MM), '20', num((BH-q[1])*MM), '30','0.0');
4307    }
4308    push('0','SEQEND','8',layer);
4309  };
4310  emitPoly([[0,0],[B.bw,0],[B.bw,BH],[0,BH]], true, 'Chegara');
4311  for(const pl of polys){
4312    if(pl.pts.length > 1) emitPoly(pl.pts, pl.closed, 'Naqsh');
4313  }
4314  push('0','ENDSEC','0','EOF');
4315  return L.join('\r\n') + '\r\n'; /* R12 o'qigichlari CRLF ni kutadi */
4316 }
4317
4318 /* -----
4319 EPS – Encapsulated PostScript
4320 -----
4321 Lazer apparatlarining import ro'yxatida EPS odatda BIRINCHI turadi va eng
4322 ishonchli format: u oddiy MATN, ichida faqat to'g'ri chiziqlar va
4323 moveto/lineto buyruqlari bo'ladi.
4324 Birlik – PostScript punkti (1 pt = 1/72 dyuym = 0.352778 mm), shuning
4325 uchun mm → pt: x72/25.4. BoundingBox aynan detal o'lchamiga teng.
4326 ----- */
4327 function buildEPS(wMm, hMm, opts){
4328   opts = opts || {};
4329   const B = boardFor(wMm, hMm);
4330   const polys = laserPolys(wMm, hMm, opts.fill);
4331   const K = B.k*72/25.4; /* ichki birlik → punkt */
4332   const BH = B.bh;
4333   const Wp = B.bw*K, Hp = BH*K;
4334   const kerf = (opts.kerf || 0.1)*72/25.4;
4335   const n2 = v => (Math.round(v*100)/100).toString();
4336   const out = [];
4337   out.push('!PS-Adobe-3.0 EPSF-3.0');
4338   out.push('%Creator: NaqshAI');
4339   out.push('%Title: naqsh ' + wMm + 'x' + hMm + ' mm');
4340   out.push('%BoundingBox: 0 0 ' + Math.ceil(Wp) + ' ' + Math.ceil(Hp));
4341   out.push('%HiResBoundingBox: 0 0 ' + n2(Wp) + ' ' + n2(Hp));
4342   out.push('%DocumentData: Clean7Bit');
4343   out.push('%LanguageLevel: 2');
4344   out.push('%EndComments');
4345   out.push('%BeginProlog');
4346   out.push('/m {moveto} bind def /l {lineto} bind def');
4347   out.push('/h {closepath} bind def /S {stroke} bind def');
4348   out.push('%EndProlog');
4349   out.push('%Page: 1 1');
4350   out.push('gsave');
4351   out.push('1 0 0 setrgbcolor'); /* qizil = kesish konturi */
4352   out.push(n2(kerf) + ' setlinewidth');
4353   out.push('1 setlinecap 1 setlinejoin');
4354   const emit = (pts, closed)=>{
4355     if(pts.length < 2) return;
4356     let line = 'newpath ' + n2(pts[0][0]*K) + ' ' + n2((BH - pts[0][1])*K) + ' m';
4357     for(let i = 1; i < pts.length; i++){
4358       line += ' ' + n2(pts[i][0]*K) + ' ' + n2((BH - pts[i][1])*K) + ' l';
4359     }
4360     if(closed) line += ' h';
4361     out.push(line + ' S');
4362   };
4363   if(opts.frame) emit([[0,0],[B.bw,0],[B.bw,BH],[0,BH]], true);
4364   for(const pl of polys) emit(pl.pts, pl.closed);
4365   out.push('grestore');
4366   out.push('%Trailer');
4367   out.push('%EOF');
4368   return out.join('\n') + '\n';
4369 }
4370 /* Yuklab olish helper */
4371 function _dataURLtoBlob(dataURL){
4372   const comma = dataURL.indexOf(',');
4373   const meta = dataURL.slice(0, comma), b64 = dataURL.slice(comma+1);
4374   const mime = (meta.match(/data:(^;+)?/)||[])[1] || 'application/octet-stream';
4375   const bin = atob(b64);
4376   const arr = new Uint8Array(bin.length);
4377   for(let i=0;i<bin.length;i++) arr[i] = bin.charCodeAt(i);
4378   return new Blob([arr], {type:mime});
4379 }
4380 /* claude.ai artifact preview'da <a download> ishlamaydi – mavjud bo'lsa
4381 "downloads" capability orqali, aks holda (haqiqiy joylashtirishda)
4382 odatiy brauzer usuli bilan saqlaydi. */
4383 async function _dlBlob(content, mime, filename, successMsg){
4384   const blob = (content instanceof Blob) ? content : new Blob([content], {type:mime});
4385   if(window.claude && typeof window.claude.use === 'function'){
4386     let downloads = null;
4387     try{ downloads = await window.claude.use('downloads'); }catch(e){}
4388     if(downloads){
4389       try{

```

```

4389     await downloads.save({filename, data: blob});
4390     if(successMsg) ntf(successMsg, 'ok');
4391     return true;
4392   }catch(err){
4393     const code = err && err.code;
4394     if(code === 'declined') return false;
4395     if(code === 'rejected_extension' || code === 'extension_not_enabled'){
4396       /* Claude'ning ko'rish oynasi faqat cheklangan ro'yxatdagi
4397         kengaytmalarni saqlashga ruxsat beradi (png, jpg, txt, json,
4398         md, svg, pdf...). Sanoat formatlari – TIFF, BMP, WIF, DXF –
4399         bu ro'yxatda YO'Q. Bu ilovaning kamchiligi emas, ko'rish
4400         oynasining cheklovi: haqiqiy saytda hammasi normal yuklanadi.
4401         DXF va WIF – MATN fayllar, shuning uchun ularni .txt sifatida
4402         saqlab berish mumkin: foydalanuvchi nomini o'zgartirsa bo'ldi. */
4403       const ext = (filename.split('.').pop() || '').toLowerCase();
4404       if(ext === 'dxf' || ext === 'wif' || ext === 'eps'){
4405         try{
4406           await downloads.save({ filename: filename + '.txt', data: blob });
4407           ntf('Ko'rish oynasi .' + ext + ' ni saqlay olmaydi – fayl «' +
4408             filename + '.txt» nomi bilan saqlandi. Nomidagi oxirgi ' +
4409             '«.txt» ni o'chiring, ichidagisi to'liq to'lg'ri.', 'ok');
4410           return true;
4411         }catch(e2){ if(e2 && e2.code === 'declined') return false; }
4412       }
4413       ntf('Bu formatni Claude ko'rish oynasida saqlab bo'lmaydi (faqat ' +
4414         'png/jpg/txt/pdf kabi turlar mumkin). Sanoat fayllari uchun ' +
4415         'ilovani o'z saytingizdan oching – u yerda TIFF, BMP, WIF va ' +
4416         'DXF normal yuklanadi.', 'err');
4417       return false;
4418     }
4419     if(code === 'too_large'){
4420       ntf('Fayl juda katta (16 MB dan ortiq) – kichikroq rapport yoki ' +
4421         'past chuqurlik tanlang', 'err');
4422       return false;
4423     }
4424     ntf('Yuklab olishda xato: ' + (err && err.message || code || "noma'lum"), 'err');
4425     return false;
4426   }
4427 }
4428 }
4429 try{
4430   const url = URL.createObjectURL(blob);
4431   const a = document.createElement('a');
4432   a.href = url;
4433   a.download = filename;
4434   a.style.display = 'none';
4435   document.body.appendChild(a);
4436   a.click();
4437   setTimeout(()=>{
4438     if(a.parentNode) a.parentNode.removeChild(a);
4439     URL.revokeObjectURL(url);
4440   }, 1000);
4441   if(successMsg) ntf(successMsg, 'ok');
4442   return true;
4443 }catch(e){
4444   ntf('Yuklab olishda xato: ' + e.message, 'err');
4445   return false;
4446 }
4447 }
4448 /* PNG ga FIZIK O'LCHAM (pHYs) bo'lagini yozish.
4449 Canvas.toBlob() PNG ga zichlik ma'lumotini qo'shmaydi. Shu sababli
4450 fayl nomida "300dpi" yozilgan bo'lsa ham, CorelDraw / Illustrator /
4451 bosmaxona dasturlari uni 96 dpi deb oladi va 2048 px rasm 17.3 sm
4452 o'rniga 54 sm bo'lib joylashadi – ya'ni o'lcham 3 barobar xato.
4453 Quyida IHDR dan keyin standart pHYs bo'lagi qo'shiladi. */
4454 let _crcTab = null;
4455 function crc32(buf){
4456   if(!_crcTab){
4457     _crcTab = new Uint32Array(256);
4458     for(let n = 0; n < 256; n++){
4459       let c = n;
4460       for(let k = 0; k < 8; k++) c = (c & 1) ? (0xEDB88320 ^ (c >>> 1)) : (c >>> 1);
4461       _crcTab[n] = c >>> 0;
4462     }
4463   }
4464   let c = 0xFFFFFFFF;
4465   for(let i = 0; i < buf.length; i++) c = _crcTab[(c ^ buf[i]) & 0xFF] ^ (c >>> 8);
4466   return (c ^ 0xFFFFFFFF) >>> 0;
4467 }
4468 function pngSetDPI(bytes, dpi){
4469   const sig = [137,80,78,71,13,10,26,10];
4470   for(let i = 0; i < 8; i++) if(bytes[i] !== sig[i]) return bytes;
4471   const dvIn = new DataView(bytes.buffer, bytes.byteOffset, bytes.byteLength);
4472   /* allaqachon pHYs bo'lsa – tegmaymiz */
4473   let off = 8;
4474   while(off + 8 <= bytes.length){
4475     const len = dvIn.getUint32(off);
4476     const type = String.fromCharCode(bytes[off+4], bytes[off+5], bytes[off+6], bytes[off+7]);
4477     if(type === 'pHYs') return bytes;
4478     if(type === 'IDAT' || type === 'IEND') break;
4479     off += 12 + len;

```

```

4480 }
4481 const ihdrEnd = 8 + 12 + dvIn.getUint32(8); /* imzo + IHDR */
4482 const ppu = Math.round((dpi || 300)/0.0254); /* piksel/metr */
4483 const chunk = new Uint8Array(21);
4484 const dv = new DataView(chunk.buffer);
4485 dv.setUint32(0, 9);
4486 chunk.set([0x70, 0x48, 0x59, 0x73], 4); /* 'pHYs' */
4487 dv.setUint32(8, ppu); dv.setUint32(12, ppu);
4488 chunk[16] = 1; /* birlik = metr */
4489 dv.setUint32(17, crc32(chunk.subarray(4, 17)));
4490 const out = new Uint8Array(bytes.length + 21);
4491 out.set(bytes.subarray(0, ihdrEnd), 0);
4492 out.set(chunk, ihdrEnd);
4493 out.set(bytes.subarray(ihdrEnd, ihdrEnd + 21));
4494 return out;
4495 }
4496 function exportPNG(w, h, dpi, transparent, cb, fill){
4497   w = Math.max(16, w|0); h = Math.max(16, h|0) || w;
4498   const off = document.createElement('canvas');
4499   off.width = w; off.height = h;
4500   const octx = off.getContext('2d');
4501   if(!octx){ nt('Canvas yaratib bo\'lmadi', 'err'); return; }
4502   const pal = PAL[P.pal] || PAL.gold;
4503   P._bg = pal.bg;
4504   if(!transparent){
4505     octx.fillStyle = pal.bg;
4506     octx.fillRect(0, 0, w, h);
4507   }
4508   /* 'tile' – butun yuzani to'ldiradi (to'rtburchakda ham to'g'ri ishlaydi),
4509   'medal' – markazda doira medalyon. Ilgari bu tanlov yo'q edi va
4510   tessellatsiya o'chirilgan bo'lsa har doim doira chiqardi. */
4511   const doTile = (fill === 'medal') ? false : (fill === 'tile' ? true : TESS.on);
4512   if(doTile){
4513     const saveTess = TESS.on; TESS.on = true;
4514     drawTessellation(octx, w, h, P);
4515     TESS.on = saveTess;
4516   } else {
4517     drawPattern(octx, w/2, h/2, Math.min(w, h)*0.46, P);
4518   }
4519   if(off.toBlob){
4520     off.toBlob(async b=>{
4521       if(!b){ if(cb) cb(); return; }
4522       let blob = b;
4523       try{
4524         const withDpi = pngSetDPI(new Uint8Array(await b.arrayBuffer()), dpi);
4525         blob = new Blob([withDpi], {type:'image/png'});
4526       }catch(e){ console.warn('[png] pHys qo'shilmadi', e); }
4527       _d1Blob(blob, 'image/png', 'naqsh_'+P.type+'-'+w+'x'+h+'px_'+dpi+'dpi.png',
4528         'PNG yuklab olindi □ – ' + w + 'x' + h + ' px, ' + dpi + ' dpi, ' +
4529         (w/dpi*25.4).toFixed(0) + 'x' + (h/dpi*25.4).toFixed(0) + ' mm');
4530       if(cb) cb();
4531     }, 'image/png');
4532   } else {
4533     const dataURL = off.toDataURL('image/png');
4534     const a = document.createElement('a');
4535     a.href = dataURL; a.download = 'naqsh_'+P.type+'.png';
4536     document.body.appendChild(a); a.click();
4537     setTimeout(()=>{ if(a.parentNode) a.parentNode.removeChild(a); }, 500);
4538     if(cb) cb();
4539   }
4540 }
4541
4542 /* ----- Eksport modal ----- */
4543 /* =====
4544 SANOAT EKSPORTI – Jakkard to'quv dastgohlari va lazer kesish apparatlari
4545 -----
4546 Jakkard uchun eng muhim shart: naqsh RAPPORT (repeat) bo'lishi, ya'ni
4547 chetlari bir-biriga aynan tutashishi kerak. Aks holda mato bo'ylab har
4548 takrorlanishda UZILISH chizig'i paydo bo'ladi.
4549
4550 Bu yerdagi yechim uzilishni "yashirmaydi" – matematik jihatdan yo'q qiladi:
4551 kanvas o'lchami panjaraning ANIQ DAVRIGA (period) teng qilib olinadi va
4552 naqsh barcha qo'shni panjara tugunlarida chiziladi. Natijada tasvir
4553 ta'rifi bo'yicha davriy bo'ladi – chetlari kafolatlangan holda tutashadi.
4554
4555 Simmetriya: p4 / p4m / p6 / p6m (Fyodorov devor guruhlari).
4556 p4, p4m → to'g'ri burchakli rapport c × c
4557 p6, p6m → to'g'ri burchakli rapport c × c√3
4558 ===== */
4559
4560 /* Panjara davriga teng, chok-suz rapport kanvasi.
4561 Qaytadi: {canvas, w, h} – canvas o'zi bitta to'liq rapport. */
4562 /* Rapport asosining o'lchami.
4563 c JUFT olinadi, geksagonalda H0 – c·√3 ga eng yaqin JUFT son.
4564 Sababi o'lchangan: H0 toq bo'lsa panjaraning yarim qadami (c/2, H0/2)
4565 yarim pikselga tushadi va geksagonal yarim-davr simmetriyasi buziladi –
4566 c=192 (H0=333) da mos kelmaslik 4.3–6.5% edi, H0 juft bo'lgan
4567 c=128 (222) va c=768 (1330) da esa 0.0–0.2%. Eng yaqin juft songa
4568 yaxlitlash tomonlar nisbatini atigi ≤0.2% o'zgartiradi, lekin har bir
4569 ilmoq butun pikselga tushadi – Jakkardda aynan shu muhim. */
4570 function rapportBase(px, hex){

```

```

4571 let c = Math.max(16, px|0);
4572 if(c & 1) c++;
4573 let H0 = c;
4574 if(hex){
4575     H0 = 2*Math.round(c*Math.sqrt(3)/2);
4576     if(H0 < 16) H0 = 16;
4577 }
4578 return { c, H0 };
4579 }
4580
4581 /* HAQIQIY KO'ZGULI RAPPORT (p4m / p6m).
4582   Ilgari ko'zgu panjaraning navbatdagi tugunlariga qo'llanardi — natijada
4583   rapportda aniq ko'zgu o'qi HOSIL BO'LMASDI (o'lchovda eng mos o'q ham
4584   o'rtacha 12% farq berardi, ya'ni "ko'zguli" yorlig'i haqiqatga mos
4585   emasdi). To'g'ri usul — asos plitkani o'z chetlariga nisbatan aks
4586   ettirish:
4587       R = [ T   Tx ]      Tx — x bo'yicha akslangan T
4588           [ Ty  Txy ]      Ty — y bo'yicha akslangan T
4589   Bunda R ning oxirgi ustuni aynan birinchi ustuniga teng
4590   (R[2W-1] = T[0] = R[0]), demak chok MATEMATIK jihatdan nol, ko'zgu
4591   o'qlari esa aniq. To'quvda bu "ko'zguli rapport" deb ataladi va
4592   uzilishni butunlay yo'q qiladi. */
4593 function mirrorTile(t){
4594     const W = t.w, H = t.h;
4595     const cv = document.createElement('canvas');
4596     cv.width = W*2; cv.height = H*2;
4597     const c2 = cv.getContext('2d');
4598     if(!c2) return t;
4599     c2.imageSmoothingEnabled = false;
4600     const quad = (a, d, e, f)=>{ /* setTransform(a,0,0,d,e,f) */
4601         c2.setTransform(a, 0, 0, d, e, f);
4602         c2.drawImage(t.canvas, 0, 0);
4603     };
4604     quad( 1, 1, 0, 0); /* T */
4605     quad(-1, 1, W*2, 0); /* Tx */
4606     quad( 1, -1, 0, H*2); /* Ty */
4607     quad(-1, -1, W*2, H*2); /* Txy */
4608     c2.setTransform(1, 0, 0, 1, 0, 0);
4609     return { canvas: cv, w: W*2, h: H*2 };
4610 }
4611
4612 /* Rapport o'lchamini CHIZMASDAN oldindan hisoblash — interfeys paneli
4613   foydalanuvchiga aynan fayldagi raqamni ko'rsatishi uchun.
4614   (Ilgari panel har doim nazariy cxc ni ko'rsatardi, girih oilasida esa
4615   fayl butunlay boshqa o'lchamda chiqardi — masalan panel "192x333"
4616   deganda fayl 219x253 bo'lardi. Dastgoh sozlashda bu jiddiy xato.) */
4617 function rapportSize(px, group){
4618     const g = GRIDS[group] || GRIDS.p4;
4619     const { c, H0 } = rapportBase(px, g.hex);
4620     let W = c, H = H0;
4621     if(typeof PIC_PATTERN !== 'undefined' && PIC_PATTERN[P.type] && !picIsRadial(picTileOf(P.type))){
4622         try{
4623             const t0 = picSeamlessTile({ ...P, tile: picTileOf(P.type) }, Math.max(24, Math.round(c/3)), 1);
4624             if(t0 && t0.w > 2 && t0.h > 2){
4625                 W = Math.round(t0.w*Math.max(1, Math.round(W/t0.w)));
4626                 H = Math.round(t0.h*Math.max(1, Math.round(H/t0.h)));
4627             }
4628         }catch(e){ console.warn('[rapport] o'lcham bashorati', e); }
4629     }
4630     if(g.mirror){ W *= 2; H *= 2; }
4631     return { w:W, h:H };
4632 }
4633
4634 function makeSeamlessRepeatCore(px, group, opts){
4635     opts = opts || {};
4636     const g = GRIDS[group] || GRIDS.p4;
4637     const hex = !!g.hex;
4638     /* Ko'zgu bu yerda QO'LLANMAYDI: asos plitka har doim sof siljish
4639     panjarasida chiziladi, ko'zguli rapportni makeSeamlessRepeat()
4640     mirrorTile() orqali yasaydi — shunda ko'zgu o'qi aniq chiqadi. */
4641     const base = rapportBase(px, hex);
4642     const c = base.c;
4643
4644     /* ---- RAPPORT O'LCHAMI -----
4645     Sof siljish (p4/p6): rapport = panjaraning bitta davri.
4646     p4 → c × c           p6 → c × round(c√3)
4647     Ko'zguli guruhlar (p4m/p6m) shu asos plitkani aks ettirish yo'li bilan
4648     ikki barobar katta rapport beradi — mirrorTile() ga qarang. */
4649     const H0 = base.H0;
4650     const W = c, H = H0;
4651
4652     const cv = document.createElement('canvas');
4653     cv.width = W; cv.height = H;
4654     const ctx = cv.getContext('2d');
4655     if(!ctx) return null;
4656
4657     /* Naqsh O'Z rangida chiziladi; 1-bitga aylantirish keyin yorqinlik
4658     bo'yicha (0tsu) bajariladi. Ilgari bu yerda barcha palitra uyalari
4659     qora qilingandi — natijada to'ldirishga asoslangan naqshlar
4660     (Voronoi, Zellige...) butunlay qora blokka aylanib qolardi. */
4661     const savePalObj = PAL[P.pal] || PAL.gold;

```

```

4662 const pal = PAL[P.pal] || PAL.gold;
4663 P._bg = pal.bg;
4664 ctx.fillStyle = pal.bg;
4665 ctx.fillRect(0, 0, W, H);
4666
4667 /* ---- PANJARA VEKTORLARI -----
4668 Geksagonalda vertikal qadam BUTUN H0/2 dan olinadi (c*√3/2 emas):
4669 c*√3 kasr son, uni piksel to'riga yaxlitlash davriylikni buzadi va
4670 chetda ~0.4 piksellik siljish — ko'zga tashlanadigan chiziqli beradi.
4671 H0 dan kelib chiqilsa 2*v2 - v1 = (0, H0) aynan butun bo'ladi. */
4672 const v1 = [c, 0];
4673 const v2 = hex ? [c/2, H0/2] : [0, c];
4674
4675 /* ---- PIC / MAYDON NAQSHLARI -----
4676 Girih oilasi va boshqa "maydon" turlari har nusxada O'Z ORQA FONINI
4677 (to'la doira) chizadi. Panjarada nusxalar ustma-ust tushgani uchun
4678 keyingi doira oldingisining lentalarini O'CHIRIB yuboradi — natijada
4679 bir tekis rangli blok chiqadi. Shuning uchun ular medalyon nusxalari
4680 sifatida emas, UZLUKSIZ MAYDON sifatida chiziladi. */
4681 if(typeof PIC_PATTERN !== 'undefined' && PIC_PATTERN[P.type] && !picIsRadial(picTileOf(P.type))){
4682   const pp = { ...P, tile: picTileOf(P.type) };
4683   /* bir davrli aniq plitka — matematik jihatdan chok-suz */
4684   const unit0 = Math.max(24, Math.round(c/3));
4685   const t0 = picSeamlessTile(pp, unit0, 1);
4686   if(t0 && t0.w > 2 && t0.h > 2){
4687     /* rapportni davrning BUTUN karralisiga yaxlitlaymiz — shundagina
4688     takrorlash chetlarida siljish bo'lmaydi */
4689     const nx = Math.max(1, Math.round(W / t0.w));
4690     const ny = Math.max(1, Math.round(H / t0.h));
4691     const W2 = Math.round(t0.w*nx), H2 = Math.round(t0.h*ny);
4692     cv.width = W2; cv.height = H2;
4693     const c2 = cv.getContext('2d');
4694     c2.fillStyle = pal.bg; c2.fillRect(0, 0, W2, H2);
4695     const pat = c2.createPattern(t0.canvas, 'repeat');
4696     if(pat){ c2.fillStyle = pat; c2.fillRect(0, 0, W2, H2); }
4697     PAL[P.pal] = savePalObj;
4698     return { canvas: cv, w: W2, h: H2 };
4699   }
4700 }
4701 /* Maydon turlari uchun uzluksiz-maydon rejimi — LEKIN u har doim ham
4702 yaxshi natija bermaydi (ba'zilar panjara nusxalari bilan chiroyliroq
4703 chiqadi). Shuning uchun bu yerda majburlamaymiz: pastdagi
4704 makeSeamlessRepeat() o'rovchisi ikkala usulni sinab, o'lchab,
4705 yaxshirog'ini tanlaydi (opts.field bayrog'i orqali). */
4706 if(opts.field && typeof FIELD_TESS !== 'undefined' && FIELD_TESS[P.type]){
4707   const nCell = Math.max(1, Math.round(c/128));
4708   const cellpx = W/nCell;
4709   const fp = { ...P, _field:true, _cellpx:cellpx, lay:1 };
4710   ctx.save();
4711   ctx.translate(W/2, H/2);
4712   drawPattern(ctx, 0, 0, Math.hypot(W,H)/2*1.02, { ...fp, rot:0 });
4713   ctx.restore();
4714   PAL[P.pal] = savePalObj;
4715   return { canvas: cv, w: W, h: H };
4716 }
4717
4718 const Rt = c*(hex ? 0.64 : 0.76);
4719 const tp = { ...P, lay: Math.min(2, P.lay) };
4720 if(typeof HEAVY_TESS !== 'undefined' && HEAVY_TESS[P.type]) tp.cmp = Math.min(3, tp.cmp);
4721 /* Siyrak naqshlar zichroq joylashtiriladi. Qadam 1/BUTUN bo'lishi shart —
4722 aks holda quyi panjara rapport davriga mos tushmaydi. 1/3 da rapport
4723 ichiga aynan 3 (ko'zguli holatda 6) qadam sig'adi. */
4724 const K = (typeof SPARSE_TESS !== 'undefined' && SPARSE_TESS[P.type]) ? 3 : 1;
4725 const w1 = [v1[0]/K, v1[1]/K], w2 = [v2[0]/K, v2[1]/K];
4726
4727 const saveNoring = (typeof NORING !== 'undefined') ? NORING : 0;
4728 NORING = Rt*0.84;
4729
4730 /* Kanvasni (+ chekka xaxira) qoplaydigan barcha tugunlarni chizamiz.
4731 Kanvas o'lchami panjara davriga teng bo'lgani uchun natija ta'rifi
4732 bo'yicha davriy — chetlari kafolatlangan holda tutashadi. */
4733 const marg = Rt*1.7;
4734 const span = Math.ceil((Math.max(W, H) + marg*2) / (c/K)) + 3;
4735 for(let n = -span; n <= span; n++){
4736   for(let m = -span; m <= span; m++){
4737     const tx = n*w1[0] + m*w2[0];
4738     const ty = n*w1[1] + m*w2[1];
4739     if(tx < -marg || tx > W + marg) continue;
4740     if(ty < -marg || ty > H + marg) continue;
4741     ctx.save();
4742     ctx.translate(tx, ty);
4743     drawPattern(ctx, 0, 0, Rt, tp);
4744     ctx.restore();
4745   }
4746 }
4747 NORING = saveNoring;
4748 PAL[P.pal] = savePalObj;
4749 return { canvas: cv, w: W, h: H };
4750 }
4751
4752 /* Rapport generatori — o'rovchi.

```

```

4753 Ba'zi naqshlar panjara nusxalari bilan, ba'zilar uzluksiz maydon bilan
4754 yaxshiroq chiqadi. Taxmin qilish o'rniga ikkalasini ham chizib, natijani
4755 O'LCHAYMIZ: chok-suzlik va 1-bitdagi to'ldirish darajasi bo'yicha
4756 yaroqlisini tanlaymiz. Shu bois yangi naqsh turi qo'shilganda ham
4757 eksport o'z-o'zidan to'g'ri ishlaydi. */
4758 function makeSeamlessRepeat(px, group, opts){
4759     opts = opts || {};
4760     const cans = [];
4761     const tryOne = (o)=>{
4762         try{
4763             const t = makeSeamlessRepeatCore(px, group, o);
4764             if(!t) return;
4765             const v = verifySeamless(t.canvas);
4766             const b = canvasToBilevel(t.canvas, null, false);
4767             let ink = 0;
4768             for(let i = 0; i < b.grid.length; i++) ink += b.grid[i];
4769             const pct = 100*ink/b.grid.length;
4770             /* deyarli bo'sh yoki deyarli to'la = yaroqsiz (naqsh ko'rinmaydi) */
4771             const degenerate = (pct < 1 || pct > 99);
4772             cans.push({ t, ok:v.ok, degenerate, pct, score:(v.ok?0:2) + (degenerate?4:0) });
4773         }catch(e){ console.error(['rapport', e]); }
4774     };
4775     tryOne({ ...opts, field:false });
4776     if(typeof FIELD_TESS !== 'undefined' && FIELD_TESS[P.type]) tryOne({ ...opts, field:true });
4777     if(!cans.length) return null;
4778     cans.sort((a,b)=> a.score - b.score);
4779     const g = GRIDS[group] || GRIDS.p4;
4780     /* ko'zguli guruhlarda asos plitka aks ettiriladi – chok aynan nol */
4781     return g.mirror ? mirrorTile(cans[0].t) : cans[0].t;
4782 }
4783
4784 /* Rapportning haqiqatan chok-suz ekanini TEKSHIRISH.
4785 Chap/o'ng va yuqori/pastki chetlarni bir piksel siljitib solishtiradi:
4786 davriy tasvirda ular aynan mos tushishi shart. */
4787 function verifySeamless(cv){
4788     const ctx = cv.getContext('2d');
4789     if(!ctx) return { ok:false, reason:'context yo\'q' };
4790     const W = cv.width, H = cv.height;
4791     const D = ctx.getImageData(0, 0, W, H).data;
4792     const px = (x,y)=> ((y*W + x) << 2);
4793     const dPix = (i,j)=> Math.abs(D[i]-D[j]) + Math.abs(D[i+1]-D[j+1]) + Math.abs(D[i+2]-D[j+2]);
4794
4795     /* Davriy tasvirda x=W ustuni x=0 ga teng. Demak tiling'da x=W-1 va x=0
4796     00'SHNI piksellar bo'ladi – ular orasidagi farq tasvirning odatdagi
4797     qo'shni piksellar farqidan oshmasligi kerak.
4798     Etalon sifatida BUTUN tasvir bo'yicha o'rtacha qo'shni farq olinadi
4799     (bitta ustun juftligi tasodifan tekis joyga tushib, noto'g'ri
4800     "uzilish" xulosasini berishi mumkin). */
4801     let sumX = 0, sumY = 0, nX = 0, nY = 0;
4802     for(let y = 0; y < H; y++){
4803         for(let x = 0; x + 1 < W; x++){ sumX += dPix(px(x,y), px(x+1,y)); nX++; }
4804     }
4805     for(let y = 0; y < H; y++){
4806         for(let x = 0; x < W; x++){ sumY += dPix(px(x,y), px(x,y+1)); nY++; }
4807     }
4808     const baseX = sumX/Math.max(1,nX)/3, baseY = sumY/Math.max(1,nY)/3;
4809
4810     let eX = 0, eY = 0;
4811     for(let y = 0; y < H; y++) eX += dPix(px(W-1,y), px(0,y));
4812     for(let x = 0; x < W; x++) eY += dPix(px(x,H-1), px(x,0));
4813     const edgeX = eX/H/3, edgeY = eY/W/3;
4814
4815     /* chet farqi o'rtacha qo'shni farqdan 2 barobardan ko'p oshmasligi kerak */
4816     const okX = edgeX <= Math.max(4, baseX*2 + 2);
4817     const okY = edgeY <= Math.max(4, baseY*2 + 2);
4818     return { ok: okX && okY, edgeX:+edgeX.toFixed(2), edgeY:+edgeY.toFixed(2),
4819             baseX:+baseX.toFixed(2), baseY:+baseY.toFixed(2) };
4820 }
4821
4822 /* Kanvasni 1-bitli (ko'tarilgan/tushirilgan ip) to'rga aylantirish.
4823 Jakkarda har piksel = bitta ilmoq: 1 = tanlangan (naqsh), 0 = fon. */
4824 /* Otsu usuli – gistogrammani ikkiga ajratuvchi ENG YAXSHI chegarani
4825 avtomatik topadi (sinflararo dispersiyani maksimallashtiradi).
4826 Shu tufayli har qanday palitra va naqsh turida struktura saqlanadi. */
4827 function otsuThreshold(lum){
4828     const hist = new Uint32Array(256);
4829     for(let i = 0; i < lum.length; i++) hist[lum[i]]++;
4830     const total = lum.length;
4831     let sum = 0;
4832     for(let t = 0; t < 256; t++) sum += t*hist[t];
4833     let sumB = 0, wB = 0, best = 0, thr = 128;
4834     for(let t = 0; t < 256; t++){
4835         wB += hist[t];
4836         if(!wB) continue;
4837         const wF = total - wB;
4838         if(!wF) break;
4839         sumB += t*hist[t];
4840         const mB = sumB/wB, mF = (sum - sumB)/wF;
4841         const between = wB*wF*(mB - mF)*(mB - mF);
4842         if(between > best){ best = between; thr = t; }
4843     }

```

```

4844     return thr;
4845 }
4846 function canvasToBilevel(cv, threshold, invert){
4847     const ctx = cv.getContext('2d');
4848     const W = cv.width, H = cv.height;
4849     const d = ctx.getImageData(0, 0, W, H).data;
4850     const lum = new Uint8Array(W*H);
4851     for(let i = 0, p = 0; i < d.length; i += 4, p++){
4852         lum[p] = (0.299*d[i] + 0.587*d[i+1] + 0.114*d[i+2]) | 0;
4853     }
4854     /* threshold berilmasa – avtomatik (Otsu) */
4855     const th = (threshold == null || threshold <= 0) ? otsuThreshold(lum) : threshold;
4856     const grid = new Uint8Array(W*H);
4857     for(let p = 0; p < lum.length; p++){
4858         let on = lum[p] < th ? 1 : 0;    /* qorong'i piksel = ip ko'tarilgan */
4859         if(invert) on = on ? 0 : 1;
4860         grid[p] = on;
4861     }
4862     return { grid, w:W, h:H, threshold:th };
4863 }
4864
4865 /* -----
4866     TIFF (baseline, siqilmagan) – Jakkard CAD tizimlarining asosiy formati
4867     1-bit (bilevel), 8-bit kulrang yoki 24-bit RGB
4868     ----- */
4869 function encodeTIFF(w, h, opts){
4870     opts = opts || {};
4871     const mode = opts.mode || 'bilevel';    /* bilevel | gray | rgb */
4872     const dpi = opts.dpi || 300;
4873     let spp, bps, photo, raw;
4874
4875     if(mode === 'rgb'){
4876         spp = 3; bps = [8,8,8]; photo = 2;
4877         raw = opts.rgb;    /* Uint8Array w*h*3 */
4878     } else if(mode === 'gray'){
4879         spp = 1; bps = [8]; photo = 1;    /* BlackIsZero */
4880         raw = opts.gray;    /* Uint8Array w*h */
4881     } else {
4882         spp = 1; bps = [1]; photo = 0;    /* WhiteIsZero: 0=oq, 1=qora */
4883         const rowBytes = (w + 7) >> 3;
4884         raw = new Uint8Array(rowBytes*h);
4885         const g = opts.grid;
4886         for(let y = 0; y < h; y++){
4887             for(let x = 0; x < w; x++){
4888                 if(g[y*w + x]) raw[y*rowBytes + (x >> 3)] |= (0x80 >> (x & 7));
4889             }
4890         }
4891     }
4892     const rowBytes = (mode === 'bilevel') ? ((w + 7) >> 3) : w*spp;
4893     const stripBytes = rowBytes*h;
4894
4895     /* teglar (tag raqami bo'yicha o'sish tartibida bo'lishi SHART) */
4896     const tags = [];
4897     const T = { SHORT:3, LONG:4, RATIONAL:5 };
4898     const add = (tag, type, count, value) => tags.push({tag, type, count, value});
4899
4900     add(256, T.LONG, 1, w);    /* ImageWidth */
4901     add(257, T.LONG, 1, h);    /* ImageLength */
4902     add(258, T.SHORT, spp, bps);    /* BitsPerSample */
4903     add(259, T.SHORT, 1, 1);    /* Compression = none */
4904     add(262, T.SHORT, 1, photo);    /* PhotometricInterpretation */
4905     add(273, T.LONG, 1, 0);    /* StripOffsets – keyin to'ldiriladi */
4906     add(277, T.SHORT, 1, spp);    /* SamplesPerPixel */
4907     add(278, T.LONG, 1, h);    /* RowsPerStrip */
4908     add(279, T.LONG, 1, stripBytes);    /* StripByteCounts */
4909     add(282, T.RATIONAL, 1, [dpi, 1]);    /* XResolution */
4910     add(283, T.RATIONAL, 1, [dpi, 1]);    /* YResolution */
4911     add(296, T.SHORT, 1, 2);    /* ResolutionUnit = inch */
4912
4913     const n = tags.length;
4914     const ifdSize = 2 + n*12 + 4;
4915     let extra = 0;
4916     const typeSize = t => (t === T.SHORT ? 2 : t === T.RATIONAL ? 8 : 4);
4917     tags.forEach(t=>{
4918         const bytes = typeSize(t.type)*t.count;
4919         if(bytes > 4) extra += bytes + (bytes & 1);
4920     });
4921     const ifdOff = 8;
4922     const extraOff = ifdOff + ifdSize;
4923     const dataOff = extraOff + extra;
4924     const buf = new ArrayBuffer(dataOff + stripBytes);
4925     const dv = new DataView(buf);
4926     const u8 = new Uint8Array(buf);
4927
4928     /* sarlavha: little-endian */
4929     u8[0] = 0x49; u8[1] = 0x49;
4930     dv.setUint16(2, 42, true);
4931     dv.setUint32(4, ifdOff, true);
4932     dv.setUint16(ifdOff, n, true);
4933
4934     let ePos = extraOff;

```

```

4935 tags.forEach((t, i)=>{
4936   const off = ifd0ff + 2 + i*12;
4937   if(t.tag === 273) t.value = data0ff; /* StripOffsets */
4938   dv.setUint16(off, t.tag, true);
4939   dv.setUint16(off+2, t.type, true);
4940   dv.setUint32(off+4, t.count, true);
4941   const bytes = typeSize(t.type)*t.count;
4942   if(bytes <= 4){
4943     if(t.type === T.SHORT){
4944       const arr = Array.isArray(t.value) ? t.value : [t.value];
4945       arr.forEach((v,k)=> dv.setUint16(off+8+k*2, v, true));
4946     } else {
4947       dv.setUint32(off+8, t.value, true);
4948     }
4949   } else {
4950     dv.setUint32(off+8, ePos, true);
4951     if(t.type === T.RATIONAL){
4952       dv.setUint32(ePos, t.value[0], true);
4953       dv.setUint32(ePos+4, t.value[1], true);
4954       ePos += 8;
4955     } else if(t.type === T.SHORT){
4956       t.value.forEach((v,k)=> dv.setUint16(ePos+k*2, v, true));
4957       ePos += bytes + (bytes & 1);
4958     }
4959   }
4960 });
4961 dv.setUint32(ifd0ff + 2 + n*12, 0, true); /* keyingi IFD yo'q */
4962 u8.set(raw, data0ff);
4963 return u8;
4964 }
4965
4966 /* -----
4967    BMP — 1-bit yoki 24-bit (ko'p Jakkard CAD dasturlari o'qiydi)
4968    ----- */
4969 function encodeBMP(w, h, opts){
4970   opts = opts || {};
4971   const mode = opts.mode || 'bilevel';
4972   const dpi = opts.dpi || 300;
4973   const ppm = Math.round(dpi/0.0254);
4974   const bpp = (mode === 'rgb') ? 24 : 1;
4975   const rowRaw = (mode === 'rgb') ? w*3 : ((w + 7) >> 3);
4976   const rowPad = (rowRaw + 3) & ~3; /* qatorlar 4 baytga tekislanadi */
4977   const palBytes = (bpp === 1) ? 8 : 0;
4978   const data0ff = 14 + 40 + palBytes;
4979   const size = data0ff + rowPad*h;
4980   const buf = new ArrayBuffer(size);
4981   const dv = new DataView(buf);
4982   const u8 = new Uint8Array(buf);
4983
4984   u8[0] = 0x42; u8[1] = 0x4D; /* 'BM' */
4985   dv.setUint32(2, size, true);
4986   dv.setUint32(10, data0ff, true);
4987   dv.setUint32(14, 40, true); /* BITMAPINFOHEADER */
4988   dv.setInt32(18, w, true);
4989   dv.setInt32(22, h, true); /* musbat = pastdan yuqoriga */
4990   dv.setUint16(26, 1, true);
4991   dv.setUint16(28, bpp, true);
4992   dv.setUint32(30, 0, true); /* siqilmagan */
4993   dv.setUint32(34, rowPad*h, true);
4994   dv.setUint32(38, ppm, true);
4995   dv.setUint32(42, ppm, true);
4996   if(bpp === 1){
4997     dv.setUint32(46, 2, true); /* palitrada 2 rang */
4998     /* 0 = oq (fon), 1 = qora (naqsh) — BGRA */
4999     u8[54]=255; u8[55]=255; u8[56]=255; u8[57]=0;
5000     u8[58]=0; u8[59]=0; u8[60]=0; u8[61]=0;
5001   }
5002   const g = opts.grid, rgb = opts.rgb;
5003   for(let y = 0; y < h; y++){
5004     const srcY = h - 1 - y; /* BMP teskari tartibda */
5005     let o = data0ff + y*rowPad;
5006     if(bpp === 1){
5007       for(let x = 0; x < w; x++){
5008         if(g[srcY*w + x]) u8[o + (x >> 3)] |= (0x80 >> (x & 7));
5009       }
5010     } else {
5011       for(let x = 0; x < w; x++){
5012         const s = (srcY*w + x)*3;
5013         u8[o++] = rgb[s+2]; u8[o++] = rgb[s+1]; u8[o++] = rgb[s]; /* BGR */
5014       }
5015     }
5016   }
5017   return u8;
5018 }
5019
5020 /* -----
5021    WIF — Weaving Information File (dastgoh drafti)
5022    Piksel to'ridan haqiqiy draft chiqariladi: bir xil ustunlar → bitta ramka
5023    (shaft), bir xil qatorlar → bitta pedal (treadle), kesishmasi → tie-up.
5024    Bu ajratish har doim ANIQ: bir xil ustunlar ta'rifi bo'yicha bir xil
5025    ko'tariladi, shuning uchun tie-up ziddiyatsiz chiqadi.

```



```

5026 ----- */
5027 function buildWIF(grid, w, h, meta){
5028   meta = meta || {};
5029   const colKey = new Array(w), rowKey = new Array(h);
5030   for(let x = 0; x < w; x++){
5031     let s = '';
5032     for(let y = 0; y < h; y++) s += grid[y*w + x] ? '1' : '0';
5033     colKey[x] = s;
5034   }
5035   for(let y = 0; y < h; y++){
5036     let s = '';
5037     for(let x = 0; x < w; x++) s += grid[y*w + x] ? '1' : '0';
5038     rowKey[y] = s;
5039   }
5040   const shafts = [], shaftIx = {}, treadles = [], treadleIx = {};
5041   colKey.forEach(k=>{ if(!(k in shaftIx)){ shaftIx[k] = shafts.length; shafts.push(k); } });
5042   rowKey.forEach(k=>{ if(!(k in treadleIx)){ treadleIx[k] = treadles.length; treadles.push(k); } });
5043
5044   const nS = shafts.length, nT = treadles.length;
5045   const pad2 = n => (n < 10 ? '0'+n : ''+n);
5046   const d = new Date();
5047   const dateStr = pad2(d.getDate())+'.'+pad2(d.getMonth()+1)+'.'+d.getFullYear();
5048
5049   let s = '';
5050   s += '[WIF]\n';
5051   s += 'Version=1.1\n';
5052   s += 'Date=' + dateStr + '\n';
5053   s += 'Developers=NaqshAI\n';
5054   s += 'Source Program=NaqshAI\n';
5055   s += 'Source Version=27.1\n';
5056
5057   s += '[CONTENTS]\n';
5058   s += 'COLOR PALETTE=yes\nTEXT=yes\nWEAVING=yes\nWARP=yes\nWEFT=yes\n';
5059   s += 'COLOR TABLE=yes\nTHREADING=yes\nTIEUP=yes\nTREADLING=yes\n\n';
5060
5061   s += '[TEXT]\n';
5062   s += 'Title=' + (meta.title || 'NaqshAI naqsh') + '\n';
5063   s += 'Author=NaqshAI\n\n';
5064
5065   s += '[WEAVING]\n';
5066   s += 'Rising Shed=yes\n';
5067   s += 'Treadles=' + nT + '\n';
5068   s += 'Shafts=' + nS + '\n\n';
5069
5070   s += '[COLOR PALETTE]\nEntries=2\nForm=RGB\nRange=0,255\n\n';
5071   s += '[COLOR TABLE]\nn1=255,255,255\nn2=0,0,0\n\n';
5072
5073   s += '[WARP]\nThreads=' + w + '\nUnits=Decipoints\nColor=2\n\n';
5074   s += '[WEFT]\nThreads=' + h + '\nUnits=Decipoints\nColor=1\n\n';
5075
5076   s += '[THREADING]\n';
5077   for(let x = 0; x < w; x++) s += (x+1) + '=' + (shaftIx[colKey[x]] + 1) + '\n';
5078   s += '\n[TREADLING]\n';
5079   for(let y = 0; y < h; y++) s += (y+1) + '=' + (treadleIx[rowKey[y]] + 1) + '\n';
5080
5081   s += '\n[TIEUP]\n';
5082   for(let t = 0; t < nT; t++){
5083     const rk = treadles[t];
5084     const lifted = [];
5085     for(let sIdx = 0; sIdx < nS; sIdx++){
5086       /* shu ramkaga tegishli istalgan ustunni olamiz */
5087       const x = colKey.indexOf(shafts[sIdx]);
5088       if(rk[x] === '1') lifted.push(sIdx + 1);
5089     }
5090     s += (t+1) + '=' + lifted.join(',') + '\n';
5091   }
5092   return { wif: s, shafts: nS, treadles: nT };
5093 }
5094
5095 /* -----
5096 LAZER: DXF va SVG – haqiqiy mm o'lchamida, yopiq konturlar bilan
5097 ----- */
5098 function buildLaserSVG(wMm, hMm, opts){
5099   opts = opts || {};
5100   const B = boardFor(wMm, hMm);
5101   const kerf = opts.kerf || 0.1; /* hairline – lazer dasturlari kesish deb tanidi */
5102   let out = '<?xml version="1.0" encoding="UTF-8"?>\n' +
5103     '<!-- NaqshAI – lazer kesish uchun. Qizil = kesish konturi. O\'lcham: ' +
5104     wMm + 'x' + hMm + ' mm -->\n' +
5105     '<svg xmlns="http://www.w3.org/2000/svg" width="' + wMm + 'mm" height="' + hMm + 'mm" ' +
5106     'viewBox="0 0 ' + num(B.bw) + ' ' + num(B.bh) + '">\n' +
5107     '<g id="kesish" stroke="#ff0000" stroke-width="' + (kerf/B.k).toFixed(3) + '" fill="none" ' +
5108     'stroke-linecap="round" stroke-linejoin="round">\n';
5109   /* Taxta ichidagi yo'llar Bezier ko'rinishida qoladi (CorelDraw'da
5110     tahrirlash uchun silliqroq), chegaradan chiqadiganlari qirqilgan
5111     poliliniya sifatida chiqadi – DXF bilan aynan bir xil detal bo'lsin. */
5112   for(const pl of laserPolys(wMm, hMm, opts.fill)){
5113     out += '<path d="M' + pl.pts.map(q => num(q[0]) + ' ' + num(q[1])).join(' L') +
5114       (pl.closed ? ' Z' : '') + '" />\n';
5115   }
5116   if(opts.frame){

```

```

5117     out += '<rect x="0" y="0" width="' + num(B.bw) + ' " height="' + num(B.bh) + '"/>\n';
5118   }
5119   out += '</g>\n</svg>';
5120   return out;
5121 }
5122
5123 const EXP = { format:'png', size:2048, dpi:300, trans:false,
5124               jqsize:512, jqgroup:'p4', jqdepth:1, mm:100, invert:false,
5125               fill:'tile' };
5126
5127 /* Format tanlanganda qaysi sozlama guruhlari ko'rinishini boshqaradi */
5128 function expSyncGroups(){
5129   const f = EXP.format;
5130   const isRaster = (f === 'png');
5131   const isJq = (f === 'tiff' || f === 'bmp' || f === 'wif');
5132   const isVector = (f === 'svg' || f === 'dxf' || f === 'eps');
5133   const show = (id, on) => { const el = $(id); if(el) el.style.display = on ? '' : 'none'; };
5134   show('grp-size', isRaster);
5135   show('grp-trans', isRaster);
5136   show('grp-jq', isJq);
5137   show('grp-mm', isVector);
5138   show('grp-fill', isRaster || isVector);
5139   /* WIF faqat 1-bit bo'la oladi – draft ip ko'tarilishidan iborat */
5140   const dep = $('exp-jqdepth');
5141   if(dep) dep.style.opacity = (f === 'wif') ? '.45' : '1';
5142   expJqInfo();
5143 }
5144
5145 /* Rapport haqida jonli ma'lumot – nechta ilmoq, taxminiy mato o'lchami */
5146 function expJqInfo(){
5147   const el = $('jq-info');
5148   if(!el) return;
5149   if(!(EXP.format === 'tiff' || EXP.format === 'bmp' || EXP.format === 'wif')){ el.textContent = ''; return; }
5150   const g = GRIDS[EXP.jqgroup] || GRIDS.p4;
5151   /* Fayldagi HAQIQIY o'lcham ko'rsatiladi (nazariy cxc emas):
5152   girih oilasida rapport plitka davriga yaxlitlanadi va nazariy
5153   qiymatdan farq qiladi. */
5154   const sz = rapportSize(EXP.jqsize, EXP.jqgroup);
5155   const W = sz.w, H = sz.h;
5156   const dens = 100; /* taxminiy: 100 ilmoq/dyuym */
5157   el.innerHTML = 'Rapport: <b>' + W + ' × ' + H + '</b> ilmoq' +
5158     (g.mirror ? ' (ko\'zguli rapport – asos plitka aks ettirilgan, 2x katta)' : '') +
5159     (g.hex ? ' (geksagonal: balandlik ≈ c·√3)' : '') +
5160     '<br>' + (W/dens*25.4).toFixed(0) + ' × ' + (H/dens*25.4).toFixed(0) + ' mm (100 ilmoq/dyuym zichlikda)' +
5161     (EXP.format === 'wif' ? '<br>WIF – ramka/pedal soni fayl yaratilganda hisoblanadi.' : '');
5162 }
5163
5164 function segWire(id, key, isNum){
5165   const box = $(id);
5166   if(!box) return;
5167   box.addEventListener('click', e=>{
5168     const b = e.target.closest('button');
5169     if(!b) return;
5170     box.querySelectorAll('button').forEach(x=>x.classList.remove('sel'));
5171     b.classList.add('sel');
5172     EXP[key] = isNum ? parseInt(b.dataset.v,10) : b.dataset.v;
5173     /* Format ikkita segmentga bo'lingan – biri tanlansa ikkinchisi bo'shatiladi */
5174     if(key === 'format'){
5175       const other = (id === 'exp-format') ? $('exp-format2') : $('exp-format');
5176       if(other) other.querySelectorAll('button').forEach(x=>x.classList.remove('sel'));
5177     }
5178     expSyncGroups();
5179   });
5180 }
5181
5182 /* Jakkard eksporti – chok-suz rapport, so'ng tanlangan formatga kodlash */
5183 async function exportJacquard(){
5184   const inv = $('exp-invert');
5185   EXP.invert = inv ? inv.checked : false;
5186   const depth = (EXP.format === 'wif') ? 1 : EXP.jqdepth;
5187   const tile = makeSeamlessRepeat(EXP.jqsize, EXP.jqgroup, {});
5188   if(!tile){ ntf('Rapport yaratib bo'lmadi', 'err'); return; }
5189
5190   /* Chok-suzlikni HAQIQATAN tekshiramiz – "ishonamiz" emas, o'lchaymiz */
5191   const chk = verifySeamless(tile.canvas);
5192   if(!chk.ok){
5193     console.warn('[jacquard] chetlar mos kelmadi', chk);
5194     ntf('Diqqat: rapport chetlarida farq aniqlandi – natijani tekshiring', 'err');
5195   }
5196
5197   const W = tile.w, H = tile.h;
5198   /* Interfeys paneli va fayl bir xil raqamni ko'rsatishi shart –
5199   farq chiqsa bu dasturiy xato, jimgina o'tkazib yubormaymiz. */
5200   const pred = rapportSize(EXP.jqsize, EXP.jqgroup);
5201   if(pred.w !== W || pred.h !== H)
5202     console.error('[jacquard] o'lcham bashorati mos emas', pred, {W, H});
5203   const base = 'naqsh_' + P.type + '_' + EXP.jqgroup + '_' + W + 'x' + H;
5204
5205   if(EXP.format === 'wif'){
5206     const { grid } = canvasToBilevel(tile.canvas, null, EXP.invert);
5207     const r = buildWIF(grid, W, H, { title: (getPattern(P.type)||{}).nm });

```

```

5208     _dlBlob(r.wif, 'text/plain', base + '.wif',
5209             'WIF yuklab olindi □ - ' + r.shafits + ' ramka, ' + r.treadles + ' pedal');
5210     if(r.shafits > 40){
5211         ntf('Eslatma: ' + r.shafits + ' ramka – bu Jakkard uchun normal, lekin oddiy dastgohda ko\'p', 'err');
5212     }
5213     closeExport();
5214     return;
5215 }
5216
5217 const ctx = tile.canvas.getContext('2d');
5218 const img = ctx.getImageData(0, 0, W, H).data;
5219 let opts = { dpi: EXP.dpi };
5220 if(depth === 1){
5221     opts.mode = 'bilevel';
5222     const bl = canvasToBilevel(tile.canvas, null, EXP.invert);
5223     opts.grid = bl.grid;
5224     /* Sifat nazorati: 1-bit natija butunlay bir rangga aylanib qolmasin */
5225     let ink = 0;
5226     for(let i = 0; i < bl.grid.length; i++) ink += bl.grid[i];
5227     const pct = 100*ink/bl.grid.length;
5228     if(pct < 0.5 || pct > 99.5){
5229         ntf('Diqqat: 1-bit natija deyarli bir rangli (' + pct.toFixed(1) + '%) – palitra yoki naqshni o\'zgartiring'
5230         , 'err');
5231     }
5232 } else if(depth === 8){
5233     opts.mode = 'gray';
5234     const gray = new Uint8Array(W*H);
5235     for(let i = 0, p = 0; i < img.length; i += 4, p++){
5236         gray[p] = Math.round(0.299*img[i] + 0.587*img[i+1] + 0.114*img[i+2]);
5237     }
5238     opts.gray = gray;
5239 } else {
5240     opts.mode = 'rgb';
5241     const rgb = new Uint8Array(W*H*3);
5242     for(let i = 0, p = 0; i < img.length; i += 4, p += 3){
5243         rgb[p] = img[i]; rgb[p+1] = img[i+1]; rgb[p+2] = img[i+2];
5244     }
5245     opts.rgb = rgb;
5246 }
5247 const bytes = (EXP.format === 'tiff') ? encodeTIFF(W, H, opts) : encodeBMP(W, H, opts);
5248 const ext = (EXP.format === 'tiff') ? 'tif' : 'bmp';
5249 const mime = (EXP.format === 'tiff') ? 'image/tiff' : 'image/bmp';
5250 _dlBlob(new Blob([bytes], {type:mime}), mime, base + '_' + depth + 'bit.' + ext,
5251         ext.toUpperCase() + ' yuklab olindi □ – chok-suz rapport ' + W + 'x' + H);
5252 closeExport();
5253 }
5254
5255 /* Raqamli maydonlardan o'lchamni o'qish – bo'sh yoki noto'g'ri bo'lsa
5256 segmentdagi tanlangan qiymatga qaytadi. */
5257 function expDims(){
5258     const rd = (id, def, lo, hi)=>{
5259         const el = $(id);
5260         if(!el) return def;
5261         const v = parseFloat(el.value);
5262         if(!isFinite(v)) return def;
5263         return Math.min(hi, Math.max(lo, Math.round(v)));
5264     };
5265     return {
5266         mmW: rd('exp-mm-w', EXP.mm, 5, 5000),
5267         mmH: rd('exp-mm-h', EXP.mm, 5, 5000),
5268         pxW: rd('exp-px-w', EXP.size, 16, 12000),
5269         pxH: rd('exp-px-h', EXP.size, 16, 12000),
5270         jqc: rd('exp-jq-c', EXP.jqsize, 32, 4096)
5271     };
5272 }
5273 function doExport(){
5274     const chk = $('exp-trans');
5275     EXP.trans = chk ? chk.checked : false;
5276     const D = expDims();
5277     if(EXP.format === 'tiff' || EXP.format === 'bmp' || EXP.format === 'wif'){
5278         EXP.jqsize = D.jqc;
5279         exportJacquard();
5280         return;
5281     }
5282     if(EXP.format === 'png'){
5283         exportPNG(D.pxW, D.pxH, EXP.dpi, EXP.trans, null, EXP.fill);
5284     } else if(EXP.format === 'svg'){
5285         const svg = buildLaserSVG(D.mmW, D.mmH, { frame:true, fill:EXP.fill });
5286         _dlBlob(svg, 'image/svg+xml', 'naqsh_'+P.type+'_'+D.mmW+'x'+D.mmH+'mm.svg',
5287             'SVG yuklab olindi □ - ' + D.mmW + 'x' + D.mmH + ' mm');
5288     } else if(EXP.format === 'eps'){
5289         const eps = buildEPS(D.mmW, D.mmH, { frame:true, fill:EXP.fill });
5290         _dlBlob(eps, 'application/postscript', 'naqsh_'+P.type+'_'+D.mmW+'x'+D.mmH+'mm.eps',
5291             'EPS yuklab olindi □ - ' + D.mmW + 'x' + D.mmH + ' mm');
5292     } else {
5293         const dxf = buildDXF(D.mmW, D.mmH, EXP.fill);
5294         _dlBlob(dxf, 'application/dxf', 'naqsh_'+P.type+'_'+D.mmW+'x'+D.mmH+'mm.dxf',
5295             'DXF (CNC/lazer) yuklab olindi □ - ' + D.mmW + 'x' + D.mmH + ' mm');
5296     }
5297     closeExport();

```

```

5298 }
5299 /* Raqamli maydonlarni EXP dan to'ldirish — oyna ochilganda ular hech qachon
5300 tanlangan tugmadan farq qilib qolmasin (ikki manba bo'lsa, qaysi biri
5301 fayl o'lchamini belgilagani noaniq bo'lib qoladi). */
5302 function expSyncInputs(){
5303   const set = (id, v)=>{ const el = $(id); if(el && el.value === '') el.value = String(v); };
5304   const force = (id, v)=>{ const el = $(id); if(el) el.value = String(v); };
5305   const mmw = $('exp-mm-w'), pxw = $('exp-px-w'), jqc = $('exp-jq-c');
5306   if(mmw && parseFloat(mmw.value) !== EXP.mm && !mmw.dataset.touched){
5307     force('exp-mm-w', EXP.mm); force('exp-mm-h', EXP.mm);
5308   }
5309   if(pxw && parseFloat(pxw.value) !== EXP.size && !pxw.dataset.touched){
5310     force('exp-px-w', EXP.size); force('exp-px-h', EXP.size);
5311   }
5312   if(jqc && parseFloat(jqc.value) !== EXP.jqsize && !jqc.dataset.touched){
5313     force('exp-jq-c', EXP.jqsize);
5314   }
5315   set('exp-mm-w', EXP.mm); set('exp-mm-h', EXP.mm);
5316   set('exp-px-w', EXP.size); set('exp-px-h', EXP.size);
5317   set('exp-jq-c', EXP.jqsize);
5318 }
5319 function openExport(){ expSyncInputs(); expSyncGroups(); const m = $('modal-export'); if(m) m.classList.add('open'
); }
5320 function closeExport(){ const m = $('modal-export'); if(m) m.classList.remove('open'); }
5321 /* =====
5322 3D KO'RISH (Three.js r128)
5323 ===== */
5324 let threeReady = false;
5325 let T3 = null; /* {scene,camera,renderer,group,mesh,mat,amb,hemi,key,fill,auto,kind} */
5326 const PH = { img:null, shape:'doira', size:45, frame:12, text:'', date:'' };
5327
5328 function latheFromProfile(profile, scale){
5329   const pts = profile.map(q=>new THREE.Vector2(q[0]*scale, q[1]*scale));
5330   return new THREE.LatheGeometry(pts, 96);
5331 }
5332 /* PLANAR UV — tepadan proyeksiya: naqsh idish markazida SIMMETRIK yotadi
5333 (LatheGeometry ning standart UV si teksturani aylana bo'ylab cho'zib yuboradi) */
5334 function planarUV(geo){
5335   geo.computeBoundingBox();
5336   const bb = geo.boundingBox;
5337   const size = Math.max(bb.max.x-bb.min.x, bb.max.z-bb.min.z) || 1;
5338   const pos = geo.attributes.position, uv = geo.attributes.uv;
5339   if(!pos || !uv) return geo;
5340   for(let i=0;i<pos.count;i++){
5341     uv.setXY(i, 0.5 + pos.getX(i)/size, 0.5 - pos.getZ(i)/size);
5342   }
5343   uv.needsUpdate = true;
5344   return geo;
5345 }
5346 /* =====
5347 ME'MORIY OB'YEKTLAR — ustun, darvoza, peshtoq
5348 -----
5349 Ko'p qismli guruhlar: har qism o'z materiali va o'z naqsh zichligi bilan.
5350
5351 NAQSH. Tekstura — HAQIQIY RAPPORT (makeSeamlessRepeat, chok-suzligi 160
5352 holatda o'lchangan) + RepeatWrapping. UV diapazoni geometriya turiga qarab
5353 har xil (brauzerda o'lchangan):
5354   Extrude / Shape → UV = DUNYO koordinatasi → repeat = 1/plitka
5355   Box / Cylinder / Lathe → UV = 0..1 → repeat = plitka SONI
5356   stripFromOutline → UV = (yoy uzunligi, chuqurlik)
5357 Aylanma yuzalarda u BUTUN son bo'lishi shart — aks holda buyum orqasida
5358 ulanish chizig'i chiqadi.
5359
5360 MIQYOS. Haqiqiy koshinkorlikda modul mayda: 4 birlik enli peshtoqda
5361 ~10 ta plitka. Shuning uchun asosiy maydon plitkasi TU = 0.40, bandlar esa
5362 o'z enidan kelib chiqadi (bandda aynan bitta plitka yotadi) — naqsh nafis
5363 va me'moriy o'lchamga mos chiqadi.
5364
5365 O'LCHOV. Har ob'yekt asosi LOKAL y = 0 da quriladi va guruh FLOOR_Y ga
5366 qo'yiladi, shunda buyum polda turadi.
5367 ===== */
5368 const ARCH_KINDS = {ustun:1, darvoza:1, peshtoq:1};
5369 const FLOOR_Y = -1.66;
5370 const TU = 0.40; /* asosiy maydon plitkasi (dunyo birligi) */
5371
5372 function archMat(kind, rx, ry, tint){
5373   let m;
5374   if(kind === 'stone'){
5375     m = new THREE.MeshStandardMaterial({color:0xd6c7a4, roughness:0.86, metalness:0.02});
5376   } else if(kind === 'metal'){
5377     m = new THREE.MeshStandardMaterial({color:0xa87c34, roughness:0.34, metalness:0.9});
5378   } else if(kind === 'dark'){
5379     m = new THREE.MeshStandardMaterial({color:0x2a1d10, roughness:0.85, metalness:0.05});
5380   } else if(kind === 'wood'){
5381     m = T3.matWood.clone();
5382   } else {
5383     m = T3.mat.clone();
5384   }
5385   m.side = THREE.FrontSide;
5386   if(kind === 'tile' || kind === 'wood'){
5387     if(tint !== undefined){

```

```

5388     m.color = new THREE.Color(tint);
5389     /* tinted yuzalar porlab ketmasin – chuqurlik sezilsin */
5390     if(m.emissive){ m.emissive = new THREE.Color(tint); m.emissiveIntensity = 0.05; }
5391   }
5392   m.userData.rep = [rx || 1, (ry === undefined ? rx : ry) || 1];
5393   T3.archMats.push(m);
5394 }
5395 return m;
5396 }
5397
5398 /* TO'RT MARKAZLI (Temuriylar) SINIQ RAVOQ.
5399    $(cx + hw)^2 = cx^2 + rise^2 \rightarrow cx = (rise^2 - hw^2)/(2 \cdot hw)$ .
5400   Ravoq SINIQ bo'lishi uchun rise > hw shart. */
5401 function archOutline(hw, spring, apex, n){
5402   n = n || 48;
5403   const rise = apex - spring;
5404   const cx = (rise*rise - hw*hw)/(2*hw);
5405   const R = cx + hw;
5406   const aEnd = Math.atan2(rise, -cx);
5407   const pts = [];
5408   for(let i = 0; i <= n; i++){
5409     const a = Math.PI + (aEnd - Math.PI)*(i/n);
5410     pts.push([cx + R*Math.cos(a), spring + R*Math.sin(a)]);
5411   }
5412   for(let i = n - 1; i >= 0; i--){
5413     const a = Math.PI + (aEnd - Math.PI)*(i/n);
5414     pts.push([-cx + R*Math.cos(a), spring + R*Math.sin(a)]);
5415   }
5416   return pts;
5417 }
5418
5419 /* Ravoq atrofidagi bo'rtma band (archivolt) shakli */
5420 function archBandShape(ow, spring, apex, bw, n){
5421   n = n || 44;
5422   const outer = archOutline(ow + bw, spring, apex + bw, n);
5423   const inner = archOutline(ow, spring, apex, n);
5424   const sh = new THREE.Shape();
5425   sh.moveTo(-(ow + bw), 0);
5426   for(const q of outer) sh.lineTo(q[0], q[1]);
5427   sh.lineTo(ow + bw, 0);
5428   sh.closePath();
5429   const hole = new THREE.Path();
5430   hole.moveTo(-ow, 0);
5431   for(const q of inner) hole.lineTo(q[0], q[1]);
5432   hole.lineTo(ow, 0);
5433   hole.closePath();
5434   sh.holes.push(hole);
5435   return sh;
5436 }
5437
5438 /* Kontur bo'ylab tunnel yuzasi (ravoq soffiti). UV = (yoy uzunligi, chuqurlik) */
5439 function stripFromOutline(pts, depth){
5440   const g = new THREE.BufferGeometry();
5441   const n = pts.length;
5442   const pos = [], uv = [], idx = [];
5443   let len = 0; const s = [0];
5444   for(let i = 1; i < n; i++){
5445     len += Math.hypot(pts[i][0]-pts[i-1][0], pts[i][1]-pts[i-1][1]);
5446     s.push(len);
5447   }
5448   for(let i = 0; i < n; i++){
5449     pos.push(pts[i][0], pts[i][1], 0);
5450     pos.push(pts[i][0], pts[i][1], -depth);
5451     uv.push(s[i], 0, s[i], depth);
5452   }
5453   for(let i = 0; i < n - 1; i++){
5454     const a = i*2, b = i*2+1, c = (i+1)*2, d = (i+1)*2+1;
5455     idx.push(a, b, c, b, d, c);
5456   }
5457   g.setAttribute('position', new THREE.Float32BufferAttribute(pos, 3));
5458   g.setAttribute('uv', new THREE.Float32BufferAttribute(uv, 2));
5459   g.setIndex(idx);
5460   g.computeVertexNormals();
5461   return g;
5462 }
5463
5464 /* MUQARNAS KATAGI – botiq chig'anoq (stalaktit).
5465   Ilgari kataklar oddiy PARALLELEPIPED edi va muqarnas "g'ishtlar uyumi"dek
5466   ko'rinaridi. Haqiqiy muqarnas kataklari – kichik yarim gumbazchalar;
5467   shuning uchun bu yerda sferaning bir bo'lagi ishlatiladi. */
5468 function muqCellGeo(r){
5469   return new THREE.SphereGeometry(r, 12, 8, -Math.PI/2, Math.PI, 0, Math.PI*0.62);
5470 }
5471
5472 /* -----
5473   MUQARNAS TOQCHASI – HAQIQIY KATAK GEOMETRIYASI
5474   -----
5475   Sfera bo'lagi tashqaridan qavariq MUNCHOQ bo'lib ko'rinaridi – muqarnas esa
5476   aksincha: har katak ichkariga O'YILGAN toqcha, og'zi siniq ravoq shaklida,
5477   ichi yarim gumbaz bilan yopiladi. Ustun boshiga qaralganda ko'z aynan
5478   shu qator-qator botiq toqchalarni va ular orasidagi tik qovurg'alarni

```

```

5479 ko'radi.
5480 Bu yerda katak parametrik yuza sifatida quriladi:
5481  $c = \cos(v \cdot \pi / 2)$ ,  $s = \sin(v \cdot \pi / 2)$ 
5482  $P(u, v) = (ox(u) \cdot c, yf + (oy(u) - yf) \cdot c, -d \cdot s)$ 
5483  $v = 0$  da yuza aynan og'iz konturi (siniq ravoq),  $v = 1$  da esa u bitta
5484 nuqtaga – toqcha tubiga yig'iladi. Bo'ylama kesim chorak aylana bo'lgani
5485 uchun tub silliq gumbaz bo'lib chiqadi, o'tkir burchak qolmaydi.
5486 UV:  $u$  – og'iz bo'ylab yoy uzunligi,  $v$  – chuqurlik. Shuning uchun naqsh
5487 toqcha ichiga oqib kiradi, chetida uzilmaydi. */
5488 function muqNicheGeo(hw, h, depth, uSeg, vSeg){
5489   uSeg = uSeg || 20; vSeg = vSeg || 6;
5490   const out = archOutline(hw, 0, h, Math.max(6, uSeg >> 1));
5491   const n = out.length;
5492   /* og'iz bo'ylab yoy uzunligi (UV uchun) */
5493   const sArr = [0];
5494   let tot = 0;
5495   for(let i = 1; i < n; i++){
5496     tot += Math.hypot(out[i][0]-out[i-1][0], out[i][1]-out[i-1][1]);
5497     sArr.push(tot);
5498   }
5499   const yf = h*0.54; /* gumbaz markazi */
5500   const pos = [], uv = [], idx = [];
5501   for(let j = 0; j <= vSeg; j++){
5502     const v = j/vSeg;
5503     const c = Math.cos(v*Math.PI/2), sn = Math.sin(v*Math.PI/2);
5504     for(let i = 0; i < n; i++){
5505       pos.push(out[i][0]*c, yf + (out[i][1]-yf)*c, -depth*sn);
5506       uv.push(sArr[i]/(tot || 1), 1 - v);
5507     }
5508   }
5509   for(let j = 0; j < vSeg; j++){
5510     for(let i = 0; i < n - 1; i++){
5511       const a = j*n + i, b = a + 1, c2 = a + n, d2 = c2 + 1;
5512       idx.push(a, c2, b, b, c2, d2);
5513     }
5514   }
5515   const g = new THREE.BufferGeometry();
5516   g.setAttribute('position', new THREE.Float32BufferAttribute(pos, 3));
5517   g.setAttribute('uv', new THREE.Float32BufferAttribute(uv, 2));
5518   g.setIndex(idx);
5519   g.computeVertexNormals();
5520   return g;
5521 }
5522 /* Toqcha og'zini o'rab turuvchi QOVURG'A (ravog yuzasi).
5523 Toqchalar yonma-yon terilganda ular orasida ingichka tik qovurg'a
5524 qoladi – muqarnasning o'sha tanish "asal uyasi" to'ri aynan shu.
5525 Qovurg'a ravog bandi shaklida chiqariladi. */
5526 function muqRibGeo(hw, h, bw, depth){
5527   /* Qovurg'a MAYDA detal – u ko'zga faqat toqchalar orasidagi chiziq
5528   bo'lib ko'rinadi. Shuning uchun kontur nuqtalari va qiyalik
5529   bosqichlari kam olinadi: bir qovurg'a ~1500 uchburchakdan ~400 ga
5530   tushdi, peshtoqda esa 78 ta qovurg'a bor. */
5531   const sh = archBandShape(hw, 0, h, bw, 10);
5532   return new THREE.ExtrudeGeometry(sh, {depth:depth, bevelEnabled:true,
5533     bevelThickness:bw*0.45, bevelSize:bw*0.40, bevelSegments:1, curveSegments:5});
5534 }
5535
5536 /* Ustun boshi uchun muqarnas – aylana bo'ylab yaruslar */
5537 function muqarnasCap(r0, tiers, height, mat){
5538   const grp = new THREE.Group();
5539   let r = r0, y = 0;
5540   const hT = height/tiers;
5541   for(let t = 0; t < tiers; t++){
5542     const n = 10 + t*4;
5543     const cr = Math.min(hT*0.62, TAU*r/n*0.52);
5544     const geo = muqCellGeo(cr);
5545     for(let i = 0; i < n; i++){
5546       const a = TAU*(i + 0.5)/n + (t % 2 ? Math.PI/n : 0);
5547       const cell = new THREE.Mesh(geo, mat);
5548       cell.position.set(Math.sin(a)*r, y + hT*0.5, Math.cos(a)*r);
5549       cell.rotation.y = a;
5550       cell.rotation.x = -0.12;
5551       grp.add(cell);
5552     }
5553     r += hT*0.48; y += hT;
5554   }
5555   return grp;
5556 }
5557
5558 /* USTUN/GULDASTA BOSHI – kengayuvchi KORBEL.
5559 Kataknlarni aylana bo'ylab sochib chiqish bu miqyosda ishlamadi: ular
5560 bir-biriga tegmay, "to'kilaotgan bo'laklar"dek ko'rinardi. Haqiqiy
5561 Xiva ustun boshi – ketma-ket kengayuvchi o'yma halqalar; ular yaxlit
5562 massa hosil qiladi va tepasidagi yostiqni ko'taradi. Eng yuqorida
5563 bitta zich muqarnas qatori qo'shiladi. */
5564 function corbelCap(r0, tiers, height, matA, matB, matCell){
5565   const grp = new THREE.Group();
5566   const hT = height/tiers;
5567   let rb = r0;
5568   for(let t = 0; t < tiers; t++){
5569     const rt = rb + hT*0.72;

```

```

5570     const ring = new THREE.Mesh(new THREE.CylinderGeometry(rt, rb, hT*0.94, 32),
5571                                   (t % 2) ? matB : matA);
5572     ring.position.y = t*hT + hT/2;
5573     grp.add(ring);
5574     rb = rt;
5575   }
5576   if(matCell){
5577     const n = Math.max(12, Math.round(TAU*rb/(hT*0.62)));
5578     const cr = Math.min(hT*0.42, TAU*rb/n*0.48);
5579     const geo = muqCellGeo(cr);
5580     for(let i = 0; i < n; i++){
5581       const a = TAU*(i + 0.5)/n;
5582       const cell = new THREE.Mesh(geo, matCell);
5583       cell.position.set(Math.sin(a)*(rb - cr*0.5), tiers*hT + cr*0.6, Math.cos(a)*(rb - cr*0.5));
5584       cell.rotation.y = a;
5585       cell.rotation.x = Math.PI;
5586       grp.add(cell);
5587     }
5588   }
5589   return grp;
5590 }
5591
5592 /* -----
5593 MUQARNAS USTUN BOSHI (haqiqiy o'yma stalaktit kapiteli)
5594 ----- */
5595
5596 Ilgari bu yerda oddiy kengayuvchi halqalar (korbel) turardi: yaxlit massa
5597 berardi, lekin muqarnasga o'xshamasdi — o'yma emas, tornada aylantirilgan
5598 detaldek ko'rinardi.
5599 Haqiqiy muqarnas kapiteli quyidagicha ishlaydi:
5600   • yaruslar yuqoriga ko'tarilgan sari tashqariga chiqadi (korbel);
5601   • har yarus aylana bo'ylab bir qator TOQCHAdan iborat;
5602   • qo'shni yaruslar yarim qadamga surilgan — shuning uchun ustki
5603     toqchani tubi pastki ikki toqcha orasidagi qovurg'a ustiga tushadi;
5604   • yaruslar orasida ingichka javon (polka) bo'ladi — soya chizig'i
5605     aynan shundan chiqadi va ko'z yaruslarni ajratadi;
5606   • hamma narsa orqadan yaxlit konus tana bilan yopiladi, teshik qolmaydi.
5607 Toqchalar soni har yarusda radiusga qarab hisoblanadi, shuning uchun
5608 katak eni hamma yarusda deyarli bir xil bo'ladi — asl o'ymadagidek. */
5609 function muqarnasCapital(r0, tiers, height, matBody, matA, matB, matShelf){
5610   const grp = new THREE.Group();
5611   const hT = height/tiers;
5612   /* Kengayish TEKIS emas: pastki yaruslar oz, yuqoridagilar ko'p chiqadi.
5613     Bir xil qadamda kapitel "teskari zinapoya"dek qotib ko'rinardi;
5614     o'sib boruvchi qadam esa haqiqiy stalaktit korbelning yumshoq
5615     yoyilishini beradi. Yig'indi kengayish height*0.62 ga teng. */
5616   const fSum = tiers > 1 ? tiers : 1;
5617   const flare0f = t => height*0.62/fSum * (0.55 + 0.90*(tiers > 1 ? t/(tiers-1) : 0));
5618   let rb = r0;
5619   for(let t = 0; t < tiers; t++){
5620     const flare = flare0f(t);
5621     const rt = rb + flare;
5622     const y0 = t*hT;
5623     /* 1) TANA — orqadagi yaxlit konus. To'q rangda: toqcha tubi qorong'i
5624       bo'lsa, o'yma chuqurligi ko'zga aniq bilinadi. */
5625     const body = new THREE.Mesh(
5626       new THREE.CylinderGeometry(rt*0.98, rb*0.98, hT*1.02, 44, 1, true), matBody);
5627     body.position.y = y0 + hT/2;
5628     grp.add(body);
5629     /* 2) TOQCHALAR — aylana bo'ylab, qo'shni yarus yarim qadamga surilgan.
5630       Katak eni yarus balandligiga yaqin (haqiqiy muqarnasda ham shunday),
5631       shuning uchun radius o'sgani sari kataklar SONI ortadi, o'lchami
5632       emas — naqsh miqyosi hamma yarusda bir xil qoladi. */
5633     const cellW = hT*1.02;
5634     const n = Math.max(7, Math.round(TAU*rt/cellW));
5635     const hw = TAU*rt/n*0.5; /* katak yarim eni — qo'shnisiga tegib turadi */
5636     const hCell = hT*0.99;
5637     const depth = Math.max(hw*0.95, flare*1.25);
5638     const niche = muqNicheGeo(hw*0.90, hCell, depth);
5639     const rib = muqRibGeo(hw*0.90, hCell, hw*0.14, depth*0.26);
5640     const mN = (t % 2) ? matB : matA;
5641     const off = (t % 2) ? Math.PI/n : 0;
5642     for(let i = 0; i < n; i++){
5643       const a = TAU*i/n + off;
5644       const ca = Math.cos(a), sa = Math.sin(a);
5645       const cell = new THREE.Mesh(niche, mN);
5646       cell.position.set(sa*rt, y0 + hT*0.015, ca*rt);
5647       cell.rotation.y = a; /* og'iz tashqariga qaraydi */
5648       grp.add(cell);
5649       const rm = new THREE.Mesh(rib, matShelf || matBody);
5650       rm.position.set(sa*rt, y0 + hT*0.015, ca*rt);
5651       rm.rotation.y = a;
5652       grp.add(rm);
5653     }
5654     /* 3) JAVON — yarus tagidagi ingichka chiqiq: soya chizig'i beradi va
5655       ustki qator pastkisiga "tayangan"dek ko'rinadi */
5656     const shelf = new THREE.Mesh(
5657       new THREE.CylinderGeometry(rt + hw*0.16, rt - flare*0.35, hT*0.055, 52),
5658       matShelf || matBody);
5659     shelf.position.y = y0 + hT*0.012;
5660     grp.add(shelf);
5661     rb = rt;

```

```

5661 }
5662 /* Tapa gardish – kapitelni yopadi va yostiqni ko'taradi */
5663 const top = new THREE.Mesh(
5664   new THREE.CylinderGeometry(rb + hT*0.18, rb + hT*0.02, hT*0.20, 52), matShelf || matBody);
5665 top.position.y = tiers*hT + hT*0.09;
5666 grp.add(top);
5667 grp.userData.rTop = rb + hT*0.18;
5668 grp.userData.h = tiers*hT + hT*0.19;
5669 return grp;
5670 }
5671
5672 /* AYVON RAVOQ BOSHI – ichma-ich chekinuvchi ravoq qovurg'alari.
5673   Katakarni ravoq egrisi bo'ylab sochib chiqish yaxshi natija bermadi:
5674   old tomondan ular alohida munchoqlardek ko'rinardi. Temuriylar
5675   ayvonlarida ravoq boshi aslida ICHMA-ICH JOYLASHGAN, orqaga chekinuvchi
5676   ravoq bandlaridan iborat – quyida aynan shu quriladi. Har band oldingisidan
5677   kichikroq va chuqurroq, ranglari navbatlashadi. */
5678 function archRibs(ow, spring, apex, tiers, z0, dz, matA, matB){
5679   const grp = new THREE.Group();
5680   for(let t = 0; t < tiers; t++){
5681     const k = 1 - t*0.085;
5682     const bw = 0.13*k;
5683     const sh = archBandShape(ow*k, spring, spring + (apex - spring)*k, bw);
5684     const rib = new THREE.Mesh(
5685       new THREE.ExtrudeGeometry(sh, {depth:Math.abs(dz)*0.9, bevelEnabled:false, curveSegments:18}),
5686       (t % 2) ? matB : matA);
5687     rib.position.z = z0 + t*dz;
5688     grp.add(rib);
5689   }
5690   return grp;
5691 }
5692
5693 /* MUQARNAS KORBEL BANDI – gorizontal qator kataklar (ravoq yelkasida yoki
5694   guldasta boshida). Katakarni zich terilgani uchun yaxlit asal uyasi
5695   ko'rinishini beradi. */
5696 function muqarnasBand(width, n, tiers, cellR, mat){
5697   const grp = new THREE.Group();
5698   const geo = muqCellGeo(cellR);
5699   for(let t = 0; t < tiers; t++){
5700     const cnt = n - t*2;
5701     if(cnt < 2) break;
5702     const step = width/cnt;
5703     for(let i = 0; i < cnt; i++){
5704       const cell = new THREE.Mesh(geo, mat);
5705       cell.position.set(-width/2 + step*(i + 0.5), t*cellR*1.35, -t*cellR*0.9);
5706       cell.rotation.x = Math.PI;
5707       grp.add(cell);
5708     }
5709   }
5710   return grp;
5711 }
5712
5713 /* ----- USTUN
5714   Xiva Juma masjidi uslubidagi o'yma yog'och ustun. Nisbatlari haqiqiyga
5715   yaqin: tana diametri ~0.28, balandligi ~2.9 → ingichkalik ~10:1. */
5716 function buildUstun(){
5717   const g = new THREE.Group();
5718   const R = 0.145, Hs = 2.75;
5719
5720   /* sakkiz qirrali tosh poya – Xiva ustunlari aynan shunday poyada turadi */
5721   const poya = new THREE.Mesh(new THREE.CylinderGeometry(0.40, 0.46, 0.24, 8), archMat('stone'));
5722   poya.position.y = 0.12; g.add(poya);
5723   const poya2 = new THREE.Mesh(new THREE.CylinderGeometry(0.31, 0.38, 0.14, 8), archMat('stone'));
5724   poya2.position.y = 0.31; g.add(poya2);
5725
5726   /* kuza – vazasimon o'yma asos */
5727   const kuza = new THREE.Mesh(
5728     latheFromProfile([[0.13,0],[0.22,0.05],[0.27,0.13],[0.25,0.22],[0.19,0.31],[0.155,0.40],[0.145,0.46]], 1),
5729     archMat('wood', 6, 1));
5730   kuza.position.y = 0.38; g.add(kuza);
5731
5732   /* tana – entazis bilan; naqsh aylana bo'ylab 4, balandligi bo'ylab 14 */
5733   const prof = [];
5734   for(let i = 0; i <= 20; i++){
5735     const t = i/20;
5736     prof.push([R*(1 - 0.16*t + 0.055*Math.sin(Math.PI*t)), t*Hs]);
5737   }
5738   /* Naqsh zichligi kamaytirildi (4x14 → 3x9): oldingi zichlikda o'yma
5739     mato naqshidek mayda chiqib, chuqurligi ko'rinmasdi. */
5740   const tana = new THREE.Mesh(latheFromProfile(prof, 1), archMat('wood', 3, 9));
5741   tana.position.y = 0.84; g.add(tana);
5742
5743   /* o'yma bo'g'inlar – halqa + ingichka bronza gardish */
5744   for(const [y, rr] of [[1.05, 0.150], [2.05, 0.140], [3.05, 0.131]]){
5745     const band = new THREE.Mesh(new THREE.CylinderGeometry(rr*1.16, rr*1.16, 0.10, 32),
5746       archMat('wood', 5, 1, 0x8a4f22));
5747     band.position.y = y; g.add(band);
5748     for(const dy of [-0.055, 0.055]){
5749       const ring = new THREE.Mesh(new THREE.TorusGeometry(rr*1.18, 0.012, 8, 40), archMat('metal'));
5750       ring.rotation.x = Math.PI/2; ring.position.y = y + dy; g.add(ring);
5751     }

```



```

5752     }
5753
5754     /* MUQARNAS BOSH — o'yilgan stalaktit kapitел.
5755         Toqchalar tanadan kengroq bo'lgani uchun ular ostiga silliq o'tish
5756         (kova) qo'yiladi, aks holda kapitел poyada "osilib" turgandek
5757         ko'rinadi. */
5758     const kova = new THREE.Mesh(
5759         latheFromProfile([R*0.99,0],[R*1.02,0.05],[R*1.10,0.09],[R*1.20,0.11]], 1),
5760         archMat('wood', 5, 1, 0x8a4f22));
5761     kova.position.y = 3.50; g.add(kova);
5762
5763     const cap = muqarnasCapital(R*1.18, 4, 0.62,
5764         archMat('wood', 8, 1, 0x6b3c17),          /* orqa tana — to'q */
5765         archMat('wood', 1, 1, 0xc9975a),          /* toqcha A */
5766         archMat('wood', 1, 1, 0xb87a3c),          /* toqcha B */
5767         archMat('wood', 6, 1, 0x8a4f22));          /* javon/qovurg'a */
5768     cap.position.y = 3.61; g.add(cap);
5769     const capTop = cap.position.y + (cap.userData.h || 0.52);
5770     const rTop = cap.userData.rTop || R*1.9;
5771
5772     /* yostiқ (bracket) va tosh bosh — kapitел kengligiga moslashadi */
5773     const yostiқ = new THREE.Mesh(new THREE.BoxGeometry(rTop*1.85, 0.09, rTop*0.95), archMat('wood', 3, 1));
5774     yostiқ.position.y = capTop + 0.045; g.add(yostiқ);
5775     const yostiқ2 = new THREE.Mesh(new THREE.BoxGeometry(rTop*2.15, 0.06, rTop*1.10), archMat('wood', 4, 1, 0x8a4f22
5776 ));
5777     yostiқ2.position.y = capTop + 0.120; g.add(yostiқ2);
5778     const bosh = new THREE.Mesh(new THREE.BoxGeometry(rTop*2.40, 0.08, rTop*1.24), archMat('stone'));
5779     bosh.position.y = capTop + 0.190; g.add(bosh);
5780
5781     g.userData.h = capTop + 0.24;
5782     return g;
5783 }
5784
5785 /* ----- DARVOZA
5786     Koshin ramkadagi ikki tabaqali o'yma yog'och darvoza. Tabaqalar
5787     haqiqiy duradgorlikdagidek: tik tayanchlar (stil), ko'ndalang
5788     bog'lamalar (rels) va ular orasida botiқ panellar. */
5789 function buildDarvoza(){
5790     const g = new THREE.Group();
5791     const W = 3.0, H = 4.0, D = 0.34;
5792     const ow = 1.02, oh = 2.30, apexY = H - 0.30;
5793
5794     const ap = archOutline(ow, oh, apexY, 40);
5795     const opening = new THREE.Path();
5796     opening.moveTo(-ow, 0);
5797     for(const q of ap) opening.lineTo(q[0], q[1]);
5798     opening.lineTo(ow, 0);
5799     opening.closePath();
5800
5801     const sh = new THREE.Shape();
5802     sh.moveTo(-W/2, 0); sh.lineTo(W/2, 0); sh.lineTo(W/2, H); sh.lineTo(-W/2, H); sh.closePath();
5803     sh.holes.push(opening);
5804     const frame = new THREE.Mesh(
5805         new THREE.ExtrudeGeometry(sh, {depth:D, bevelEnabled:false, curveSegments:24}),
5806         archMat('tile', 1/TU, 1/TU, 0x24518f));
5807     frame.position.z = -D; g.add(frame);
5808
5809     /* archivolt */
5810     const archv = new THREE.Mesh(
5811         new THREE.ExtrudeGeometry(archBandShape(ow, oh, apexY, 0.16),
5812             {depth:0.06, bevelEnabled:false, curveSegments:20}),
5813         archMat('tile', 1/0.16, 1/0.16, 0xc99b3f));
5814     archv.position.z = 0.004; g.add(archv);
5815
5816     /* soffit — ichkarida yorug'lik kam */
5817     const soffit = new THREE.Mesh(stripFromOutline(ap, D + 0.10),
5818         archMat('tile', 1/TU, 1/TU, 0x8d96a4));
5819     soffit.material.side = THREE.DoubleSide;
5820     g.add(soffit);
5821
5822     /* timpan */
5823     const tsh = new THREE.Shape();
5824     tsh.moveTo(-ow, oh);
5825     for(const q of ap) tsh.lineTo(q[0], q[1]);
5826     tsh.closePath();
5827     const timpan = new THREE.Mesh(new THREE.ShapeGeometry(tsh, 24),
5828         archMat('tile', 1/(TU*0.7), 1/(TU*0.7), 0x7fc7c0));
5829     timpan.position.z = -0.10; g.add(timpan);
5830
5831     /* ---- tabaqalar: stil + rels + botiқ panel ---- */
5832     const lw = ow - 0.02, lh = oh, zL = -0.17;
5833     const stileW = lw*0.13, railH = lh*0.055;
5834     for(const sgn of [-1, 1]){
5835         const cxL = sgn*lw/2;
5836         const leaf = new THREE.Mesh(new THREE.BoxGeometry(lw, lh, 0.06),
5837             archMat('wood', 1, 3, 0x6f3d1a)); /* botiқ fon */
5838         leaf.position.set(cxL, lh/2, zL - 0.03); g.add(leaf);
5839
5840         /* tik tayanchlar */
5841         for(const sx of [-1, 1]){
5842             const st2 = new THREE.Mesh(new THREE.BoxGeometry(stileW, lh, 0.075),

```

```

5842         archMat('wood', 1, 8, 0x8a4f22));
5843     st2.position.set(cxL + sx*(lw/2 - stileW/2), lh/2, zL + 0.01); g.add(st2);
5844 }
5845 /* ko'ndalang bog'lamalar - 4 ta, orasida 3 panel */
5846 for(let i = 0; i <= 3; i++){
5847     const y = lh*(0.04 + i*0.31);
5848     const rl = new THREE.Mesh(new THREE.BoxGeometry(lw, railH, 0.075),
5849         archMat('wood', 3, 1, 0x8a4f22));
5850     rl.position.set(cxL, y, zL + 0.01); g.add(rl);
5851 }
5852 /* botiq panellar - nozik naqsh bilan */
5853 for(let i = 0; i < 3; i++){
5854     const y = lh*(0.195 + i*0.31);
5855     const pnl = new THREE.Mesh(new THREE.BoxGeometry(lw - 2*stileW - 0.02, lh*0.24, 0.05),
5856         archMat('wood', 1, 1, 0xc98a45));
5857     pnl.position.set(cxL, y, zL); g.add(pnl);
5858     /* panel atrofidagi ingichka gardish */
5859     const brd = new THREE.Mesh(new THREE.BoxGeometry(lw - 2*stileW + 0.02, lh*0.26, 0.035),
5860         archMat('dark'));
5861     brd.position.set(cxL, y, zL - 0.012); g.add(brd);
5862 }
5863 /* bronza mixlar - tayanch va bog'lamalar bo'ylab */
5864 const stud = archMat('metal');
5865 const studGeo = new THREE.SphereGeometry(0.022, 10, 8);
5866 for(let i = 0; i <= 3; i++){
5867     for(const sx of [-1, 1]){
5868         const s2 = new THREE.Mesh(studGeo, stud);
5869         s2.position.set(cxL + sx*(lw/2 - stileW/2), lh*(0.04 + i*0.31), zL + 0.05);
5870         g.add(s2);
5871     }
5872 }
5873 /* halqa tutqich + rozetka */
5874 const plate = new THREE.Mesh(new THREE.CylinderGeometry(0.055, 0.055, 0.02, 16), archMat('metal'));
5875 plate.rotation.x = Math.PI/2; plate.position.set(sgn*0.26, 1.30, zL + 0.05); g.add(plate);
5876 const ring = new THREE.Mesh(new THREE.TorusGeometry(0.10, 0.018, 10, 30), archMat('metal'));
5877 ring.position.set(sgn*0.26, 1.22, zL + 0.055); g.add(ring);
5878 }
5879
5880 const ostona = new THREE.Mesh(new THREE.BoxGeometry(W + 0.34, 0.14, D + 0.44), archMat('stone'));
5881 ostona.position.set(0, 0.07, -D/2); g.add(ostona);
5882 g.userData.h = H;
5883 return g;
5884 }
5885
5886 /* ----- PESHTOQ
5887     Registon uslubidagi darvozaxona. */
5888 function buildPeshtoq(){
5889     const g = new THREE.Group();
5890     const W = 4.0, H = 5.0, D = 0.42;
5891     const ow = 1.15, spring = 2.05, apex = 3.95;
5892     const nicheD = 1.20;
5893     const band = 0.38; /* hoshiya (kitoba) eni */
5894
5895     const ap = archOutline(ow, spring, apex, 44);
5896     const opening = new THREE.Path();
5897     opening.moveTo(-ow, 0);
5898     for(const q of ap) opening.lineTo(q[0], q[1]);
5899     opening.lineTo(ow, 0);
5900     opening.closePath();
5901
5902     /* 1) asosiy koshin maydoni */
5903     const iw = W/2 - band, ih = H - band;
5904     const inner = new THREE.Shape();
5905     inner.moveTo(-iw, 0); inner.lineTo(iw, 0); inner.lineTo(iw, ih); inner.lineTo(-iw, ih); inner.closePath();
5906     inner.holes.push(opening);
5907     const face = new THREE.Mesh(
5908         new THREE.ExtrudeGeometry(inner, {depth:D, bevelEnabled:false, curveSegments:24}),
5909         archMat('tile', 1/TU, 1/TU));
5910     face.position.z = -D; g.add(face);
5911
5912     /* 2) HOSHIYA (kitoba) - to'q ko'k, bandda aynan bitta plitka yotadi */
5913     const outer = new THREE.Shape();
5914     outer.moveTo(-W/2, 0); outer.lineTo(W/2, 0); outer.lineTo(W/2, H); outer.lineTo(-W/2, H); outer.closePath();
5915     const cut = new THREE.Path();
5916     cut.moveTo(-iw, 0); cut.lineTo(iw, 0); cut.lineTo(iw, ih); cut.lineTo(-iw, ih); cut.closePath();
5917     outer.holes.push(cut);
5918     const hoshiya = new THREE.Mesh(
5919         new THREE.ExtrudeGeometry(outer, {depth:D + 0.10, bevelEnabled:false}),
5920         archMat('tile', 1/band, 1/band, 0x24518f));
5921     hoshiya.position.z = -D; g.add(hoshiya);
5922     /* hoshiya chetidagi ingichka oq gardish */
5923     for(const [w2, h2, z2] of [[W, H, -D + D + 0.105]]){
5924         const eg = new THREE.Mesh(new THREE.BoxGeometry(w2 + 0.05, 0.045, 0.05), archMat('stone'));
5925         eg.position.set(0, h2 + 0.02, z2); g.add(eg);
5926     }
5927
5928     /* 3) archivoIt */
5929     const archv = new THREE.Mesh(
5930         new THREE.ExtrudeGeometry(archBandShape(ow, spring, apex, 0.20),
5931             {depth:0.07, bevelEnabled:false, curveSegments:20}),
5932         archMat('tile', 1/0.20, 1/0.20, 0xc99b3f));

```

```

5933 archv.position.z = 0.005; g.add(archv);
5934
5935 /* 4) ayvon ichki yuzalari — yorug'lik kam, to'qroq tus */
5936 const INNER = 0x93a3b8;
5937 const soffit = new THREE.Mesh(stripFromOutline(ap, nicheD),
5938                               archMat('tile', 1/TU, 1/TU, INNER));
5939 soffit.material.side = THREE.DoubleSide;
5940 soffit.position.z = -D; g.add(soffit);
5941
5942 for(const sgn of [-1, 1]){
5943   const wall = new THREE.Mesh(new THREE.PlaneGeometry(nicheD, spring),
5944                               archMat('tile', nicheD/TU, spring/TU, INNER));
5945   wall.material.side = THREE.DoubleSide;
5946   wall.rotation.y = sgn*Math.PI/2;
5947   wall.position.set(sgn*ow, spring/2, -D - nicheD/2);
5948   g.add(wall);
5949 }
5950
5951 const backSh = new THREE.Shape();
5952 backSh.moveTo(-ow, 0);
5953 for(const q of ap) backSh.lineTo(q[0], q[1]);
5954 backSh.lineTo(ow, 0); backSh.closePath();
5955 const back = new THREE.Mesh(new THREE.ShapeGeometry(backSh, 24),
5956                              archMat('tile', 1/TU, 1/TU, 0x74869c));
5957 back.position.z = -D - nicheD; g.add(back);
5958
5959 /* 5) ravog boshi — ichma-ich chekinuvchi qovurg'alar + yelkada muqarnas */
5960 const ribs = archRibs(ow*0.99, spring, apex, 5, -D - 0.02, -0.19,
5961                       archMat('tile', 1/0.13, 1/0.13, 0xbf9648),
5962                       archMat('tile', 1/0.13, 1/0.13, 0x2f639f));
5963 g.add(ribs);
5964 /* Yelka muqarnasi ayvon YON DEVORLARIDA — ilgari u ochiq joyni kesib
5965    o'tuvchi tokchadek ko'rinardi. */
5966 for(const sgn of [-1, 1]){
5967   const corb = muqarnasBand(nicheD*0.8, 6, 2, 0.062, archMat('tile', 1, 1, 0xbf9648));
5968   corb.rotation.y = sgn*Math.PI/2;
5969   corb.position.set(sgn*(ow - 0.06), spring - 0.06, -D - nicheD/2);
5970   g.add(corb);
5971 }
5972
5973 /* 6) guldasta — burchak yarim minorallari: poya, tana, muqarnas boshi */
5974 for(const sgn of [-1, 1]){
5975   const gx = sgn*(W/2 - 0.06), gz = 0.19;
5976   const foot = new THREE.Mesh(new THREE.CylinderGeometry(0.27, 0.30, 0.26, 28), archMat('stone'));
5977   foot.position.set(gx, 0.13, gz); g.add(foot);
5978   const gd = new THREE.Mesh(new THREE.CylinderGeometry(0.21, 0.23, H - 0.55, 32, 1),
5979                              archMat('tile', 3, Math.round((H - 0.55)/TU), 0x24518f));
5980   gd.position.set(gx, 0.26 + (H - 0.55)/2, gz); g.add(gd);
5981   /* Guldasta boshi ham haqiqiy muqarnas: ilgari bu yerda kengayuvchi
5982      halqalar turardi va minora boshi "etak"dek ko'rinardi. */
5983   const gcap = muqarnasCapital(0.20, 3, 0.27,
5984                                archMat('tile', 8, 1, 0x1e4a7d), /* orqa tana — to'q ko'k */
5985                                archMat('tile', 1, 1, 0xbf9648), /* toqcha A — oltin */
5986                                archMat('tile', 1, 1, 0x2f639f), /* toqcha B — ko'k */
5987                                archMat('tile', 6, 1, 0xbf9648)); /* javon */
5988   gcap.position.set(gx, H - 0.34, gz); g.add(gcap);
5989   const gr = gcap.userData.rTop || 0.30;
5990   const gyTop = H - 0.34 + (gcap.userData.h || 0.27);
5991   /* muqarnas ustidagi gardish — bosh yaxlit ko'rinsin */
5992   const gring = new THREE.Mesh(new THREE.CylinderGeometry(gr*0.98, gr, 0.06, 32), archMat('stone'));
5993   gring.position.set(gx, gyTop + 0.03, gz); g.add(gring);
5994   const top = new THREE.Mesh(new THREE.CylinderGeometry(gr*0.86, gr*0.98, 0.16, 32), archMat('stone'));
5995   top.position.set(gx, gyTop + 0.14, gz); g.add(top);
5996 }
5997
5998 /* 7) yelka medalyonlari */
5999 for(const sgn of [-1, 1]){
6000   const med = new THREE.Mesh(new THREE.CircleGeometry(0.24, 40),
6001                               archMat('tile', 1/0.24, 1/0.24, 0xd8b45a));
6002   med.position.set(sgn*(iw - 0.38), ih - 0.46, 0.012); g.add(med);
6003   const rim = new THREE.Mesh(new THREE.TorusGeometry(0.245, 0.018, 8, 40), archMat('metal'));
6004   rim.position.set(sgn*(iw - 0.38), ih - 0.46, 0.012); g.add(rim);
6005 }
6006
6007 /* 8) poydevor va gulband */
6008 const plinth = new THREE.Mesh(new THREE.BoxGeometry(W + 0.5, 0.30, D + nicheD + 0.5), archMat('stone'));
6009 plinth.position.set(0, 0.15, -D - nicheD/2 + 0.1); g.add(plinth);
6010 const cornice = new THREE.Mesh(new THREE.BoxGeometry(W + 0.30, 0.18, D + 0.34), archMat('stone'));
6011 cornice.position.set(0, H + 0.09, -D + 0.06); g.add(cornice);
6012
6013 g.userData.h = H + 0.2;
6014 return g;
6015 }
6016
6017 function buildArch(kind){
6018   T3.archMats = [];
6019   const g = kind === 'ustun' ? buildUstun()
6020       : kind === 'darvoza' ? buildDarvoza()
6021       : buildPeshtoq();
6022   /* KAFOLAT: o'raladigan yuzalarda (Cylinder / Lathe / Sphere / Torus) naqsh
6023      aylana bo'ylab BUTUN son marta takrorlanishi SHART — aks holda buyum

```

```

6024     orqasida ulanish chizig'i chiqadi. Qism qurayotganda bu oson unutiladi
6025     (guldasta boshidagi halqalarda 8.33 chiqib qolgandi), shuning uchun
6026     tekshiruv geometriya turi ma'lum bo'lgan yagona joyda – shu yerda –
6027     markazlashtirilgan. */
6028     g.traverse(o=>{
6029       if(o.isMesh){ o.castShadow = true; o.receiveShadow = true; }
6030       if(!o.isMesh || !o.material || !o.material.userData || !o.material.userData.rep) return;
6031       if(!/Cylinder|Lathe|Sphere|Torus/.test((o.geometry && o.geometry.type) || '')) return;
6032       const r = o.material.userData.rep;
6033       r[0] = Math.max(1, Math.round(r[0]));
6034     });
6035     g.position.y = FLOOR_Y;
6036     return g;
6037   }
6038
6039   /* Lagan – planar UV (medalyon tepadan); piyola/kosa/tuvak – standart UV (friz aylanaga o'raladi, haqiqiy sopolchi
likdagidek) */
6040   const PLATE_KINDS = {lagan:1};
6041   const FRIEZE_KINDS = {piyola:1, kosa:1, tuvak:1, stakan:1, shar:1};
6042   function buildGeometry(kind){
6043     switch(kind){
6044       case 'panel': return new THREE.PlaneGeometry(3,3);
6045       case 'lagan': return planarUV(latheFromProfile([[0,0],[0.9,0.02],[1.3,0.12],[1.45,0.22],[1.5,0.3]], 1.15));
6046       case 'piyola': return latheFromProfile([[0,0],[0.35,0.02],[0.5,0.08],[0.62,0.38],[0.72,0.82],[0.74,1.0]], 1.4);
6047       case 'kosa': return latheFromProfile([[0,0],[0.5,0.03],[0.95,0.25],[1.15,0.6],[1.2,0.82]], 1.25);
6048       case 'tuvak': return latheFromProfile([[0,0],[0.55,0.02],[0.6,0.1],[0.75,0.9],[0.95,1.05]], 1.3);
6049       case 'stakan': return latheFromProfile([[0,0],[0.42,0.02],[0.5,0.08],[0.56,0.7],[0.6,1.15],[0.62,1.3]], 1.15);
6050       /* friz UV – yon aylanaga o'raladi */
6051       case 'shar': return new THREE.SphereGeometry(1.35, 64, 48);
6052       default: return new THREE.SphereGeometry(1.3, 48, 32);
6053     }
6054   }
6055   /* -----
6056   3D TILING
6057   Naqsh yuza bo'ylab CHOK-SUZ va BIR TEKIS taqsimlanishi kerak.
6058   Ilgari tekstura 1024 px kanvasga atigi 3 katak qilib chizilar va butun
6059   buyumga BIR MARTA yopishtirilardi – natijada piyolada 2-3 ta ulkan naqsh
6060   chiqardi (foydalanuvchi ko'rsatgan holat). Endi:
6061   · tekstura HAQIQIY RAPPORT (makeSeamlessRepeat – chok-suzligi 160
6062   holatda o'lchangan),
6063   · takrorlanish soni buyum o'lchamidan hisoblanadi, shuning uchun naqsh
6064   har buyumda bir xil FIZIK o'lchamda chiqadi,
6065   · aylanma yuzalarda u bo'yicha son BUTUN bo'ladi – aks holda buyum
6066   orqasida ulanish chizig'i paydo bo'lardi.
6067   ----- */
6068   const TILE3D = 0.62; /* naqsh plitkasining dunyo birligidagi o'lchami */
6069   let TILE_AR = 1; /* oxirgi rapportning bo'y/en nisbati */
6070
6071   /* Yuzaning v yo'nalishi bo'ylab HAQIQIY uzunligi.
6072   Lathe va sferada v profil (meridian) bo'ylab boradi va bu uzunlik bbox
6073   balandligidan sezilarli katta bo'ladi: piyolada profil 2.06, balandlik esa
6074   atigi 1.4. Bbox dan hisoblansa naqsh vertikal bo'yicha ~60% cho'zilib
6075   ketadi – buyumda aynan shu ko'rinaradi. Shuning uchun uzunlik geometriyaning
6076   O'ZIDAN o'lchanadi: u=0 ustunidagi nuqtalar v bo'yicha tartiblanib,
6077   ketma-ket masofalar yig'iladi. */
6078   function vSpanOf(geo){
6079     const pos = geo.attributes && geo.attributes.position;
6080     const uv = geo.attributes && geo.attributes.uv;
6081     if(!pos || !uv) return null;
6082     const col = [];
6083     for(let i = 0; i < uv.count; i++){
6084       if(uv.getX(i) < 0.02) col.push([uv.getY(i), pos.getX(i), pos.getY(i), pos.getZ(i)]);
6085     }
6086     if(col.length < 3) return null;
6087     col.sort((a, b)=> a[0] - b[0]);
6088     let L = 0;
6089     for(let i = 1; i < col.length; i++){
6090       L += Math.hypot(col[i][1]-col[i-1][1], col[i][2]-col[i-1][2], col[i][3]-col[i-1][3]);
6091     }
6092     return L > 1e-4 ? L : null;
6093   }
6094
6095   function tileRepeatFor(kind, geo){
6096     if(!geo) return [1, 1];
6097     if(!geo.boundingBox) geo.computeBoundingBox();
6098     const bb = geo.boundingBox;
6099     const sx = bb.max.x - bb.min.x, sy = bb.max.y - bb.min.y, sz = bb.max.z - bb.min.z;
6100     const vTile = TILE3D*(TILE_AR || 1); /* naqsh cho'zilmaligi uchun */
6101     const iv = v => Math.max(1, Math.round(v));
6102     if(kind === 'panel') return [iv(sx/TILE3D), iv(sy/vTile)];
6103     if(kind === 'lagan') return [iv(sx/TILE3D), iv(sz/vTile)];
6104     /* aylanma yuzalar: u – aylana bo'ylab (BUTUN son SHART, aks holda buyum
6105     orqasida ulanish chizig'i chiqadi), v – profil uzunligi bo'ylab */
6106     const R = Math.max(sx, sz)/2;
6107     const vLen = vSpanOf(geo) || sy;
6108     return [Math.max(3, Math.round(TAU*R/TILE3D)), iv(vLen/vTile)];
6109   }
6110
6111   function makePatternTexture(){
6112     const S = 1024;
6113     const cnv = document.createElement('canvas');

```

```

6112   cnv.width = cnv.height = S;
6113   const octx = cnv.getContext('2d');
6114   if(!octx) return cnv;
6115   const pal = PAL[P.pal] || PAL.gold;
6116   P._bg = pal.bg;
6117   octx.fillStyle = pal.bg;
6118   octx.fillRect(0,0,S,S);
6119   const kind = T3 ? T3.kind : 'lagan';
6120   if(T3D.tess){
6121     let rep = null;
6122     try{ rep = makeSeamlessRepeat(320, T3D.group || 'p4', {}); }catch(e){ console.warn('[3d] rapport', e); }
6123     if(rep && rep.canvas && rep.w > 4 && rep.h > 4){
6124       TILE_AR = rep.h/rep.w;
6125       cnv.width = rep.w; cnv.height = rep.h;
6126       const c2 = cnv.getContext('2d');
6127       c2.drawImage(rep.canvas, 0, 0);
6128       /* Rasm (foto) bu rejimda qo'shilmaydi: tekstura ko'p marta
6129          takrorlangani uchun rasm ham takrorlanib ketardi. */
6130       return cnv;
6131     }
6132     /* zaxira yo'l – rapport chiqmasa eski usul */
6133     TILE_AR = 1;
6134     const keepG = TESS.group, keepC = TESS.cell, keepB = TESS.showCell;
6135     TESS.group = T3D.group;
6136     TESS.cell = Math.round(S/3);
6137     TESS.showCell = false;
6138     drawTessellation(octx, S, S, P);
6139     TESS.group = keepG; TESS.cell = keepC; TESS.showCell = keepB;
6140   } else if(TESS.on){
6141     drawTessellation(octx, S, S, P);
6142   } else if(FRIEZE_KINDS[kind]){
6143     /* FRIZ: gorizontal bandlar – buyum aylanasiga chok-suz o'raladi
6144        (haqiqiy piyola/kosa banddagi naqsh bilan bezaladi) */
6145     drawFriezeComposition(octx, S, P);
6146   } else if(PLATE_KINDS[kind]){
6147     /* CHINNI KOMPOZITSIYASI: hoshiya bandlari + markaziy medalyon */
6148     drawPlateComposition(octx, S, P);
6149   } else {
6150     drawPattern(octx, S/2, S/2, S*0.46, P);
6151   }
6152   compositePhoto(octx, S);
6153   return cnv;
6154 }
6155 function photoShapePath(octx, cx, cy, r){
6156   octx.beginPath();
6157   if(PH.shape==='doira'){
6158     circle(octx, cx, cy, r);
6159   } else if(PH.shape==='sakkiz'){
6160     const pts=[]; for(let i=0;i<8;i++) pts.push([cx+Math.cos(Math.PI/8+TAU*i/8)*r, cy+Math.sin(Math.PI/8+TAU*i/8)*
6161   r]);
6162     polyPath(octx, pts);
6163   } else {
6164     polyPath(octx, [[cx-r,cy-r],[cx+r,cy-r],[cx+r,cy+r],[cx-r,cy+r]]);
6165   }
6166 }
6167 /* FOTO COMPOSITING: rasm markazda, naqsh – ramka */
6168 function compositePhoto(octx, S){
6169   if(!PH.img) return;
6170   const cx = S/2, cy = S/2;
6171   const r = S*PH.size/200;
6172   octx.save();
6173   photoShapePath(octx, cx, cy, r);
6174   octx.clip();
6175   const iw = PH.img.naturalWidth || PH.img.width || 1;
6176   const ih = PH.img.naturalHeight || PH.img.height || 1;
6177   const box = r*2;
6178   const sc = Math.max(box/iw, box/ih);
6179   const dw = iw*sc, dh = ih*sc;
6180   octx.drawImage(PH.img, cx-dw/2, cy-dh/2, dw, dh);
6181   octx.restore();
6182   const pal = PAL[P.pal] || PAL.gold;
6183   octx.save();
6184   /* tashqi bezakli ramka: qalin band + mini yulduzlar */
6185   photoShapePath(octx, cx, cy, r + PH.frame*0.6);
6186   octx.strokeStyle = pal.c[0];
6187   octx.lineWidth = PH.frame;
6188   octx.lineJoin = 'round';
6189   octx.stroke();
6190   photoShapePath(octx, cx, cy, r + PH.frame*1.1);
6191   octx.strokeStyle = pal.c[1];
6192   octx.lineWidth = Math.max(1.5, PH.frame*0.3);
6193   octx.stroke();
6194   photoShapePath(octx, cx, cy, r - PH.frame*0.12);
6195   octx.strokeStyle = '#ffffff';
6196   octx.lineWidth = Math.max(1.5, PH.frame*0.22);
6197   octx.stroke();
6198   if(PH.shape==='doira'){
6199     const m = 16;
6200     for(let i=0;i<m;i++){
6201       const a = TAU*i/m;
6202       const q = [cx+Math.cos(a)*(r+PH.frame*0.6), cy+Math.sin(a)*(r+PH.frame*0.6)];

```

```

6202     octx.beginPath(); circle(octx, q[0], q[1], Math.max(1.6, PH.frame*0.16));
6203     octx.fillStyle = pal.c[4];
6204     octx.fill();
6205 }
6206 }
6207 octx.restore();
6208 /* SUVENIR MATNI: rasm tagida SIMMETRIK – doira ramkada yoy bo'ylab */
6209 const txt = (PH.text||'').trim(), dt = (PH.date||'').trim();
6210 if(txt || dt){
6211     octx.save();
6212     octx.translate(cx, cy);
6213     const fp = Math.max(22, S*0.05);
6214     if(txt){
6215         if(PH.shape==='doira'){
6216             arcText(octx, txt.toUpperCase(), r + PH.frame*1.3 + fp*0.72, fp, pal.c[0], '#ffffff');
6217         } else {
6218             octx.font = '700 '+fp+'px Georgia, "Times New Roman", serif';
6219             octx.textAlign = 'center'; octx.textBaseline = 'middle';
6220             const ty = r + PH.frame*1.3 + fp*0.75;
6221             octx.lineWidth = fp*0.2; octx.strokeStyle = '#ffffff'; octx.lineJoin='round';
6222             octx.strokeText(txt.toUpperCase(), 0, ty);
6223             octx.fillStyle = pal.c[0];
6224             octx.fillText(txt.toUpperCase(), 0, ty);
6225         }
6226     }
6227     if(dt){
6228         const fp2 = fp*0.66;
6229         octx.font = 'italic 600 '+fp2+'px Georgia, serif';
6230         octx.textAlign = 'center'; octx.textBaseline = 'middle';
6231         const ty2 = r + PH.frame*1.3 + (txt ? fp*1.75 : fp*0.75);
6232         octx.lineWidth = fp2*0.22; octx.strokeStyle = '#ffffff'; octx.lineJoin='round';
6233         octx.strokeText(dt, 0, ty2);
6234         octx.fillStyle = pal.c[2];
6235         octx.fillText(dt, 0, ty2);
6236         /* ikki yon bezak nuqta */
6237         const dw = octx.measureText(dt).width;
6238         dot(octx, -dw/2 - fp2*0.7, ty2, fp2*0.14, pal.c[1]);
6239         dot(octx, dw/2 + fp2*0.7, ty2, fp2*0.14, pal.c[1]);
6240     }
6241     octx.restore();
6242 }
6243 }
6244 /* Yoy bo'ylab matn – pastki yoyda, chapdan o'ngga, tik holatda */
6245 function arcText(ctx, text, r, fontPx, fillC, strokeC){
6246     ctx.save();
6247     ctx.font = '700 '+fontPx+'px Georgia, "Times New Roman", serif';
6248     ctx.textAlign = 'center'; ctx.textBaseline = 'middle';
6249     const chars = text.split('');
6250     const widths = chars.map(ch => ctx.measureText(ch).width * 1.08);
6251     const total = widths.reduce((a,b)=>a+b, 0);
6252     let a = Math.PI/2 + (total/2)/r; /* chapdan (katta burchak) boshlab */
6253     for(let i=0;i<chars.length;i++){
6254         a -= widths[i]/2/r;
6255         ctx.save();
6256         ctx.translate(Math.cos(a)*r, Math.sin(a)*r);
6257         ctx.rotate(a - Math.PI/2); /* harf tepasi markazga qaraydi – tik o'qiladi */
6258         ctx.lineWidth = fontPx*0.2; ctx.strokeStyle = strokeC; ctx.lineJoin='round';
6259         ctx.strokeText(chars[i], 0, 0);
6260         ctx.fillStyle = fillC;
6261         ctx.fillText(chars[i], 0, 0);
6262         ctx.restore();
6263         a -= widths[i]/2/r;
6264     }
6265     ctx.restore();
6266 }
6267 /* Me'moriy yuzalar uchun CHOK-SUZ plitka.
6268 Oddiy buyumlarda tekstura buyumga moslab chiziladi (medalyon, friz), me'moriy
6269 yuzada esa naqsh yuza bo'ylab KO'P MARTA takrorlanadi – shuning uchun u
6270 haqiqiy rapport bo'lishi shart, aks holda har takrorlanishda chok chizig'i
6271 ko'rinadi. Shu bois bu yerda eksportdagi bir xil makeSeamlessRepeat()
6272 ishlatiladi: uning chok-suzligi 160 holatda o'lchab tekshirilgan. */
6273 function makeArchTileCanvas(){
6274     try{
6275         const t = makeSeamlessRepeat(256, T3D.group || 'p4', {});
6276         if(t && t.canvas) return t.canvas;
6277     }catch(e){ console.warn('[3d] rapport', e); }
6278     return makePatternTexture();
6279 }
6280 /* -----
6281 O'YMA EFFEKTI – NAQSHDAN NORMAL XARITA
6282 -----
6283 Ilgari naqsh ob'yektga faqat RANG (map + emissiveMap) sifatida qo'yilardi.
6284 Shuning uchun ustun "bo'yalgan" ko'rinaradi: yorug'lik naqsh chizig'ida
6285 sinmaydi, soya tushmaydi – o'yma sezilmaydi.
6286 Bu yerda naqsh tasviridan balandlik xaritasi olinadi (yorqinlik: och joy
6287 – ko'tarilgan yuza, to'q joy – o'yilgan chuqurcha), so'ng Sobel bilan
6288 sirt normali hisoblanadi. Natijada yorug'lik har chiziq qirrasida
6289 sinadi va yuza haqiqatan o'yilgandek ko'rinadi.
6290 Balandlik oldindan biroz xiralashtiriladi: aks holda bir piksellik
6291 chiziqlar o'tkir "tunuka" qirra beradi, o'yma esa yumshoq qirrali. */
6292 function makeNormalCanvas(src, strength){

```

```

6293 const W = src.width, H = src.height;
6294 const sc = document.createElement('canvas');
6295 sc.width = W; sc.height = H;
6296 const sx = sc.getContext('2d');
6297 if(!sx) return null;
6298 sx.drawImage(src, 0, 0);
6299 let d;
6300 try{ d = sx.getImageData(0, 0, W, H).data; }catch(e){ return null; }
6301 /* yorqinlik → balandlik */
6302 const h = new Float32Array(W*H);
6303 for(let i = 0, n = W*H; i < n; i++){
6304     const o = i*4;
6305     h[i] = (0.299*d[o] + 0.587*d[o+1] + 0.114*d[o+2])/255;
6306 }
6307 /* 3x3 silliqlash — chekkalar takrorlanuvchi (rapport uzilmasin) */
6308 const hb = new Float32Array(W*H);
6309 const at = (x, y) => h[((y % H) + H) % H * W + (((x % W) + W) % W)];
6310 for(let y = 0; y < H; y++) for(let x = 0; x < W; x++){
6311     hb[y*W+x] = (at(x-1,y-1) + 2*at(x,y-1) + at(x+1,y-1) +
6312                 2*at(x-1,y) + 4*at(x,y) + 2*at(x+1,y) +
6313                 at(x-1,y+1) + 2*at(x,y+1) + at(x+1,y+1))/16;
6314 }
6315 const bt = (x, y) => hb[((y % H) + H) % H * W + (((x % W) + W) % W)];
6316 const out = sx.createImageData(W, H);
6317 const od = out.data;
6318 const k = (strength === undefined ? 2.6 : strength);
6319 for(let y = 0; y < H; y++) for(let x = 0; x < W; x++){
6320     /* Sobel */
6321     const gx = (bt(x+1,y-1) + 2*bt(x,y-1) + bt(x-1,y-1)) -
6322               (bt(x+1,y+1) + 2*bt(x,y+1) + bt(x-1,y+1));
6323     const gy = (bt(x-1,y+1) + 2*bt(x,y+1) + bt(x+1,y+1)) -
6324               (bt(x-1,y-1) + 2*bt(x,y-1) + bt(x+1,y-1));
6325     let nx = -gx*k, ny = -gy*k, nz = 1;
6326     const l = Math.sqrt(nx*nx + ny*ny + 1) || 1;
6327     nx /= l; ny /= l; nz /= l;
6328     const o = (y*W + x)*4;
6329     od[o] = (nx*0.5 + 0.5)*255;
6330     od[o+1] = (ny*0.5 + 0.5)*255;
6331     od[o+2] = (nz*0.5 + 0.5)*255;
6332     od[o+3] = 255;
6333 }
6334 sx.putImageData(out, 0, 0);
6335 return sc;
6336 }
6337 /* 0'yilgan joy yaltiramaydi — chuqurchada chang va g'adir-budurlik ko'p.
6338    Balandlik xaritasidan g'adirlik xaritasi: och (ko'tarilgan) joy
6339    silliqroq, to'q (o'yilgan) joy g'adirroq. */
6340 function makeRoughCanvas(src, lo, hi){
6341     const W = src.width, H = src.height;
6342     const sc = document.createElement('canvas');
6343     sc.width = W; sc.height = H;
6344     const sx = sc.getContext('2d');
6345     if(!sx) return null;
6346     sx.drawImage(src, 0, 0);
6347     let img;
6348     try{ img = sx.getImageData(0, 0, W, H); }catch(e){ return null; }
6349     const d = img.data;
6350     const a = (lo === undefined ? 0.34 : lo), b = (hi === undefined ? 0.86 : hi);
6351     for(let i = 0, n = W*H; i < n; i++){
6352         const o = i*4;
6353         const v = (0.299*d[o] + 0.587*d[o+1] + 0.114*d[o+2])/255;
6354         const r = Math.round((b + (a - b)*v)*255); /* och → a, to'q → b */
6355         d[o] = d[o+1] = d[o+2] = r; d[o+3] = 255;
6356     }
6357     sx.putImageData(img, 0, 0);
6358     return sc;
6359 }
6360 function update3DTexture(){
6361     if(!threeReady || !T3 || !T3.mat) return;
6362     const tex = new THREE.CanvasTexture(makePatternTexture());
6363     tex.anisotropy = 8;
6364     tex.wrapS = THREE.RepeatWrapping;
6365     tex.wrapT = THREE.RepeatWrapping;
6366     if(THREE.sRGBEncoding !== undefined) tex.encoding = THREE.sRGBEncoding;
6367     /* Tiling yoqilganda naqsh buyum o'lchamiga qarab ko'p marta takrorlanadi
6368        (tileRepeatFor izohiga qarang). Yakka naqsh rejimida — bir marta. */
6369     if(T3D.tess && T3.mesh && T3.mesh.geometry && !ARCH_KINDS[T3.kind]){
6370         const r = tileRepeatFor(T3.kind, T3.mesh.geometry);
6371         tex.repeat.set(r[0], r[1]);
6372         T3._rep = r;
6373     } else {
6374         tex.repeat.set(1, 1);
6375         T3._rep = [1, 1];
6376     }
6377     const old = T3.mat.map;
6378     for(const m of (T3.mats || [T3.mat])){
6379         m.map = tex;
6380         if(m.emissive) m.emissiveMap = tex; /* glazur: soyada ham naqsh o'qiladi */
6381         m.needsUpdate = true;
6382     }
6383     if(old && old.dispose) old.dispose();

```

```

6384
6385 /* me'moriy qismlar: bitta rapport kanvasi, lekin har qism o'z zichligi
6386 bilan — repeat teksturaga tegishli bo'lgani uchun har material o'z
6387 nusxasini oladi (rasm xotirada baham ko'riladi) */
6388 const arch = T3.archMats || [];
6389 if(!arch.length) return;
6390 const cnv = makeArchTileCanvas();
6391 const base = new THREE.CanvasTexture(cnv);
6392 base.wrapS = base.wrapT = THREE.RepeatWrapping;
6393 base.anisotropy = 8;
6394 if(THREE.sRGBEncoding !== undefined) base.encoding = THREE.sRGBEncoding;
6395 /* O'YMA: naqshdan normal va g'adirlik xaritalari. Bitta marta
6396 hisoblanadi va hamma me'moriy material baham ko'radi (faqat
6397 repeat har materialda o'ziniki, shuning uchun nusxa olinadi). */
6398 const nCnv = makeNormalCanvas(cnv, 2.6);
6399 const rCnv = makeRoughCanvas(cnv, 0.34, 0.88);
6400 const nBase = nCnv ? new THREE.CanvasTexture(nCnv) : null;
6401 const rBase = rCnv ? new THREE.CanvasTexture(rCnv) : null;
6402 for(const t of [nBase, rBase]) if(t){
6403   t.wrapS = t.wrapT = THREE.RepeatWrapping;
6404   t.anisotropy = 8; /* normal/rough — chiziqli fazo, encoding qo'yilmaydi */
6405 }
6406 for(const m of arch){
6407   const rep = m.userData.rep || [1, 1];
6408   const t2 = base.clone();
6409   t2.needsUpdate = true;
6410   t2.wrapS = t2.wrapT = THREE.RepeatWrapping;
6411   t2.repeat.set(rep[0], rep[1]);
6412   const prev = m.map, prevN = m.normalMap, prevR = m.roughnessMap;
6413   m.map = t2;
6414   if(m.emissive) m.emissiveMap = t2;
6415   if(nBase){
6416     const tn = nBase.clone();
6417     tn.needsUpdate = true;
6418     tn.wrapS = tn.wrapT = THREE.RepeatWrapping;
6419     tn.repeat.set(rep[0], rep[1]);
6420     m.normalMap = tn;
6421     /* o'yma chuqurligi — yog'ochda chuqurroq, koshinda yuzaroq */
6422     const dp = m.userData.carve === undefined ? 1 : m.userData.carve;
6423     m.normalScale = new THREE.Vector2(dp, dp);
6424   }
6425   if(rBase){
6426     const tr = rBase.clone();
6427     tr.needsUpdate = true;
6428     tr.wrapS = tr.wrapT = THREE.RepeatWrapping;
6429     tr.repeat.set(rep[0], rep[1]);
6430     m.roughnessMap = tr;
6431   }
6432   m.needsUpdate = true;
6433   for(const q of [prev, prevN, prevR]) if(q && q.dispose) q.dispose();
6434 }
6435 }
6436 const PAINT = { raf:0, active:false };
6437 /* NAQSH BUYUMDA JONLI CHIZILADI: tekstura progressiv ochiladi, buyum sekin aylanadi */
6438 function animatePaint3D(){
6439   if(!threeReady || !T3){ ntf('Avval 3D sahifasini oching', 'err'); return; }
6440   if(typeof cancelAnimationFrame === 'function') cancelAnimationFrame(PAINT.raf);
6441   const full = makePatternTexture();
6442   /* rapport kvadrat bo'lmasligi mumkin (p6/p6m) — ikkala o'lchamni olamiz */
6443   const S = full.width || 1024, SH = full.height || S;
6444   const cnv = document.createElement('canvas');
6445   cnv.width = S; cnv.height = SH;
6446   const ctx = cnv.getContext('2d');
6447   if(!ctx) return;
6448   const tex = new THREE.CanvasTexture(cnv);
6449   tex.anisotropy = 8;
6450   tex.wrapS = tex.wrapT = THREE.RepeatWrapping;
6451   if(T3._rep) tex.repeat.set(T3._rep[0], T3._rep[1]);
6452   if(THREE.sRGBEncoding !== undefined) tex.encoding = THREE.sRGBEncoding;
6453   const old = T3.mat.map;
6454   for(const m of (T3.mats || [T3.mat])){
6455     m.map = tex;
6456     if(m.emissive) m.emissiveMap = tex;
6457     m.needsUpdate = true;
6458   }
6459   if(old && old.dispose) old.dispose();
6460   const pal = PAL[P.pal] || PAL.gold;
6461   const frieze = !!FRIEZE_KINDS[T3.kind];
6462   const nowFn = () => (window.performance ? performance.now() : Date.now());
6463   const t0 = nowFn(), dur = 2600;
6464   PAINT.active = true;
6465   const step = () =>{
6466     const t = Math.min(1, (nowFn()-t0)/dur);
6467     const e = 1 - Math.pow(1-t, 2.1);
6468     ctx.clearRect(0,0,S,SH);
6469     ctx.fillStyle = pal.bg;
6470     ctx.fillRect(0,0,S,SH);
6471     ctx.save();
6472     let a1 = 0;
6473     if(frieze){
6474       ctx.beginPath(); ctx.rect(0,0,S*e,SH); ctx.clip();

```



```

6475     } else {
6476         a1 = -Math.PI/2 + e*TAU;
6477         ctx.beginPath();
6478         ctx.moveTo(S/2,SH/2);
6479         ctx.arc(S/2,SH/2, Math.max(S,SH)*0.72*Math.min(1, 0.3+e), -Math.PI/2, a1);
6480         ctx.closePath();
6481         ctx.clip();
6482     }
6483     ctx.drawImage(full,0,0);
6484     ctx.restore();
6485     if(t < 1){
6486         ctx.save();
6487         ctx.shadowColor = 'rgba(224,189,106,.95)';
6488         ctx.shadowBlur = 28;
6489         ctx.fillStyle = '#e0bd6a';
6490         let bx, by;
6491         if(frieze){ bx = S*e; by = SH*0.5; }
6492         else { const br = Math.max(S,SH)*0.35; bx = S/2 + Math.cos(a1)*br; by = SH/2 + Math.sin(a1)*br; }
6493         ctx.beginPath(); ctx.arc(bx,by,11,0,TAU); ctx.fill();
6494         ctx.restore();
6495     }
6496     tex.needsUpdate = true;
6497     if(T3.group) T3.group.rotation.y += 0.006; /* chizish davomida ohista aylanish */
6498     if(t < 1){
6499         PAINT.raf = requestAnimationFrame(step);
6500     } else {
6501         PAINT.active = false;
6502         ctx.clearRect(0,0,S,SH);
6503         ctx.drawImage(full,0,0);
6504         tex.needsUpdate = true;
6505         ntf('Naqsh buyumga tushirildi ☐', 'ok');
6506     }
6507 };
6508 step();
6509 }
6510 function setLights(preset){
6511     if(!T3) return;
6512     const cfg = {
6513         warm: {amb:0x9c8867, ai:0.5, hemi:0.75, key:0xffe2b8, ki:1.25, fill:0xffc890, fi:0.45, bg:0xf0e9da},
6514         cold: {amb:0x66788c, ai:0.5, hemi:0.7, key:0xd6ecff, ki:1.2, fill:0x9dbdd8, fi:0.45, bg:0xe8eef2},
6515         studio:{amb:0xffffffff, ai:0.45,hemi:0.65, key:0xffffffff, ki:1.3, fill:0xffffffff, fi:0.55, bg:0xf3f3ef},
6516         dark: {amb:0x24301f, ai:0.2, hemi:0.15, key:0xd8e0c0, ki:0.7, fill:0x445533, fi:0.2, bg:0x181e18}
6517     }[preset] || {amb:0xffffffff, ai:0.45, hemi:0.6, key:0xffffffff, ki:1.25, fill:0xffffffff, fi:0.5, bg:0xf3f3ef};
6518     T3.amb.color.setHex(cfg.amb); T3.amb.intensity = cfg.ai;
6519     T3.hemi.intensity = cfg.hemi;
6520     T3.key.color.setHex(cfg.key); T3.key.intensity = cfg.ki;
6521     T3.fill.color.setHex(cfg.fill); T3.fill.intensity = cfg.fi;
6522     T3.scene.background = new THREE.Color(cfg.bg);
6523     if(T3.scene.fog) T3.scene.fog.color.setHex(cfg.bg);
6524 }
6525 function disposeObject3D(obj){
6526     if(!obj) return;
6527     if(obj.traverse){
6528         obj.traverse(o=>{ if(o.isMesh && o.geometry && o.geometry.dispose) o.geometry.dispose(); });
6529     } else if(obj.geometry && obj.geometry.dispose){
6530         obj.geometry.dispose();
6531     }
6532 }
6533 /* Har buyum uchun kamera kadri: me'moriy ob'yektlar baland, shuning uchun
6534 kamera uzoqroqqa suriladi va nigoh markazi yuqoriroqqa ko'tariladi —
6535 aks holda ustun yoki peshtoq kadriga sig'maydi. */
6536 /* look — nigoh markazi, eye — kamera balandligi, d — masofa.
6537 Balandlikning yarmiga qaraymiz va shu balandlikdan bir oz yuqoridan
6538 turamiz: aks holda kamera tikka pastga qaraydi va bino "yiqilib"
6539 ko'rinadi. d ≈ balandlik×1.7 — 42° ko'rish burchagida to'liq sig'adi. */
6540 const CAM_FRAME = {
6541     ustun: {look: FLOOR_Y + 2.20, eye: FLOOR_Y + 2.55, d: 7.4},
6542     darvoza: {look: FLOOR_Y + 2.00, eye: FLOOR_Y + 2.30, d: 7.0},
6543     peshtoq: {look: FLOOR_Y + 2.55, eye: FLOOR_Y + 2.90, d: 9.0}
6544 };
6545 function setObject(kind){
6546     if(!T3) return;
6547     if(T3.mesh){
6548         T3.group.remove(T3.mesh);
6549         disposeObject3D(T3.mesh);
6550     }
6551     T3.kind = kind;
6552     const cf = CAM_FRAME[kind] || {look:0, eye:1.9, d:4.9};
6553     T3.camLook = cf.look; T3.camEye = cf.eye; T3.camDist = cf.d;
6554     T3.camera.position.z = cf.d;
6555     /* marmar poydevor faqat idishlar uchun — me'moriy ob'yekt polda turadi */
6556     if(T3.ped) T3.ped.visible = !ARCH_KINDS[kind];
6557     const obj = ARCH_KINDS[kind] ? buildArch(kind)
6558         : new THREE.Mesh(buildGeometry(kind), T3.mat);
6559     if(!ARCH_KINDS[kind]) T3.archMats = [];
6560     if(kind==='lagan') obj.rotation.x = 0.5; /* lagan yuzi tomoshabinga qaragan */
6561     if(kind==='panel') obj.rotation.x = 0;
6562     /* Me'moriy ob'yektlar O'ZIGA soya tashlaydi: muqarnas yaruslari,
6563 ravoq bandlari va o'yma toqchalar chuqurligi aynan shu ichki
6564 soyadan o'qiladi. Ilgari bu yerda receiveShadow hammaga false
6565 qilinardi va buildArch qo'ygan qiymatni bekor qilardi — natijada

```

```

6566     kapitel yassi, "bo'yalgan" ko'rinardi.
6567     Yakka buyumlarda (lagan, ko'za) ichki soya kerak emas — ular yaxlit
6568     yuza, o'ziga soya tushirsa faqat shovqin chiqadi. */
6569     const selfShadow = !!ARCH_KINDS[kind];
6570     obj.traverse ? obj.traverse(o=>{ if(o.isMesh){ o.castShadow = true; o.receiveShadow = selfShadow; } })
6571         : (obj.castShadow = true);
6572     T3.mesh = obj;
6573     obj.scale.setScalar(0.01);
6574     T3.intro = { t:0 }; /* kirish springi — buyum "tug'iladi" */
6575     T3.group.add(obj);
6576     update3DTexture(); /* obyektga mos kompozitsiya (chinni/panel) */
6577 }
6578 function init3D(){
6579     if(threeReady) return; /* renderer faqat bir marta yaratiladi */
6580     const wrap = $('three-wrap');
6581     if(!wrap) return;
6582     if(typeof THREE === 'undefined'){
6583         ntf("Three.js yuklanmadi — internet aloqasini tekshiring", 'err');
6584         return;
6585     }
6586     const W = wrap.clientWidth || 800, H = wrap.clientHeight || 500;
6587     const scene = new THREE.Scene();
6588     const camera = new THREE.PerspectiveCamera(42, W/H, 0.1, 100);
6589     camera.position.set(0, 1.9, 4.9);
6590     camera.lookAt(0, 0, 0);
6591     const renderer = new THREE.WebGLRenderer({antialias:true, preserveDrawingBuffer:true});
6592     renderer.setSize(W, H);
6593     renderer.setPixelRatio(Math.min(2, window.devicePixelRatio||1));
6594     if(THREE.SRGBEncoding !== undefined) renderer.outputEncoding = THREE.SRGBEncoding; /* to'g'ri rang */
6595     renderer.shadowMap.enabled = true;
6596     renderer.shadowMap.type = THREE.PCFSoftShadowMap;
6597     wrap.insertBefore(renderer.domElement, wrap.firstChild);
6598     /* yorug'lik tizimi: hemisfera + kalit (soyali) + to'ldiruvchi */
6599     const amb = new THREE.AmbientLight(0xffffff, 0.45);
6600     const hemi = new THREE.HemisphereLight(0xff6e6e, 0x8a7a5c, 0.6);
6601     const key = new THREE.DirectionalLight(0xffffff, 1.25);
6602     key.position.set(3, 5, 4);
6603     key.castShadow = true;
6604     /* Soya xaritasi 2048: 1024 da 8x8 maydonga 128 tekse/birlik tushardi,
6605     ya'ni bitta muqarnas katagiga ~19 tekse — o'yma ichidagi soya
6606     "pog'ona-pog'ona" chiqib, chuqurlik o'rniga shovqin berardi.
6607     bias/normalBias — egri yuzalarda o'z-o'ziga soya tushirish
6608     artefaktini (shadow acne) yo'qotadi. */
6609     key.shadow.mapSize.width = 2048;
6610     key.shadow.mapSize.height = 2048;
6611     key.shadow.camera.near = 1; key.shadow.camera.far = 15;
6612     key.shadow.camera.left = -4; key.shadow.camera.right = 4;
6613     key.shadow.camera.top = 4; key.shadow.camera.bottom = -4;
6614     key.shadow.bias = -0.0005;
6615     if(key.shadow.normalBias !== undefined) key.shadow.normalBias = 0.015;
6616     const fillL = new THREE.DirectionalLight(0xffffff, 0.5);
6617     fillL.position.set(-4, 1.5, -3);
6618     scene.add(amb, hemi, key, fillL);
6619     /* — HOVLI MUHITI: naqshli tosh pol + marmar poydevor + tuman — */
6620     const floorCnv = document.createElement('canvas');
6621     floorCnv.width = floorCnv.height = 512;
6622     const fctx = floorCnv.getContext('2d');
6623     if(fctx){
6624         fctx.fillStyle = '#e7dfcb';
6625         fctx.fillRect(0,0,512,512);
6626         /* och rangdagi Hankin girih poli (3-rasm uslubida) */
6627         fctx.save();
6628         fctx.translate(256,256);
6629         fctx.globalAlpha = 0.5;
6630         try{
6631             const fp = {...P, type:'hankin8', pal:'earth', sw:1.6, dens:0.6, spir:0.45, lay:1, rot:0, skew:0};
6632             const keepBg = P._bg; P._bg = '#e7dfcb';
6633             drawHankin8(fctx, 300, fp, 1.6, '#b9a377', '#cbbb92', '#a8946a', '#efe7d2');
6634             P._bg = keepBg;
6635         }catch(e){}
6636         fctx.restore();
6637     }
6638     const floorTex = new THREE.CanvasTexture(floorCnv);
6639     floorTex.wrapS = floorTex.wrapT = THREE.RepeatWrapping;
6640     floorTex.repeat.set(3,3);
6641     const ground = new THREE.Mesh(
6642         new THREE.CircleGeometry(9, 64),
6643         new THREE.MeshStandardMaterial({map:floorTex, roughness:0.92, metalness:0})
6644     );
6645     ground.rotation.x = -Math.PI/2;
6646     ground.position.y = -1.66;
6647     ground.receiveShadow = true;
6648     scene.add(ground);
6649     /* oq marmar 8 qirrali poydevor (Xazrati Imom ustun poydevorlari uslubida) */
6650     const ped = new THREE.Mesh(
6651         new THREE.CylinderGeometry(1.2, 1.45, 0.5, 8),
6652         new THREE.MeshStandardMaterial({color:0xf2eee4, roughness:0.45, metalness:0.02})
6653     );
6654     ped.position.y = -1.41;
6655     ped.castShadow = true;
6656     ped.receiveShadow = true;

```

```

6657 scene.add(ped);
6658 scene.fog = new THREE.Fog(0xf0e9da, 9, 20);
6659 const group = new THREE.Group();
6660 scene.add(group);
6661 /* havoda suzuvchi oltin gard zararlari – sahna "nafas oladi" */
6662 const DUST_N = 130;
6663 const dPos = new Float32Array(DUST_N*3);
6664 const dSpd = new Float32Array(DUST_N);
6665 for(let i=0;i<DUST_N;i++){
6666   dPos[i*3] = (Math.random()-0.5)*7;
6667   dPos[i*3+1] = -1.6 + Math.random()*4.2;
6668   dPos[i*3+2] = (Math.random()-0.5)*7;
6669   dSpd[i] = 0.0018 + Math.random()*0.0045;
6670 }
6671 const dGeo = new THREE.BufferGeometry();
6672 dGeo.setAttribute('position', new THREE.BufferAttribute(dPos, 3));
6673 const dust = new THREE.Points(dGeo, new THREE.PointsMaterial({
6674   color:0xe0bd6a, size:0.05, transparent:true, opacity:0.55,
6675   depthWrite:false, sizeAttenuation:true
6676 }));
6677 scene.add(dust);
6678 /* SIRLANGAN CHINNI materiali (glazur) */
6679 const MatClass = THREE.MeshPhysicalMaterial || THREE.MeshStandardMaterial;
6680 const mat = new MatClass({side:THREE.DoubleSide, roughness:0.26, metalness:0.03});
6681 if(mat.clearcoat !== undefined){ mat.clearcoat = 0.7; mat.clearcoatRoughness = 0.22; }
6682 mat.emissive = new THREE.Color(0xffffffff);
6683 mat.emissiveIntensity = 0.22; /* sopol o'z rangida "nafas oladi" – qoraymaydi */
6684 /* o'yma yog'och: naqsh teksturasi jigarrang tus bilan (ustun/peshtoq uchun) */
6685 const matWood = new MatClass({side:THREE.DoubleSide, roughness:0.55, metalness:0.02, color:0xb06a2f});
6686 if(matWood.clearcoat !== undefined){ matWood.clearcoat = 0.3; matWood.clearcoatRoughness = 0.4; }
6687 matWood.emissive = new THREE.Color(0xb06a2f);
6688 matWood.emissiveIntensity = 0.12;
6689 const matStone = new THREE.MeshStandardMaterial({color:0xflede2, roughness:0.5, metalness:0.02});
6690 const matDark = new THREE.MeshStandardMaterial({color:0x151009, roughness:1});
6691 T3 = {scene, camera, renderer, group, mesh:null, mat, matWood, matStone, matDark, mats:[mat, matWood], archMats:
[], amb, hemi, key, fill:fillL, auto:false, kind:'lagan', dust:{pts:dust,pos:dPos,spd:dSpd,n:DUST_N}, parX:0, parY
:0, intro:null, ped, camLook:0, camEye:1.9, camDist:4.9};
6692 threeReady = true;
6693 const objSel = $('sel-obj');
6694 setObject(objSel ? objSel.value : 'lagan');
6695 const lightSel = $('sel-light');
6696 setLights(lightSel ? lightSel.value : 'warm');
6697 /* boshqaruv: drag = aylantirish, g'ildirak = zoom */
6698 let dragging = false, lx = 0, ly = 0;
6699 const dom = renderer.domElement;
6700 dom.style.touchAction = 'none';
6701 dom.addEventListener('pointerdown', e=>{ dragging = true; lx = e.clientX; ly = e.clientY; dom.setPointerCapture
&& dom.setPointerCapture(e.pointerId); });
6702 dom.addEventListener('pointermove', e=>{
6703   /* yumshoq parallaks – kamera kursorga ergashadi */
6704   const rct = dom.getBoundingClientRect() ? dom.getBoundingClientRect() : {left:0,top:0,width:800,height:500};
6705   T3.parX = ((e.clientX - rct.left)/Math.max(1,rct.width) - 0.5)*0.8;
6706   T3.parY = -((e.clientY - rct.top)/Math.max(1,rct.height) - 0.5)*0.45;
6707   if(!dragging) return;
6708   group.rotation.y += (e.clientX - lx)*0.01;
6709   group.rotation.x += (e.clientY - ly)*0.008;
6710   group.rotation.x = Math.max(-1.4, Math.min(1.4, group.rotation.x));
6711   lx = e.clientX; ly = e.clientY;
6712 });
6713 dom.addEventListener('pointerup', ()=>{ dragging = false; });
6714 dom.addEventListener('pointercancel', ()=>{ dragging = false; });
6715 dom.addEventListener('wheel', e=>{
6716   e.preventDefault();
6717   camera.position.z = Math.max(2, Math.min(12, camera.position.z + e.deltaY*0.004));
6718 }, {passive:false});
6719 const animate = ()=>{
6720   requestAnimationFrame(animate);
6721   const page = $('page-3d');
6722   if(!page || !page.classList.contains('active')) return;
6723   const tNow = (window.performance ? performance.now() : Date.now())*0.001;
6724   if(T3.auto) group.rotation.y += 0.007;
6725   /* buyum havoda ohista suzadi */
6726   if(!dragging) group.position.y = Math.sin(tNow*0.9)*0.045;
6727   /* kirish springi – easeOutBack */
6728   if(T3.intro && T3.mesh){
6729     T3.intro.t = Math.min(1, T3.intro.t + 0.04);
6730     const k = T3.intro.t;
6731     const c1 = 1.70158, c3 = c1 + 1;
6732     const sc = 1 + c3*Math.pow(k-1,3) + c1*Math.pow(k-1,2);
6733     T3.mesh.scale.setScalar(Math.max(0.01, sc));
6734     if(k >= 1) T3.intro = null;
6735   }
6736   /* oltin gard – yuqoriga suzadi */
6737   if(T3.dust){
6738     const p = T3.dust.pos;
6739     for(let i=0;i<T3.dust.n;i++){
6740       p[i*3+1] += T3.dust.spd[i];
6741       p[i*3] += Math.sin(tNow + i)*0.0006;
6742       if(p[i*3+1] > 2.8) p[i*3+1] = -1.6;
6743     }
6744     T3.dust.pts.geometry.attributes.position.needsUpdate = true;

```

```

6745     }
6746     /* parallax kamera */
6747     camera.position.x += (T3.parX - camera.position.x)*0.05;
6748     const eyeY = (T3.camEye === undefined ? 1.9 : T3.camEye);
6749     camera.position.y += (eyeY + T3.parY - camera.position.y)*0.05;
6750     camera.lookAt(0, T3.camLook || 0, 0);
6751     renderer.render(scene, camera);
6752   };
6753   animate();
6754   window.addEventListener('resize', ()=>{
6755     const w2 = wrap.clientWidth||800, h2 = wrap.clientHeight||500;
6756     camera.aspect = w2/h2;
6757     camera.updateProjectionMatrix();
6758     renderer.setSize(w2, h2);
6759   });
6760 }
6761 function screenshot3D(){
6762   if(!threeReady || !T3){ ntf('Avval 3D sahifasini oching', 'err'); return; }
6763   try{
6764     T3.renderer.render(T3.scene, T3.camera);
6765     const url = T3.renderer.domElement.toDataURL('image/png');
6766     _d1Blob(_dataURLtoBlob(url), 'image/png', 'naqsh_3d_'+P.type+'.png', '3D surat saqlandi ');
6767   }catch(e){
6768     ntf('Surat olishda xato: '+e.message, 'err');
6769   }
6770 }
6771
6772 /* =====
6773    AR PROYEKSIYA
6774    ===== */
6775 let arStream = null, arActive = false, arNeedsRedraw = true;
6776 let arPatCanvas = null;
6777 const AR = { surface:'devor', op:0.82, size:0.55, rot:0, x:0, y:0, simMode:false };
6778
6779 /* Kamerasiz Simulyatsiya foni: kamera mavjud bo'lmagan yoki ruxsat
6780 berilmagan muhitlarda (masalan, ushbu ilova preview sifatida
6781 ko'rsatilganda) foydalanuvchi baribir naqshni sirt ustida ko'ra oladi. */
6782 function drawARBackdrop(ctx, W, H){
6783   ctx.save();
6784   if(AR.surface==='pol'){
6785     const g = ctx.createLinearGradient(0,0,0,H);
6786     g.addColorStop(0,'#8a6a44'); g.addColorStop(1,'#4a3620');
6787     ctx.fillStyle = g; ctx.fillRect(0,0,W,H);
6788     ctx.strokeStyle = 'rgba(0,0,0,.18)'; ctx.lineWidth = 2;
6789     for(let i=-4;i<=4;i++){
6790       ctx.beginPath(); ctx.moveTo(W/2 + i*W*0.09, H*0.15); ctx.lineTo(W/2 + i*W*0.3, H); ctx.stroke();
6791     }
6792   } else if(AR.surface==='gilam'){
6793     ctx.fillStyle = '#5a2e2e'; ctx.fillRect(0,0,W,H);
6794     ctx.strokeStyle = 'rgba(255,255,255,.06)'; ctx.lineWidth = 1;
6795     for(let x=0;x<W;x+=18){ ctx.beginPath(); ctx.moveTo(x,0); ctx.lineTo(x,H); ctx.stroke(); }
6796     for(let y=0;y<H;y+=18){ ctx.beginPath(); ctx.moveTo(0,y); ctx.lineTo(W,y); ctx.stroke(); }
6797   } else if(AR.surface==='piyola'){
6798     const g = ctx.createRadialGradient(W/2,H*0.42,H*0.05,W/2,H*0.42,H*0.5);
6799     g.addColorStop(0,'#e8e0c8'); g.addColorStop(0.7,'#c9bf9e'); g.addColorStop(1,'#8f8365');
6800     ctx.fillStyle = g; ctx.fillRect(0,0,W,H);
6801   } else { /* devor */
6802     const g = ctx.createLinearGradient(0,0,0,H);
6803     g.addColorStop(0,'#efe9d8'); g.addColorStop(1,'#d8cfb2');
6804     ctx.fillStyle = g; ctx.fillRect(0,0,W,H);
6805   }
6806   ctx.fillStyle = 'rgba(255,255,255,.85)';
6807   ctx.font = '600 12px Manrope, sans-serif';
6808   ctx.textAlign = 'center';
6809   ctx.fillText('SIMULYATSIYA – kamerasiz ko'rinish', W/2, H-16);
6810   ctx.restore();
6811 }
6812 function startARSim(){
6813   const errBox = $('ar-err'); if(errBox) errBox.textContent = '';
6814   AR.simMode = true;
6815   arActive = true;
6816   arNeedsRedraw = true;
6817   const video = $('ar-video'); if(video) video.style.display = 'none';
6818   const card = $('ar-start-card'); if(card) card.style.display = 'none';
6819   const bar = $('ar-bar'); if(bar){ bar.hidden = false; bar.style.display = 'flex'; }
6820   arLoop();
6821   ntf('Simulyatsiya rejimi yoqildi ', 'ok');
6822 }
6823
6824 /* AR uchun ikki xil tasvir kerak:
6825   • MEDALYON – devor va piyolada bitta naqsh sifatida yotadi;
6826   • CHOK-SUZ MAYDON – gilam va polda naqsh takrorlanadi, shuning uchun u
6827     haqiqiy rapportdan yig'iladi. Ilgari bu yerda ham medalyon ishlatilib,
6828     u 3x3 qilib takrorlanardi – natijada plitkalar orasida UZILISH va
6829     bo'sh joylar ko'rinaradi. */
6830 let arTileCanvas = null, arTileKey = '';
6831 function arTileField(W, H){
6832   const key = P.type + '|' + P.pal + '|' + P.sym + '|' + P.cmp + '|' + P.dens + '|' +
6833     P.sw + '|' + P.lay + '|' + (T3D.group || 'p4') + '|' + W + 'x' + H;
6834   if(arTileCanvas && arTileKey === key) return arTileCanvas;
6835   let rep = null;

```

```

6836 try{ rep = makeSeamlessRepeat(256, T3D.group || 'p4', {}); }catch(e){ console.warn('[ar] rapport', e); }
6837 const cnv = document.createElement('canvas');
6838 cnv.width = W; cnv.height = H;
6839 const c2 = cnv.getContext('2d');
6840 if(!c2) return null;
6841 if(rep && rep.canvas){
6842   const pat = c2.createPattern(rep.canvas, 'repeat');
6843   if(pat){ c2.fillStyle = pat; c2.fillRect(0, 0, W, H); }
6844 } else {
6845   /* zaxira – rapport chiqmasa medalyon bilan to'ldiramiz */
6846   const t = Math.min(W, H)/2;
6847   for(let y = 0; y < H; y += t) for(let x = 0; x < W; x += t)
6848     drawPattern(c2, x + t/2, y + t/2, t*0.5, P);
6849 }
6850 arTileCanvas = cnv; arTileKey = key;
6851 return cnv;
6852 }
6853 function renderArPattern(){
6854   if(!arPatCanvas){
6855     arPatCanvas = document.createElement('canvas');
6856     arPatCanvas.width = arPatCanvas.height = 512;
6857   }
6858   const octx = arPatCanvas.getContext('2d');
6859   if(!octx) return;
6860   octx.clearRect(0,0,512,512); /* fon TO'LIQ SHAFFOF – hech qanday fill yo'q */
6861   drawPattern(octx, 256, 256, 236, P);
6862   arTileKey = ''; /* maydon ham qayta yig'ilsin */
6863   arNeedsRedraw = false;
6864 }
6865 function drawArSurface(ctx, W, H){
6866   const size = Math.min(W,H)*AR.size;
6867   const cx = W/2 + AR.x, cy = H/2 + AR.y;
6868   ctx.save();
6869   ctx.globalAlpha = AR.op;
6870   if(AR.surface==='devor'){
6871     ctx.translate(cx,cy);
6872     ctx.rotate(AR.rot*Math.PI/180);
6873     ctx.drawImage(arPatCanvas, -size/2, -size/2, size, size);
6874   } else if(AR.surface==='piyola'){
6875     /* elliptik deformatsiya: naqsh pastga qarab siqiladi */
6876     ctx.translate(cx,cy);
6877     ctx.rotate(AR.rot*Math.PI/180);
6878     const rows = 36, srcH = 512/rows;
6879     let dy = -size/2;
6880     for(let j=0;j<rows;j++){
6881       const t = j/rows;
6882       const k = 1 - 0.5*t*t; /* pastga qarab siqilish */
6883       const wRow = size*(0.72 + 0.28*Math.sqrt(Math.max(0,1-t*t)));
6884       const hRow = (size/rows)*k;
6885       ctx.drawImage(arPatCanvas, 0, j*srcH, 512, srcH, -wRow/2, dy, wRow, hRow+0.7);
6886       dy += hRow;
6887     }
6888   } else if(AR.surface==='pol'){
6889     /* perspektiv trapetsiya: uzoqda tor, yaqinida keng.
6890     Manba CHOK-SUZ maydon – shuning uchun qatorlar orasida uzilish yo'q. */
6891     ctx.translate(cx,cy);
6892     const src = arTileField(1024, 1024) || arPatCanvas;
6893     const SH = src.height, SW = src.width;
6894     const rows = 36, srcH = SH/rows;
6895     let dy = -size/2;
6896     for(let j=0;j<rows;j++){
6897       const t = j/rows;
6898       const k = 0.35 + 0.75*t;
6899       const wRow = size*1.25*k;
6900       const hRow = (size/rows)*k;
6901       ctx.save();
6902       ctx.rotate(AR.rot*Math.PI/180*0.2);
6903       ctx.drawImage(src, 0, j*srcH, SW, srcH, -wRow/2, dy, wRow, hRow+0.7);
6904       ctx.restore();
6905       dy += hRow;
6906     }
6907   } else { /* gilam: CHOK-SUZ maydon – rapport takrorlanadi, uzilish yo'q */
6908     ctx.translate(cx,cy);
6909     ctx.rotate(AR.rot*Math.PI/180);
6910     const fld = arTileField(1024, 1024);
6911     if(fld){
6912       const pat = ctx.createPattern(fld, 'repeat');
6913       const half = size*0.85;
6914       if(pat){
6915         ctx.save();
6916         /* naqsh AR.size ga qarab kattalashadi */
6917         const k = size/512;
6918         ctx.scale(k, k);
6919         ctx.fillStyle = pat;
6920         ctx.fillRect(-half/k, -half/k, 2*half/k, 2*half/k);
6921         ctx.restore();
6922       } else {
6923         ctx.drawImage(fld, -half, -half, 2*half, 2*half);
6924       }
6925     }
6926   }

```

```

6927   ctx.restore();
6928 }
6929 function arLoop(){
6930   if(!arActive) return;
6931   const overlay = $('ar-overlay');
6932   const stage = $('ar-stage');
6933   if(overlay && stage){
6934     if(overlay.width !== stage.clientWidth || overlay.height !== stage.clientHeight){
6935       overlay.width = stage.clientWidth||300;
6936       overlay.height = stage.clientHeight||300;
6937     }
6938     const ctx = overlay.getContext('2d');
6939     if(ctx){
6940       if(arNeedsRedraw || !arPatCanvas) renderArPattern();
6941       ctx.clearRect(0,0,overlay.width,overlay.height);
6942       if(AR.simMode) drawArBackdrop(ctx, overlay.width, overlay.height);
6943       drawArSurface(ctx, overlay.width, overlay.height);
6944     }
6945   }
6946   requestAnimationFrame(arLoop);
6947 }
6948 function startAR(){
6949   const errBox = $('ar-err');
6950   if(errBox) errBox.textContent = '';
6951   if(!navigator.mediaDevices || !navigator.mediaDevices.getUserMedia){
6952     /* Brauzer kamerani faqat XAVFSIZ MUHITDA beradi (https yoki localhost).
6953     Oddiy http:// da navigator.mediaDevices umuman mavjud bo'lmaydi va
6954     ilgari bu "brauzeringiz qo'llab-quvvatlamaydi" deb ko'rsatilardi -
6955     sabab noto'g'ri aytilgan bo'lardi. */
6956     const secure = (typeof isSecureContext !== 'undefined') ? isSecureContext
6957       : (location.protocol === 'https:' || location.hostname === 'localhost');
6958     const msg = secure
6959       ? 'Brauzeringiz kamerani qo'llab-quvvatlamaydi.'
6960       : 'Kamera faqat HTTPS orqali ishlaydi. Saytni https:// bilan oching '
6961       + '(GitHub Pages allaqachon HTTPS). Hozircha «Simulyatsiya» tugmasidan foydalaning.';
6962     if(errBox) errBox.textContent = msg;
6963     ntf(msg, 'err');
6964     return;
6965   }
6966   navigator.mediaDevices.getUserMedia({video:{facingMode:'environment'}})
6967   .catch(()=>navigator.mediaDevices.getUserMedia({video:true}))
6968   .then(stream=>{
6969     arStream = stream;
6970     const video = $('ar-video');
6971     if(video){
6972       video.srcObject = stream;
6973       video.style.display = ''; /* simulyatsiyadan keyin qaytariladi */
6974       /* autoplay atributi bor, lekin ba'zi brauzerlar aniq play() talab
6975       qiladi - xatosi jimgina yutilmasin */
6976       const pr = video.play();
6977       if(pr && pr.catch) pr.catch(e=>{
6978         console.warn('[ar] video.play', e);
6979         if(errBox) errBox.textContent = 'Videoni boshlab bo'lmadi: ' + e.name;
6980       });
6981     }
6982     arActive = true;
6983     arNeedsRedraw = true;
6984     const card = $('ar-start-card'); if(card) card.style.display = 'none';
6985     const bar = $('ar-bar'); if(bar){ bar.hidden = false; bar.style.display = 'flex'; }
6986     arLoop();
6987     ntf('AR rejim yoqildi ☑', 'ok');
6988   })
6989   .catch(err=>{
6990     let msg = 'Kamera xatosi: ' + err.name;
6991     if(err.name === 'NotAllowedError') msg = 'Kameraga ruxsat berilmadi – brauzer sozlamalaridan ruxsat bering'
+ (location.protocol==='file:' ? ' (agar file:// da bloklangan bo'lsa, HTTPS/GitHub Pages orqali oching)' : '') +
+ '.';
6992     else if(err.name === 'NotFoundError') msg = 'Kamera topilmadi.';
6993     else if(err.name === 'NotReadableError') msg = 'Kamera band – boshqa ilova ishlatayotgan bo'lishi mumkin.';
6994     if(errBox) errBox.textContent = msg;
6995     ntf(msg, 'err');
6996   });
6997 }
6998 function stopAR(){
6999   arActive = false;
7000   AR.simMode = false;
7001   if(arStream){
7002     arStream.getTracks().forEach(tr=>tr.stop()); /* streamni to'xtatish */
7003     arStream = null;
7004   }
7005   const video = $('ar-video');
7006   if(video){ video.srcObject = null; video.style.display = ''; }
7007   const overlay = $('ar-overlay');
7008   if(overlay){
7009     /* keyingi safar boshlanguncha eski kadr ko'rinib qolmasin */
7010     const octx = overlay.getContext('2d');
7011     if(octx) octx.clearRect(0, 0, overlay.width, overlay.height);
7012   }
7013   const card = $('ar-start-card'); if(card) card.style.display = 'flex';
7014   /* .ar-bar{display:flex} qoidasi [hidden] ustidan ustunlik qiladi –
7015   shuning uchun inline style bilan ham aniq yashiramiz */

```

```

7016   const bar = $('ar-bar'); if(bar){ bar.hidden = true; bar.style.display = 'none'; }
7017 }
7018 function captureAR(){
7019   const video = $('ar-video'), overlay = $('ar-overlay');
7020   if(!overlay || !arActive){ ntf('Avval kamerani yoqing yoki simulyatsiyani boshlang', 'err'); return; }
7021   const W = (!AR.simMode && video && video.videoWidth) || overlay.width || 640;
7022   const H = (!AR.simMode && video && video.videoHeight) || overlay.height || 480;
7023   const off = document.createElement('canvas');
7024   off.width = W; off.height = H;
7025   const octx = off.getContext('2d');
7026   if(!octx) return;
7027   if(AR.simMode){
7028     drawArBackdrop(octx, W, H);
7029   } else if(video){
7030     octx.drawImage(video, 0, 0, W, H);
7031   }
7032   /* overlay object-fit:cover ga mos masshtabda ustiga chiziladi */
7033   const sc = Math.max(W/overlay.width, H/overlay.height);
7034   const dw = overlay.width*sc, dh = overlay.height*sc;
7035   octx.drawImage(overlay, (W-dw)/2, (H-dh)/2, dw, dh);
7036   if(off.toBlob){
7037     off.toBlob(b=>{ if(b) _d1Blob(b, 'image/png', 'naqsh_ar.png', 'AR surat saqlandi '); }, 'image/png');
7038   }
7039 }
7040
7041 /* =====
7042   AI CHAT — lokal NLP parser, rasm-natija markazli
7043   ===== */
7044 const CHAT_TYPES = [
7045   ['girihi', 'girihi'], ['xatam', 'xatam'], ['rozetta', 'rozetta'], ['yulduz', 'yulduz'],
7046   ['muqarnas', 'muqarnas'], ['katak', 'katak8'], ['olti burchak', 'hexa'], ['asal', 'hexa'], ['zanjira', 'zanjira'],
7047   ['hankin 6', 'hankin6'], ['hankin olti', 'hankin6'], ['hankin', 'hankin8'], ['sakkiz qirrali', 'panelgirihi'], ['panel', 'panelgirihi'],
7048   ['islamiy', 'islamiy'], ['rumi', 'rumi'], ['hatayi', 'hatayi'], ['palak', 'palak'], ['shamsa', 'shamsa'],
7049   ['arabesk', 'arabesk'], ['arabesque', 'arabesk'], ['novda', 'scroll'], ['scroll', 'scroll'], ['medalyon', 'medallion'], ['medallion', 'medallion'],
7050   ['celtic', 'celtic'], ['kelt', 'celtic'], ['mandala', 'mandala'], ['penrose', 'penrose'], ['zellige', 'zellige'], ['zelij', 'zellige'],
7051   ['batik', 'batik'], ['paisley', 'paisley'], ['buta', 'paisley'], ['greek', 'greek'], ['meander', 'greek'], ['xitoy', 'xitoy'],
7052   ['koch', 'koch'], ['qor parcha', 'koch'], ['sierpinski', 'sierpinski'], ['dragon', 'dragon'], ['ajdar', 'dragon'],
7053   ['hilbert', 'hilbert'], ['spirograf', 'spirograph'], ['spirograph', 'spirograph'], ['lissajous', 'lissajous'], ['lissajous', 'lissajous'],
7054 ];
7055 const CHAT_COLORS = [
7056   ['oltin', 'gold'], ['tilla', 'gold'], ['feruza', 'turquoise'], ['turkuaz', 'turquoise'], ['registon', 'turquoise'],
7057   ['zumrad', 'emerald'], ['yashil', 'emerald'], ['yoqut', 'ruby'], ['qizil', 'ruby'],
7058   ['ko'k', 'navy'], ['kok', 'navy'], ['navy', 'navy'], ['tuproq', 'earth'], ['jigarrang', 'earth']
7059 ];
7060 /* Matn -> barqaror hash (bir xil matn = bir xil naqsh) */
7061 function hashStr(str){
7062   let h = 5381;
7063   for(let i=0; i<str.length; i++){ h = ((h<<5)+h) + str.charCodeAt(i) >>> 0; }
7064   return h >>> 0;
7065 }
7066 function parseChat(txt){
7067   const q = (txt||'').toLowerCase();
7068   const np = {...P}; delete np._bg;
7069   /* 1) HAR QANDAY MATN naqsh tug'diradi: matn hashi -> deterministik parametrlar
7070      (nano-banana uslubi: "salom dunyo" ham, ism ham o'z noyob naqshiga ega) */
7071   const seed = hashStr(q);
7072   const rnd = mulberry32(seed);
7073   const ids = PATTERNS.map(x=>x.id);
7074   const pals = Object.keys(PAL);
7075   np.type = ids[Math.floor(rnd()*ids.length)];
7076   np.pal = pals[Math.floor(rnd()*pals.length)];
7077   np.sym = 5 + Math.floor(rnd()*10);
7078   np.cmp = 2 + Math.floor(rnd()*6);
7079   np.sw = +(1.2 + rnd()*2.6).toFixed(1);
7080   np.ir = +(0.2 + rnd()*0.5).toFixed(2);
7081   np.spir = +(rnd()).toFixed(2);
7082   np.dens = +(0.3 + rnd()*0.6).toFixed(2);
7083   np.lay = 1 + Math.floor(rnd()*3);
7084   np.skew = 0;
7085   np.rot = Math.floor(rnd()*360);
7086   np._seed = seed % 100000;
7087   /* 2) Kalit so'zlar seedni ANIQLASHTIRADI (nomi, rangi, soni aytilsa — shu bo'ladi) */
7088   for(const [kw, id] of CHAT_TYPES){ if(q.indexOf(kw)>=0){ np.type = id; break; } }
7089   /* rang */
7090   for(const [kw, id] of CHAT_COLORS){ if(q.indexOf(kw)>=0){ np.pal = id; break; } }
7091   /* sonlar: "8 simmetriya", "12 ta" */
7092   const m = q.match(/(\d+)\s*(ta|simmetriya|simmetriyal|burchak|qirra)/);
7093   if(m){ np.sym = Math.max(3, Math.min(16, parseInt(m[1], 10))); }
7094   else { const m2 = q.match(/b{1,2}\b/); if(m2){ const v=parseInt(m2[1], 10); if(v>=3 && v<=16) np.sym = v; } }
7095 }
7096 /* sifatlar */
7097 if(q.indexOf('zich')>=0) np.dens = 0.8;
7098 if(q.indexOf('siyrak')>=0) np.dens = 0.3;
7099 if(q.indexOf('murakkab')>=0) np.cmp = 6;
7100 if(q.indexOf('sodda')>=0) np.cmp = 2;
7101 if(q.indexOf('ingichka')>=0) np.sw = 1;

```

```

7101   if(q.indexOf('qalin')>=0)      np.sw   = 4;
7102   if(q.indexOf('spiral')>=0)      np.spir = 0.8;
7103   if(q.indexOf('qatlam')>=0)      np.lay  = 4;
7104   if(q.indexOf('tessell')>=0 || q.indexOf('tiling')>=0 || q.indexOf('takror')>=0) np._tess = true;
7105   if(q.indexOf('p4m')>=0) np._grid='p4m'; else if(q.indexOf('p6m')>=0) np._grid='p6m';
7106   else if(q.indexOf('p6')>=0) np._grid='p6'; else if(q.indexOf('p4')>=0) np._grid='p4';
7107   return np;
7108 }
7109 function chatCard(np, query){
7110   const log = $('chat-log');
7111   if(!log) return;
7112   const card = document.createElement('div');
7113   card.className = 'chat-row';
7114   const inner = document.createElement('div');
7115   inner.className = 'ai-card';
7116   const cnv = document.createElement('canvas');
7117   cnv.width = cnv.height = 600; /* 2x – CorelDraw kabi tiniq preview */
7118   inner.appendChild(cnv);
7119   const octx = cnv.getContext('2d');
7120   if(octx){
7121     const pal = PAL[np.pal] || PAL.gold;
7122     const keepBg = P._bg;
7123     P._bg = pal.bg;
7124     octx.fillStyle = pal.bg;
7125     octx.fillRect(0,0,600,600);
7126     if(np._tess){
7127       const keepG = TESS.group, keepC = TESS.cell, keepS = TESS.showCell;
7128       TESS.group = np._grid || TESS.group;
7129       TESS.cell = 200; TESS.showCell = false;
7130       drawTessellation(octx, 600, 600, np);
7131       TESS.group = keepG; TESS.cell = keepC; TESS.showCell = keepS;
7132     } else {
7133       drawPattern(octx, 300, 300, 276, np);
7134     }
7135     P._bg = keepBg;
7136   }
7137   const meta = document.createElement('div');
7138   meta.className = 'meta';
7139   const pt = getPattern(np.type), pal2 = PAL[np.pal] || PAL.gold;
7140   meta.innerHTML = 'Turi: <b>'+pt.nm+'</b> · Simmetriya: <b>'+np.sym+'</b> · Palitra: <b>'+pal2.nm+'</b> · Murakka
blik: <b>'+np.cmp+'</b>' + (np._tess ? ' · <b>Tessellatsiya '+ (np._grid || TESS.group) + '</b>' : '') + (np._seed !== un
defined ? ' · Seed <b>'+np._seed+'</b>' : '');
7141   inner.appendChild(meta);
7142   const acts = document.createElement('div');
7143   acts.className = 'acts';
7144   const bOpen = document.createElement('button');
7145   bOpen.className = 'btn sm'; bOpen.textContent = 'Studio da ochish';
7146   bOpen.addEventListener('click', ()=>{
7147     const clean = {...np}; delete clean._tess; delete clean._grid; delete clean._seed;
7148     Object.assign(P, clean);
7149     if(np._tess){
7150       TESS.on = true;
7151       if(np._grid) TESS.group = np._grid;
7152       const chk = $('chk-tess'); if(chk) chk.checked = true;
7153       const sel = $('sel-tess'); if(sel) sel.value = TESS.group;
7154     }
7155     syncSliders(); redraw();
7156     showPage('studio');
7157     ntf('Naqsh Studio ga yuklandi', 'ok');
7158   });
7159   const bVar = document.createElement('button');
7160   bVar.className = 'btn sm ghost'; bVar.textContent = 'Yana variant';
7161   bVar.addEventListener('click', ()=>{
7162     const v = {...np};
7163     v.spir = +(Math.random()).toFixed(2);
7164     v.dens = +(0.2+Math.random()*0.7).toFixed(2);
7165     v.ir   = +(0.15+Math.random()*0.6).toFixed(2);
7166     v.cmp  = 1+Math.floor(Math.random()*8);
7167     v.rot  = Math.floor(Math.random()*360);
7168     v.lay  = 1+Math.floor(Math.random()*4);
7169     chatCard(v, query);
7170     saveChatHistory();
7171   });
7172   const bDL = document.createElement('button');
7173   bDL.className = 'btn sm gold'; bDL.textContent = 'Yuklab olish';
7174   bDL.addEventListener('click', ()=>{
7175     if(cnv.toBlob) cnv.toBlob(b=>{ if(b) _dlBlob(b,'image/png','naqsh_chat_'+np.type+'.png', 'Rasm yuklab olindi
'); }, 'image/png');
7176   });
7177   acts.appendChild(bOpen); acts.appendChild(bVar); acts.appendChild(bDL);
7178   inner.appendChild(acts);
7179   card.appendChild(inner);
7180   card._np = np;
7181   log.appendChild(card);
7182   log.scrollTop = log.scrollHeight;
7183 }
7184 function chatUserMsg(txt){
7185   const log = $('chat-log');
7186   if(!log) return;
7187   const row = document.createElement('div');
7188   row.className = 'chat-row user';

```



```

7189     const b = document.createElement('div');
7190     b.className = 'bubble-user';
7191     b.textContent = txt;
7192     row.appendChild(b);
7193     row._q = txt;
7194     log.appendChild(row);
7195     log.scrollTop = log.scrollHeight;
7196   }
7197   function saveChatHistory(){
7198     const log = $('chat-log');
7199     if(!log) return;
7200     const hist = [];
7201     Array.prototype.forEach.call(log.children, ch=>{
7202       if(ch._q) hist.push({q:ch._q});
7203       else if(ch._np) hist.push({p:ch._np});
7204     });
7205     lsSet('naqsh_chat', hist.slice(-40));
7206   }
7207   function restoreChatHistory(){
7208     const hist = lsGet('naqsh_chat', []);
7209     hist.forEach(item=>{
7210       if(item.q) chatUserMsg(item.q);
7211       else if(item.p) chatCard(item.p, '');
7212     });
7213   }
7214   /* =====
7215     AI ERKIN TASVIR – Nano Banana (Gemini) orqali, Cloudflare Worker
7216     proksisi bilan. API kalit hech qachon shu faylda saqlanmaydi –
7217     faqat foydalanuvchi o'zi joylashtirgan Worker manzili + token
7218     localStorage'da turadi.
7219     ===== */
7220   let CHAT_MODE = 'exact'; /* 'exact' | 'ai' */
7221   const AI_MODEL_DEFAULT = 'gemini-2.5-flash-image';
7222   const lsGetStr = (k, d) => { try{ return (localStorage.getItem(k) || d || '').trim(); }catch(e){ return d || ''; }
7223   };
7224   const lsSetStr = (k, v) => { try{ localStorage.setItem(k, (v||'').trim()); return true; }catch(e){ return false; }
7225   };
7226   function getAiConn(){ return lsGetStr('naqsh_ai_conn', 'direct') === 'worker' ? 'worker' : 'direct'; }
7227   function setAiConn(v){ return lsSetStr('naqsh_ai_conn', v === 'worker' ? 'worker' : 'direct'); }
7228   function getAiWorkerUrl(){ return lsGetStr('naqsh_ai_worker', ''); }
7229   function setAiWorkerUrl(v){ return lsSetStr('naqsh_ai_worker', v); }
7230   function getAiAppToken(){ return lsGetStr('naqsh_ai_token', ''); }
7231   function setAiAppToken(v){ return lsSetStr('naqsh_ai_token', v); }
7232   function getAiKey(){ return lsGetStr('naqsh_ai_key', ''); }
7233   function setAiKey(v){ return lsSetStr('naqsh_ai_key', v); }
7234   function getAiModel(){ return lsGetStr('naqsh_ai_model', AI_MODEL_DEFAULT) || AI_MODEL_DEFAULT; }
7235   function setAiModel(v){ return lsSetStr('naqsh_ai_model', v); }
7236   /* AI rasm rejimi ishlashga tayyormi? */
7237   function aiReady(){
7238     return getAiConn() === 'worker' ? !!getAiWorkerUrl() : !!getAiKey();
7239   }
7240   function aiShowConn(which){
7241     const w = (which === 'worker');
7242     const seg = $('ai-conn-seg');
7243     if(seg) seg.querySelectorAll('button').forEach(b=> b.classList.toggle('sel', (b.dataset.v === (w?'worker':'direc
7244     t'))));
7245     const d = $('ai-direct-box'), wb = $('ai-worker-box');
7246     if(d) d.hidden = w;
7247     if(wb) wb.hidden = !w;
7248   }
7249   function openAiSetup(){
7250     const u=$('ai-worker-url'), t=$('ai-app-token'), k=$('ai-key'), m=$('ai-model');
7251     if(u) u.value = getAiWorkerUrl();
7252     if(t) t.value = getAiAppToken();
7253     if(k) k.value = getAiKey();
7254     if(m) m.value = getAiModel();
7255     aiShowConn(getAiConn());
7256     const md=$('modal-ai-setup'); if(md) md.classList.add('open');
7257   }
7258   function closeAiSetup(){ const m=$('modal-ai-setup'); if(m) m.classList.remove('open'); }
7259   function aiLoadingCard(){
7260     const log = $('chat-log');
7261     if(!log) return null;
7262     const card = document.createElement('div');
7263     card.className = 'ai-card ai-loading';
7264     card.innerHTML = '<div class="ai-spin"></div><div> Nano Banana tasvirni chizyapti... (2\u2011soniya)</div>';
7265     log.appendChild(card);
7266     log.scrollTop = log.scrollHeight;
7267     return card;
7268   }
7269   /* Gemini javobidan birinchi rasmni ajratib olish.
7270     Worker ham, to'g'ridan-to'g'ri Google ham shu funksiyaga tushadi:
7271     Worker {image,mime} qaytaradi, Google esa candidates[].content.parts[]
7272     ichida inlineData beradi. Ikkalasini ham qo'llab-quvvatlaymiz. */
7273   function aiExtractImage(data){
7274     if(!data) return null;
7275     if(data.image) return { b64: data.image, mime: data.mime || 'image/png' };
7276     const cand = data.candidates;

```

```

7277     if(Array.isArray(cands)){
7278         for(const c of cands){
7279             const parts = c && c.content && c.content.parts;
7280             if(!Array.isArray(parts)) continue;
7281             for(const p of parts){
7282                 const inl = p.inlineData || p.inline_data;
7283                 if(inl && inl.data) return { b64: inl.data, mime: inl.mimeType || inl.mime_type || 'image/png' };
7284             }
7285         }
7286     }
7287     return null;
7288 }
7289 /* Google xato javobini tushunarli xabarga aylantirish */
7290 function aiApiErr(status, data){
7291     const msg = (data && data.error && data.error.message) || '';
7292     console.error(['ai'], status, data);
7293     const m = msg.toLowerCase();
7294
7295     if(status === 400 && /api key not valid/.test(m))
7296         return { short:'API kalit noto\'g\'ri – tekshirib qayta kiriting', detail:msg };
7297     if(status === 403)
7298         return { short:'Kalitga ruxsat yo\'q – Google AI Studio\'da kalitni tekshiring', detail:msg };
7299     if(status === 404)
7300         return { short:'Model topilmadi – «Model» maydonidagi nomni tekshiring', detail:msg };
7301
7302     if(status === 429){
7303         /* Google 429 ni ikki xil holatda qaytaradi va ular butunlay boshqacha:
7304             1) haqiqiy tezlik chegarasi – kutib qayta urinsa bo'ladi;
7305             2) bu model bepul rejada UMUMAN yo'q (quota_limit_value: 0) –
7306                kutish yordam bermaydi, to'lov (billing) yoqilishi kerak.
7307             Ikkinchisini "birozdan so'ng urining" deb ko'rsatish – chalg'itish. */
7308         const freeTier = /free[_ ]tier|billing|plan and billing|quota_limit_value.*?0\b/.test(m);
7309         if(freeTier) return {
7310             short:'Bu model bepul rejada mavjud emas – Google hisobingizda to'lov (billing) yoqilishi kerak',
7311             detail:msg, hint:'billing'
7312         };
7313         return { short:'So'rovlar chegarasi – 1–2 daqiqadan so'ng qayta urining', detail:msg };
7314     }
7315     if(status >= 500)
7316         return { short:'Google serverida vaqtincha nosozlik – qayta urining', detail:msg };
7317     return { short: msg ? ('Xato: ' + msg.slice(0,120)) : ('Xato: HTTP ' + status), detail:msg };
7318 }
7319
7320 async function aiGenerate(prompt){
7321     if(!aiReady()){
7322         ntf('Avval o AI sozlash – API kalitini kiriting', 'err');
7323         openAiSetup();
7324         return;
7325     }
7326     const card = aiLoadingCard();
7327     /* e – matn yoki {short, detail, hint}. Google'ning O'Z xabari yashirilmaydi:
7328         aynan o'sha matn muammoning haqiqiy sababini aytadi. */
7329     const fail = (e)=>{
7330         const short = (typeof e === 'string') ? e : e.short;
7331         const detail = (typeof e === 'string') ? '' : (e.detail || '');
7332         const hint = (typeof e === 'string') ? '' : (e.hint || '');
7333         if(card){
7334             card.classList.remove('ai-loading');
7335             let html = '<div class="ai-err">▲ ' + escHTML(short) + '</div>';
7336             if(hint === 'billing'){
7337                 html += '<div class="meta" style="line-height:1.6">Nima qilish kerak:<br> +
7338                     '1. <b>aistudio.google.com/apikey</b> → kalitingiz qaysi loyihaga tegishli ekanini ko'ring<br> +
7339                     '2. <b>console.cloud.google.com/billing</b> → o'sha loyihaga to'lov usulini ulang<br> +
7340                     '3. Yoki « AI sozlash → Model» da bepul rejada mavjud boshqa modelni sinang<br> +
7341                     '<i>Eslatma: «Aniq naqsh» rejimi bepul va cheksiz ishlaydi.</i></div>';
7342             }
7343             if(detail){
7344                 html += '<details class="meta"><summary>Google javobi (texnik)</summary> +
7345                     '<div style="white-space:pre-wrap;word-break:break-word;margin-top:6px"> +
7346                     escHTML(detail.slice(0, 600)) + '</div></details>';
7347             }
7348             card.innerHTML = html;
7349         }
7350         ntf(short, 'err');
7351     };
7352     try{
7353         let res, data;
7354         if(getAiConn() === 'worker'){
7355             res = await fetch(getAiWorkerUrl(), {
7356                 method: 'POST',
7357                 headers: { 'Content-Type': 'application/json', 'X-App-Token': getAiAppToken() },
7358                 body: JSON.stringify({ prompt })
7359             });
7360             data = await res.json().catch(()=>({}));
7361             if(!res.ok){
7362                 /* Worker xatoni matn sifatida uzatadi ({error:"..."}), Google esa
7363                 obyekt sifatida ({error:{message}}). Ikkalasini ham aiApiErr
7364                 tushunadigan bitta shaklga keltiramiz – shunda bepul reja/billing
7365                 holati proksi orqali ham to'g'ri aniqlanadi. */
7366                 const e0 = data && data.error;
7367                 fail(aiApiErr(res.status, (typeof e0 === 'string') ? {error:{message:e0}} : data));

```

```

7368         return;
7369     }
7370 } else {
7371     const model = encodeURIComponent(getAiModel());
7372     const url = 'https://generativelanguage.googleapis.com/v1beta/models/' + model + ':generateContent';
7373     res = await fetch(url, {
7374         method: 'POST',
7375         headers: { 'Content-Type': 'application/json', 'x-goog-api-key': getAiKey() },
7376         body: JSON.stringify({ contents: [{ parts: [{ text: prompt }] }] });
7377     });
7378     data = await res.json().catch(() => ({}));
7379     if(!res.ok){ fail(aiApiErr(res.status, data)); return; }
7380 }
7381
7382 const img0 = aiExtractImage(data);
7383 if(!img0){
7384     /* model matn qaytargan bo'lishi mumkin – uni ko'rsatamiz */
7385     let txt = '';
7386     try{
7387         const parts = data.candidates[0].content.parts;
7388         txt = parts.map(p=>p.text).filter(Boolean).join(' ');
7389     }catch(e){}
7390     fail(txt ? ('Model rasm o\'rniga matn qaytardi: ' + txt.slice(0,160))
7391           : 'Javobda rasm topilmadi – «Model» nomi rasm chizadigan model ekanini tekshiring');
7392     return;
7393 }
7394
7395 if(card){
7396     card.classList.remove('ai-loading');
7397     const img = document.createElement('img');
7398     img.src = 'data:' + img0.mime + ';base64,' + img0.b64;
7399     card.innerHTML = '';
7400     card.appendChild(img);
7401     const meta = document.createElement('div');
7402     meta.className = 'meta';
7403     meta.innerHTML = '□ <b>Nano Banana</b> · "' + escHTML(prompt) + '"';
7404     card.appendChild(meta);
7405     const acts = document.createElement('div');
7406     acts.className = 'acts';
7407     const bdl = document.createElement('button');
7408     bdl.className = 'btn sm gold'; bdl.textContent = 'Yuklab olish';
7409     bdl.addEventListener('click', ()=>{
7410         dlBlob(_dataURLtoBlob(img.src), img0.mime, 'naqshai_ai_' + Date.now() + '.png', 'Rasm yuklab olindi □');
7411     });
7412     acts.appendChild(bdl);
7413     card.appendChild(acts);
7414 }
7415 const log = $('chat-log'); if(log) log.scrollTop = log.scrollHeight;
7416 }catch(e){
7417     console.error('[ai]', e);
7418     fail(/failed to fetch|networkerror/i.test(e.message) ? 'Serverga ulanib bo\'lmadi – internetni tekshiring'
7419       : ('Xatolik: ' + e.message));
7420 }
7421 }
7422 }
7423 function wireAiSetup(){
7424     const seg = $('chat-mode-seg');
7425     if(seg) seg.addEventListener('click', e=>{
7426         const b = e.target.closest('button');
7427         if(!b) return;
7428         CHAT_MODE = b.dataset.v;
7429         seg.querySelectorAll('button').forEach(x=>x.classList.remove('sel'));
7430         b.classList.add('sel');
7431         if(CHAT_MODE === 'ai' && !aiReady()){
7432             ntf('AI rasm rejimi uchun API kalit kerak (bir marta)', 'err');
7433             openAiSetup();
7434         }
7435         const inp = $('chat-input');
7436         if(inp) inp.placeholder = CHAT_MODE === 'ai'
7437           ? "Masalan: quyosh nurida oltin girih naqshli chinni laganning fotosi"
7438           : "Masalan: 8 simmetriyali oltin girih chiz";
7439     });
7440     const setupBtn = $('btn-ai-setup'); if(setupBtn) setupBtn.addEventListener('click', openAiSetup);
7441     const cancel = $('ai-setup-cancel'); if(cancel) cancel.addEventListener('click', closeAiSetup);
7442     const back = $('modal-ai-setup'); if(back) back.addEventListener('click', e=>{ if(e.target===back) closeAiSetup() });
7443     const cseg = $('ai-conn-seg');
7444     if(cseg) cseg.addEventListener('click', e=>{
7445         const b = e.target.closest('button');
7446         if(b) aiShowConn(b.dataset.v);
7447     });
7448     const save = $('ai-setup-save'); if(save) save.addEventListener('click', ()=>{
7449         const chosen = $('ai-worker-box') && !($('ai-worker-box').hidden) ? 'worker' : 'direct';
7450         setAiConn(chosen);
7451         const u = $('ai-worker-url'), t = $('ai-app-token'), k = $('ai-key'), m = $('ai-model');
7452         setAiWorkerUrl(u?u.value: ''); setAiAppToken(t?t.value: '');
7453         setAiKey(k?k.value: ''); setAiModel(m?m.value: '');
7454         closeAiSetup();
7455         ntf(aiReady() ? 'Saqlandi ✓ – AI rasm rejimi tayyor' : 'Saqlandi, lekin kalit/manzil bo\'sh', aiReady()?'ok':'err');
7456     });

```

```

7457 }
7458 function sendChat(){
7459   const inp = $('chat-input');
7460   if(!inp) return;
7461   const txt = (inp.value||'').trim();
7462   if(!txt) return;
7463   inp.value = '';
7464   chatUserMsg(txt);
7465   if(CHAT_MODE === 'ai'){
7466     aiGenerate(txt);
7467     return;
7468   }
7469   const np = parseChat(txt);
7470   setTimeout(()=>{ chatCard(np, txt); saveChatHistory(); }, 120);
7471 }
7472 const CHAT_CHIPS = [
7473   '8 simmetriyali oltin girih chiz',
7474   'Feruza islamiy, zich',
7475   'Ismingizni yozing – shaxsiy naqsh',
7476   '12 ta mandala, murakkab',
7477   "Ko'k penrose, sodda",
7478   'Oltin girih tessellatsiya p6m',
7479   'Samarqand 2026'
7480 ];
7481 function buildChips(){
7482   const box = $('chat-chips');
7483   if(!box) return;
7484   box.innerHTML = '';
7485   CHAT_CHIPS.forEach(txt=>{
7486     const b = document.createElement('button');
7487     b.textContent = txt;
7488     b.addEventListener('click', ()=>{
7489       const inp = $('chat-input');
7490       if(txt.indexOf('Ismingizni')===0){
7491         if(inp){ inp.value=''; inp.placeholder='Ismingizni yozing...'; inp.focus() && inp.focus(); }
7492         return;
7493       }
7494       if(inp){ inp.value = txt; }
7495       sendChat();
7496     });
7497     box.appendChild(b);
7498   });
7499 }
7500
7501 /* =====
7502   NAZARIYA SAHIFASI – INTERAKTIV DARSLIK
7503   Har naqsh: jonli preview + model + formulalar + algoritm + kod
7504   ===== */
7505 function escHTML(t){
7506   return String(t).replace(/&/g, '&amp;').replace(/</g, '&lt;').replace(/>/g, '&gt;');
7507 }
7508 const THEORY = {
7509   girih:{
7510     model:`Kanonik islomiy rozetta: {n/k} yulduz o'zagi + kite-gulbarglar halqasi. Chiziqlar lenta (ribbon) sifatida
7511     – ikki qoramtir kontur orasidagi band – chiziladi va navbatlashib to'qiladi.` ,
7512     frm:`P_i = (R*cos θ_i, R*sin θ_i), θ_i = 2πi/n – π/2`,
7513     `k = {n/2} – 1 (vatar qadami: P_i → P_{i+k})`,
7514     `r_2 = R(0.36 + ir*0.28) – yulduz uchlari; r_1 = r_2(0.66 – spir*0.28)`,
7515     `Kite: A(θ, r_2) → L(θ-w, r_m) → T(θ, R_0) → R_r(θ+w, r_m), w = (π/n)*0.86*(0.72+dens*0.55)`],
7516     algo:`Aylanada n ta nuqta hisoblanadi`,
7517     `Har yo'nalishda kite-gulbarg to'ldiriladi (4 bo'rtgan Bezier yon)`,
7518     `starPath(n, r_2, r_1) – yulduz o'zagi to'ldiriladi`,
7519     `{n/k} vatarlar IKKI O'TISHDA lenta qilib chiziladi: juftlari ostda, toqlari fon-uzilish (ribbonI) bilan us
7520     tida → to'qish`,
7521     `Juft vatarlarning markaziy 42–58% segmenti yana ustidan – weave yakunlanadi`,
7522     `Tashqi halqa lentasi + tugunlarda mini 8-yulduzlar`,
7523     code:`const pts=[]; for(let i=0;i<n;i++) pts.push(pol(TAU*i/n-Math.PI/2, R0));
7524     const chord=(i)=>{ const A=pts[i], B=pts[(i+k)%n];
7525     ctx.beginPath(); ctx.moveTo(A[0],A[1]); ctx.lineTo(B[0],B[1]); };
7526     for(let pass=0; pass<2; pass++)
7527     for(let i=pass; i<n; i+=2){ chord(i); ribbonI(ctx,c0,edge,bw); }`,
7528     par:`<b>sym</b>→n, <b>ir</b>→r_2 (yulduz o'lchami), <b>dens</b>→gulbarg eni w, <b>spir</b>→yulduz o'tkirligi r_1, <
7529     b>cmp>5</b>→ichki mini-rozetta`,
7530     hankin8:{
7531       model:`Hankin usuli (polygons-in-contact): tekislik 4.8.8 Arximed plitkalashi (sakkizburchak + kvadrat) bilan qop
7532       lanadi; strapwork plitkalardan avtomatik "tug'iladi" – hech bir chiziq qo'lda joylanmaydi.` ,
7533       frm:`oR = s/2 / cos(π/8) – oktagon radiusi (s – panjara qadami)`,
7534       `Qirra o'rtasi M_i dan ikki nur: yo'nalish d_i ± δ, δ = π(0.2 + spir*0.18)`,
7535       `Kesishma: M_i + t*(cos(d_i+δ), sin(d_i+δ)) = M_{i+1} + u*(cos(d_{i+1}+π-δ), ...)`],
7536       `Kvadrat uchlari = 4 qo'shni oktagonning eng yaqin burchaklari`,
7537       algo:`Panjara bo'ylab oktagon va oraliq kvadrat plitkalar quriladi`,
7538       `Har plitkada: qirralar o'rtalari M_i va yo'nalishlari d_i topiladi`,
7539       `Qo'shni nurlar juftlari kesishtiriladi (rayX) → [M_i, X, M_{i+1}] strap`,
7540       `Plitka fonlari navbat rangda to'ldiriladi`,
7541       `Oktagon straplari oddiy lenta, kvadratniki ribbonI (to'qish) bilan ustidan`,
7542       code:`function hankinStraps(poly, ang){
7543       const m=poly.length, mids=[], dirs=[], straps=[];
7544       for(let i=0;i<m;i++){ const A=poly[i],B=poly[(i+1)%m];
7545       mids.push([(A[0]+B[0])/2,(A[1]+B[1])/2]);
7546       dirs.push(Math.atan2(B[1]-A[1],B[0]-A[0])); }
7547       for(let i=0;i<m;i++){ const j=(i+1)%m;

```

```

7544     const X=rayX(mids[i],dirs[i]+ang, mids[j],dirs[j]+Math.PI-ang);
7545     if(X) straps.push([mids[i],X,mids[j]]); }
7546     return straps; }`,
7547     par: `<b>dens</b>->panjara zichligi (katak soni), <b>spir</b>->kontakt burchagi 6 (yulduz o'tkirligi)`},
7548     hankin6:{
7549     model: `Hankin usuli geksagonal 6.6.6 panjarada: har muntazam olti burchakdan kontakt burchagiga qarab 6 qirrali y
7550     ulduz hosil bo'ladi.` ,
7551     frm: `Markazlar:  $x = g_x \cdot \sqrt{3} \cdot s + (g_y \bmod 2) \cdot \sqrt{3} \cdot s/2$ ,  $y = g_y \cdot 1.5 \cdot s$ ,
7552     `Hex uchlari:  $C + s \cdot (\cos(\pi/6 + 2\pi i/6), \sin(\pi/6 + 2\pi i/6))$ `,
7553     `6 =  $\pi(0.24 + \text{spir} \cdot 0.16)$ `],
7554     algo: `Asal uyasi panjarasi quriladi (qatorlar yarim qadam siljigan)`,
7555     `Har hexga hankinStraps(6) qo'llanadi`,
7556     `Fonlar  $(g_x + 2g_y) \bmod 3$  bo'yicha 3 rangda almashadi`,
7557     code: `for(let gy=-N;gy<=N;gy++) for(let gx=-N;gx<=N;gx++){
7558     const x=gx*dx+(gy%2*dx/2:0), y=gy*dy;
7559     const h=[]; for(let i=0;i<6;i++){
7560     h.push([x+Math.cos(Math.PI/6+TAU*i/6)*s, y+Math.sin(...)*s]);
7561     strokeStraps(ctx, hankinStraps(h,ang), band, edge, bw); }`,
7562     par: `<b>dens</b>->hex o'lchami s, <b>spir</b>->6 burchagi`,
7563     panelgirih:{
7564     model: `Klassik kvadrat o'quv paneli: ikki kvadrat 45° buralib 8 qirrali yulduz beradi; oq lenta straplar, ramka b
7565     andlari va burchak choraklari – o'zbek naqqoshlik maktabining "sakkiz qirrali girih" darsligi.` ,
7566     frm: `Yulduz: kvadrat1( $\theta_0 = -\pi/2$ ) u kvadrat2( $\theta_0 = -\pi/4$ ), uchlari radiusi r`,
7567     `Ichki oktagon radiusi  $\approx 0.545r$  (kvadratlar kesishmasi)`,
7568     `starPath(8, r, 0.765r) – {8} yulduz maydoni`,
7569     algo: `Tashqi ramka: 2 kvadrat lenta band + orasida mini-rozetka nuqtalari`,
7570     `Burchak choraklari Bezier bilan to'ldiriladi`,
7571     `8 kite-gulbarg + yulduz maydoni fill`,
7572     `Kvadrat1 ostda, kvadrat2 ribbonI ustida, keyin kvadrat1 ning 2 qarama-qarshi tomoni yana ustidan → haqiqiy
7573     weave`,
7574     code: `const sq=(rot)=>{const q=[];for(let i=0;i<4;i++)q.push(pol(rot+TAU*i/4,r));return q};
7575     ctx.beginPath(); polyPath(ctx, sq(-Math.PI/2)); ribbon(ctx,band,edge,bw);
7576     ctx.beginPath(); polyPath(ctx, sq(-Math.PI/2+Math.PI/4)); ribbonI(ctx,band,edge,bw);`,
7577     par: `<b>dens</b>->ramka mini-rozetkalar soni`,
7578     xatam:{
7579     model: `Xatam yulduzi: ikki kvadrat 45° ustma-ust – 8 qirrali yulduz; to'ldirilgan yulduz maydoni, oktagon o'zak v
7580     a to'qilgan lenta kvadratlar.` ,
7581     frm: `Kvadrat uchlari:  $\text{pol}(\theta_0 + 2\pi i/4, r)$ ,  $\theta_0 \in \{-\pi/4, 0\}$ `,
7582     `Yulduz ichki nisbat:  $r_{in}/r_{out} = 0.765 \approx \cos(\pi/8)^2$ `,
7583     algo: `8-yulduz maydoni fill → ichki oktagon fill`,
7584     `Kvadrat A lenta, kvadrat B ribbonI, A ning 28–72% segmentlari qayta ustidan`,
7585     `Uchlardan tashqi halqaga nurlar + halqa band`,
7586     code: `ctx.beginPath(); starPath(ctx, 8, r, r*0.765, rot); fillA(ctx,c1,0.55);
7587     ctx.beginPath(); polyPath(ctx,A); ribbon(ctx,c0,edge,bw);
7588     ctx.beginPath(); polyPath(ctx,B); ribbonI(ctx,c1,shade(c1,.5),bw);`,
7589     par: `<b>cmp</b>->5</b>->ikkinchi ichki daraja (0.5 masshtab, 22.5° burilgan)`},
7590     rozetta:{
7591     model: `Gulbarg halqalari: har barg ikki kubik Bezier yon + o'tkir uch; bandlar konsentrik – tashqi katta barglar,
7592     oraliqda kichiklari, markazda halqa+yulduzcha.` ,
7593     frm: `Barg asosi: ( $\theta \neq w$ ,  $r_{in}$ ), uchi: ( $\theta$ ,  $r_{out}$ )`,
7594     `Yon nazorat nuqtalari: ( $\theta \neq 1.12w$ ,  $r_{in} + 0.42\Delta r$ ) va ( $\theta \neq 0.5w$ ,  $0.93r_{out}$ )`,
7595     `w =  $(\pi/n)(0.55 + \text{dens} \cdot 0.35)$ `,
7596     algo: `n ta tashqi barg: petalPath fill + kontur + o'rta tomir`,
7597     `n ta oraliq kichik barg (yarim qadam siljigan)`,
7598     `Markaz: lenta halqa (radius ir ga bog'liq) + n-yulduzcha + cmp bo'yicha nuqta halqalari`,
7599     code: `function petalPath(ctx,a,rIn,rOut,hw){
7600     const bl=pol(a-hw,rIn), tip=pol(a,rOut), br=pol(a+hw,rIn);
7601     ctx.moveTo(bl[0],bl[1]);
7602     ctx.bezierCurveTo(...c1,...c2, tip[0],tip[1]); // chap yon
7603     ctx.bezierCurveTo(...c1,...c2, br[0],br[1]); // o'ng yon
7604     ctx.closePath(); }`,
7605     par: `<b>sym</b>->barglar n, <b>dens</b>->barg eni, <b>ir</b>->markaz halqasi, <b>cmp</b>->nuqta halqalari`,
7606     yulduz:{
7607     model: `n/k yulduz poligoni to'qilgan lentalarda: har uch Pi dan P{i+k} ga vatar; qo'shni vatarlar kesishmasi
7608     markaziy n-burchakni beradi va u to'ldiriladi.` ,
7609     frm: `Vatar: Pi → P{(i+k) mod n},  $k = \lfloor n/2 \rfloor - 1 + \lfloor \text{spir} \cdot 1.5 \rfloor$ ,
7610     `Ichki uch:  $X = \text{segX}(P_i, P_{i+k}, P_{i+1}, P_{i+1-k})$ `,
7611     algo: `Vatarlarning ketma-ket kesishmalari hisoblanib ichki poligon fill qilinadi`,
7612     `Vatarlar 2 o'tishda ribbonI bilan to'qiladi`,
7613     `Juftlarning o'rta segmenti ustidan; halqa band + tugun nuqtalar`,
7614     code: `const inner=[];
7615     for(let i=0;i<n;i++){
7616     const X = segX(pts[i], pts[(i+k)%n],
7617     pts[(i+1)%n], pts[(i+1-k)%n]);
7618     if(X) inner.push(X); }
7619     ctx.beginPath(); polyPath(ctx, inner); fillA(ctx,c1,0.85);`,
7620     par: `<b>sym</b>->n, <b>spir</b>->k qadami (yulduz "o'ramliligi")`,
7621     muqarnas:{
7622     model: `Pog'onali arkada modeli: har daraja radiusi geometrik kamayadi (0.74 koeffitsiyent), darajada m ta tokcha
7623     (nisha) – har biri ikki kubik Bezier ravoq, uchida osilgan tomchi.` ,
7624     frm: `rl =  $0.94R \cdot 0.74^l$ ,  $l = 0 \dots (\text{cmp}+1)$ ,
7625     `ml = n + 2l tokcha; osilish:  $M = \text{pol}(\theta_{mid}, r_l - \text{drop})$ ,  $\text{drop} = r_l(0.14 + 0.2 \cdot \text{ir})$ `,
7626     algo: `Har darajada tokcha fonlari (ravoq + yoy bilan yopiq) navbat rangda`,
7627     `Ravoq chizig'i yaxlit lenta sifatida`,
7628     `Osilgan tomchi nuqtalari; rang darajalar bo'ylab lerpColor bilan o'tadi`,
7629     `Markazda yulduz o'zak`,
7630     code: `for(let l=0;l<levels;l++){
7631     const r=R*0.94*Math.pow(0.74,l), m=n+l*2, drop=r*(0.14+ir*0.2);
7632     for(let i=0;i<m;i++){
7633     const A=pol(a0,r), B=pol(a1,r), M=pol((a0+a1)/2, r-drop);
7634     ctx.bezierCurveTo(...,-M...); // ikki Bezier ravoq A-M-B

```

```

7628   } ribbon(ctx, lerpColor(c0,c1,l/levels), ...); }`,
7629   par: <b>sym</b>->bazaviy tokcha soni, <b>cmp</b>->darajalar, <b>ir</b>->osilish chuqurligi`,
7630   katak8:{
7631     model:`4.8.8 Arximed tessellatsiyasi: har uchda bitta kvadrat + ikki oktagon uchrashadi. Kvadrat uchlari oktagon
burchaklaridan olinadi – plitkalar matematik zich yopishadi.` ,
7632     frm:[`s – qadam; oR = s/2/cos(π/8)` ,
7633           `Kvadrat yarim dioganali q ≈ 1.35·(s/2 – oR·cos(π/8))`],
7634     algo:[`Panjara bo'ylab oktagonlar (fill + lenta kontur, ichida 8-yulduzcha)` ,
7635           `Oraliq nuqtalarda 45° kvadratlar fill` ,
7636           `Doira clip + tashqi halqa (yakka rejimda)`],
7637     code:`for(let gy=0;gy<=cells;gy++) for(let gx=0;gx<=cells;gx++){
7638       const x=-half+gx*s, y=-half+gy*s;
7639       const oct=[]; for(let i=0;i<8;i++){
7640         oct.push([x+Math.cos(Math.PI/8+TAU*i/8)*oR, y+Math.sin(...)*oR]);
7641         ctx.beginPath(); polyPath(ctx,oct); fillA(...); ribbon(...); }`,
7642     par: <b>dens</b>->katak soni, <b>cmp</b>->ichki yulduzchalar`,
7643     hexa:{
7644       model:`Asal uyasi (p6m panjara): qatorlar dx/2 siljigan hex to'r; har katak ichida 6 barg rozetkasi.` ,
7645       frm:[`dx = √3·s, dy = 1.5·s; x = g_x·dx + (g_y mod 2)·dx/2`],
7646       algo:[`Hex panjara quriladi, kataklar 3 rangda davriy bo'yaladi` ,
7647             `Lenta konturlar; cmp≥4 da har katakka 6 ta barg + markaz nuqta`],
7648       code:`const pts=[]; for(let i=0;i<6;i++){
7649         pts.push([x+Math.cos(Math.PI/6+TAU*i/6)*s, y+Math.sin(...)*s]);
7650         polyPath(ctx,pts); fillA(...); ribbon(ctx,c0,edge,bw);` ,
7651       par: <b>dens</b>->katak o'lchami, <b>cmp</b>->ichki barglar`,
7652       zanjira:{
7653         model:`Interlock halqalar zanjiri: n ta halqa aylana bo'ylab, radiusi qo'shnilar kesishadigan qilib tanlanadi; har
n keyingi halqa ribbonI bilan oldingisini "kesib" o'tadi, oxirida bitta yoy zanjirni yopadi.` ,
7654         frm:[`r_r = (π·r_c/n)(0.92 + dens·0.35) – halqa radiusi` ,
7655               `Markazlar: pol(2πi/n – π/2, r_c), r_c = 0.68R`],
7656         algo:[`Halqalar ketma-ket ribbonI bilan chiziladi – har biri oldingisining ustidan` ,
7657               `1-halqaning oxirgi halqa tomonidagi yoyi (arcB) qayta ustidan → zanjir yopiladi` ,
7658               `Markazda medalyon: lenta halqa + yulduz`],
7659         code:`for(let i=0;i<n;i++){
7660           const c=centers[i];
7661           ctx.beginPath(); circle(ctx,c[0],c[1],rr);
7662           ribbonI(ctx, i%2?c1:c0, shade(col,.5), bw); }
7663         arcB(ctx, centers[0][0], centers[0][1], rr, dir-0.7, dir+0.7);
7664         ribbonI(ctx, c0, edge, bw); // zanjirni yopish` ,
7665         par: <b>sym</b>->halqalar soni, <b>dens</b>->halqa kesishuvi`,
7666         islimiy:{
7667           model:`Logarifmik spiral novdalar: o'sish qonuni r(θ)=a·e^{bθ}; poya torayuvchi (3 o'tishli qalinlik), barglar ta
ngens burchagida navbatma-navbat, uchida uch bargli palmetta kurtak.` ,
7668           frm:[`r(θ) = a·e^{bθ}, b = 0.1 + spir·0.28, a = L·e^{-b·θ_max}` ,
7669                 `θ_max = π(1.2 + cmp·0.18); barg burchagi = atan2(Δy,Δx) ± π/2.5`],
7670           algo:[`SpiralPts: θ bo'ylab namuna nuqtalar → Catmull-Rom → kubik Bezier` ,
7671                 `stem(): bir yo'l 3 marta kamayuvchi qalinlikda – taper illyuziyasi` ,
7672                 `Har k-nuqtada fineLeaf (fill + kontur + o'rta tomir), o'lcham uchga qarab o'sadi` ,
7673                 `Oxirida budAt – 3 barg fan; cmp≥3 da qarshi kichik gajak`],
7674           code:`function spiralPts(b,thMax,L,mir){
7675             const a=L/Math.exp(b*thMax), pts=[];
7676             for(let i=0;i<=steps;i++){
7677               const th=thMax*i/steps, r=a*Math.exp(b*th);
7678               pts.push([Math.cos(mir*th)*r, Math.sin(mir*th)*r]); }
7679             return pts; }`,
7680           par: <b>sym</b>->novdalar, <b>spir</b>->b (o'ralish), <b>cmp</b>->θ_max, <b>dens</b>->barg zichligi`,
7681           rumi:{
7682             model:`Klassik rumi (buralgan barg) motivi: 4 kubik Bezierdan iborat ilmoqli yopiq shakl, tok halqasi bo'ylab tas
hqi-katta / ichki-kichik juftlikda; barglar orasidan chetga S-novdalar chiqadi – tessellatsiyada qo'shnilar bilan
tutashadi.` ,
7683             frm:[`Motiv: (0,0)→(0.94L,-0.1L) yuqori yon, ilmoq uchi (0.7L, 0.13L), ichki qaytish → yopiq` ,
7684                   `Bog'lovchi novda: r(t)=0.96r_c+(1.06R-0.96r_c)t, θ(t)=θ_0+0.16·sin(πt)·(±1)`],
7685             algo:[`Tok halqasi lenta bilan` ,
7686                   `Har pozitsiyada katta rumi barg (tashqariga) + kichigi (ichkariga), ichida ilmoq chizig'i` ,
7687                   `Barglar orasidan chetga S-novda + o'rta va uch barglar` ,
7688                   `Markazda yulduz rozetkasi`],
7689             code:`function rumiLeafPath(ctx,L){
7690               ctx.moveTo(0,0);
7691               ctx.bezierCurveTo(L*.12,-L*.36, L*.68,-L*.42, L*.94,-L*.1);
7692               ctx.bezierCurveTo(L*.04,L*.04, L*.9,L*.2, L*.7,L*.13); // ilmoq
7693               ctx.bezierCurveTo(L*.52,L*.06, L*.42,L*.2, L*.28,L*.16);
7694               ctx.bezierCurveTo(L*.12,L*.1, L*.04,L*.05, 0,0);
7695               ctx.closePath(); }`,
7696             par: <b>sym</b>->motivlar soni, <b>dens</b>->barg o'lchami L, <b>skew</b>->burilish`,
7697             hatayi:{
7698               model:`Stilizatsiyalangan lotus: har gulbarg 3 Bezier bosqich – keng asos → bel (58% balandlikda) → o'tkir uch; i
kki band + urug'don halqasi.` ,
7699               frm:[`Bel nuqtalari: (θ±0.62w, r_in + 0.58Δr); uch: (θ, r_out)` ,
7700                     `m = 5 + [cmp≥3] + [cmp≥5] gulbarg`],
7701               algo:[`Tashqi m gulbarg (3-segment yo'l) + o'rta tomirlar` ,
7702                     `Oraliq kichik gulbarglar (yarim qadam)` ,
7703                     `Urug'don: lenta halqa + nuqtalar doirasi + markaz`],
7704               code:`ctx.moveTo(bl...); // asos chap
7705               ctx.bezierCurveTo(c1..., wl..., wl...); // asos-bel
7706               ctx.bezierCurveTo(wl..., c2..., tip...); // bel-uch
7707               ctx.bezierCurveTo(c3..., wr..., wr...); // uch-bel (o'ng)
7708               ctx.bezierCurveTo(wr..., c4..., br...); // bel-asos` ,
7709               par: <b>cmp</b>->gulbarg soni, <b>dens</b>->barg eni, <b>sym</b>->urug'don nuqtalari`,
7710               palak:{
7711               model:`Nilufar: 3 konsentrik gulbarg halqasi jadval bilan beriladi – har halqaning (r_in, r_out, en, siljish, ran
g) parametrlari; halqalar yarim qadam navbatlashadi.` ,

```

```

7712 frm:[`Halqa1: 0.55R-0.95R; Halqa2: 0.32R-0.6R (siljish ½); Halqa3: 0.14R-0.36R`],
7713 algo:[`Har halqada n barg petalPath bilan, fill ranglari navbat`,
7714 `cmp halqalar sonini boshqaradi; markaz lenta + nuqta`],
7715 code:`const rings=[{rIn:.55*R,rOut:.95*R,off:0},
7716 {rIn:.32*R,rOut:.6*R, off:.5},
7717 {rIn:.14*R,rOut:.36*R,off:0}];
7718 for(const rg of rings) for(let i=0;i<n;i++)
7719 petalPath(ctx, TAU*(i+rg.off)/n - Math.PI/2, rg.rIn, rg.rOut, rg.w);`,
7720 par:`<b>sym</b>->barglar, <b>cmp</b>->halqalar soni`,
7721 shamsa:{
7722 model:`Quyosh medalyoni – 3 band kompozitsiya: markaziy to'ldirilgan yulduz, 2n alanga-nur (uzun/qisqa navbat, to
mchi-barg shaklida), tashqi scallop-gulbarg bandi.`,
7723 frm:[`Nurlar: boshi (0, 0.4R), uzunligi R.{0.36, 0.22} navbat`,
7724 `Tashqi band: 2n gulbarg 0.8R-0.955R`],
7725 algo:[`Tashqi halqa lentasi + gulbarg bandi`,
7726 `2n nur-barg (leaf, 2 Bezier) – juftlari uzun`,
7727 `Markaz: lenta halqa + katta yulduz fill + ichki qarama-qarshi yulduzcha`],
7728 code:`for(let i=0;i<n*2;i++){
7729 const a=TAU*i/(n*2)-Math.PI/2, long=i%2===0;
7730 const s0=pol(a, R*0.4), len=R*(long?0.36:0.22);
7731 ctx.beginPath(); leaf(ctx,s0[0],s0[1],a,len,len*0.14);
7732 fillA(ctx, long?c0:c2, 0.9); }`,
7733 par:`<b>sym</b>->n (nurlar = 2n)`},
7734 arabesk:{
7735 model:`Ogee (S-egri) to'ri: ichki n tugundan tashqi (yarim qadam siljigan) n tugunga ikki oila S-egri – nazorat n
uqtalari o'rta chiziqning normal bo'ylab offset siljiriladi; tutashuvlarda palmetta.`,
7736 frm:[`S-egri nazorat: 0.6A+0.4M ± n`off, off = 0.09R(0.6+spir)`],
7737 `Tugunlar: ichki (0_i, 0.4R), tashqi (0_i+π/n, 0.88R)`],
7738 algo:[`Birinci oila (+off) lenta, ikkinchi oila (-off) ribbonI – kesishuvda to'qiladi`,
7739 `Tashqi tugunlarda budAt palmetta, ichkilarida barg juftligi`,
7740 `Markaz halqa + yulduz`],
7741 code:`ctx.moveTo(A[0],A[1]);
7742 ctx.bezierCurveTo(
7743 A[0]*.6+mx*.4 + nx/L*off, A[1]*.6+my*.4 + ny/L*off,
7744 B[0]*.6+mx*.4 - nx/L*off, B[1]*.6+my*.4 - ny/L*off,
7745 B[0], B[1]);`,
7746 par:`<b>sym</b>->tugunlar, <b>spir</b>->S-egri chuqurligi`,
7747 scroll:{
7748 model:`Bargli novda: asosiy S-poya bitta kubik Bezier; barglar egri bo'ylab B(t) formulasidan namuna olinib, tang
ens burchagiga ±π/2.3 da navbatlashadi; uchida log-spiral gajak.`,
7749 frm:[`B(t) = (1-t)³P₀ + 3(1-t)²tP₁ + 3(1-t)t²P₂ + t³P₃`,
7750 `Barg burchagi: atan2(B(t+ε)-B(t)) ± π/2.3`],
7751 algo:[`P₀...P₃ nazorat nuqtalari (S-shakl) → stem taper bilan`,
7752 `t_j = j/(m+1) da cubicAt namunalari, fineLeaf navbat tomonlarda, o'lcham t bilan o'sadi`,
7753 `Uchida spiralPts gajak`],
7754 code:`function cubicAt(p0,p1,p2,p3,t){ const u=1-t;
7755 return [u*u*u*p0[0]+3*u*u*t*p1[0]+3*u*t*t*p2[0]+t*t*t*p3[0],
7756 u*u*u*p0[1]+3*u*u*t*p1[1]+3*u*t*t*p2[1]+t*t*t*p3[1]];`,
7757 par:`<b>sym</b>->novdalar, <b>dens</b>->barg soni, <b>spir</b>->barg o'lchami, <b>skew</b>->S chuqurligi`,
7758 medallion:{
7759 model:`Ko'p bandli medalyon – kompozitsion model: tashqi lenta halqa → scallop (chig'anoq) band → gulbarg band →
markazda 0.5 masshtabda to'liq girih rozetta chaqiriladi.`,
7760 frm:[`Scallop: har segment A-M(0.82R)→B ikki Bezier; gulbarglar 0.5R-0.8R`],
7761 algo:[`4 band tashqaridan ichkariga ketma-ket`,
7762 `drawGirih(ctx, R*0.95, ...) 0.5 masshtabli kontekstda qayta ishlatiladi – kod kompozitsiyasi`],
7763 code:`ctx.save(); ctx.scale(0.5,0.5);
7764 drawGirih(ctx, R*0.95, {...p, sym:n}, sw*1.6, c0,c1,c2,c3);
7765 ctx.restore();`,
7766 par:`girih parametrlarining barchasi ichki o'zakka o'tadi`,
7767 gi:{model:`Yuqori tartibli funksiya kompozitsiyasi: makeCombo(fA,fB) – tashqi uslub to'liq R da, ichki uslub 0.5 m
ashtab + yarim qadam burilish bilan chiziladi. Bu yerda: Girih (tashqi) + Islimiy (ichki).`,
7768 frm:[`f(ctx) = fA(ctx,R) ∘ [scale(0.5)·rot(π/n)] fB(ctx,R)`],
7769 algo:[`fA chaqiriladi`, `Kontekst 0.5 ga masshtablanadi, π/n ga buriladi`, `fB rang juftlari almashgan holda chaqir
iladi`],
7770 code:`function makeCombo(fA,fB){
7771 return function(ctx,R,p,sw,c0,c1,c2,c3){
7772 fA(ctx,R,p,sw,c0,c1,c2,c3);
7773 ctx.save(); ctx.scale(0.5,0.5); ctx.rotate(Math.PI/p.sym);
7774 fB(ctx,R,p,sw*1.7, c1,c0,c3,c2);
7775 ctx.restore(); }; },
7776 par:`ikkala uslubning parametrlari birga ishlaydi`,
7777 sg:{model:`makeCombo(Shamsa, Girih): tashqi quyosh medalyoni ichida 0.5 masshtabda girih yulduz-tugun.`,frm:[`f =
Shamsa ∘ scale(0.5)·Girih`],algo:[`Shamsa to'liq R da`, `Girih 0.5R da yarim qadam burilib`],code:`makeCombo(drawSh
amsa, drawGirih)`},par:`ikkala uslub parametrlari`,
7778 mi:{model:`makeCombo(Muqarnas, Islimiy): pog'onali arkada ichida spiral novdalar.`,frm:[`f = Muqarnas ∘ scale(0.5)
·Islimiy`],algo:[`Muqarnas darajalari`, `Islimiy spirallari markazda`],code:`makeCombo(drawMuqarnas, drawIslimiy)`},
par:`ikkala uslub parametrlari`,
7779 xr:{model:`makeCombo(Xatam, Rumi): xatam yulduzi ichida rumi barglari halqasi.`,frm:[`f = Xatam ∘ scale(0.5)·Rumi`
],algo:[`Xatam ramkasi`, `Rumi halqasi markazda`],code:`makeCombo(drawXatam, drawRumi)`},par:`ikkala uslub parametrl
ari`},
7780 celtic:{
7781 model:`Ikki tolali torus o'rmasi (plait): tolalar r(θ)=r_m + a·sin(kθ+φ) sinusoid halqalar, φ∈{0,π}; kesishishlar
sin=0 nuqtalarida; har tola 2k segmentga bo'linib, (tola+j) juftligi bo'yicha ost/ust tartibida chiziladi – matem
atik aniq navbatlashgan to'qish.`,
7782 frm:[`r_A(θ)=r_m+a·sin(kθ), r_B(θ)=r_m+a·sin(kθ+π)`],
7783 `Kesishishlar: 0_j = jπ/k, j = 0..2k-1`,
7784 `Ust/ost: (strand + j) mod 2`,
7785 algo:[`Har tola 0∈[jπ/k,(j+1)π/k] segmentlarga bo'linadi (smoothPath)`,
7786 `1-o'tish: juft paritetli segmentlar oddiy lenta (ostda)`,
7787 `2-o'tish: toq paritetli segmentlar ribbonI (fon-uzilish) bilan ustida`,
7788 `Ichki/tashqi chegara bandlari`],

```

```

7789 code:`for(let pass=0; pass<2; pass++)
7790 for(let strand=0; strand<2; strand++)
7791   for(let j=0; j<2*k; j++){
7792     if(((strand+j)%2)!==pass) continue;
7793     strandSeg(strand*Math.PI, j);
7794     pass ? ribbonI(ctx,col,edge,bw) : ribbon(ctx,col,edge,bw); `},
7795 par:`<b>sym</b>→k (kesishishlar), <b>dens</b>→amplituda a va band eni`,
7796 mandala:{
7797   model:`Konsentrik halqalar generatori: halqa indeksi k bo'yicha element turi kind = k mod 5 davriy almashadi – nu
qtalar, gulbarglar, to'ldinli kontur, barglar, doira.`;
7798   frm:`r_k = R(0.12 + 0.85·k/(K-1)); m_k = n(1 + 1.5k/K)`,
7799   `To'ldin: r(θ) = r_k(1 + 0.035·sin(mθ + k))`,
7800   algo:`K = 6..12 halqa (cmp+dens dan)`,`Har halqada kind bo'yicha element chiziladi`,`Ranglar 4 lik siklda`,
7801   code:`for(let k=0;k<rings;k++){
7802     const r=R*(0.12+0.85*k/(rings-1)), kind=k%5;
7803     if(kind===1) /* gulbarglar */ petalPath(...);
7804     else if(kind===2) /* to'ldin */ smoothPath(ctx, wavePts, true);
7805     ... `},
7806   par:`<b>sym</b>→bazaviy m, <b>cmp,dens</b>→halqalar soni, <b>spir</b>→barg o'lchami`,
7807   penrose:{
7808     model:`P2 aperiodik qoplama – Robinson uchburchaklari subdividesi: har "qizil" uchburchak 2 ga, har "ko'k" 3 ga b
o'linadi, bo'lish nuqtalari oltin nisbat φ bilan; boshlanish – 10 uchburchakli g'ildirak.`;
7809     frm:`φ = (1+√5)/2 ≈ 1.618; P = A + (B-A)/φ`,
7810     `Qizil → {(C,P,B), ko'k(P,C,A)}; Ko'k → {ko'k(R,C,A), qizil(Q,R,B), ko'k(R,Q,A)}`,
7811     algo:`10 ta yarim-uchburchak g'ildiragi (har ikkinchisi aks)`,
7812     `depth = cmp marta subdivide`,
7813     `Turi bo'yicha 2 rangda fill + ingichka kontur`,
7814     code:`for(const [ty,A,B,C] of tris){
7815       if(ty===0){ const P=div(A,B,PHI);
7816         next.push([0,C,P,B],[1,P,C,A]); }
7817       else { const Q=div(B,A,PHI), R2=div(B,C,PHI);
7818         next.push([1,R2,C,A],[0,Q,R2,B],[1,R2,Q,A]); } `},
7819     par:`<b>cmp</b>→rekursiya chuqurligi (uchburchaklar ~ 3.6^cmp)`,
7820     zellige:{
7821       model:`Marokash kafeli: romb panjara, har rombnig 4 tomoni navbatma-navbat ichkari/tashqariga bo'rtgan Bezier; 5
rang (g_x + 2g_y) mod 5 davriyligida.`;
7822       frm:`Markazlar: x=(g_x+g_y)·0.8s, y=(g_y-g_x)·0.5s`,
7823       `Bo'rtish: nazorat = o'rta + n·bow·(-1)^i, bow = 0.2s(0.4+spir)`,
7824       algo:`Romb panjara quriladi`,`curvedQuad: 4 Bezier tomon, ishorasi almashib`,`5 rangli fill + kontur`,
7825       code:`for(let i=0;i<4;i++){ const A=q[i], B=q[(i+1)%4];
7826         const sgn = i%2 ? 1 : -1;
7827         const cx2 = mx + nx/L*bow*sgn, cy2 = my + ny/L*bow*sgn;
7828         ctx.bezierCurveTo(A+0.7(c-A)..., B+0.7(c-B)..., B[0],B[1]); `},
7829       par:`<b>dens</b>→panjara zichligi, <b>spir</b>→bo'rtish, <b>cmp≥5</b>→markaz nuqtalari`,
7830       batik:{
7831         model:`To'ldin modulyatsiyali konsentrik halqalar: r(θ) = r_k + A·sin(nθ + φ_k); nuqtalar Catmull-Rom orqali kubi
k Bezierga aylantiriladi; oraliqlarda 5-nuqtali gul motivlari.`;
7832         frm:`r(θ) = r_k + A·sin(nθ + 1.3k + skew), A = 0.035R(0.6+dens)`,
7833         algo:`K = 3+cmp halqa, har biri ≥48 namunali smoothPath`,`Toq halqalar orasida gul-nuqtalar`,`Ranglar navbat`,
7834         code:`const pts=[];
7835         for(let i=0;i<steps;i++){ const a=TAU*i/steps;
7836           pts.push(pol(a, rk + A*Math.sin(n*a + k*1.3))); }
7837         ctx.beginPath(); smoothPath(ctx, pts, true); st(ctx,col,w);`,
7838         par:`<b>sym</b>→to'ldin chastotasi n, <b>cmp</b>→halqalar, <b>dens</b>→amplituda, <b>skew</b>→faza`,
7839         paisley:{
7840           model:`Boteh (buta): 4 kubik Bezierdan asimetrik, uchi ilmoqli tomchi; ichida log-spiral, ichki hoshiya konturi
(0.82 masshtab), uchida 5-nuqta gul, chetida nuqta bezaklar.`;
7841           frm:`Kontur: (0, .5L)→(-W,-.35L)→ilmoq(.38W,-.6L)→(.12W,-.28L)→yopiq`,
7842           `Ichki spiral: r=a·e^{(0.16+0.2spir)θ}`,
7843           algo:`n pozitsiyaga buriladi/siljiydi`,`Boteh fill + ribbonI + 0.82 masshtabli ichki kontur`,`Spiral, uch guli,
cmp≥4 da chet nuqtalari`,
7844           code:`ctx.moveTo(0, L*0.5);
7845           ctx.bezierCurveTo(-W, L*.5, -W, -L*.35, -W*.25, -L*.62);
7846           ctx.bezierCurveTo(-W*.05,-L*.75, W*.3,-L*.78, W*.38,-L*.6);
7847           ctx.bezierCurveTo(W*.44,-L*.46, W*.1,-L*.42, W*.12,-L*.28);
7848           ctx.bezierCurveTo(W*.55,-L*.05, W*.62, L*.32, 0, L*.5);`,
7849           par:`<b>sym</b>→butalar, <b>dens</b>→o'lcham, <b>spir</b>→ichki spiral, <b>cmp≥4</b>→chet nuqtalari`,
7850           greek:{
7851             model:`Meander (labirint): birlik "kalit" yo'li 8 nuqtali lokal koordinatalarda beriladi va affin almashtirish bi
lan har tomonga u marta takrorlanadi; ramkalar konsentrik.`;
7852             frm:`Birlik: [(0,0),(.72,0),(.72,.72),(.24,.72),(.24,.3),(.48,.3),(.48,.52),(1,.52)]`,
7853             `Nuqta = B + d·x + n·h·y (d – tomon yo'nalishi, n – ichki normal)`,
7854             algo:`K ramka, har birida 4 tomon`,`Har tomonda u birlik meander affin ko'chiriladi`,`Markazda 4-yulduz`,
7855             code:`for(const q of unit){
7856               ctx.lineTo(bx + dx*q[0] + hx*q[1],
7857                 by + dy*q[0] + hy*q[1]); `},
7858             par:`<b>sym,dens</b>→birliklar soni, <b>cmp</b>→ramkalar, <b>ir</b>→markaz yulduzi`,
7859             xitoy:{
7860               model:`Fretwork panjara: kvadrat kataklar to'ri, har katakda T-motiv (gx+gy) mod 4 · 90° buralgan; burchak qavsla
r bilan boyitiladi.`;
7861               frm:`T: tik (0,-t)→(0,.2t), gorizontaal (±b,.2t), oyoqlar (±b,.2t)→(±b,.62t)`,
7862               algo:`Tashqi ramka + panjara chiziqlari`,`Har katak markazida buralgan T`,`cmp≥4 da burchak qavslar`,
7863               code:`ctx.save(); ctx.translate(cx,cy);
7864               ctx.rotate(((gx+gy)%4)*Math.PI/2);
7865               ctx.moveTo(0,-tLen); ctx.lineTo(0,tLen*0.2);
7866               ctx.moveTo(0,tBar,tLen*0.2); ctx.lineTo(tBar,tLen*0.2); ...`,
7867               par:`<b>dens</b>→kataklar, <b>ir</b>→T o'lchami, <b>skew</b>→burilish, <b>cmp≥4</b>→qavslar`,
7868               koch:{
7869                 model:`Koch qor parchasi – IFS/rekursiv almashtirish: har segment 4 ga bo'linadi, o'rtadagi ikkitasi 60° uchburch
ak yasaydi. Fraktal o'lcham D = log4/log3.`;
7870                 frm:`P2 = P1 + Rot(-60°)·(B-A)/3; D = log 4 / log 3 ≈ 1.262`,

```



```

7871 algo:[`Teng tomonli uchburchakdan boshlanadi`,`depth marta: har segment → 4 segment`,`Yopiq kontur fill + lenta`]
7872 ,
7873 code:`const ang = Math.atan2(dy,dx) - Math.PI/3;
7874 const P2 = [P1[0]+Math.cos(ang)*L, P1[1]+Math.sin(ang)*L];
7875 next.push(A, P1, P2, P3);`,
7876 par:`<b>cmp</b>→rekursiya (segmentlar 3·4^cmp)`},
7877 sierpinski:{
7878 model:`Sierpinski uchburchagi: o'rta nuqtalar orqali 3 kichik uchburchakka rekursiv bo'linish; D = log3/log2`,`
7879 frm:[`mid(U,V) = (U+V)/2; D = log 3 / log 2 ≈ 1.585`],
7880 algo:[`depth=0 da uchburchak fill`,`Aks holda 3 burchak uchburchagiga rekursiya`],
7881 code:`const rec=(a,b,c,d)=>{
7882   if(d===0){ polyPath(ctx,[a,b,c]); fillA(...); return; }
7883   rec(a, mid(a,b), mid(a,c), d-1);
7884   rec(mid(a,b), b, mid(b,c), d-1);
7885   rec(mid(a,c), mid(b,c), c, d-1); }`,`
7886 par:`<b>cmp</b>→chuqurlik (3^cmp uchburchak)`},
7887 dragon:{
7888 model:`Heighway ajdar egri – L-sistema: F→F+G, G→F-G, burilish ±90°; satr iteratsiyadan keyin chelakdek yig'ilib,
7889 chegara qutisiga masshtablanadi`,`
7890 frm:[`Uzunlik: 2^N segment; burchak ±π/2`,`
7891 `Fit: masshtab = 1.7R / max(Δx, Δy)`],
7892 algo:[`Satr N marta qayta yoziladi`,`Toshbaqa-yurish: F/G oldinga, +/- burilish`,`Bounding box → markazlash va ma
7893 sshtab`],
7894 code:`for(const ch of seq){
7895   if(ch==='F' || ch==='G'){ x+=Math.cos(ang); y+=Math.sin(ang);
7896   pts.push([x,y]); }
7897   else if(ch==='+') ang += Math.PI/2;
7898   else if(ch==='-') ang -= Math.PI/2; }`,`
7899 par:`<b>cmp</b>→iteratsiyalar (8+cmp)`},
7900 hilbert:{
7901 model:`Hilbert fazoni to'ldiruvchi egri: d-indeksdan (x,y) ga Gray-kod asosidagi iterativ almashtirish (kvadrant
7902 aniqlash + aks/burish); 4^order nuqta ketma-ket ulanadi`,`
7903 frm:[`N = 2^order; d ∈ [0, N^2)`],
7904 `Har bosqichda: r_x=1&(t/2), r_y=1&(t&t_x); r_y=0 bo'lsa aks+almashtirish`,`
7905 algo:[`d = 0..N^2-1 uchun d2xy hisoblanadi`,`Nuqtalar polyline bilan ulanadi`,`Markazlash va masshtab`],
7906 code:`const d2xy=(d)=>{ let rx,ry,t=d,x=0,y=0;
7907   for(let s=1;s<N;s*=2){
7908     rx=1&(t/2); ry=1&(t^rx);
7909     if(ry===0){ if(rx===1){x=s-1-x;y=s-1-y;}
7910     const tmp=x; x=y; y=tmp; }
7911     x+=s*rx; y+=s*ry; t=Math.floor(t/4); }
7912   return [x,y]; }`,`
7913 par:`<b>cmp</b>→order (nuqtalar 4^order)`},
7914 spirograph:{
7915 model:`Epitrochoid – g'ildirak ichida g'ildirak: R_0 radiusli halqa ichida r g'ildirak dumalaydi, qalam d masofada
7916 ; kNum/kDen nisbat necha aylanishda yopilishini belgilaydi`,`
7917 frm:[`x = (R_0-r)cos t + d·cos((R_0-r)t/r)`],
7918 `y = (R_0-r)sin t - d·sin((R_0-r)t/r)`],
7919 `r = R_0·kNum/kDen; yopilish: t ∈ [0, 2π·kDen)`],
7920 algo:[`t bo'ylab zich namunalar`,`Max radiusga qarab masshtab`,`smoothPath (Catmull-Bezier) bilan silliq chizilad
7921 i`],
7922 code:`for(let i=0;i<=steps;i++){
7923   const t=TAU*loops*i/steps;
7924   pts.push([(R0-r)*Math.cos(t)+d*Math.cos((R0-r)*t/r),
7925     (R0-r)*Math.sin(t)-d*Math.sin((R0-r)*t/r)]); }`,`
7926 par:`<b>sym,cmp</b>→kNum/kDen nisbati, <b>spir</b>→d (qalam masofasi)`},
7927 lissajous:{
7928 model:`Lissajou figurasi: ikki perpendikulyar garmonik tebranish superpozitsiyasi; a:b chastota nisbati figura tur
7929 ini, δ faza siljishi shaklini belgilaydi`,`
7930 frm:[`x = A·sin(a·t + δ), y = B·sin(b·t)`],
7931 `δ = (π/2)(1 + skew)`],
7932 algo:[`400 namunali yopiq smoothPath`,`dens>0.5 da 0.7 masshtabli ikkinchi kontur`],
7933 code:`for(let i=0;i<=steps;i++){ const t=TAU*i/steps;
7934   pts.push([R*Math.sin(a*t+del), R*Math.sin(b*t)]); }
7935 smoothPath(ctx, pts, true);`,`
7936 par:`<b>sym</b>→a, <b>cmp</b>→b-a, <b>skew</b>→δ, <b>dens</b>→ikkinchi kontur`},
7937 voronoi:{
7938 model:`Voronoi diagrammasi yarim-tekislik kesish usulida: har seed hujayrasi boshlang'ich 16-burchakdan boshlab,
7939 boshqa har bir seed bilan o'rta perpendikulyar bo'ylab Sutherland-Hodgman kesish orqali olinadi. Seedlar determini
7940 stik PRNG (mulberry32) dan – bir xil parametrlar = bir xil mozaika`,`
7941 frm:[`Hujayra(S) = n_T { q : (q-M)·(T-S) ≤ 0 }, M=(S+T)/2`,`
7942 `PRNG urug'i: 1000 + 31·sym + 7·cmp + 13·[100·dens] + ...`],
7943 algo:[`m = 8+4cmp+14dens seed doira ichida (√ taqsimot)`],
7944 `Har seed uchun poligon barcha bisektorlar bilan kesiladi`,`
7945 `5 palitra rangida fill + kontur`],
7946 code:`function clipHalfPlane(poly, px,py, nx,ny){
7947   const out=[];
7948   for(let i=0;i<poly.length;i++){
7949     const A=poly[i], B=poly[(i+1)%poly.length];
7950     const da=(A[0]-px)*nx+(A[1]-py)*ny;
7951     const db=(B[0]-px)*nx+(B[1]-py)*ny;
7952     if(da<=0) out.push(A);
7953     if(da*db<0){ const t=da/(da-db);
7954       out.push([A[0]+(B[0]-A[0])*t, A[1]+(B[1]-A[1])*t]); } }
7955   return out; }`,`
7956 par:`<b>sym,cmp,dens,spir</b>→PRNG urug'i va seedlar soni`
7957 };
7958 function buildTheory(){
7959   const wrap = $('theory-wrap');
7960   if(!wrap) return;
7961   let html = '<h1>Naqsh matematikasi va algoritmlari</h1>' +

```

```

7953     <p class="lead">Har bir naqshni oching: qaysi <b>model</b>, qaysi <b>formulalar</b> va qanday <b>kod</b> uni
        hosil qilayotganini jonli preview bilan birga ko'rasiz. Barcha kod parchalari platformaning haqiqiy manbasidan ol
        ingan.</p>;
7954     /* ===== Umumiy qurilish bloklari ===== */
7955     html += '<div class="th-card"><h3>Umumiy qurilish bloklari – barcha naqshlar shulardan quriladi</h3>' +
7956     '<div class="th-item"><b>Qutb koordinatalari:</b> <span class="formula">pol(θ, r) = (r·cos θ, r·sin θ)</span>
        – n-karra simmetriya θi = 2πi/n burchaklarida takrorlash orqali olinadi.</div>' +
7957     '<div class="th-item"><b>Kubik Bezier:</b> <span class="formula">B(t) = (1-t)3P0 + 3(1-t)2tP1 + 3(1-t)t2P2 + t
        3P3</span> – barcha organik egri chiziqlarning yagona asosi; hatto doiralar ham 4 Bezier segment (k = 0.5523·r naz
        orat masofasi) bilan chiziladi, shuning uchun SVG eksportda faqat C buyruqlari chiqadi.</div>' +
7958     '<div class="th-item"><b>Catmull-Rom → Bezier:</b> nuqtalar orqali silliq egri: nazorat nuqtalari <span class=
        "formula">C1 = P1 + (P2-P0)/6, C2 = P2 - (P3-P1)/6</span> – spiral, to'lqin va epitrochoidlarda ishlatiladi (smoo
        thPath).</div>' +
7959     '<div class="th-item"><b>Lenta (ribbon):</b> bitta yo'l 3 marta chiziladi – qora hoshiya (w), band rangi (w-m
        ax(1.8, 0.42w)), glazur yaltiroq'i (0.14w) – haqiqiy strapwork ko'rinishi.</div>' +
7960     '<div class="th-item"><b>Interlacing (to'qish):</b> ribbonI avval fon rangida kengroq (w+0.55w) chizadi – ost
        idagi lenta kesishuvda "uziladi", ustidan o'tish effekti tug'iladi. Chizish tartibi (2 o'tish, paritet bo'yich
        a) navbatlashgan weave beradi.</div>' +
7961     '<div class="th-item"><b>Nur kesishmasi:</b> <span class="formula">rayX: P+t·d^ = Q+u·ê → t = ((Q-P)×ê)/(d^×ê)
        </span> – g'irih va Hankin straplarining yuragi.</div>' +
7962     '<div class="th-item"><b>Qatlamlash:</b> qatlamlar ichma-ich medalyonlar: masshtab 0.42^l, burilish (l mod 2)·
        π/n – yarim qadam qatlamlarni interlock qiladi, ustma-ust loyqa bo'lmaydi.</div>' +
7963     '<div class="th-item"><b>Tessellatsiya:</b> <span class="formula">T(n,m) = n·v1 + m·v2</span>; p4: v1=(c,0), v
        2=(0,c); p6: v1=(c/2, c√3/2); p4m/p6m – shaxmat ko'zgusi. Panjara-naqshlar (Hankin, Katak-8, Zellige...) uzluksiz y
        axlit maydon sifatida, medalyonlar bleed (R tile = 0.62c &gt; c/2) bilan chiziladi.</div>' +
7964     '<div class="th-item"><b>SVG/DXF eksport:</b> soxta kontekst (SvgCtx) har bezierCurveTo ni 2×3 affin matritsa
        steki orqali kuzatib SVG C buyrug'iga yozadi; arc() 4/3·tan(Δθ/4) formulasi bilan Bezierga aylanadi; DXF uchun Be
        zier 32 nuqtali LWPOLYLINE ga tabaqalanadi.</div>' +
7965     '</div>;
7966     Object.keys(CATS).forEach(cat=>{
7967     html += '<h2 class="th-cat">' + CATS[cat] + '</h2>';
7968     PATTERNS.filter(pt=>pt.cat===cat).forEach(pt=>{
7969     const T = THEORY[pt.id] || {};
7970     const modelTxt = T.model || pt.th;
7971     html += '<details class="th-pat" data-id="'+pt.id+'"><summary>' +
7972     '<canvas data-pt="'+pt.id+'" width="200" height="200"></canvas>' +
7973     '<span class="th-head"><b>'+pt.nm+'</b><em>'+modelTxt+'</em></span>' +
7974     '<span class="th-open"></span></summary><div class="th-body">';
7975     if(T.frm && T.frm.length){
7976     html += '<h5>Matematik formulalar</h5>';
7977     T.frm.forEach(f=>{ html += '<div class="frm">'+escHTML(f)+'</div>'; });
7978     }
7979     if(T.algo && T.algo.length){
7980     html += '<h5>Algoritmlar</h5><ol class="algo">';
7981     T.algo.forEach(a=>{ html += '<li>'+a+'</li>'; });
7982     html += '</ol>';
7983     }
7984     if(T.code){
7985     html += '<h5>Kod (platforma manbasidan)</h5><pre class="code">'+escHTML(T.code)+'</pre>';
7986     }
7987     if(T.par){
7988     html += '<div class="th-par"><b>Slayderlar:</b> '+T.par+'</div>';
7989     }
7990     html += '<button class="btn sm gold th-studio" data-id="'+pt.id+'" style="margin-top:12px"> Studio da sinab
        ko'rish</button>';
7991     html += '</div></details>';
7992     });
7993     });
7994     wrap.innerHTML = html;
7995     renderTheoryCanvases(wrap);
7996     wrap.addEventListener('click', e=>{
7997     const b = e.target && e.target.closest ? e.target.closest('.th-studio') : null;
7998     if(!b) return;
7999     P.type = b.dataset.id;
8000     syncSliders();
8001     redraw(true);
8002     showPage('studio');
8003     ntf(getPattern(P.type).nm + ' – Studio da ochildi', 'ok');
8004     });
8005     }
8006     function renderTheoryCanvases(wrap){
8007     const list = wrap.querySelectorAll ? wrap.querySelectorAll('canvas[data-pt]') : [];
8008     const keepBg = P._bg;
8009     Array.prototype.forEach.call(list, cnv=>{
8010     const octx = cnv.getContext ? cnv.getContext('2d') : null;
8011     if(!octx) return;
8012     const id = cnv.dataset ? cnv.dataset.pt : null;
8013     const pal = PAL.gold;
8014     P._bg = pal.bg;
8015     octx.fillStyle = pal.bg;
8016     octx.fillRect(0,0,200,200);
8017     try{
8018     drawPattern(octx, 100, 100, 92, {
8019     type:id, sym:8, cmp:4, sw:1.7, ir:0.38, spir:0.4,
8020     dens:0.55, lay:1, skew:0, rot:0, pal:'gold', fill:'dual'
8021     });
8022     }catch(e){}
8023     });
8024     P._bg = keepBg;
8025     }
8026

```

```

8027  /* =====
8028  NAVIGATSIYA + INIT
8029  ===== */
8030  function showPage(name){
8031    document.querySelectorAll('.page').forEach(pg=>{
8032      pg.classList.toggle('active', pg.id === 'page-'+name);
8033    });
8034    document.querySelectorAll('nav.tabs button').forEach(b=>{
8035      b.classList.toggle('active', b.dataset.page === name);
8036    });
8037    if(name !== 'ar' && arActive) stopAR(); /* sahifadan chiqilganda kamera o'chadi */
8038    if(name === '3d'){ init3D(); if(threeReady) update3DTexture(); }
8039    if(name === 'studio') setTimeout(resizeCanvas, 30);
8040  }
8041  function wireUI(){
8042    const tabs = $('tabs');
8043    if(tabs) tabs.addEventListener('click', e=>{
8044      const b = e.target.closest('button');
8045      if(b && b.dataset.page) showPage(b.dataset.page);
8046    });
8047    /* accordion */
8048    document.querySelectorAll('.acc > .acc-h').forEach(h=>{
8049      h.addEventListener('click', ()=>{
8050        const acc = h.parentNode;
8051        if(acc) acc.classList.toggle('open');
8052      });
8053    });
8054    /* mobil sheet */
8055    const sideT = $('side-toggle'), side = $('side');
8056    if(sideT && side) sideT.addEventListener('click', ()=> side.classList.toggle('open'));
8057    /* studio boshqaruvlari */
8058    const rndB = $('btn-random'); if(rndB) rndB.addEventListener('click', randomize);
8059    const saveB = $('btn-save'); if(saveB) saveB.addEventListener('click', saveCurrent);
8060    const fillSel = $('sel-fill'); if(fillSel) fillSel.addEventListener('change', ()=>{ P.fill = fillSel.value; PIC_
8061    TILE_CACHE = {}; redraw(); });
8062    const tessChk = $('chk-tess'); if(tessChk) tessChk.addEventListener('change', ()=>{ TESS.on = tessChk.checked; r
8063    edraw(); });
8064    const tessSel = $('sel-tess'); if(tessSel) tessSel.addEventListener('change', ()=>{ TESS.group = tessSel.value;
8065    if(TESS.on) redraw(); });
8066    const cellSl = $('sl-cell'); if(cellSl) cellSl.addEventListener('input', ()=>{
8067      TESS.cell = parseInt(cellSl.value,10);
8068      const v = $('v-cell'); if(v) v.textContent = TESS.cell+'px';
8069      if(TESS.on) redraw();
8070    });
8071    const cellB = $('chk-cellb'); if(cellB) cellB.addEventListener('change', ()=>{ TESS.showCell = cellB.checked; if
8072    (TESS.on) redraw(); });
8073    const bleedB = $('chk-bleed'); if(bleedB) bleedB.addEventListener('change', ()=>{ TESS.bleed = bleedB.checked; i
8074    f(TESS.on) redraw(); });
8075    const nrB = $('chk-noring'); if(nrB) nrB.addEventListener('change', ()=>{ TESS.noring = nrB.checked; if(TESS.on)
8076    redraw(); });
8077    const g3 = $('sel-3dgrid'); if(g3) g3.addEventListener('change', ()=>{
8078      if(g3.value === 'off'){ T3D.tess = false; }
8079      else { T3D.tess = true; T3D.group = g3.value; }
8080      if(threeReady) update3DTexture();
8081    });
8082    /* eksport modal */
8083    const expB = $('btn-export'); if(expB) expB.addEventListener('click', openExport);
8084    const expC = $('exp-cancel'); if(expC) expC.addEventListener('click', closeExport);
8085    const expG = $('exp-go'); if(expG) expG.addEventListener('click', doExport);
8086    const back = $('modal-export'); if(back) back.addEventListener('click', e=>{ if(e.target===back) closeExport();
8087    });
8088    segWire('exp-format','format',false);
8089    segWire('exp-format2','format',false);
8090    segWire('exp-size','size',true);
8091    segWire('exp-dpi','dpi',true);
8092    segWire('exp-jqsize','jqsize',true);
8093    segWire('exp-jqgroup','jqgroup',false);
8094    segWire('exp-jqdepth','jqdepth',true);
8095    segWire('exp-mm','mm',true);
8096    segWire('exp-fill','fill',false);
8097    /* Tayyor o'lchamni bosganda raqamli maydonlar ham yangilanadi –
8098    shunda foydalanuvchi qaysi qiymat ketayotganini har doim ko'radi. */
8099    const linkSeg = (segId, aId, bId)>{
8100      const box = $(segId); if(!box) return;
8101      box.addEventListener('click', e=>{
8102        const b = e.target.closest('button'); if(!b) return;
8103        const v = b.dataset.v;
8104        const a = $(aId), c = $(bId);
8105        if(a){ a.value = v; delete a.dataset.touched; }
8106        if(c){ c.value = v; delete c.dataset.touched; }
8107      });
8108    };
8109    linkSeg('exp-mm', 'exp-mm-w', 'exp-mm-h');
8110    linkSeg('exp-size', 'exp-px-w', 'exp-px-h');
8111    linkSeg('exp-jqsize', 'exp-jq-c', null);
8112    /* Raqam qo'lda o'zgartirilsa – tayyor tugmalardan tanlovni olib tashlaymiz,
8113    chunki endi qiymat ularnikiga mos kelmasligi mumkin. */
8114    const unsel = (inputIds, segId)>{
8115      for(const id of inputIds){
8116        const el = $(id); if(!el) continue;
8117        el.addEventListener('input', ()=>{

```

```

8111     el.dataset.touched = '1';
8112     const box = $(segId); if(!box) return;
8113     const a = $(inputIds[0]), b = inputIds[1] ? $(inputIds[1]) : null;
8114     const same = a && (!b || a.value === b.value) ? a.value : null;
8115     box.querySelectorAll('button').forEach(x=>{
8116       x.classList.toggle('sel', same !== null && x.dataset.v === same);
8117     });
8118     if(segId === 'exp-jqsize'){
8119       const v = parseInt(el.value, 10);
8120       if(isFinite(v)){ EXP.jqsize = Math.min(4096, Math.max(32, v)); expJqInfo(); }
8121     }
8122   });
8123 }
8124 };
8125 unsel(['exp-mm-w', 'exp-mm-h'], 'exp-mm');
8126 unsel(['exp-px-w', 'exp-px-h'], 'exp-size');
8127 unsel(['exp-jq-c', 'exp-jqsize');
8128 /* 3D */
8129 const objSel = $('sel-obj'); if(objSel) objSel.addEventListener('change', ()=>{ if(threeReady) setObject(objSel.
value); });
8130 const lightSel = $('sel-light'); if(lightSel) lightSel.addEventListener('change', ()=>{ if(threeReady) setLights
(lightSel.value); });
8131 const rotB = $('btn-rotate'); if(rotB) rotB.addEventListener('click', ()=>{
8132   if(!threeReady){ ntf('3D hali yuklanmadi', 'err'); return; }
8133   T3.auto = !T3.auto;
8134   rotB.textContent = T3.auto ? '□ To\'xtatish' : '□ Avto-aylanish';
8135 });
8136 const shotB = $('btn-shot'); if(shotB) shotB.addEventListener('click', screenshot3D);
8137 const paintB = $('btn-paint'); if(paintB) paintB.addEventListener('click', animatePaint3D);
8138 const photoB = $('btn-photo'); photoF = $('photo-file');
8139 if(photoB && photoF) photoB.addEventListener('click', ()=> photoF.click());
8140 if(photoF) photoF.addEventListener('change', ()=>{
8141   const f = photoF.files && photoF.files[0];
8142   if(!f) return;
8143   const reader = new FileReader();
8144   reader.onload = ()=>{
8145     const img = new Image();
8146     img.onload = ()=>{
8147       PH.img = img;
8148       const panel = $('photo-panel'); if(panel) panel.hidden = false;
8149       if(threeReady) update3DTexture();
8150       ntf('Rasm ramkaga joylandi □', 'ok');
8151     };
8152     img.onerror = ()=> ntf('Rasm ochilmadi', 'err');
8153     img.src = reader.result;
8154   };
8155   reader.onerror = ()=> ntf('Fayl o\'qilmadi', 'err');
8156   reader.readAsDataURL(f);
8157   photoF.value = '';
8158 });
8159 const frameSel = $('sel-frame'); if(frameSel) frameSel.addEventListener('change', ()=>{ PH.shape = frameSel.valu
e; if(threeReady) update3DTexture(); });
8160 const psize = $('sl-psize'); if(psize) psize.addEventListener('input', ()=>{
8161   PH.size = parseInt(psize.value, 10);
8162   const v = $('v-psize'); if(v) v.textContent = PH.size+'%';
8163   if(threeReady) update3DTexture();
8164 });
8165 const pframe = $('sl-pframe'); if(pframe) pframe.addEventListener('input', ()=>{
8166   PH.frame = parseInt(pframe.value, 10);
8167   const v = $('v-pframe'); if(v) v.textContent = PH.frame;
8168   if(threeReady) update3DTexture();
8169 });
8170 const ptext = $('inp-ptext'); if(ptext) ptext.addEventListener('input', ()=>{
8171   PH.text = ptext.value;
8172   if(threeReady) update3DTexture();
8173 });
8174 const pdate = $('inp-pdate'); if(pdate) pdate.addEventListener('input', ()=>{
8175   PH.date = pdate.value;
8176   if(threeReady) update3DTexture();
8177 });
8178 const photoDel = $('btn-photo-del'); if(photoDel) photoDel.addEventListener('click', ()=>{
8179   PH.img = null;
8180   const panel = $('photo-panel'); if(panel) panel.hidden = true;
8181   if(threeReady) update3DTexture();
8182   ntf('Rasm o\'chirildi', 'ok');
8183 });
8184 /* AR */
8185 const arStart = $('btn-ar-start'); if(arStart) arStart.addEventListener('click', startAR);
8186 const arSim = $('btn-ar-sim'); if(arSim) arSim.addEventListener('click', startARSim);
8187 const arStop = $('btn-ar-stop'); if(arStop) arStop.addEventListener('click', stopAR);
8188 const arCap = $('btn-ar-capture'); if(arCap) arCap.addEventListener('click', captureAR);
8189 const arSurf = $('ar-surface'); if(arSurf) arSurf.addEventListener('change', ()=>{ AR.surface = arSurf.value; })
;
8190 const arOp = $('ar-op'); if(arOp) arOp.addEventListener('input', ()=>{ AR.op = parseInt(arOp.value, 10)/100; });
8191 const arSize = $('ar-size'); if(arSize) arSize.addEventListener('input', ()=>{ AR.size = parseInt(arSize.value, 1
0)/100; });
8192 const arRot = $('ar-rot'); if(arRot) arRot.addEventListener('input', ()=>{ AR.rot = parseInt(arRot.value, 10); })
;
8193 const overlay = $('ar-overlay');
8194 if(overlay){
8195   let dragging = false, lx = 0, ly = 0;

```

```

8196     overlay.addEventListener('pointerdown', e=>{ dragging = true; lx = e.clientX; ly = e.clientY; overlay.setPoint
erCapture && overlay.setPointerCapture(e.pointerId); });
8197     overlay.addEventListener('pointermove', e=>{
8198         if(!dragging) return;
8199         AR.x += e.clientX - lx; AR.y += e.clientY - ly;
8200         lx = e.clientX; ly = e.clientY;
8201     });
8202     overlay.addEventListener('pointerup', ()=>{ dragging = false; });
8203     overlay.addEventListener('pointercancel', ()=>{ dragging = false; });
8204 }
8205 window.addEventListener('pagehide', stopAR);
8206 /* Chat */
8207 const sendB = $('btn-chat-send'); if(sendB) sendB.addEventListener('click', sendChat);
8208 const inp = $('chat-input'); if(inp) inp.addEventListener('keydown', e=>{ if(e.key==='Enter') sendChat(); });
8209 window.addEventListener('resize', ()=>{
8210     const pg = $('page-studio');
8211     if(pg && pg.classList.contains('active')) resizeCanvas();
8212 });
8213 }
8214 /* =====
8215     AUTENTIFIKATSIYA — ikki rejimli
8216     =====
8217     1) SUPABASE rejimi (production): haqiqiy PostgreSQL database,
8218     bcrypt parol xeshi (Supabase auth.users ichida), Google OAuth,
8219     rollar, status, parol tiklash, email tasdiqlash, audit log.
8220     Yoqish: quyidagi SUPABASE_URL/SUPABASE_ANON_KEY ni to'ldiring
8221     yoki ilovadan "Server sozlash" orqali kiriting.
8222     2) LOKAL rejim (zaxira): server sozlanmaganda ilova baribir
8223     ishlaydi — hisoblar shu qurilmada localStorage'da qoladi.
8224     Demo/oflayn uchun. Bunda admin panel va parol tiklash yo'q.
8225
8226     anon key MAXFIY EMAS — u brauzerga chiqarilishi Supabase
8227     arxitekturasida normal; haqiqiy himoya Row Level Security (RLS)
8228     orqali serverda amalga oshiriladi (supabase/schema.sql ga qarang).
8229     service_role kalitini esa BU YERGA HECH QACHON yozmang.
8230     ===== */
8231 const SUPABASE_URL = ''; /* masalan: https://abcdefgh.supabase.co */
8232 const SUPABASE_ANON_KEY = ''; /* "anon public" kaliti */
8233
8234 const AUTH = { mode:'local', client:null, user:null, profile:null, ready:false };
8235
8236 function sbCfg(){
8237     let url = SUPABASE_URL, key = SUPABASE_ANON_KEY;
8238     try{
8239         url = (localStorage.getItem('naqsh_sb_url') || url || '').trim();
8240         key = (localStorage.getItem('naqsh_sb_key') || key || '').trim();
8241     }catch(e){}
8242     return { url, key };
8243 }
8244 function sbCfgSet(url, key){
8245     try{
8246         localStorage.setItem('naqsh_sb_url', (url||'').trim());
8247         localStorage.setItem('naqsh_sb_key', (key||'').trim());
8248         return true;
8249     }catch(e){ return false; }
8250 }
8251 function sbEnabled(){ return AUTH.mode === 'supabase' && !!AUTH.client; }
8252
8253 /* Texnik xatoni foydalanuvchiga tushunarli o'zbekcha xabarga aylantiradi.
8254     Batafsil xato faqat konsolga (developer uchun) yoziladi. */
8255 function authErrMsg(err){
8256     const raw = (err && (err.message || err.error_description || '')) || '';
8257     console.error('[auth]', err);
8258     const m = raw.toLowerCase();
8259     if(m.includes('invalid login credentials')) return 'Email yoki parol noto\'g\'ri';
8260     if(m.includes('email not confirmed')) return 'Email tasdiqlanmagan — pochtagizdagi havolani bosing';
8261     if(m.includes('user already registered') || m.includes('already been registered')) return 'Bu email allaqachon ro\'yxatdan o\'tgan';
8262     if(m.includes('password should be')) return 'Parol kamida 6 belgi bo\'lishi kerak';
8263     if(m.includes('rate limit') || m.includes('too many')) return 'Juda ko\'p urinish — bir necha daqiqadan so\'ng qayta urinin
g';
8264     if(m.includes('failed to fetch') || m.includes('networkerror')) return 'Serverga ulanib bo\'lmadi — internetni tekshiring';
8265     return 'Xatolik yuz berdi — birozdan so\'ng qayta urining';
8266 }
8267
8268 async function authLogEvent(event, meta){
8269     if(!sbEnabled()) return;
8270     try{
8271         await AUTH.client.from('auth_events').insert({
8272             user_id: AUTH.user ? AUTH.user.id : null,
8273             email: AUTH.user ? AUTH.user.email : null,
8274             event, meta: meta || null
8275         });
8276     }catch(e){ console.error('[auth] log', e); }
8277 }
8278
8279 async function authLoadProfile(){
8280     if(!sbEnabled() || !AUTH.user){ AUTH.profile = null; return null; }
8281     try{

```

```

8285     const { data, error } = await AUTH.client.from('profiles').select('*').eq('id', AUTH.user.id).single();
8286     if(error) throw error;
8287     AUTH.profile = data;
8288     return data;
8289   }catch(e){ console.error('[auth] profile', e); AUTH.profile = null; return null; }
8290 }
8291
8292 /* Bloklangan hisob kira olmasligi kerak. Supabase auth.users status'ni
8293    bilmaydi, shuning uchun kirgandan keyin profil status'ini tekshiramiz. */
8294 async function authEnforceStatus(){
8295   if(!sbEnabled() || !AUTH.profile) return true;
8296   const st = AUTH.profile.status;
8297   if(st === 'blocked' || st === 'deactivated'){
8298     const msg = st === 'blocked' ? 'Hisob bloklangan – administrator bilan bog\'laning'
8299       : 'Hisob faolsizlantirilgan';
8300     await AUTH.client.auth.signOut();
8301     AUTH.user = null; AUTH.profile = null;
8302     ntf(msg, 'err');
8303     return false;
8304   }
8305   return true;
8306 }
8307
8308 async function authBootstrap(){
8309   const { url, key } = sbCfg();
8310   if(url && key && typeof supabase !== 'undefined' && supabase.createClient){
8311     try{
8312       AUTH.client = supabase.createClient(url, key, {
8313         auth:{ persistSession:true, autoRefreshToken:true, detectSessionInUrl:true }
8314       });
8315       AUTH.mode = 'supabase';
8316       const { data } = await AUTH.client.auth.getSession();
8317       AUTH.user = data && data.session ? data.session.user : null;
8318       if(AUTH.user){ await authLoadProfile(); await authEnforceStatus(); await cloudLoadPatterns(); }
8319       AUTH.client.auth.onAuthStateChange(async (evt, session)=>{
8320         const wasUser = AUTH.user;
8321         AUTH.user = session ? session.user : null;
8322         if(AUTH.user){
8323           await authLoadProfile();
8324           /* yangi kirish bo'lsa – mehmon naqshlarini ko'chiramiz, so'ng yuklaymiz */
8325           if(!wasUser){
8326             const moved = await cloudMigrateGuest();
8327             if(moved) ntf(moved + ' ta naqsh hisobingizga ko\'chirildi 🔄', 'ok');
8328           }
8329           await cloudLoadPatterns();
8330         } else {
8331           AUTH.profile = null; CLOUD.patterns = null;
8332         }
8333         updateUserBtn(); renderGallery(); renderProfileCard();
8334         if(evt === 'PASSWORD_RECOVERY') openNewPassword();
8335       });
8336     }catch(e){
8337       console.error('[auth] init', e);
8338       AUTH.mode = 'local'; AUTH.client = null;
8339     }
8340   }
8341   AUTH.ready = true;
8342   updateUserBtn();
8343 }
8344
8345 /* ----- LOKAL rejim (zaxira) ----- */
8346 function usersGet(){ return lsGet('naqsh_users', {}); }
8347 function usersSet(u){ return lsSet('naqsh_users', u); }
8348 function sessionGet(){ return lsGet('naqsh_session', null); }
8349 function sessionSet(v){
8350   try{
8351     if(v===null) localStorage.removeItem('naqsh_session');
8352     else localStorage.setItem('naqsh_session', JSON.stringify(v));
8353   }catch(e){}
8354 }
8355 async function hashPass(pass, salt){
8356   try{
8357     const data = new TextEncoder().encode(salt + '::naqshai::' + pass);
8358     const buf = await crypto.subtle.digest('SHA-256', data);
8359     return Array.from(new Uint8Array(buf)).map(b=>b.toString(16).padStart(2,'0')).join('');
8360   }catch(e){
8361     return 'f' + hashStr(salt + '::naqshai::' + pass).toString(16); /* zaxira yo'l */
8362   }
8363 }
8364 function randSalt(){
8365   let s = '';
8366   for(let i=0;i<16;i++) s += Math.floor(Math.random()*36).toString(36);
8367   return s + Date.now().toString(36);
8368 }
8369
8370 /* ----- Umumiy API (UI shu nomlarni chaqiradi) ----- */
8371 function currentUser(){
8372   if(!sbEnabled()){
8373     if(!AUTH.user) return null;
8374     const p = AUTH.profile || {};
8375     const nm = [p.first_name, p.last_name].filter(Boolean).join(' ')

```

```

8376         || (AUTH.user.email||'').split('@')[0];
8377     return {
8378         login: AUTH.user.email, name: nm,
8379         created: p.created_at ? Date.parse(p.created_at) : Date.now(),
8380         role: p.role || 'user', status: p.status || 'active',
8381         provider: p.provider || 'email', verified: !!p.email_verified
8382     };
8383 }
8384 const s = sessionGet();
8385 if(!s) return null;
8386 const u = usersGet();
8387 return u[s] ? {login:s, name:u[s].name, created:u[s].created, role:'user', status:'active', provider:'email', ve
8388 rified:true} : null;
8389 }
8390 function isAdmin(){
8391     const u = currentUser();
8392     return !!u && (u.role === 'admin' || u.role === 'super_admin');
8393 }
8394 async function registerUser(name, login, pass, pass2, lastName){
8395     name = (name||'').trim(); login = (login||'').trim().toLowerCase();
8396     lastName = (lastName||'').trim();
8397     if(!name || !login){ ntf('Ism va email kiritilishi shart', 'err'); return false; }
8398     if(!/^[^s@]+@[^\s@]+\.[^\s@]{2,}$/.test(login)){ ntf('Email formati noto\'g\'ri', 'err'); return false; }
8399     if((pass||'').length < 6){ ntf('Parol kamida 6 belgi', 'err'); return false; }
8400     if(pass !== pass2){ ntf('Parollar mos emas', 'err'); return false; }
8401 }
8402 if(sbEnabled()){
8403     try{
8404         const { data, error } = await AUTH.client.auth.signUp({
8405             email: login, password: pass,
8406             options:{ data:{ first_name:name, last_name:lastName }, emailRedirectTo: location.href.split('#')[0] }
8407         });
8408         if(error){ ntf(authErrMsg(error), 'err'); return false; }
8409         if(data && data.user && !data.session){
8410             ntf('Email manzilingizni tasdiqlang – pochtagizga havola yuborildi ☐', 'ok');
8411             return true;
8412         }
8413         AUTH.user = data.session ? data.session.user : null;
8414         await authLoadProfile();
8415         return true;
8416     }catch(e){ ntf(authErrMsg(e), 'err'); return false; }
8417 }
8418 }
8419 const users = usersGet();
8420 if(users[login]){ ntf('Bu email allaqachon ro\'yxatdan o\'tgan', 'err'); return false; }
8421 const salt = randSalt();
8422 const hash = await hashPass(pass, salt);
8423 const guest = lsGet('naqsh_saved', []);
8424 const full = [name, lastName].filter(Boolean).join(' ');
8425 users[login] = { name: full, salt, hash, created:Date.now(), patterns:guest.slice(0,48) };
8426 if(!usersSet(users)){ ntf('Saqlashda xato (localStorage)', 'err'); return false; }
8427 sessionSet(login);
8428 return true;
8429 }
8430 }
8431 async function loginUser(login, pass){
8432     login = (login||'').trim().toLowerCase();
8433     if(sbEnabled()){
8434         try{
8435             const { data, error } = await AUTH.client.auth.signInWithPassword({ email: login, password: pass||'' });
8436             if(error){
8437                 /* xavfsizlik: hisob bor-yo'qligini oshkor qilmaymiz */
8438                 ntf(authErrMsg(error), 'err');
8439                 try{ await AUTH.client.from('auth_events').insert({ email: login, event:'failed_login' }); }catch(_){}
8440                 return false;
8441             }
8442             AUTH.user = data.user;
8443             await authLoadProfile();
8444             if(!(await authEnforceStatus())) return false;
8445             try{ await AUTH.client.rpc('touch_last_login'); }catch(_){}
8446             await authLogEvent('login');
8447             return true;
8448         }catch(e){ ntf(authErrMsg(e), 'err'); return false; }
8449     }
8450 }
8451 const users = usersGet();
8452 const rec = users[login];
8453 if(!rec){ ntf('Email yoki parol noto\'g\'ri', 'err'); return false; }
8454 const hash = await hashPass(pass||'', rec.salt);
8455 if(hash !== rec.hash){ ntf('Email yoki parol noto\'g\'ri', 'err'); return false; }
8456 sessionSet(login);
8457 return true;
8458 }
8459 }
8460 async function logoutUser(){
8461     if(sbEnabled()){
8462         await authLogEvent('logout');
8463         try{ await AUTH.client.auth.signOut(); }catch(e){}
8464         AUTH.user = null; AUTH.profile = null;
8465     } else {

```

```

8466     sessionSet(null);
8467 }
8468 updateUserBtn();
8469 renderGallery();
8470 closeProfile();
8471 ntf('Hisobdan chiqildi', 'ok');
8472 }
8473
8474 async function changePassword(oldP, newP, newP2){
8475     if((newP||'').length < 6){ ntf('Yangi parol kamida 6 belgi', 'err'); return false; }
8476     if(newP !== newP2){ ntf('Yangi parollar mos emas', 'err'); return false; }
8477
8478     if(sbEnabled()){
8479         try{
8480             /* joriy parolni tekshirish uchun qayta kirishga urinamiz */
8481             const email = AUTH.user && AUTH.user.email;
8482             const { error: e1 } = await AUTH.client.auth.signInWithPassword({ email, password: oldP||'' });
8483             if(e1){ ntf('Joriy parol noto\'g\'ri', 'err'); return false; }
8484             const { error: e2 } = await AUTH.client.auth.updateUser({ password: newP });
8485             if(e2){ ntf(authErrMsg(e2), 'err'); return false; }
8486             await authLogEvent('password_changed');
8487             ntf('Parol muvaffaqiyatli o\'zgartirildi', 'ok');
8488             return true;
8489         }catch(e){ ntf(authErrMsg(e), 'err'); return false; }
8490     }
8491
8492     const u = currentUser();
8493     if(!u){ ntf('Avval hisobga kiring', 'err'); return false; }
8494     const users = usersGet();
8495     const rec = users[u.login];
8496     const oldHash = await hashPass(oldP||'', rec.salt);
8497     if(oldHash !== rec.hash){ ntf('Joriy parol noto\'g\'ri', 'err'); return false; }
8498     rec.salt = randSalt();
8499     rec.hash = await hashPass(newP, rec.salt);
8500     usersSet(users);
8501     ntf('Parol muvaffaqiyatli o\'zgartirildi', 'ok');
8502     return true;
8503 }
8504
8505 /* ----- Parolni tiklash ----- */
8506 async function requestPasswordReset(email){
8507     email = (email||'').trim().toLowerCase();
8508     if(!/^[^\s@]+@[^\s@]+\.[^\s@]{2,}$/.test(email)){ ntf('Email formati noto\'g\'ri', 'err'); return false; }
8509     if(!sbEnabled()){
8510         ntf('Parolni tiklash uchun server (Supabase) sozlanishi kerak', 'err');
8511         return false;
8512     }
8513     try{
8514         const { error } = await AUTH.client.auth.resetPasswordForEmail(email, {
8515             redirectTo: location.href.split('#')[0]
8516         });
8517         if(error){ ntf(authErrMsg(error), 'err'); return false; }
8518         /* xavfsizlik: email ro'yxatda bor-yo'qligini oshkor qilmaymiz */
8519         ntf('Agar bunday hisob mavjud bo\'lsa, tiklash havolasi yuborildi', 'ok');
8520         return true;
8521     }catch(e){ ntf(authErrMsg(e), 'err'); return false; }
8522 }
8523
8524 async function applyNewPassword(newP, newP2){
8525     if((newP||'').length < 6){ ntf('Parol kamida 6 belgi', 'err'); return false; }
8526     if(newP !== newP2){ ntf('Parollar mos emas', 'err'); return false; }
8527     if(!sbEnabled()){ ntf('Server sozlanmagan', 'err'); return false; }
8528     try{
8529         const { error } = await AUTH.client.auth.updateUser({ password: newP });
8530         if(error){ ntf(authErrMsg(error), 'err'); return false; }
8531         await authLogEvent('password_reset');
8532         ntf('Parol yangilandi — endi kirishingiz mumkin', 'ok');
8533         return true;
8534     }catch(e){ ntf(authErrMsg(e), 'err'); return false; }
8535 }
8536
8537 /* ----- Google orqali kirish (Supabase OAuth) ----- */
8538 async function googleSignIn(){
8539     if(!sbEnabled()){
8540         ntf('Google orqali kirish uchun avval o Server sozlash (Supabase) ni bajaring', 'err');
8541         const box = $('gcid-box'); if(box) box.hidden = false;
8542         return;
8543     }
8544     try{
8545         const { error } = await AUTH.client.auth.signInWithOAuth({
8546             provider: 'google',
8547             options:{ redirectTo: location.href.split('#')[0] }
8548         });
8549         if(error) ntf(authErrMsg(error), 'err');
8550     }catch(e){ ntf(authErrMsg(e), 'err'); }
8551 }
8552
8553 /* =====
8554 ADMIN PANEL — foydalanuvchilarni boshqarish
8555 Diqqat: bu yerdagi tekshiruvlar faqat KO'RINISH uchun. Haqiqiy
8556 ruxsat serverda — Supabase RLS siyosatlari va protect_privileged_columns
8557 triggeri bilan ta'minlanadi (supabase/schema.sql). Oddiy foydalanuvchi
8558 so'rov yuborsa ham server rad etadi.

```



```

8557 ===== */
8558 const ADM = { rows: [], q:'', status:'', role:'' };
8559
8560 function openAdmin(){
8561   if(!isEnabled()){ ntf('Admin panel uchun server (Supabase) sozlanishi kerak', 'err'); return; }
8562   if(!isAdmin()){ ntf('Ruxsat yo\'q', 'err'); return; }
8563   const m=$( 'modal-admin' ); if(m) m.classList.add('open');
8564   admLoad();
8565 }
8566 function closeAdmin(){ const m=$( 'modal-admin' ); if(m) m.classList.remove('open'); }
8567
8568 async function admLoad(){
8569   if(!isEnabled() || !isAdmin()) return;
8570   try{
8571     const [{ data: rows, error: e1 }, { data: stats }] = await Promise.all([
8572       AUTH.client.from('profiles').select('*').order('created_at', { ascending:false }).limit(500),
8573       AUTH.client.rpc('admin_stats')
8574     ]);
8575     if(e1) throw e1;
8576     ADM.rows = rows || [];
8577     admRenderStats(stats);
8578     admRender();
8579   }catch(e){ ntf(authErrMsg(e), 'err'); }
8580 }
8581
8582 function admRenderStats(st){
8583   const box = $( 'adm-stats' ); if(!box) return;
8584   if(!st){ box.innerHTML = ''; return; }
8585   const cell = (v,l)=> '<div class="adm-stat"><b>'+(v==null?'-':v)+'</b><span>'+escHTML(l)+'</span></div>';
8586   box.innerHTML =
8587     cell(st.total, 'Jami foydalanuvchi') + cell(st.active, 'Faol') +
8588     cell(st.today, 'Bugun qo\'shilgan') + cell(st.blocked, 'Bloklangan') +
8589     cell(st.via_google, 'Google orqali') + cell(st.via_email, 'Email orqali');
8590 }
8591
8592 function admRender(){
8593   const t = $( 'adm-table' ); if(!t) return;
8594   const q = ADM.q.trim().toLowerCase();
8595   const list = ADM.rows.filter(r=>{
8596     if(ADM.status && r.status !== ADM.status) return false;
8597     if(ADM.role && r.role !== ADM.role) return false;
8598     if(q){
8599       const hay = ((r.email||'')+ '+(r.first_name||'')+ '+(r.last_name||')).toLowerCase();
8600       if(hay.indexOf(q) < 0) return false;
8601     }
8602     return true;
8603   });
8604   const me = currentUser();
8605   const superA = me && me.role === 'super_admin';
8606   const ROLES = ['user', 'researcher', 'teacher', 'admin', 'super_admin'];
8607   const fmt = d => d ? new Date(d).toLocaleString('uz-UZ', {year:'numeric', month:'2-digit', day:'2-digit', hour:'2-digit', minute:'2-digit'}) : '-';
8608
8609   let html = '<thead><tr><th>Email</th><th>Ism</th><th>Rol</th><th>Status</th><th>Provider</th>'
8610     + ' <th>Ro\'yxatdan o\'tgan</th><th>Oxirgi kirish</th><th>Amal</th></tr></thead><tbody>';
8611   if(!list.length){
8612     html += '<tr><td colspan="8" style="text-align:center;padding:18px;color:var(--text-soft)">Hech narsa topilmadi</td></tr>';
8613   }
8614   list.forEach(r=>{
8615     const nm = [r.first_name, r.last_name].filter(Boolean).join(' ') || '-';
8616     const canEdit = superA || r.role !== 'super_admin';
8617     const roleSel = '<select data-act="role" data-id="'+r.id+'"+(canEdit?'': 'disabled')>'+
8618       ROLES.filter(x => superA || x !== 'super_admin' || r.role === 'super_admin')
8619       .map(x=>'<option value="'+x+'"+(x===r.role?' selected':'')>'+x+'</option>').join('') +
8620       '</select>';
8621     const blocked = r.status === 'blocked';
8622     const btn = canEdit
8623       ? '<button class="btn sm "+(blocked?'': 'danger')+" data-act="'+(blocked?'unblock':'block')+" data-id="'+r.id+'">'
8624       + (blocked ? 'Blokdan chiqar' : 'Bloklash') + '</button>'
8625       : '<button class="btn sm ghost" data-act="deact" data-id="'+r.id+'"+(r.status==='deactivated'? 'disabled' : '')>Faolsizlantirish</button>'
8626       : '<span style="color:var(--text-soft);font-size:12px"></span>';
8627     html += '<tr>'
8628       + '<td>'+escHTML(r.email||'')+ (r.email_verified? '<span title="Email tasdiqlangan"></span>':'' )</td>'
8629       + '<td>'+escHTML(nm)</td>'
8630       + '<td>'+roleSel</td>'
8631       + '<td><span class="adm-badge "+escHTML(r.status)+'">'+escHTML(r.status)+'</span></td>'
8632       + '<td>'+escHTML(r.provider||'email')</td>'
8633       + '<td>'+fmt(r.created_at)</td>'
8634       + '<td>'+fmt(r.last_login)</td>'
8635       + '<td>'+btn</td>'
8636       + '</tr>';
8637   });
8638   html += '</tbody>';
8639   t.innerHTML = html;
8640 }
8641
8642 async function admSetStatus(id, status){
8643   try{

```

```

8644     const { error } = await AUTH.client.from('profiles').update({ status }).eq('id', id);
8645     if(error) throw error;
8646     ntf('Status yangilandi: ' + status, 'ok');
8647     await admLoad();
8648   }catch(e){ ntf(authErrMsg(e), 'err'); }
8649 }
8650 async function admSetRole(id, role){
8651   try{
8652     const { error } = await AUTH.client.from('profiles').update({ role }).eq('id', id);
8653     if(error) throw error;
8654     ntf('Rol yangilandi: ' + role, 'ok');
8655     await admLoad();
8656   }catch(e){ ntf(authErrMsg(e), 'err'); await admLoad(); }
8657 }
8658
8659 function wireAdmin(){
8660   const t = $('#adm-table');
8661   if(t){
8662     t.addEventListener('click', e=>{
8663       const b = e.target.closest('button[data-act]');
8664       if(!b) return;
8665       const id = b.dataset.id;
8666       if(b.dataset.act === 'block') admSetStatus(id, 'blocked');
8667       if(b.dataset.act === 'unblock') admSetStatus(id, 'active');
8668       if(b.dataset.act === 'deact') admSetStatus(id, 'deactivated');
8669     });
8670     t.addEventListener('change', e=>{
8671       const sel = e.target.closest('select[data-act="role"]');
8672       if(sel) admSetRole(sel.dataset.id, sel.value);
8673     });
8674   }
8675   const q=$('#adm-q'); if(q) q.addEventListener('input', ()=>{ ADM.q = q.value; admRender(); });
8676   const st=$('#adm-status');if(st) st.addEventListener('change', ()=>{ ADM.status = st.value; admRender(); });
8677   const rl=$('#adm-role'); if(rl) rl.addEventListener('change', ()=>{ ADM.role = rl.value; admRender(); });
8678   const rf=$('#adm-refresh');if(rf) rf.addEventListener('click', admLoad);
8679   const cl=$('#adm-close'); if(cl) cl.addEventListener('click', closeAdmin);
8680   const bk=$('#modal-admin');if(bk) bk.addEventListener('click', e=>{ if(e.target===bk) closeAdmin(); });
8681 }
8682
8683 /* ----- Parolni tiklash oynalari ----- */
8684 function openForgot(){ closeAuth(); const m=$('#modal-forgot'); if(m) m.classList.add('open'); }
8685 function closeForgot(){ const m=$('#modal-forgot'); if(m) m.classList.remove('open'); }
8686 function openNewPassword(){ const m=$('#modal-newpass'); if(m) m.classList.add('open'); }
8687 function closeNewPassword(){ const m=$('#modal-newpass'); if(m) m.classList.remove('open'); }
8688
8689 function wireResetUI(){
8690   const f=$('#btn-forgot'); if(f) f.addEventListener('click', openForgot);
8691   const fc=$('#fg-cancel'); if(fc) fc.addEventListener('click', closeForgot);
8692   const fb=$('#modal-forgot'); if(fb) fb.addEventListener('click', e=>{ if(e.target===fb) closeForgot(); });
8693   const fs=$('#fg-send');
8694   if(fs) fs.addEventListener('click', async ()=>{
8695     const el=$('#fg-email');
8696     if(await requestPasswordReset(el?el.value:')) closeForgot();
8697   });
8698   const nc=$('#np-cancel'); if(nc) nc.addEventListener('click', closeNewPassword);
8699   const ns=$('#np-save');
8700   if(ns) ns.addEventListener('click', async ()=>{
8701     const a=$('#np-1'), b=$('#np-2');
8702     if(await applyNewPassword(a?a.value:'', b?b.value:')){
8703       if(a) a.value=''; if(b) b.value='';
8704       closeNewPassword();
8705     }
8706   });
8707 }
8708
8709 /* Auth modalidagi rejim izohi – foydalanuvchi qaysi rejimda ekanini bilsin */
8710 function updateAuthNote(){
8711   const n = $('#auth-mode-note');
8712   if(!n) return;
8713   n.innerHTML = sbEnabled()
8714     ? 'Hisoblar serverda (Supabase) saqlanadi; parol bcrypt bilan xeshlanadi va bizga ko\'rinmaydi.'
8715     : 'Server sozlanmagan – hisob faqat SHU QURILMADA saqlanadi (demo rejimi). Haqiqiy hisoblar, Google kirish
8716 va parol tiklash uchun yuqoridagi «Server sozlash» ni bajaring.';
8717 }
8718
8719 /* ----- Auth UI ----- */
8720 /* Modal har safar KIRISH tabida ochiladi. Aks holda foydalanuvchi bir marta
8721 "Ro'yxatdan o'tish" tabiga o'tsa, tab holati saqlanib qolar va keyingi
8722 ochilishlarda kirish formasi (#auth-login-box) yashirin qolardi –
8723 tashqaridan bu "login umuman ishlaymapti" bo'lib ko'rinadi. */
8724 function authShowTab(which){
8725   const seg = $('#auth-seg');
8726   if(seg) seg.querySelectorAll('button').forEach(b=>{
8727     b.classList.toggle('sel', b.dataset.v === which);
8728   });
8729   const lb=$('#auth-login-box'), rb=$('#auth-reg-box');
8730   if(lb) lb.hidden = which !== 'login';
8731   if(rb) rb.hidden = which !== 'reg';
8732   const err=$('#gcid-box'); if(err) err.hidden = true;
8733 }
8734 function openAuth(tab){

```

```

8734     authShowTab(tab === 'reg' ? 'reg' : 'login');
8735     updateAuthNote();
8736     const m=$( 'modal-auth'); if(m) m.classList.add('open');
8737 }
8738 function closeAuth(){ const m=$( 'modal-auth'); if(m) m.classList.remove('open'); }
8739 function openProfile(){ renderProfile(); const m=$( 'modal-profile'); if(m) m.classList.add('open'); }
8740 function closeProfile(){ const m=$( 'modal-profile'); if(m) m.classList.remove('open'); }
8741 function updateUserBtn(){
8742     const b = $( 'btn-user');
8743     if(!b) return;
8744     const u = currentUser();
8745     b.textContent = u ? (' ' + u.name) : ' Kirish';
8746 }
8747 function renderProfile(){
8748     const body = $( 'profile-body');
8749     if(!body) return;
8750     const u = currentUser();
8751     if(!u){ body.innerHTML = '<p>Hisobga kirilmagan.</p>'; return; }
8752     const pats = getSavedList();
8753     const d = new Date(u.created);
8754     const dateStr = d.getDate()+'.'+(d.getMonth()+1)+'.'+d.getFullYear();
8755     const cloud = (typeof cloudActive === 'function') && cloudActive();
8756     let html = '<div class="prof-row"><span>Ism</span><b>'+escHTML(u.name)+'</b></div>' +
8757     '<div class="prof-row"><span>Email</span><b>'+escHTML(u.login)+'</b></div>' +
8758     '<div class="prof-row"><span>Ro'yxatdan o'tish usuli</span><b>' +
8759     (u.provider === 'google' ? 'Google' : 'Email va parol') + '</b></div>' +
8760     '<div class="prof-row"><span>A'zo bo'lgan sana</span><b>'+dateStr+'</b></div>' +
8761     '<div class="prof-row"><span>Yaratilgan naqshlar</span><b>'+pats.length+' ta</b></div>' +
8762     '<div class="prof-row"><span>Naqshlar qayerda</span><b>' +
8763     (cloud ? ' Hisobingizda – boshqa qurilmada ham ochiladi'
8764     : ' Faqat shu qurilmada') + '</b></div>';
8765     html += '<div class="prof-sec"><h4>Mening naqshlarim</h4><div class="gal-grid" id="prof-gal">';
8766     html += '</div>' + (pats.length ? '' : '<div class="gal-empty">Studio da « Saqlash» bosing – naqshlar shu yerda
to\planadi.</div>') + '</div>';
8767     html += '<div class="prof-sec"><h4>Sozlamalar – parolni o'zgartirish</h4>' +
8768     '<div class="fld"><label>Joriy parol</label><input id="cp-old" type="password" autocomplete="current-password"
></div>' +
8769     '<div class="fld"><label>Yangi parol</label><input id="cp-new" type="password" autocomplete="new-password"></d
iv>' +
8770     '<div class="fld"><label>Yangi parolni tasdiqlang</label><input id="cp-new2" type="password" autocomplete="new
-password"></div>' +
8771     '<button class="btn sm" id="btn-cp"> Parolni o'zgartirish</button></div>';
8772     if(isAdmin()){
8773         html += '<div class="prof-sec"><h4>Administrator</h4>' +
8774         '<button class="btn sm gold" id="btn-admin"> Foydalanuvchilarni boshqarish</button></div>';
8775     }
8776     html += '<div class="prof-sec"><button class="btn sm danger" id="btn-logout"> Hisobdan chiqish</button></div>';
8777     body.innerHTML = html;
8778     /* naqsh thumb lari */
8779     const grid = $( 'prof-gal');
8780     if(grid){
8781         pats.forEach(item=>{
8782             const dv = document.createElement('div');
8783             dv.className = 'gal-item';
8784             const img = document.createElement('img');
8785             img.src = item.thumb;
8786             img.alt = 'naqsh';
8787             dv.appendChild(img);
8788             dv.addEventListener('click', ()=>{
8789                 Object.assign(P, item.p);
8790                 syncSliders(); redraw();
8791                 closeProfile();
8792                 showPage('studio');
8793                 ntf('Naqsh yuklandi', 'ok');
8794             });
8795             grid.appendChild(dv);
8796         });
8797     }
8798     const cpBtn = $( 'btn-cp');
8799     if(cpBtn) cpBtn.addEventListener('click', async ()=>{
8800         const o=$( 'cp-old'), n=$( 'cp-new'), n2=$( 'cp-new2');
8801         const ok = await changePassword(o.o.value, n.n.value, n2.n2.value);
8802         if(ok){ if(o)o.value=''; if(n)n.value=''; if(n2)n2.value=''; }
8803     });
8804     const lo = $( 'btn-logout');
8805     if(lo) lo.addEventListener('click', logoutUser);
8806     const ab = $( 'btn-admin');
8807     if(ab) ab.addEventListener('click', ()=>{ closeProfile(); openAdmin(); });
8808 }
8809 function wireAuth(){
8810     const ub = $( 'btn-user');
8811     if(ub) ub.addEventListener('click', ()=>{ currentUser() ? openProfile() : openAuth(); });
8812     const seg = $( 'auth-seg');
8813     if(seg) seg.addEventListener('click', e=>{
8814         const b = e.target.closest('button');
8815         if(!b) return;
8816         authShowTab(b.dataset.v);
8817     });
8818     const c1=$( 'auth-cancel'); if(c1) c1.addEventListener('click', closeAuth);
8819     const c2=$( 'auth-cancel2'); if(c2) c2.addEventListener('click', closeAuth);
8820     const back=$( 'modal-auth'); if(back) back.addEventListener('click', e=>{ if(e.target===back) closeAuth(); });

```

```

8821   const pback=$( 'modal-profile' ); if(pback) pback.addEventListener('click', e=>{ if(e.target===pback) closeProfile
8822   (); });
8823   const pc=$( 'profile-close' ); if(pc) pc.addEventListener('click', closeProfile);
8824   const doL=$( 'btn-do-login' );
8825   if(doL) doL.addEventListener('click', async ()=>{
8826     const l=$( 'al-login' ), p=$( 'al-pass' );
8827     if(await loginUser(l?.value:'', p?.value: '')){
8828       closeAuth(); updateUserBtn(); renderGallery();
8829       ntf('Xush kelibsiz, ' + currentUser().name + '! +', 'ok');
8830       if(p) p.value='';
8831     }
8832   });
8833   const doR=$( 'btn-do-register' );
8834   if(doR) doR.addEventListener('click', async ()=>{
8835     const n=$( 'au-name' ), ln=$( 'au-last' ), l=$( 'au-login' ), p=$( 'au-pass' ), p2=$( 'au-pass2' );
8836     if(await registerUser(n?.value:'', l?.value:'', p?.value:'', p2?.value:'', ln?.value: '')){
8837       closeAuth(); updateUserBtn(); renderGallery();
8838       ntf('Hisob yaratildi — xush kelibsiz! ☑', 'ok');
8839       if(p) p.value=''; if(p2) p2.value='';
8840     }
8841   });
8842   const alp=$( 'al-pass' );
8843   if(alp) alp.addEventListener('keydown', e=>{ if(e.key==='Enter' && doL) doL.click(); });
8844   const g1=$( 'btn-google-login' ); if(g1) g1.addEventListener('click', googleSignIn);
8845   const g2=$( 'btn-google-reg' ); if(g2) g2.addEventListener('click', googleSignIn);
8846   const gb=$( 'btn-gcid' ); if(gb) gb.addEventListener('click', ()=>{
8847     const box=$( 'gcid-box' );
8848     if(box){
8849       box.hidden = !box.hidden;
8850       if(!box.hidden){
8851         const cfg = sbCfg();
8852         const u=$( 'inp-sburl' ), k=$( 'inp-sbkey' );
8853         if(u) u.value = cfg.url; if(k) k.value = cfg.key;
8854       }
8855     }
8856   });
8857   const gs=$( 'btn-gcid-save' ); if(gs) gs.addEventListener('click', ()=>{
8858     const u=$( 'inp-sburl' ), k=$( 'inp-sbkey' );
8859     const url = u ? u.value.trim() : '', key = k ? k.value.trim() : '';
8860     if(url && !/^https:\/\/[a-z0-9-]+\.\supabase\.co\/?$/i.test(url)){
8861       ntf('Project URL odatda https://xxxx.supabase.co ko\'rinishida bo\'ladi', 'err');
8862     }
8863     if(sbCfgSet(url, key)){
8864       ntf(url && key ? 'Saqlandi ✓ — server ulanmoqda...' : 'Server sozlamalari tozalandi', 'ok');
8865       const box=$( 'gcid-box' ); if(box) box.hidden = true;
8866       authBootstrap().then(()=>{
8867         updateAuthNote();
8868         ntf(sbEnabled() ? 'Serverga ulandi ✓' : 'Lokal (demo) rejimga qaytildi', sbEnabled()?'ok':'err');
8869       });
8870     } else {
8871       ntf('Saqlab bo\'lmadi (localStorage)', 'err');
8872     }
8873   });
8874   const gc=$( 'btn-gcid-close' ); if(gc) gc.addEventListener('click', ()=>{
8875     const box=$( 'gcid-box' ); if(box) box.hidden = true;
8876   });
8877 }
8878 /* — JONLI KURSOR: yumshoq ergashuvchi halqa + bosishda to'liqin — */
8879 function initCursor(){
8880   try{
8881     if(!window.matchMedia || !matchMedia('(pointer:fine)').matches) return;
8882     if(!document.body) return;
8883     const dotEl = document.createElement('div'); dotEl.id = 'cur-dot';
8884     const ringEl = document.createElement('div'); ringEl.id = 'cur-ring';
8885     document.body.appendChild(ringEl);
8886     document.body.appendChild(dotEl);
8887     let mx=-100, my=-100, rx=-100, ry=-100;
8888     document.addEventListener('pointermove', e=>{
8889       mx = e.clientX; my = e.clientY;
8890       dotEl.style.left = mx+'px';
8891       dotEl.style.top = my+'px';
8892       const hot = e.target && e.target.closest ? e.target.closest('button,select,input,a,.pal-chip,.gal-item,canva
8893       s') : null;
8894       if(ringEl.classList) ringEl.classList.toggle('hot', !!hot);
8895     });
8896     const loop = ()=>{
8897       rx += (mx-rx)*0.16; ry += (my-ry)*0.16;
8898       ringEl.style.left = rx+'px';
8899       ringEl.style.top = ry+'px';
8900       requestAnimationFrame(loop);
8901     };
8902     loop();
8903     document.addEventListener('pointerdown', e=>{
8904       const rp = document.createElement('div');
8905       rp.className = 'cur-ripple';
8906       rp.style.left = e.clientX+'px';
8907       rp.style.top = e.clientY+'px';
8908       document.body.appendChild(rp);
8909       setTimeout(()=>{ if(rp.parentNode) rp.parentNode.removeChild(rp); }, 560);
8910     });
8911   }

```

```

8910     }catch(e){}
8911   }
8912   /* == INTRO VIDEO ==
8913   Videoni HTML fayl YONIGA "intro.mp4" nomi bilan qo'ying – avtomatik ijro etiladi.
8914   Boshqa nom/URL uchun quyidagi konstantani o'zgartiring. Bo'sh '' = intro o'chiq. */
8915   const INTRO_VIDEO_SRC = 'intro.mp4';
8916   function initIntro(after){
8917     const ov = $('intro-ov'), v = $('intro-video');
8918     let seen = false;
8919     try{ seen = sessionStorage.getItem('naqsh_intro') === '1'; }catch(e){}
8920     if(!ov || !v || !INTRO_VIDEO_SRC || seen){ after(); return; }
8921     let done = false;
8922     const finish = ()=>{
8923       if(done) return;
8924       done = true;
8925       try{ sessionStorage.setItem('naqsh_intro','1'); }catch(e){}
8926       try{ if(v.pause) v.pause(); }catch(e){}
8927       if(ov.classList) ov.classList.add('done');
8928       setTimeout(()=>{ if(ov.parentNode) ov.parentNode.removeChild(ov); }, 650);
8929       after();
8930     };
8931     v.addEventListener('ended', finish);
8932     v.addEventListener('error', finish); /* video topilmasa – darhol o'tib ketadi */
8933     const skip = $('intro-skip');
8934     if(skip) skip.addEventListener('click', finish);
8935     const mute = $('intro-mute');
8936     if(mute) mute.addEventListener('click', ()=>{
8937       v.muted = !v.muted;
8938       mute.textContent = v.muted ? '□ Ovozni yoqish' : '□ Ovozni o\'chirish';
8939     });
8940     ov.hidden = false;
8941     v.src = INTRO_VIDEO_SRC;
8942     try{
8943       const pr = v.play ? v.play() : null;
8944       if(pr && pr.catch) pr.catch(finish);
8945     }catch(e){ finish(); }
8946     /* fayl yo'q va error kelmasa ham 2s da davom etamiz */
8947     setTimeout(()=>{ if(v.readyState === 0 || v.readyState === undefined) finish(); }, 2000);
8948   }
8949   function initSplash(){
8950     try{
8951       const sp = $('splash');
8952       if(!sp) return;
8953       /* yulduz chizilish animatsiyasini qayta boshlash */
8954       try{
8955         const paths = sp.querySelectorAll ? sp.querySelectorAll('path') : [];
8956         Array.prototype.forEach.call(paths, p=>{
8957           p.style.animation = 'none';
8958           void p.getBoundingClientRect();
8959           p.style.animation = '';
8960         });
8961       }catch(e){}
8962       let seen = false;
8963       try{ seen = sessionStorage.getItem('naqsh_splash') === '1'; }catch(e){}
8964       const delay = seen ? 350 : 1900; /* birinchi tashrif – to'liq sahna, keyin qisqa */
8965       setTimeout(()=>{
8966         if(sp.classList) sp.classList.add('done');
8967         setTimeout(()=>{ if(sp.parentNode) sp.parentNode.removeChild(sp); }, 700);
8968       }, delay);
8969       sp.addEventListener('click', ()=>{
8970         if(sp.classList) sp.classList.add('done');
8971         setTimeout(()=>{ if(sp.parentNode) sp.parentNode.removeChild(sp); }, 700);
8972       });
8973       try{ sessionStorage.setItem('naqsh_splash','1'); }catch(e){}
8974     }catch(e){}
8975   }
8976   function init(){
8977     initIntro(initSplash);
8978     cvEl = $('cv');
8979     buildPatternList();
8980     buildSliders();
8981     buildPalettes();
8982     buildGirihUI();
8983     wireGirihUI();
8984     buildTheory();
8985     buildChips();
8986     renderGallery();
8987     restoreChatHistory();
8988     wireUI();
8989     wireAuth();
8990     wireAdmin();
8991     wireResetUI();
8992     wireAiSetup();
8993     updateUserBtn();
8994     authBootstrap(); /* Supabase sessiyasini tiklash (sozlangan bo'lsa) */
8995     initCursor();
8996     resizeCanvas();
8997     ntf('NaqshAI tayyor – ijodga marhamat! +', 'ok');
8998   }
8999   /* Initialization – try-catch bilan himoyalangan */
9000   document.addEventListener('DOMContentLoaded', function(){

```

```
9001     try{
9002         init();
9003     }catch(e){
9004         console.error('NaqshAI init xatosi:', e);
9005         try{ ntf('Ishga tushirishda xato: ' + e.message, 'err'); }catch(_e){}
9006     }
9007 };
9008
```